Line-Query Decision Trees: Approximation and Strong NP-Completeness

October 14, 2026

Abstract

The Line-Query Decision Tree problem is to identify a hidden planar point using only a supplied separating family of line queries. We give deterministic polynomial approximation factors below 4.464966 for maximum depth and 5.182976 for arbitrary-prior expected depth, along with quasipolynomial $(2 + \varepsilon)$ approximations for both objectives. A shared geometric structure underlies both algorithms: for flat states, Helly's theorem gives a common majority witness, and the convex hull turns each hypothesis's positive answers into a circular interval. The algorithms first discover the interval pattern containing the target, then decode its endpoints. Restricting an optimal tree to a pattern class pays for discovery, while count or entropy decreases pay for decoding. Exact scheduling and short lookahead combine these two credits.

Both exact objectives are strongly NP-complete, including expected depth under a uniform prior. Some restricted results assume a distinguished hypothesis missed by every selected cap; its answers form the all-negative route used by those algorithms. These instances admit stronger restricted algorithms: exact maximum depth for fixed-size caps, an EPTAS for weighted singleton/pair paths and cycles, and a PTAS for uniform fixed-size convex-position caps without singleton tests. Constant-factor inapproximability remains open. All constructed queries are supplied lines; an on-line third answer admits a factor-two transfer. For the distinct randomized objective $\max_h \mathbb{E}_T d_T(h)$, an expected-depth oracle and a finite mixture LP give the same polynomial and quasipolynomial factors. Each execution samples one tree from a distribution supported on at most n trees.

Contents

1 Introduction **4**
1.1 Results and the main ideas 4

2 Preliminaries and basic accounting **7**
2.1 Line-query model and objectives 7
2.2 Binary codes and ordered comparisons 11
2.3 Entropy and information inequalities 12
2.4 Restriction and recursive accounting 14
2.5 Proofs of the coding and accounting facts 16
2.6 Convex geometry and finite set systems 19
2.7 Query orderings and first-cover costs 20
2.8 Normalized queries and hull patterns 20

3 Common-witness geometry and interval tools **21**
3.1 A common majority witness 22
3.2 Adjacent supports and circular patterns 22
3.3 Interval hitting and discovery length 23

3.4 Recovering the interval endpoints . . . 25
3.5 Proofs of the geometric and interval facts . . . 26

4 Maximum-depth scheduling ingredients 30
4.1 Local credits and the priority fallback . . . 31
4.2 Technical proofs of the local ingredients . . . 33

5 Scheduling the first positive answer 37
5.1 The batch-ordering guarantee . . . 39
5.2 Proofs of the batch-ordering propositions . . . 41

6 A polynomial approximation for expected depth 46
6.1 The algorithm and its supporting propositions . . . 46
6.2 Proofs of the supporting propositions . . . 51
6.3 Calibration of the expected-depth bound . . . 63

7 Quasipolynomial approximations approaching two 66
7.1 Algorithm and the three supporting propositions . . . 67
7.2 Proofs of the supporting propositions . . . 69
7.3 A simple calibration; no optimization is needed . . . 75

8 Polynomial maximum depth with rank-subset scheduling 75
8.1 The algorithm and its guarantee . . . 75
8.2 Technical proofs of the propositions . . . 79
8.3 Parameter calibration . . . 85

9 Stronger algorithms for small caps 85
9.1 Exact maximum depth with bounded caps . . . 86
9.2 An EPTAS for weighted singleton and pair caps . . . 88

10 A PTAS for uniform shallow caps in convex position 95
10.1 Supporting propositions and the algorithm . . . 95
10.2 Technical proofs of the supporting propositions . . . 97
10.3 Calibration of the block length . . . 100

11 Randomized per-hypothesis depth and optimal mixing 100
11.1 One algorithm and its supporting propositions . . . 101
11.2 Proofs of the supporting propositions . . . 105
11.3 Calibration and the strongest resulting guarantees . . . 109

12 Allowing hypotheses on supplied lines 110

13 Strong NP-completeness 112
13.1 The geometric realization and the matching choice . . . 113
13.2 Maximum depth: bin loads become branch-height counts . . . 116
13.3 Expected depth: an exact matching formula . . . 121
13.4 Expected depth: bin loads become a quadratic penalty . . . 123

14 Uniform expected-depth hardness 133
14.1 Exact cloud replacement: normalization before accounting . . . 133
14.2 Uniform-prior strong NP-completeness . . . 139

15 Conclusion 141

A Fixed certificate for the two initial batches 142

References **145**

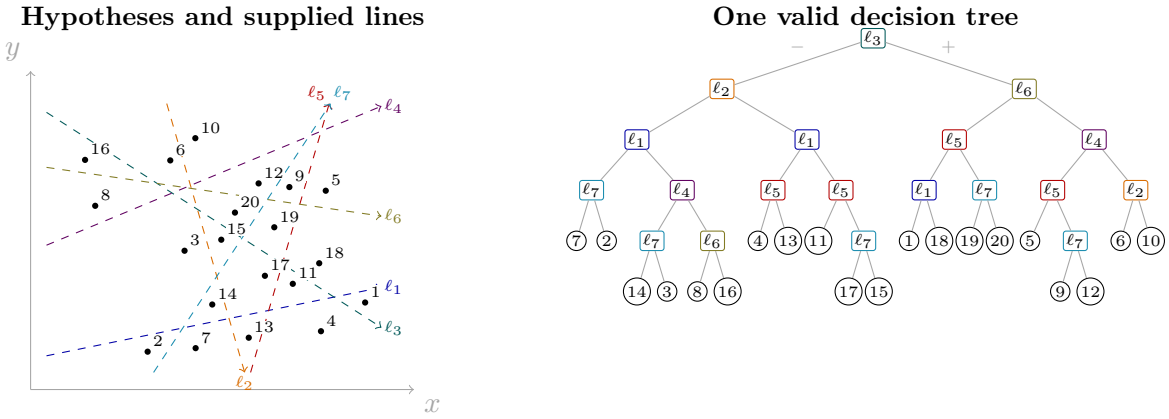


Figure 1: Seven randomly oriented supplied lines are drawn first, then twenty hypotheses are placed in distinct cells of their arrangement. The arrows orient the lines: the positive side is to the left of the arrow. Each internal node queries the correspondingly colored line. At every node the left branch is the negative answer and the right branch is the positive answer. The leaf numbers match the point labels. This valid tree has maximum depth five.

1 Introduction

In the *Line-Query Decision Tree* problem, an unknown point is promised to belong to a finite set of planar hypotheses. A query asks on which side of a specified line the point lies. The line family is part of the input, so each next query must be one of those supplied lines. Its choice may depend on all previous answers. Such a strategy is a decision tree; each leaf names the hypothesis identified by the answers on its path. Every query has unit cost.

We study two objectives for these trees. The *maximum depth* is the largest number of queries used on any hypothesis. The *expected depth* is the average under a given prior. These objectives value the same tree differently: maximum depth is governed by its hardest branch, whereas expected depth can spend more queries on low-probability hypotheses. The expected-depth algorithms accept every strictly positive rational prior in the finite input model; no lower bound on an individual hypothesis probability is assumed.

The supplied-query restriction is what makes the problem interesting. If arbitrary lines were allowed, a projection in a direction giving distinct projected coordinates would order the points, and repeated median splits would give height $\lceil \log_2 n \rceil$. A supplied family may contain no balanced split. For example, if the only useful tests isolate individual points, the worst-case strategy may have to test them one at a time and use depth $n - 1$. The information bound $\log_2 n$ gives a target; the challenge is to find a good tree using the separators that are actually available.

Figure 1 makes the input and output concrete. The left panel has seven supplied lines with varied orientations and twenty hypotheses placed in distinct cells of their arrangement. The right panel is one legal decision tree for that instance: successive line-side answers isolate a single hypothesis. Each internal node uses a line drawn on the left, and its leaves identify the same hypotheses marked by the numbered points.

1.1 Results and the main ideas

In the restricted results, an all-miss instance has a distinguished hypothesis that every selected cap misses; its answers form the all-negative route.

Setup and objective	Guarantee	Running time
General maximum depth	< 4.464966	Polynomial
General arbitrary-prior expected depth	< 5.182976	Polynomial
Either general objective	$2 + \varepsilon$	Quasipolynomial
Randomized $\max_h \mathbb{E}_T d_T(h)$	< 5.182976	Polynomial
Same randomized objective	$2 + \varepsilon$	Quasipolynomial
All-miss, fixed-size caps: maximum depth	Exact	Polynomial
All-miss, weighted singleton/pair paths and cycles: expected depth	$1 + \varepsilon$	EPTAS
All-miss, uniform fixed-size convex caps: expected depth	$1 + \varepsilon$	PTAS

Each row reports the strongest guarantee proved for that setup; the following sections develop the scheduling and coding procedures it uses. The expected-depth guarantee permits every strictly positive rational prior. For $0 < \varepsilon \leq 1$, the general quasipolynomial algorithms run in $(mn)^{O(\varepsilon^{-1} \log n)}$ time.

Both exact objectives are strongly NP-complete, including expected depth under a uniform prior. Constant-factor inapproximability remains open.

The geometric structure behind the algorithms is easiest to see in a state (the current set of hypotheses consistent with all answers so far) where no line splits much of the remaining mass. Every test then has a majority answer. The opposite, minority half-planes are small, and Helly's theorem gives a point lying on all majority sides. After translating this common witness to the origin, each query is represented by a point in a normalized plane. The vertices of their convex hull (the smallest convex set containing those points) suffice to describe the relevant caps: each original cap is covered by at most two adjacent vertex caps, and the vertex answers of any one hypothesis form a *circular interval*: a consecutive block in the cyclic order in which the hull vertices appear around the polygon, with wrap-around allowed. This turns the geometric part of the problem into scheduling and searching over circular intervals.

Maximum depth: schedule discovery against child heights. In a flat state, the algorithm first builds the recursive codes for the hull-pattern classes. It must then discover which class contains the target, so it schedules positive hull answers with deadlines set by the child codes' actual heights. To analyze this discovery cost, follow an optimal tree along the common witness path. If a class stays on that path through a long prefix, restricting the tree to the class removes those queries; the saved depth pays for the class's first positive hull answer in our schedule.

Each original query cap is covered by at most two adjacent hull caps, so the comparison ordering uses singleton and adjacent-pair blocks. Enumerating its first adjacent pair fixes a cut and turns the remaining circular demands into ordinary intervals. The priority scheduler then preserves the comparison credit at every child-height deadline. When the number of required discovery ranks is logarithmic, a subset dynamic program on the spatial Cartesian tree solves the deadline problem in polynomial time; the recursive bounds show that this logarithmic cap suffices for the stronger general factor. We charge endpoint recovery separately. If the witness path already supplies a negative neighboring answer, the decoder can reuse it to fix one endpoint, leaving a smaller height allowance for the child code.

Expected depth: pay once for learning a pattern. For expected depth, the algorithm again discovers the complete hull-answer pattern before recursing. The analysis then separates the cost of finding the pattern from the cost of identifying a hypothesis inside it. Along an optimal witness path, a class's constant prefix disappears when the tree is restricted to that class; those saved

queries pay for finding its first positive hull answer. After discovery, the entropy of the pattern bounds the endpoint information still unknown. To control discovery cost, we first find the largest weighted coverage attainable at each integer budget. A finite search over shifted geometric batches then selects a schedule with a strong first-cover bound. The endpoint decoder may query one or both endpoints of a positive block. If it asks both, the second answer provides information that can shorten later decoding; the adaptive block modes account for that gain while preserving the individual cap bounds used in the schedule analysis. Since every adjacent support block has bounded mass, some targets must wait for later blocks, where the discrete first-cover estimate gives a sharper charge. The all-negative route is charged by the waiting time to complete its schedule. Across the recursive partition, the saved optimal-depth queries and entropy decreases telescope, so each cost is charged only once.

Strong hardness: encode bin loads by matching choices. Both hardness reductions begin with strongly bounded 3-Partition: each item of size a_i must be assigned to one of p bins, and the target is load B in every bin. We encode each item's choice by a matching on its own odd path. The supplied singleton and adjacent-pair queries let the distinguished hypothesis count first-hit groups along that path; at the height threshold, the count can attain its lower bound only when the matching is maximum. Lemma 13.5 then identifies the one unmatched even-position cloud, whose location records the item's choice. The path gadgets therefore turn a tree attaining the threshold into a candidate assignment of all items.

We then calibrate the objective so that it tests this assignment. For maximum depth, cloud sizes control the remaining heights of the branches. The resulting tail histogram rules out internal unmatched positions and requires each bin load to be at most B; since the loads sum to pB, they must all equal B. For expected depth, the matching cost formula separates a load-independent base cost from a quadratic penalty in the deviations $L_b - B$, so the chosen threshold accepts precisely the balanced assignments. These two calculations make the combinatorial choice visible to the respective objectives. The integer-parabola construction realizes each singleton, pair, and private test by an explicit supplied line, completing the geometric instance. Finally, to obtain the uniform-prior result, we replace a weight w by w uniform hypotheses and give the new points private singleton tests. The cost identity adds a common internal identification term for every matching; because this term does not depend on the bin choices, the expected-depth comparison and the reduction are preserved.

Relation to earlier work. Karbasi and Zadimoghaddam's constrained binary identification problem is a close information-theoretic precedent [4]. They relate unrestricted identification to source coding and study how query constraints change the search strategy. In our setting, the common-witness and convex-hull arguments turn the supplied planar tests into circular-interval demands. Once the hull-answer pattern is known, entropy measures the endpoint information still needed to identify the hypothesis, while coverage accounts for the cost of discovering that pattern. We state and prove the entropy chain rule, log-sum, and prefix-code bounds used in this accounting. Cover and Thomas [3] give broader background on these information-theoretic tools.

The coverage analysis draws on the first-cover viewpoint in min-sum set cover. Feige, Lovász, and Tetali charge each element by the rank of its first covering set [1]; this is the objective formalized in Lemma 5.5. Gupta, Nagarajan, and Ravi use a variant of min-sum set cover to approximate optimal expected-cost decision trees, alongside their work on adaptive TSP [7]. These works provide a general first-cover accounting model. Our schedulers specialize that model to weighted circular intervals and exact coverage budgets, then combine the resulting discovery charge with a separate entropy bound for endpoint decoding.

Dasgupta's analysis of greedy active learning provides a related expected-cost comparison: it measures query progress by the probability mass removed and compares that progress with the cost of an optimal identification tree [2]. This mass-removal viewpoint guides our comparison between pattern-discovery progress and optimal expected cost. We implement that comparison with first-cover schedules for circular intervals, then use entropy to account for endpoint decoding. Adler and Heeringa make a related distinction in their comparison of decision trees with min-sum set cover: covering the competitors of an item still leaves the tree responsible for separating the items it has covered [5]. This separation supports our two-part accounting for finding a hull-answer class and identifying its member. Cicalese, Laber, and Saettler study expected and worst-case costs together for adaptive testing with general query families [6]; their joint perspective provides context for the two objectives we analyze under a supplied planar line family. For hardness context, Chakaravarthy et al. connect entity-identification trees with min-sum set cover reductions [8]. Our reductions encode matching choices on odd paths and realize the resulting tests by supplied lines on a parabola.

There is also a geometric decision-tree precedent. Arkin et al. study point probes for identifying polygonal models and use a common intersection point to obtain shallow trees [9]. This common-point method gives geometric context for our witness argument. Here, Helly's theorem produces the witness after each query is oriented by its majority side; the convex hull then yields the circular-interval family used by our schedulers.

Proof strategy.

Reading order.

1. Sections 2–3 explain the model, restriction credits, and the circular-interval geometry.
2. Sections 4–5 build the deadline and weighted-coverage primitives.
3. Sections 6, 7, and 8 assemble the best general approximation algorithms.
4. Sections 9–10 exploit bounded caps. The randomized mixture setup follows, in Section 11, connecting expected-depth oracles to mathematical programming.
5. Sections 12–14 treat boundary answers and strong NP-completeness.

Each long result first states its supporting propositions and gives the overall proof. The next subsection proves those propositions. Construction parameters are calibrated only after the structural argument.

2 Preliminaries and basic accounting

We collect the model, coding and information inequalities, and geometric terminology used by the algorithms. Reduction-specific graph and complexity definitions are given in Section 13. For a positive integer r, write $[r] = \{1, \dots, r\}$. The notation $|S|$ denotes set cardinality, while $|Q|$ denotes the length of a word, vector, or sequence when Q is such an object. The indicator $\mathbf{1}[P]$ is one if statement P holds and zero otherwise. We also write $\mathbf{1}_A(x) = \mathbf{1}[x \in A]$ for the indicator function of a set A. The floor $\lfloor x \rfloor$ and ceiling $\lceil x \rceil$ are the greatest integer at most x and the least integer at least x, respectively. For planar vectors, $x \cdot y = x_1 y_1 + x_2 y_2$ is the dot product.

2.1 Line-query model and objectives

Let $\mathcal{H} \subset \mathbb{R}^2$ be $n \geq 2$ distinct points. A finite family $\mathcal{L}$ of m supplied lines separates every pair of hypotheses, meaning that every two distinct points receive different answers to at least one line. Write a line as the zero set of an affine function $f_\ell(x) = a_\ell \cdot x + b_\ell$, where $a_\ell \in \mathbb{R}^2 \setminus \{0\}$ and $b_\ell \in \mathbb{R}$. Querying ℓ returns the sign of $f_\ell(h)$ for the hidden point h. Every query contributes one to the

depth. Unless explicitly stated otherwise, no hypothesis lies on a supplied line, so the answer is binary. Section 12 treats a third, on-line outcome.

After any sequence of queries and answers, the current *state* is the set of hypotheses whose signs on all queried lines agree with the answers received. For a state S and a supplied line ℓ, write

$$S_\ell^- = \{h \in S : f_\ell(h) < 0\}, \qquad S_\ell^+ = \{h \in S : f_\ell(h) > 0\}.$$

These are the two answer classes; the line *splits* S when both classes are nonempty.

Definition 2.1 (Line-query decision trees)**.** A binary line-query decision tree is a finite rooted tree whose internal nodes are labeled by supplied lines and whose two outgoing edges are labeled by their negative and positive answers. A hypothesis follows the edge matching its answer at each query. An *identifying* tree for S has hypothesis labels at its leaves and sends every $h \in S$ to a leaf labeled h. The root has depth zero; a leaf's depth is the number of queries on its root-to-leaf path, and the tree's *height* is its largest leaf depth.

A *partial decision tree*, or *stage*, may instead stop at leaves labeled by nonempty subsets of S. Each leaf is labeled by the hypotheses compatible with its path, so these output classes partition S. Attaching an identifying tree for each class produces a complete identifying tree.

We omit a query whenever its answer is already determined by the current candidate set; in particular, a line is never queried twice on one path. Thus every internal node of the resulting tree has two nonempty children, and we call such a tree *proper*. Every identifying tree can be put in this form by deleting one-child queries and reusing the already-known answers, so the convention preserves the optimum. It also gives a useful size bound: if a proper binary tree on n hypotheses has I internal nodes, edge counting yields $2I = I + n - 1$, hence $I = n - 1$ and every path has at most $n - 1$ queries.

A *transcript* records the queries and answers observed along a path. In a fixed binary tree the previous answers determine the next query, so the transcript can also be represented by its answer word, encoding negative as 0 and positive as 1. A *cached answer* is an answer already learned on that path; reusing it or inferring a constant answer costs no additional query. For nonempty $A \subseteq S$, the *restriction* $T|_A$ is obtained by deleting leaves outside A, deleting empty branches, and contracting queries with only one surviving outcome. It is a valid tree on A, and no surviving hypothesis's depth increases.

Definition 2.2 (Membership answers and spines)**.** A *membership interpretation* designates one answer class A of a supplied binary query. Its answer is a *hit* when $h \in A$ and a *miss* otherwise. A hit may be either affine sign; interpreting the answer changes neither the supplied line nor its cost.

The *spine* of a distinguished hypothesis is its root-to-leaf path. Another hypothesis *exits* that path at its first different answer; the attached subtree is its local exit code. If this occurs at query position j, a local depth d contributes total depth $j+d$. If every designated membership set excludes the distinguished hypothesis, its spine is an all-miss route and exits occur at first hits. A point p on no supplied line similarly determines a *witness path*, even when $p \notin \mathcal{H}$.

Let $d_T(h)$ be the number of queries on the path followed by h, stopping at its output leaf; for an identifying tree that leaf names h. For a tree T identifying a candidate set $S \subseteq \mathcal{H}$, define

$$c_{\mathrm{W}}(T) = \max_{h \in S} d_T(h).$$

For expected depth, let $\pi_h > 0$ with $\sum_h \pi_h = 1$, write $w(S) = \sum_{h \in S} \pi_h$, and define

$$c_{\mathrm{E}}(T) = \sum_{h \in S} \pi_h d_T(h).$$

We retain the original probabilities in $c_{\mathrm{E}}(T)$ when a tree identifies a restricted state. Consequently, $c_{\mathrm{E}}(T)$ is that state's contribution to the root expectation, while $c_{\mathrm{E}}(T)/w(S)$ is its conditional expected cost. At the root, $w(\mathcal{H}) = 1$, so these quantities agree. Define the corresponding optima by

$$\mathrm{OPT}_{\mathrm{W}}(S) = \min_{T \text{ identifying } S} c_{\mathrm{W}}(T), \qquad \mathrm{OPT}_{\mathrm{E}}(S) = \min_{T \text{ identifying } S} c_{\mathrm{E}}(T).$$

We set both optima to zero on the empty set and on singletons. Hypothesis weights used later in the hardness proof specify these probabilities, while query depth still counts each test once. Keeping weights unnormalized makes the expected costs of disjoint recursive states add directly.

More explicitly, if positive raw weights ω_h are supplied, put $\Omega = \sum_{h \in \mathcal{H}} \omega_h$ and $\pi_h = \omega_h/\Omega$. Then, also on a restricted state,

$$\sum_{h \in S} \omega_h d_T(h) = \Omega c_{\mathrm{E}}(T).$$

Thus normalizing weights divides both a weighted cost and its decision threshold by the same Ω, leaving the optimal tree unchanged. The weights determine hypothesis mass in the expected objective; each query still costs one.

The two objectives now differ only in how they combine the child states. If S_ℓ^- and S_ℓ^+ are the answer classes in S, their optimality recurrences on nonsingleton states are

$$\mathrm{OPT}_{\mathrm{W}}(S) = 1 + \min_{\ell \text{ splitting } S} \max\{\mathrm{OPT}_{\mathrm{W}}(S_\ell^-), \mathrm{OPT}_{\mathrm{W}}(S_\ell^+)\},$$
$$\mathrm{OPT}_{\mathrm{E}}(S) = w(S) + \min_{\ell \text{ splitting } S} \{\mathrm{OPT}_{\mathrm{E}}(S_\ell^-) + \mathrm{OPT}_{\mathrm{E}}(S_\ell^+)\}.$$

Algorithm 1 implements these recurrences on the small local states used later, and the same values serve as benchmarks for the approximation algorithms. Their different combinations follow directly from the objectives: a query adds one to every path, so the maximum-depth recurrence takes the harder child; in expected depth the query contributes the state's mass $w(S)$, after which the contributions of the two disjoint child sets add.

Algorithm 1: Exact supplied-query Bellman recursion (small local states)

Input: State S; objective $o \in \{\mathrm{W}, \mathrm{E}\}$; original weights π_h.
Output: An optimal supplied-query tree and its cost.

if $|S| \le 1$ **then**
  return the leaf of cost zero
if *the memoized value for* (S, o) *exists* **then**
  return that value and tree
foreach *supplied line* ℓ *splitting* S **do**
  $(T_-, v_-) \leftarrow \textsc{ExactTree}(S_\ell^-, o)$; $(T_+, v_+) \leftarrow \textsc{ExactTree}(S_\ell^+, o)$
  $v_\ell \leftarrow 1 + \max\{v_-, v_+\}$ if $o = \mathrm{W}$, and $v_\ell \leftarrow w(S) + v_- + v_+$ otherwise
Memoize the minimum v_ℓ and the tree joining its stored children
return the memoized cost and tree

Algorithm 1 takes $2^{O(|S|)}\,\mathrm{poly}(m, L)$ time with bit-mask states. Its exponential dependence stays within bounded local sets, while the approximation algorithms handle larger states.

All logarithms are natural unless a base is displayed, and we use $0 \ln 0 = 0$, the continuous extension at zero. Coordinates, line coefficients, and probabilities are rational and explicitly given by binary-encoded numerators and denominators. Polynomial time is measured in the total binary encoding length of this input. The *decision version* of either objective asks, given an instance and a threshold K, whether a legal identifying tree has cost at most K. For maximum depth the

threshold can be an integer; for expected depth it is rational and the prior is part of the input. An α-*approximation* for a minimization objective returns a feasible tree whose cost is at most α times the optimum on every valid input. For a deterministic algorithm, this bound applies to the particular tree it returns. Whenever an algorithm permits several choices, fix a deterministic tie-breaking rule, such as the input order.

Symbol	Meaning
$\mathcal{H}, \mathcal{L};\ n, m$	Hypothesis points, supplied lines, and their counts.
$S,\ S_\ell^-, S_\ell^+$	Current state and its negative and positive answer classes for line ℓ.
$d_T(h)$	Number of queries on the path for hypothesis h in tree T.
$c_{\mathrm{W}}(T),\ \mathrm{OPT}_{\mathrm{W}}(S)$	Maximum path depth and its minimum over identifying trees for S.
$\pi_h,\ w(S)$	Root prior probability of h and the original probability mass of state S.
$c_{\mathrm{E}}(T),\ \mathrm{OPT}_{\mathrm{E}}(S)$	Expected depth using the original probabilities and its minimum over identifying trees for S.
$\mathcal{E}(S)$	State-mass-weighted entropy of the conditional distribution on S.
$\Pi,\ \Delta_{\mathrm{E}}(S,\Pi),\ I(S,\Pi)$	A partition of S, the decrease in optimal expected depth, and the information gained from its class label.
$\omega_h,\ \Omega$	Optional raw positive weights and their total; $\pi_h = \omega_h/\Omega$.

Definition 2.3 (Balanced and flat states)**.** Fix $0 < \delta < 1/3$. A nonsingleton state S is *count-balanced* if some supplied line has at least $\delta|S|$ hypotheses on each side. Otherwise it is *count-flat*. In the flat case, every line has a unique *majority answer*, on the side containing more than half of S. We call this its *frozen answer* and write C_ℓ for the open half-plane on the opposite side. Its induced answer class $C_\ell \cap S$ is the *minority cap*, of size less than $\delta|S|$. We also use "cap" for the corresponding half-plane when discussing its geometry.

For expected depth, the same split-versus-flat distinction must track probability mass, since a small number of hypotheses can carry most of the prior. Accordingly, S is *mass-balanced* if some line has $w(S_\ell^-), w(S_\ell^+) \geq \delta w(S)$, and *mass-flat* otherwise. In a mass-flat state, freezing the mass-majority answer gives $w(C_\ell \cap S) < \delta w(S)$.

To turn each stored object into a legal supplied-query tree, the procedures use the following conventions. Tables are indexed by realized hypotheses or patterns, and ties follow input order. During tree assembly, cached answers supply later virtual questions, the origin marker records the cut position, and infeasible branches are discarded. We then contract every query whose answer is constant on the current state, leaving tests that distinguish hypotheses. The analysis for each routine gives its parameter values and proves that they meet the required bounds.

State quantities and calibration choices. The state S, its mass $w(S)$, pattern classes, and optimal costs record the instance and the algorithm's progress. Parameters such as δ, β, γ control the constructions; each algorithm states their admissible conditions before choosing values for its bound. A coefficient such as Φ_β then records how that calibration and the current state determine a particular charge.

Proof strategy.

The accounting facts below have three roles.

1. Prefix codes turn query transcripts into information lower bounds.
2. Restriction deletes queries that a known output class no longer needs.
3. Potential differences cancel across recursive states, so local charges are paid only once.

The statements are collected first. Their proofs appear together in the next technical subsection, before the geometric terminology.

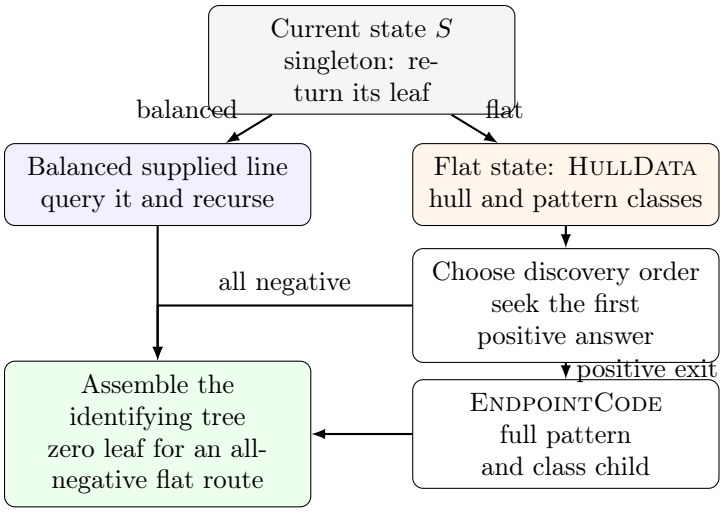


Figure 2: The common recursive structure. Arrows show how one stage feeds the next. For maximum depth, the algorithm constructs class children first and uses their heights as schedule deadlines. For expected depth, original prior masses determine the schedule scores. The flat stage has an all-negative leaf when the zero pattern is present, and both scheduling and decoding use supplied hull lines.

2.2 Binary codes and ordered comparisons

Definition 2.4 (Binary codes and prefix codes)**.** A *binary word* is a finite sequence of bits from $\{0,1\}$; $\{0,1\}^*$ includes the empty word ε, of length zero. A *binary code* on a finite nonempty alphabet $\mathcal{A}$ is an injective map $c : \mathcal{A} \to \{0,1\}^*$. The word $c(a)$ is the *codeword* for a, and $\ell_a = |c(a)|$ is its length. A word u is a *prefix* of v if v starts with all the bits of u. A code is *prefix-free*, or a *prefix code*, if no codeword is a prefix of another distinct codeword.

For a distribution (q_a) on the alphabet, the code's maximum length is $\max_a \ell_a$ and its expected length is $\sum_a q_a \ell_a$. The prefix tree of a code has one node for each prefix of a codeword, with the empty prefix at the root and edges appending 0 or 1. For a prefix code its codewords are the leaves, at depths ℓ_a. Conversely, the root-to-leaf answer words of any binary decision tree form a prefix code. Unary nodes, which have only one child, can be contracted without increasing any depth. The empty word is permissible only when the alphabet is a singleton. For example, $c(a) = 0$, $c(b) = 10$, and $c(c) = 11$ form a prefix code with lengths $1, 2, 2$.

Definition 2.5 (Alphabetic codes)**.** For an ordered alphabet $y_1 < \cdots < y_r$, a prefix code is *alphabetic* if its codewords occur in that same lexicographic order, with $0 < 1$. An *ordered threshold comparison* asks whether the unknown outcome is at most y_j for some j.

In an alphabetic prefix tree, the outcomes below each node form a consecutive range. After unary nodes are contracted, its two children separate that range at one threshold. Thus an alphabetic code can be implemented by ordered threshold comparisons whenever those comparisons are available. The code bits label the two ordered ranges; a line's raw sign answer is routed to the matching range, even if its sign encoding uses the opposite bit. Relabeling therefore only changes the names of the two outcomes. Kraft's inequality applies to every prefix code, while an executable decision-tree code must also use comparisons available in the supplied line family. Section 3.4 verifies that the endpoint-code comparisons meet this condition.

A word $b_1 \cdots b_\ell$ specifies the half-open *dyadic cell*

$$D(b_1 \cdots b_\ell) = \left[\sum_{j=1}^{\ell} b_j 2^{-j}, \sum_{j=1}^{\ell} b_j 2^{-j} + 2^{-\ell} \right) \subseteq [0,1).$$

The empty word specifies $[0,1)$. Such cells are nested or disjoint; one word is a prefix of another if and only if its cell contains the other's cell. Half-open endpoints avoid ambiguity at dyadic rationals.

Query transcripts occupy disjoint leaves of a binary code. The next inequality expresses their limited capacity; it will turn leaf depths into lower bounds on both decision-tree objectives.

Theorem 2.6 (Kraft's inequality for binary prefix codes)**.** *Every finite binary prefix code has lengths satisfying*

$$\sum_{a \in \mathcal{A}} 2^{-\ell_a} \le 1.$$

Conversely, nonnegative integer lengths $(\ell_a)_{a \in \mathcal{A}}$ are realizable by a binary prefix code if this inequality holds. The converse specifies codeword lengths; alphabetic order and implementation by a restricted query family require additional structure.

The code $0, 10, 11$ has Kraft sum $2^{-1} + 2^{-2} + 2^{-2} = 1$. More generally, equality holds for a proper binary tree because its two child cells partition each internal node's cell. The coding facts here and the information inequalities below are standard; see Cover and Thomas [3].

2.3 Entropy and information inequalities

Definition 2.7 (Entropy and conditional entropy)**.** A *probability vector* (q_i) has nonnegative entries summing to one; its *support* consists of the indices with $q_i > 0$. For a finite random variable Y with probabilities $q_y = \Pr[Y = y]$, its Shannon entropy in natural-log units is

$$H(Y) = -\sum_{y} q_y \ln q_y.$$

We also write $H(q)$ for the entropy of a probability vector. The joint entropy $H(Y, Z)$ is the entropy of the pair (Y, Z). For Z with finite support, conditional entropy is

$$H(Y \mid Z) = \sum_{z : \Pr[Z=z] > 0} \Pr[Z = z]\, H(Y \mid Z = z),$$

where the inner entropy uses $\Pr[Y = y \mid Z = z] = \Pr[Y = y, Z = z] / \Pr[Z = z]$. Conditioning on several variables means conditioning on their tuple. Entropy in bits is $H(Y)/\ln 2$. For a function g of a finite random variable, $\mathbb{E}[g(Y)] = \sum_y q_y g(y)$. The *binary entropy* is $h(t) = -t \ln t - (1-t)\ln(1-t)$ for $0 \le t \le 1$. We write $H_2(t) = h(t)/\ln 2$ when binary entropy is measured in bits.

For nonempty S, the conditional hypothesis probabilities are $q_h = \pi_h / w(S)$. Define the *weighted state entropy* by

$$\mathcal{E}(S) = w(S) H(q) = \sum_{h \in S} \pi_h \ln \frac{w(S)}{\pi_h}, \qquad \mathcal{E}(\varnothing) = 0.$$

It is zero on singletons. Like c_{E}, this quantity retains the original state mass.

To lower-bound average depth, we compare hypothesis probabilities with the capacities assigned to their codewords. Log-sum makes this comparison without any assumption on the smallest probability.

Lemma 2.8 (Log-sum inequality)**.** *For nonnegative a_i and positive b_i, with $A = \sum_i a_i > 0$ and $B = \sum_i b_i$, one has*

$$\sum_i a_i \ln \frac{a_i}{b_i} \geq A \ln \frac{A}{B},$$

with zero summands interpreted by continuity.

A stage first reveals a pattern and then identifies the hypothesis within that pattern. Since the pattern is a function of the hypothesis, the entropy chain rule separates the information spent on these two tasks. This separation lets the later endpoint decoder pay only for the information left after pattern discovery.

Lemma 2.9 (Entropy chain rule)**.** *For finite random variables,*

$$H(Y, Z) = H(Z) + H(Y \mid Z), \qquad H(Y, Z \mid W) = H(Y \mid W) + H(Z \mid Y, W).$$

If Z is determined by Y, then $H(Y) = H(Z) + H(Y \mid Z)$. Also, $0 \leq H(Y) \leq \ln |\operatorname{supp}(Y)|$ and $H(Y \mid Z) \geq 0$; in particular $0 \leq h(t) \leq \ln 2$.

An information-efficient code must also respect the queries we can ask. For ordered endpoint outcomes, alphabetic codes keep each branch consecutive and therefore make the coding bound executable by thresholds.

Lemma 2.10 (Alphabetic threshold codes)**.** *Let Y have finite ordered support, and let $H(Y)$ be its entropy with natural logarithms. There is an explicit alphabetic threshold code with depth on outcome y at most $\log_2(1/\Pr[Y = y]) + 2$. In particular its expected depth is at most*

$$\frac{H(Y)}{\ln 2} + 2.$$

This is a dyadic prefix-code construction; see Cover and Thomas [3] for the general coding background. The short proof here also verifies the extra alphabetic (order-preserving) property needed for threshold queries. An optimal expected-depth threshold tree can also be computed by a polynomial interval dynamic program. The explicit code is used when the pointwise depth bound is needed.

Proof of Lemma 2.10. Claims 2.1 and 2.2 construct the stated executable code; averaging its pointwise bound proves the expected-depth inequality. Claim 2.3 gives the optimal threshold-tree algorithm. □

Algorithm 2: Alphabetic code on ordered outcomes

Input: Ordered outcomes $y_1 < \cdots < y_r$ of positive rational masses a_i; mode *pointwise* or *expected*.
Output: A threshold-comparison tree.

if $r = 1$ **then**
  return its zero-depth leaf
if *mode is* pointwise **then**
  Normalize $q_i = a_i / \sum_j a_j$ and put $F_i = \sum_{j<i} q_j$
  foreach i **do**
    Find $d_i = \lceil \log_2(1/q_i) \rceil + 1$ by rational powers-of-two comparisons
    Take the depth-d_i dyadic cell containing $F_i + q_i/2$ as codeword i
  Build the alphabetic prefix tree and contract unary nodes
else
  Set $D_{i,i} = 0$
  for $h = 2, \ldots, r$ **do**
    foreach $[i, j]$ *of length* h **do**
      $D_{i,j} \leftarrow \sum_{a=i}^{j} a_a + min_{i \le t < j}(D_{i,t} + D_{t+1,j})$
      Store a minimizing split after y_t
  Reconstruct the stored interval splits
return the ordered tree, implementing each split by its available threshold

The pointwise mode preserves the per-outcome bound; the expected mode has $O(r^3)$ arithmetic complexity and avoids long expansions for small priors. Their distinct roles are proved in Lemma 2.10.

2.4 Restriction and recursive accounting

The next accounting step separates two sources of progress. Once a class is known, restriction contracts queries that no longer help identify its members; meanwhile, the class label itself carries information measured by entropy. These give complementary lower bounds for the work performed by a stage.

Definition 2.11 (Partitions and refinement)**.** A *partition* Π of S is a family of pairwise disjoint nonempty subsets whose union is S. We say Π *refines* $\mathcal{R}$, or $\mathcal{R}$ is *coarser* than Π, if every class of Π is contained in a class of $\mathcal{R}$. The output classes of a partial decision tree form a partition.

Revealing an output class can only make identification easier: queries whose answers are then known disappear. This observation compares recursive optima, while the coding inequalities provide a lower bound on how much work any identifying tree still requires.

Lemma 2.12. *For pairwise disjoint subsets $S_1, \ldots, S_k$ of a state S,*

$$\sum_i \mathrm{OPT}_{\mathrm{E}}(S_i) \le \mathrm{OPT}_{\mathrm{E}}(S), \qquad \mathrm{OPT}_{\mathrm{W}}(S_i) \le \mathrm{OPT}_{\mathrm{W}}(S).$$

Moreover,

$$\mathrm{OPT}_{\mathrm{E}}(S) \ge \frac{\mathcal{E}(S)}{\ln 2}, \qquad \mathrm{OPT}_{\mathrm{W}}(S) \ge \frac{\ln |S|}{\ln 2} \quad \textit{for nonempty } S.$$

Proof of Lemma 2.12. Claim 2.4 gives both restriction inequalities. Claims 2.5 and 2.6 supply the two information lower bounds. □

We use the word *potential* for a state quantity that measures remaining work. Here the relevant expected-depth potentials are $\mathrm{OPT}_{\mathrm{E}}$ and $\mathcal{E}$. When a partition is revealed, its decrement is the parent value minus the sum of the child values. For a nonempty state S and a partition Π of it, write

$$\Delta_{\mathrm{E}}(S,\Pi) = \mathrm{OPT}_{\mathrm{E}}(S) - \sum_{A\in\Pi} \mathrm{OPT}_{\mathrm{E}}(A),$$
$$I(S,\Pi) = \mathcal{E}(S) - \sum_{A\in\Pi} \mathcal{E}(A) = \sum_{A\in\Pi} w(A) \ln \frac{w(S)}{w(A)}.$$

Let X be a random hypothesis with $\Pr[X = h] = \pi_h/w(S)$, and let P be its class label under Π. Then

$$I(S,\Pi) = w(S)H(P), \qquad H(P) = -\sum_{A\in\Pi} \frac{w(A)}{w(S)} \ln \frac{w(A)}{w(S)}.$$

Thus $I(S,\Pi)$ is the state mass times the Shannon entropy of the class label, measured in natural-log units; this is the partition form of the chain rule in Lemma 2.9. Lemma 2.12 shows that $\Delta_{\mathrm{E}}(S,\Pi) \geq 0$, since revealing the class cannot increase the optimal expected cost. The entropy interpretation gives $I(S,\Pi) \geq 0$ as well. The two quantities therefore measure distinct credits from the same stage: the decrease in optimal cost and the information in its class label. The approximation proofs bound stage costs by these credits, and the cancellation below shows that summing them over the recursion charges each root-level potential only once.

Proof strategy.
Sum each potential decrement over the recursive partition tree; every nonroot state then cancels once as a child against once as a parent. Singleton leaves have zero potential, and the stage-cost bound can be summed because each query's original-probability cost is assigned to a unique stage.

Proposition 2.13 (Local charges add to a global bound)**.** *Suppose a recursive partition ends in singletons, and its stage at state S costs at most $a\Delta_{\mathrm{E}}(S,\Pi_S) + cI(S,\Pi_S)$. The complete tree then has expected cost at most $a\,\mathrm{OPT}_{\mathrm{E}}(\mathcal{H}) + c\mathcal{E}(\mathcal{H})$.*

Proof. For either potential $P = \mathrm{OPT}_{\mathrm{E}}$ or $P = \mathcal{E}$, sum over the internal state occurrences in the recursive partition tree. Every nonroot internal state occurs once as a child and once as a parent. Therefore

$$\sum_{S \text{ internal}} \left(P(S) - \sum_{A\in\Pi_S} P(A) \right) = P(\mathcal{H}) - \sum_{A \text{ leaf}} P(A)$$
$$= P(\mathcal{H}).$$

The second equality uses the singleton leaves, where both potentials are zero. Because state costs retain the original probabilities, each query's weighted contribution belongs to a unique stage. Writing $W(S)$ for that stage contribution gives

$$\begin{aligned} c_{\mathrm{E}}(T) &= \sum_{S \text{ internal}} W(S) \\ &\leq a \sum_{S \text{ internal}} \Delta_{\mathrm{E}}(S,\Pi_S) + c \sum_{S \text{ internal}} I(S,\Pi_S) \\ &= a\,\mathrm{OPT}_{\mathrm{E}}(\mathcal{H}) + c\mathcal{E}(\mathcal{H}), \end{aligned}$$

as required. □

2.5 Proofs of the coding and accounting facts

Proof of Theorem 2.6. For necessity, prefix-free codewords specify disjoint dyadic cells inside $[0,1)$, whose widths are $2^{-\ell_a}$. Summing their widths therefore gives at most the width one of the containing interval, which is the required inequality.

For sufficiency, reorder the lengths as $\ell_1 \le \cdots \le \ell_r$ and put $s_i = \sum_{j<i} 2^{-\ell_j}$. Each s_i is an integer multiple of $2^{-\ell_i}$ because every earlier length is at most ℓ_i: the ratio $2^{-\ell_j}/2^{-\ell_i} = 2^{\ell_i-\ell_j}$ is an integer. The intervals $[s_i, s_i + 2^{-\ell_i})$ are dyadic cells of the prescribed depths. They are disjoint and lie in $[0,1)$ by the assumed bound. Their binary descriptions give the required prefix code; assign them back to the original symbols. □

Proof of Lemma 2.8. Put $q_i = a_i/A$ and $r_i = b_i/B$. The inequality $-\ln x \ge 1 - x$ for $x > 0$ follows because $x - 1 - \ln x$ has its minimum zero at $x = 1$. Hence

$$\sum_{i:q_i>0} q_i \ln \frac{q_i}{r_i} \ge \sum_{i:q_i>0} (q_i - r_i) = 1 - \sum_{i:q_i>0} r_i \ge 0.$$

Here the first step applies that scalar inequality to $x = r_i/q_i$ and multiplies by q_i; the equality uses $\sum_i q_i = 1$, and the final step uses $\sum_i r_i = 1$ and $r_i \ge 0$. Since $a_i/b_i = (A/B)(q_i/r_i)$ on the positive support, multiplying by A and adding $A\ln(A/B)$ proves the assertion. □

Proof of Lemma 2.9. Write $p_z = \Pr[Z = z]$ and $q_{y|z} = \Pr[Y = y \mid Z = z]$ for $p_z > 0$. Zero-probability events contribute zero. Expanding the logarithm in the joint entropy gives

$$\begin{aligned} H(Y,Z) &= -\sum_{z:p_z>0} \sum_y p_z q_{y|z} (\ln p_z + \ln q_{y|z}) \\ &= -\sum_z p_z \ln p_z \sum_y q_{y|z} + \sum_z p_z \left(-\sum_y q_{y|z} \ln q_{y|z} \right) \\ &= H(Z) + H(Y \mid Z), \end{aligned}$$

because each conditional distribution sums to one. Under $W = w$, apply the same identity with Y first: $H(Y,Z \mid W = w) = H(Y \mid W = w) + H(Z \mid Y, W = w)$. Multiplying by $\Pr[W = w]$ and summing gives the conditional form. If Z is determined by Y, the pair (Y,Z) has the same entropy as Y. Entropy and conditional entropy are nonnegative because their summands are $-q\ln q \ge 0$ for $q \in [0,1]$. If $r = |\operatorname{supp}(Y)|$, apply Lemma 2.8 with $a_y = q_y$ and $b_y = 1/r$ on that support. Since both sums equal one,

$$0 \overset{\text{Lem. 2.8}}{\le} \sum_y q_y \ln(r q_y) = \ln r - H(Y),$$

where the equality separates $\ln(rq_y) = \ln r + \ln q_y$ and uses $\sum_y q_y = 1$. This proves the upper bound; applying it to a binary variable also gives $h(t) \le \ln 2$. □

Proof strategy.

For Lemma 2.10, place a small dyadic cell inside each probability interval. Disjoint cells preserve outcome order; their depths give the coding bound. A separate interval recurrence optimizes the executable threshold tree.

Claim 2.1 (Ordered disjoint cells give an alphabetic code)**.** *Each outcome's cumulative-probability interval contains a dyadic cell. These cells are disjoint, occur in outcome order, and form an alphabetic prefix code.*

Proof. For an ordered outcome y of probability a, let $F_y = \sum_{y'<y} \Pr[Y = y']$. Its *cumulative-probability interval* is $[F_y, F_y + a)$, with midpoint $m_y = F_y + a/2$. The *midpoint-dyadic code* chooses the dyadic cell containing m_y at depth $\ell_y = \lceil \log_2(2/a) \rceil$, in the sense of the cells defined before Theorem 2.6. If s_y is the cell's left endpoint and $w_y = 2^{-\ell_y}$ its width, then $s_y \le m_y < s_y + w_y$ and $w_y \le a/2$. Therefore

$$s_y > m_y - w_y \ge F_y, \qquad s_y + w_y \le m_y + w_y \le F_y + a.$$

The strict first inequality follows from $m_y < s_y + w_y$; both outer interval bounds then use $w_y \le a/2$ and $m_y = F_y + a/2$. Consequently no cell reaches outside its own probability interval, even when a midpoint is dyadic. Thus the entire half-open cell lies in $[F_y, F_y + a)$. These cells are disjoint and appear in outcome order. Their binary descriptions form a prefix-free alphabetic code. □

The cells have given an abstract ordered code. We now check that its branches are legal threshold comparisons and that converting the code to a tree preserves the promised depth.

Claim 2.2 (Threshold implementation preserves the depth bound)**.** *Ordered threshold queries implement the code with depth at most* $\log_2(1/\Pr[Y = y]) + 2$ *on outcome* y*.*

Proof. By Claim 2.1, outcome y has a codeword of length $\lceil \log_2(2/a) \rceil$, where $a = \Pr[Y = y]$. Since $\lceil x \rceil \le x + 1$, these lengths satisfy

$$\left\lceil \log_2 \frac{2}{a} \right\rceil \le \log_2 \frac{1}{a} + 2.$$

After unary nodes are contracted, each internal node separates two consecutive outcome ranges and is a threshold comparison, as explained after Definition 2.5. Contraction only decreases depth. Thus taking expectation of the pointwise bound gives an expected depth at most

$$\sum_y \Pr[Y = y] \left(\log_2 \frac{1}{\Pr[Y = y]} + 2 \right) = \frac{H(Y)}{\ln 2} + 2.$$

The equality uses the definition of entropy and the fact that the outcome probabilities sum to one. □

The explicit code supplies a bound for each outcome. When only average depth matters, the consecutive ranges also admit an exact optimization that avoids constructing long codewords for very small probabilities.

Claim 2.3 (An interval recurrence minimizes expected depth)**.** *Optimal expected-depth threshold comparison uses a polynomial interval dynamic program.*

Proof. To minimize expected depth, let $D_{i,j}$ be the minimum weighted depth contributed by outcomes $y_i, \ldots, y_j$, using their original probabilities $p_a = \Pr[Y = y_a]$. Set $D_{i,i} = 0$. For $i < j$, a root threshold after y_t contributes $\sum_{a=i}^{j} p_a$, so

$$D_{i,j} = \sum_{a=i}^{j} p_a + \min_{i \le t < j} \{ D_{i,t} + D_{t+1,j} \}.$$

Indeed, every outcome in the interval pays for the root query; the two disjoint child intervals then contribute their optimal weighted costs. Conversely, every legal threshold root has one of these split positions, so minimizing over t is exact. This interval dynamic program uses $O(r^3)$ arithmetic operations. The resulting optimal alphabetic tree satisfies the expected-depth bound of Claim 2.2, since that claim supplies a feasible comparison tree. Thus the interval program optimizes average depth, while the explicit midpoint-dyadic construction supplies the pointwise depth bound. □

Proof strategy.
For Lemma 2.12, contract known-answer queries to obtain feasible subproblem trees. Then use the prefix transcripts in two ways: Kraft and log-sum bound average depth; the number of binary leaves bounds maximum depth.

Claim 2.4 (Restriction cannot increase depth)**.** *Restriction to pairwise disjoint sets cannot increase their total expected optimum or any maximum-depth optimum.*

Proof. Restrict an optimal tree on S to each S_i and contract its constant queries. No hypothesis's depth increases. For an expected-depth optimal tree T, feasibility of each restriction, disjointness, and nonnegative weights give, in order,

$$\begin{aligned}\sum_i \mathrm{OPT}_{\mathrm{E}}(S_i) &\le \sum_i \sum_{h\in S_i} \pi_h d_{T|_{S_i}}(h)\\ &\le \sum_i \sum_{h\in S_i} \pi_h d_T(h) = \sum_{h\in\bigcup_i S_i} \pi_h d_T(h)\\ &\le \sum_{h\in S} \pi_h d_T(h) = \mathrm{OPT}_{\mathrm{E}}(S).\end{aligned}$$

For a maximum-depth-optimal tree T_W and nonempty S_i,

$$\mathrm{OPT}_{\mathrm{W}}(S_i) \le \max_{h\in S_i} d_{T_W|_{S_i}}(h) \le \max_{h\in S_i} d_{T_W}(h) \le \max_{h\in S} d_{T_W}(h) = \mathrm{OPT}_{\mathrm{W}}(S).$$

Here feasibility bounds the optimum by the restricted height, contraction bounds that height by the original depths on S_i, and inclusion $S_i \subseteq S$ bounds those depths by the full height. Thus the same restriction operation proves both objectives' monotonicity. Empty subsets have optimum zero by convention.

□

Restriction preserves a feasible tree on each subproblem. To quantify the cost of those trees, the next claim turns their transcripts into a lower bound that depends on the prior.

Claim 2.5 (Entropy lower-bounds expected cost)**.** *Every identifying tree has expected cost at least* $\mathcal{E}(S)/\ln 2$.

Proof. For entropy, assume $S \neq \varnothing$; the empty-state case is immediate. Let $q_h = \pi_h/w(S)$. The leaf transcripts are a binary prefix code as in Definition 2.4: a transcript cannot continue beyond a leaf. Theorem 2.6 therefore gives $\sum_h 2^{-\ell_h} \le 1$ for their lengths. Apply Lemma 2.8, dividing by $\ln 2$ to use base two, with $\ell_h = d_T(h)$ and $q_h = \pi_h/w(S)$:

$$\begin{aligned}\sum_h q_h\ell_h - \sum_h q_h \log_2 \frac{1}{q_h} &= \sum_h q_h \log_2 \frac{q_h}{2^{-\ell_h}}\\ &\overset{\text{Lem. 2.8}}{\ge} \log_2 \frac{\sum_h q_h}{\sum_h 2^{-\ell_h}} \overset{\text{Thm. 2.6}}{\ge} 0.\end{aligned}$$

The first inequality is the log-sum inequality, and the second uses $\sum_h q_h = 1$ and Kraft's bound. Hence every tree identifying S satisfies

$$c_{\mathrm{E}}(T) = w(S)\sum_h q_h d_T(h) \ge w(S)\sum_h q_h \log_2 \frac{1}{q_h} = \frac{w(S)}{\ln 2}\sum_h q_h \ln \frac{1}{q_h} = \frac{\mathcal{E}(S)}{\ln 2}.$$

The inequality is the preceding code-length bound multiplied by $w(S)$; the last equality is the definition of weighted state entropy. Minimizing over identifying trees therefore gives the expected-depth lower bound.

□

For maximum depth, probabilities are irrelevant. Counting the leaves that fit within a given height gives the corresponding lower bound.

Claim 2.6 (Binary capacity lower-bounds height)**.** *A binary tree of height d has at most 2^d leaves, so identifying S requires height at least $\log_2 |S|$.*

Proof. For the maximum-depth bound, prove by induction on d that a binary tree of height d has at most 2^d leaves. The claim is immediate for $d = 0$; for $d > 0$, the root has at most two child subtrees, each of height at most $d - 1$, so together they have at most $2 \cdot 2^{d-1} = 2^d$ leaves. Thus $|S| \le 2^d$, or $d \ge \log_2 |S| = \ln |S| / \ln 2$. □

2.6 Convex geometry and finite set systems

Definition 2.14 (Planar convexity)**.** A set is *convex* if it contains the segment joining any two of its points. A *convex combination* is a sum $\sum_j \lambda_j u_j$ with $\lambda_j \ge 0$ and $\sum_j \lambda_j = 1$. The *convex hull* $\mathrm{conv}(U)$ of a finite nonempty point set U is the set of its convex combinations. A *vertex*, or extreme point, cannot be expressed as a convex combination of other points of the hull. The hull is the convex hull of its vertices; set $\mathrm{conv}(\varnothing) = \varnothing$ by convention.

A nonempty planar hull has affine dimension zero, one, or two according as it is a point, a nontrivial segment, or has nonempty planar interior. In dimension two it is a convex polygon, whose vertices have a cyclic boundary order; its *edges* join consecutive vertices. A *supporting line* meets a convex set and leaves it entirely on one closed side. It *exposes* a point if that point is its unique intersection with the set. A finite point set is in *strictly convex position* if every point is a vertex of its hull.

An open half-plane has the form $\{x : a \cdot x > b\}$ for $a \ne 0$; replacing $>$ by $\ge$ gives its closed counterpart. A point is *strictly* on a chosen side when it lies in that side's open half-plane. A common strict *witness* for several chosen sides lies in all their open half-planes and may be any point in the plane.

The geometry will produce witnesses for every three majority sides. The planar Helly theorem is the step that turns this local consistency into a single witness for all supplied lines.

Theorem 2.15 (Planar Helly theorem)**.** *If every subfamily of at most three sets in a finite family of planar convex sets has nonempty intersection, then the whole family has nonempty intersection. The sets may be open.*

This is the standard planar form; see Matoušek [10].

Definition 2.16 (Covers and hitting sets)**.** A finite *set system* consists of a universe U, a candidate set V, and supplied subsets $F_v \subseteq U$ indexed by $v \in V$. A *cover* of U is a choice of these sets whose union contains every element of U. In the equivalent demand formulation, there is a candidate set V and a subset $D_h \subseteq V$ for each demand h. A *hitting set* is a subset $P \subseteq V$ with $P \cap D_h \ne \varnothing$ for every demand. Choosing v covers the set $F_v = \{h : v \in D_h\}$. Thus covering demands by candidates and hitting their subsets are the same incidence problem. A demand with $D_h = \varnothing$ is uncoverable.

With positive demand weights, a choice covers the sum of the weights of the demands it hits, counting each demand once. An *incidence table* records the bits $\mathbf{1}[h \in F_v]$, equivalently $\mathbf{1}[v \in D_h]$. A cardinality budget is a nonnegative integer k permitting at most k selected candidates. We may cap it at $|V|$, since there are no more distinct candidates to choose.

Definition 2.17 (Ordered and circular intervals)**.** On a finite linear order $v_1, \dots, v_M$, an *ordinary interval* is a consecutive subset $\{v_i, \dots, v_j\}$; we also allow the empty interval. Its nonempty intervals have first and last elements, called their endpoints. On a cyclic order, a *circular interval* is a consecutive block with wrap-around allowed, also permitting the empty and whole-cycle sets. Two vertices are *adjacent* when consecutive in this order, including v_M, v_1 on a cycle. An *oriented adjacent pair* specifies which of those two vertices is first. A *run* of P in a linear candidate order is a maximal nonempty consecutive block contained in P; $c(P)$ counts such runs, with $c(\varnothing) = 0$.

Cutting a cyclic order at a removed vertex turns any circular interval avoiding that vertex into an ordinary interval. When interval demands occur below, candidates are the ordered vertices, and selecting any vertex in a demand's interval hits it.

2.7 Query orderings and first-cover costs

Let a universe U have positive element weights π_h and supplied sets F_v covering U. We allow these weights to be unnormalized. A *cover ordering* $Q = (F_{v_1}, \dots, F_{v_\ell})$ is a sequence of distinct supplied sets whose union covers U, and $|Q| = \ell$ is its length. The *first-cover time*, or *latency*, of h is $t_Q(h) = \min\{j : h \in F_{v_j}\}$. The *weighted min-sum set cover* objective is

$$\mathrm{MSSC}(Q) = \sum_{h \in U} \pi_h t_Q(h).$$

Equivalently, $\mathrm{MSSC}(Q)$ is the sum of the uncovered mass before each step: a demand first covered at position $t_Q(h)$ contributes its weight to the residuals at positions $0, \dots, t_Q(h) - 1$. Interchanging these finite sums gives

$$\mathrm{MSSC}(Q) = \sum_{j=0}^{|Q|-1} w\left(U \setminus \bigcup_{r=1}^{j} F_{v_r}\right), \qquad w(E) = \sum_{h \in E} \pi_h.$$

The union for $j = 0$ is empty. *Maximum-remaining-weight greedy* repeatedly chooses the supplied set containing the largest total weight of elements not yet covered, counting each element only once.

2.8 Normalized queries and hull patterns

Definition 2.18 (Witness-relative query notation)**.** Enumerate the supplied lines as $\ell_1, \dots, \ell_m$. For a state S and a point p on no supplied line, put

$$x_h = h - p, \qquad u_i = -\frac{a_{\ell_i}}{f_{\ell_i}(p)}, \qquad C_i = \{y : u_i \cdot (y - p) > 1\}.$$

Thus x_h is a translated hypothesis vector, u_i a normalized query vector, and C_i the cap opposite p in the original plane. The corresponding dual demand is

$$R_h = \{u : u \cdot x_h > 1\}.$$

Write $K = \mathrm{conv}(\{0\} \cup \{u_i : i \in [m]\})$ and let V be its nonzero vertices, in cyclic hull order. Every $v \in V$ equals a supplied query vector; choose one representative line if several coincide, and put

$C_v = \{y : v \cdot (y - p) > 1\}$. A *support block* $J_i \subseteq V$ represents u_i as $u_i = \sum_{v \in J_i} \alpha_{iv} v$, with positive coefficients and $\sum_v \alpha_{iv} \leq 1$. We choose blocks of at most two vertices, adjacent when there are two; their existence is proved in Lemma 3.2.

The origin's hull bit is always negative because $0 \cdot x_h = 0$; we use it as a free marker that opens the cyclic order. If it lies on the hull boundary, it can be inserted at its boundary position as a free cut marker. The hull K, its vertices, and their cyclic order can be computed in polynomial time by standard planar convex-hull algorithms [11].

Indeed, substituting the definitions of u_i and x_h, and using $f_{\ell_i}(h) = f_{\ell_i}(p) + a_{\ell_i} \cdot (h - p)$, gives

$$u_i \cdot x_h - 1 = -\frac{f_{\ell_i}(h)}{f_{\ell_i}(p)}.$$

Consequently, $h \in C_i$ if and only if $u_i \in R_h$, and equality $u_i \cdot x_h = 1$ means h lies on ℓ_i. We encode the cap-side answer as 1 and the witness-side answer as 0. A *positive hull answer* is this cap-side bit, regardless of the original affine sign; the relabeling routes the answer of the same supplied query. If $x_h = 0$, the demand R_h is empty. We write $C_i \cap S$ for the original-plane hypotheses in the cap, while x_h denotes their translated vectors.

Definition 2.19 (Hull patterns and pattern classes)**.** The *hull pattern* of $h \in S$ is the bit vector

$$\sigma(h) = \big(\mathbf{1}[v \cdot x_h > 1]\big)_{v \in V}.$$

A *pattern class* consists of all hypotheses with the same vector, and Π is the partition into these classes. The *positive vertices* are the coordinates whose bits are one. Write A_0 for the class with every bit zero, taking $A_0 = \varnothing$ when no hypothesis has that pattern. When a prior is specified, the nonzero-pattern mass is $b(S) = w(S) - w(A_0)$, the mass on which a positive hull answer is possible.

A *discovery schedule* lists individual hull tests in their query order along the all-negative route. Its *first-hit time* g_A for a nonzero pattern class is the index of the first listed vertex positive on that class.

3 Common-witness geometry and interval tools

Using the notation of Definition 2.18, we prove the structural facts shared by both algorithms. Algorithm 3 records their common preprocessing. For a flat state, Helly's theorem gives a common majority witness. Relative to this witness, each query cap is covered by at most two adjacent hull caps, so every hypothesis's positive hull answers form one circular interval. The interval tools below use this structure to discover patterns and decode their endpoints.

Proof strategy.

1. Flatness makes every three majority half-planes intersect; Helly's theorem gives a common strict witness.
2. Radial projection onto the normalized query hull supplies adjacent supports; a half-plane reads one circular positive interval.
3. Interval covers discover a positive vertex, after which a query-free cut turns supplied hull tests into endpoint comparisons.

Symbol	Meaning
$p,\ x_h,\ u_i$	Strict common witness, translated hypothesis vector $h - p$, and normalized vector for supplied line ℓ_i.
$C_i,\ R_h$	Original-plane cap opposite p for query i, and the dual set of normalized queries positive on h.
$K,\ V$	Convex hull of the normalized query vectors together with 0, and its nonzero vertices in cyclic order.
$J_i,\ \alpha_{iv}$	At most two adjacent support vertices for u_i and their nonnegative coefficients, whose sum is at most one.
$\sigma(h),\ \Pi$	Hull-test bit pattern of h and the partition into equal-pattern classes.
$A_0,\ b(S)$	All-zero pattern class (empty if absent) and, with priors, the mass outside that class.
g_A	First positive test position for a nonzero pattern class A in a discovery schedule.

3.1 A common majority witness

The majority answers must describe one coherent route before we can compare our discovery schedule with an optimal tree. A common witness provides that route even when no hypothesis has all majority answers.

Lemma 3.1 (Common witness)**.** *In either kind of flat state, a point p strictly inside every majority half-plane can be computed in polynomial time.*

Proof. Flatness forces every three majority sides to intersect. Claim 3.1 and Theorem 2.15 therefore give a common strict point. Claim 3.2 then finds a rational one by a positive-margin linear program. □

3.2 Adjacent supports and circular patterns

The witness lets us express every query relative to the same origin. The next lemma identifies the two properties that make this useful: each original cap is covered by adjacent hull caps, and each target sees those hull tests as one interval.

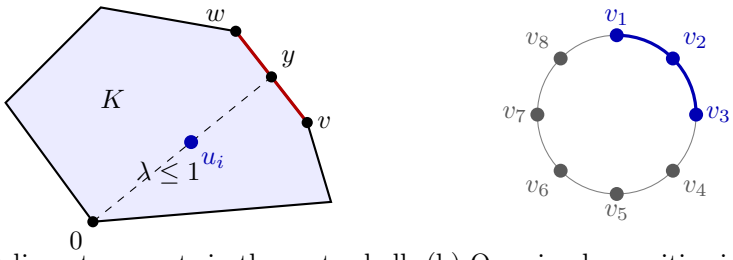
Lemma 3.2 (Hull support and pattern structure)**.** *The support blocks of Definition 2.18 can be chosen with at most two real hull vertices, adjacent when there are two. For every query,*

$$C_i \cap S \ \subseteq \ \bigcup_{v \in J_i} (C_v \cap S). \tag{1}$$

Negative support bits force the query's witness-side answer. The positive vertices of every hypothesis form a circular interval, $|A_0| \le 1$, and every nonzero pattern class lies in a positive hull cap.

Proof. Claim 3.3 projects each query vector radially to a hull edge, and Claim 3.4 shows that its adjacent endpoints cover that query's cap. Claim 3.5 then proves the interval structure and uses separability to bound the all-zero class. □

The routine first finds a point that satisfies every oriented majority inequality, then uses that common witness to normalize the supplied query normals into a planar hull. The supports J_i retain the queries represented by each hull direction; grouping hypotheses by their resulting bit vectors gives the full-pattern classes needed by the recursive algorithms. Lemmas 3.1 and 3.2 justify the witness and support invariant.



(a) Adjacent supports in the vector hull (b) One circular positive interval

Figure 3: The first panel depicts normalized query vectors: y lies on the edge vw and $u_i = \lambda y$ for $0 < \lambda \le 1$. Writing $y = \theta v + (1-\theta)w$ gives $u_i = \alpha v + \beta w$ with $\alpha + \beta = \lambda \le 1$. Consequently, if both support values on a hypothesis are at most one, then $u_i \cdot x_h \le 1$; equivalently, $C_i \subseteq C_v \cup C_w$. The second panel shows the cyclic order only; the positive hull tests of a fixed hypothesis form one interval in that order.

Algorithm 3: HULLDATA: a flat state's witness, supports, and patterns

Input: Flat state S and its count or mass measure; supplied lines.
Output: Strict witness p, full hull K, real vertices V, supports J_i, partition Π.

Orient every majority inequality as $a_i \cdot p - b_i > 0$
Maximize t subject to $a_i \cdot p - b_i \ge t$ and $0 \le t \le 1$; retain a solution with $t > 0$
Compute $u_i = -a_{\ell_i}/f_{\ell_i}(p)$ and $K = \mathrm{conv}(\{0\} \cup \{u_i\})$
Keep one supplied representative of each real hull vertex in cyclic order
foreach u_i **do**
    Extend its ray from 0 to the last point y in K
    Express y on its boundary edge and scale the coefficients by $\|u_i\|/\|y\|$; omit the origin and zero coefficients to form J_i
Compute all bits $\mathbf{1}[v \cdot (h - p) > 1]$ and group equal rows into Π
return $(p, K, V, (J_i), \Pi)$, retaining the origin's boundary position

When the normalized hull has dimension at most one, its segment endpoints provide the supports directly.

Figure 3 separates the two geometric roles: panel (a) shows how adjacent hull vertices support an original query vector, while panel (b) records only the cyclic order of the hull tests seen positively by one hypothesis.

3.3 Interval hitting and discovery length

A complete discovery route must hit every nonzero pattern. Once these patterns are intervals, we can compute its smallest possible test set; this will also provide the final budget in the batch construction.

Lemma 3.3 (Minimum interval cover)**.** *A minimum-cardinality hitting set for ordinary or circular intervals on a finite candidate order is computable in polynomial time. An empty demand is infeasible; an empty family needs no candidates.*

Proof. On a line, an exchange argument permits choosing the right endpoint of the earliest-ending demand. On a circle, enumerate one forced hit and cut there; the remaining problem is linear. The detailed exchange argument is given below. □

Priority and weight considerations can change the order in which tests are chosen. We therefore also need a length bound for greedy discovery, including the extra saving when the comparison cover

has adjacent runs. Here a demand is a nonempty interval of candidate positions, each demand I has a fixed weight $w(I) > 0$, and a candidate's remaining weight is the sum of $w(I)$ over currently unhit demands containing it. At each step greedy chooses a candidate of maximum remaining weight; ties may be resolved arbitrarily. The comparison cover P is any finite set of positions hitting every initial demand, $k = |P|$, and $c(P)$ counts its maximal consecutive blocks in the full candidate order. After a greedy choice, demands hit by that choice are deleted. The claimed bound holds for every such cover P; taking a minimum cover gives the stated special case.

Lemma 3.4 (Greedy cover length with adjacent-run credit)**.** *Let a nonempty family of positively weighted interval demands be covered by a candidate set P of size k. Let $c(P)$ be the number of runs of consecutive candidates in P, measured in the entire candidate order. Maximum-remaining-weight greedy finishes within $k + c(P) - 1$ choices, for every tie rule. In particular, a minimum cover of size k gives the usual bound $2k - 1$. For circular intervals the corresponding bound is $2k$.*

The extra term is the number of runs beyond the first. A scattered cover may force greedy to spend choices repairing several gaps, whereas a cover arranged in a few contiguous blocks has fewer such gaps to repair.

Proof strategy.
Use the potential $|P|+c(P)$ for the still-active comparison cover. An extreme choice can replace an extreme point of P without changing which positive-weight demands it hits. An interior choice partitions the unhit demands into left and right interval families. The cover parts on those sides have combined size plus run count at most the previous potential. Induction pays for the first choice and then for each nonempty side.

Proof. Induct on the comparison cover size, separating extreme choices from interior ones. Suppose first that the greedy choice v lies left of P, and let $u = \min P$. Every active interval containing v also contains u: it is covered by some $p \in P$, and the endpoint inequalities give $v < u \le p$ inside that interval. Thus the hit-demand set of v is contained in the hit-demand set of u. Maximum remaining weight gives $W(u) \le W(v)$, while containment and nonnegative weights give $W(v) \le W(u)$. Since each weight is positive, the sets are equal, so the two hit sets coincide. After querying v, every remaining demand therefore avoids u and is covered by some point of $P' = P \setminus \{u\}$. The right extreme is symmetric. Since u is an extreme point, deleting it cannot split a run, so $|P'| = k - 1$ and $c(P') \le c(P)$. If demands remain, induction bounds their greedy completion by

$$|P'| + c(P') - 1 \le k + c(P) - 2.$$

Including the current query gives at most $k + c(P) - 1$ choices. If no demand remains, the single current query already meets that bound.

Now suppose the greedy choice v is interior. Every unhit interval lies wholly to one side of v. Let P_- and P_+ be the comparison points strictly to its left and right. They cover the corresponding families. If $v \notin P$, cutting the original order in its gap preserves the total number of runs, so $c(P_-) + c(P_+) \le c(P)$ and their sizes sum to k. If $v \in P$, the side sizes sum to $k - 1$, and deleting v can split at most one run into two, so $c(P_-) + c(P_+) \le c(P) + 1$. In either case,

$$|P_-| + c(P_-) + |P_+| + c(P_+) \le k + c(P).$$

The induction hypotheses bound the two side schedules by their respective potentials minus one. Adding the current choice gives at most $k + c(P) - 1$. When only one side remains active, the unused side has at least one comparison point and hence potential at least two; this pays for the absent induction term. When neither side remains, one choice is within the bound because $k \ge 2$

in the interior case. Finally $c(P) \le k$, since each run contains a distinct point of P; therefore $k + c(P) - 1 \le 2k - 1$. For circular demands, the first greedy choice cuts the circle and leaves ordinary intervals; the original k points still cover those intervals, giving at most $2k - 1$ further choices and the bound $2k$. □

3.4 Recovering the interval endpoints

A positive vertex locates a small cap. We recover the full pattern by opening the circular order at a safe cut and comparing the interval's endpoints against the supplied queries.

Lemma 3.5 (Query-free endpoint representation)**.** *After one positive hull vertex v is known, a cut computable without queries represents every compatible positive pattern as an ordinary interval containing v. Supplied hull tests compare its endpoint coordinates. A cached negative neighbor fixes one endpoint; two negative neighbors fix the entire pattern.*

Proof strategy.
Use a cached negative neighbor as the cut when available. Otherwise the origin and an opposite hull edge give a cut that no compatible positive interval can cross. All later comparisons are actual hull tests, so no new line or anchor query is introduced.

Proof. A cached negative neighbor supplies a cut and pins one endpoint. Otherwise an opposite hull edge gives a cut without asking a query. In the resulting linear order, supplied hull tests are endpoint comparisons. Claims 3.8, 3.9, and 3.10 establish these steps, including degenerate hulls. □

A positive discovery exit is *pinned* if a neighbor of its discovered vertex has a cached negative answer. It is *singly pinned* when exactly one neighbor is known negative, and *doubly pinned* when both are. Pinning fixes one or both endpoints, as in Lemma 3.5; it introduces no additional query.

Algorithm 4: EndpointCode: legal full-pattern recovery

Input: A positive hull vertex v, compatible patterns, cached answers; code mode; optionally fixed continuation heights h_A.
Output: A supplied-query decoder whose leaves are full pattern classes.

if *the hull has dimension at most one* **then**
  return the already determined nonzero pattern
if *v has a cached negative neighbor* **then**
  Cut at that neighbor and orient so v is the first positive vertex
else
  Cut at a boundary origin, or at the opposite edge met by the ray $v0$
List the realized left and right endpoint pairs in the cut order
if *no continuation heights are supplied* **then**
  Search each unfixed endpoint using Algorithm 2; condition the second search on the first answer
else
  foreach *nonempty realized endpoint rectangle U, in increasing size* **do**
    if *U has a single realized class A* **then**
      $E(U) \leftarrow h_A$
    else
      $E(U) \leftarrow \min_\ell(1 + \max\{E(U \cap \ell^-), E(U \cap \ell^+)\})$, over proper supplied endpoint thresholds
  Reconstruct the minimizing rectangle splits, retaining full-pattern leaves and their recorded continuation heights
return the decoder; skip fixed endpoints and cached thresholds

The cut turns the cyclic endpoint problem into a linear one, and Lemma 3.5 shows that its thresholds are supplied queries, so no extra anchor query is needed. If child heights are provided, the rectangle recurrence compares each legal split by the height of its endpoint subproblems together with the continuation at the resulting pattern leaf. This is the complete-exit optimization of Section 8. Its $O(M^4)$ rectangles and $O(M)$ thresholds give polynomial running time. The routine returns pattern leaves, after which the recursive algorithm attaches each stored child once. The values $E(U)$ include the continuation heights when comparing splits, while each child's query cost is accounted for inside its attached recursive tree.

3.5 Proofs of the geometric and interval facts

The key step is to convert planar caps into executable endpoint comparisons. Radial projection gives the one-way cover implication needed by the analysis: every positive original cap is covered by its support caps. The endpoint cut then follows from the geometry or from cached answers, so it requires no additional query.

Proof of Lemma 3.1. There are two distinct tasks: obtain a common strict point from flatness, then find such a point with controlled rational encoding. The following claims separate the geometric existence argument from its algorithmic realization.

Claim 3.1 (Three majority half-planes have a common strict point)**.** *Every subfamily of at most three majority half-planes intersects.*

Proof. Use the measure defining flatness, either cardinality or probability. By Definition 2.3, each minority cap has measure less than δ times the measure of S. The union bound therefore makes the union of any three such caps have measure less than 3δ times that of S. Since $3\delta < 1$, that union omits a hypothesis. That hypothesis belongs strictly to all three majority half-planes, because no hypothesis lies on a query line. The same argument covers subfamilies of fewer than three lines. Thus every subfamily of at most three majority half-planes has nonempty intersection. Theorem 2.15 implies that the whole family has nonempty intersection. □

We now turn this strict intersection into the preprocessing algorithm's witness. A positive-margin linear program finds a rational point with polynomial encoding length, so the algorithm can compute and store it.

Claim 3.2 (A positive margin gives a rational strict witness)**.** *The common point can be found with polynomial encoding length.*

Proof. To obtain a strict rational witness, orient each majority inequality as $a_i \cdot p - b_i > 0$ and maximize t subject to $a_i \cdot p - b_i \geq t$ for every i and $0 \leq t \leq 1$. The strict point obtained from Claim 3.1 and Theorem 2.15 has a positive value in each of the finitely many inequalities. Their minimum, capped at one, supplies a feasible positive t. This is a *linear program*: its objective and constraints are linear in the unknown coordinates of p and the margin t. Rational linear programming therefore returns a strict witness with polynomial encoding length [12]. □

□

Proof of Lemma 3.2. We establish the structure in three steps. First represent each query by neighboring hull vertices; next prove the cap-cover implication; finally identify the interval patterns and the unique possible all-zero target. Each step is used by a different part of the algorithms.

Claim 3.3 (Radial boundary projection needs only adjacent supports)**.** *Each normalized vector has a nonnegative support expression whose coefficients sum to at most one.*

Proof. Extend the ray from 0 through u_i to its last point y in K. Such a point exists because the finite hull is compact and contains $u_i \neq 0$. Write $y = tu_i$ with $t \geq 1$, so $u_i = \lambda y$ for $\lambda = 1/t \in (0, 1]$. If y lies on an edge with endpoints v, w, write $y = \theta v + (1 - \theta)w$, with $0 \leq \theta \leq 1$. Then

$$u_i = (\lambda\theta)v + \lambda(1 - \theta)w, \qquad \lambda\theta + \lambda(1 - \theta) = \lambda \leq 1.$$

The coefficients are nonnegative. Delete origin endpoints and zero coefficients to obtain J_i. A boundary vertex needs only itself. In dimension at most one, the last ray point is a segment endpoint, so the same conclusion holds directly. □

The support expression is useful because it transfers a negative answer from the hull tests back to the original query. Only this one-way implication is needed to cover an optimal witness route.

Claim 3.4 (Negative support answers force the original answer)**.** *The hull-cap inclusion* (1) *holds.*

Proof. If every support has $v \cdot x_h \leq 1$, the expression from Claim 3.3 gives

$$\begin{aligned} u_i \cdot x_h &= \sum_{v \in J_i} \alpha_{iv}(v \cdot x_h) \\ &\leq \sum_{v \in J_i} \alpha_{iv} \leq 1. \end{aligned}$$

The first inequality uses the nonnegative support coefficients; the second uses their sum bound. Thus a positive original query must have a positive support, proving (1). Equality cannot occur at a hypothesis because it would put it on the original supplied line. The origin has dot product zero, so its bit is always negative and it is never queried. □

Having justified cap coverage, we now describe the states produced by hull tests. Their interval structure enables the scheduling tools; the uniqueness of the zero pattern makes the final negative route valid.

Claim 3.5 (The hull patterns are intervals with at most one zero point)**.** *Every nonzero class belongs to a supplied positive hull cap.*

Proof. The positive vertices are $V \cap R_h$. Here is why they are consecutive. Intersect the boundary line $u \cdot x_h = 1$ with the convex polygon K. Its intersection with K is empty or a segment; in a genuine crossing its two endpoints cut the polygon boundary into two arcs, one on each side of the line. The positive vertices lie on the positive arc. A tangency leaves all vertices on one side and also gives an empty or consecutive block. No hull query vertex lies on the boundary line, since that would place h on its supplied query. Removing the known-negative origin, if it is a vertex, leaves a circular interval on V. Degenerate hulls have at most two real vertices and satisfy the same assertion. By Claim 3.4, an all-zero pattern forces every original query to have the witness's answer. Two such hypotheses would therefore have identical answers on all supplied lines, contradicting separability. Finally, a nonzero class has some positive vertex v, and all its hypotheses belong to $C_v \cap S$. □

□

Proof of Lemma 3.3. For ordinary intervals, choose the right endpoint q of an earliest-ending interval I. Any cover has a point $u \in I$. Replacing u by q loses no demand containing u: its right endpoint is at least q, by the earliest-ending choice of I, and its left endpoint is at most $u \le q$. Hence it also contains q. Thus an optimal cover can choose q. Delete the intervals hit there and repeat, proving the endpoint greedy rule is exact.

For circular intervals, choose a nonempty demand and enumerate a forced candidate v in it. Delete demands hit by v and cut the cycle there. All remaining demands are ordinary intervals, so use the preceding rule and add v. Some forced choice belongs to an optimal circular cover; take the smallest resulting solution. □

Proof of Lemma 3.4. Induct on k. When $k = 1$, its sole candidate hits every demand, and any greedy maximum does so as well. Let v be the first greedy choice.

At an extreme, greedy removes the same demands as one comparison-cover point. We can therefore delete that point from the comparison cover, which gives the induction step that reduces the cover size.

Claim 3.6 (An extreme greedy choice removes an extreme cover point)**.** *The remaining demands admit the cover $P \setminus \{u\}$ with no increase in run count.*

Proof. If v lies at or outside an extreme of P, let u be the nearest extreme cover point. For example, if $v < \min P = u$, every demand containing v must also meet P, and hence must contain u: if $p \in P$ is a point of that demand, then $v < u \le p$ and the demand is an interval. Thus the demands hit by v are a subset of those hit by u, which gives $W(v) \le W(u)$ by summing nonnegative weights. Greedy maximality gives $W(u) \le W(v)$, so the scores are equal. Because every weight is strictly positive, a proper containment would make the second score strictly larger; hence the candidates hit the same demands. Thus $P \setminus \{u\}$ covers the remainder, and removing an extreme leaves its run count unchanged or smaller. Induction plus the first query gives at most $k + c(P) - 1$ choices. □

For an interior choice, the comparison cover may retain all its points. The unhit demands then split into independent left and right subproblems, whose combined size-plus-run charge supplies the required progress.

Claim 3.7 (An interior choice creates independent left and right problems)**.** *The sum of their size-plus-run charges is at most $k + c(P)$.*

Proof. Otherwise v lies strictly between the extremes of P. The unhit intervals lie entirely on one side of v. The parts P_-, P_+ strictly to its left and right cover these two families, and

$$|P_-| + c(P_-) + |P_+| + c(P_+) \le k + c(P).$$

If $v \notin P$, cutting in its gap leaves the total run count unchanged. If $v \in P$, removing it reduces size by one and can increase total run count by at most one, which proves the displayed inequality in either case. Runs are measured against the full candidate order, so truncating to the two sides creates no new gap boundary except at the removed v. Greedy may interleave the two sides, but a choice on one side covers no interval on the other; hence the choices restricted to either side are still a greedy sequence for that side. If both families are nonempty, each side has a smaller comparison cover than P. Applying the induction hypothesis to both sides and paying for the first query gives

$$1 + \big(|P_-| + c(P_-) - 1\big) + \big(|P_+| + c(P_+) - 1\big) \le k + c(P) - 1.$$

The last inequality substitutes the preceding bound on their combined size-plus-run charge. If only one is nonempty, the other cover part still has size plus run count at least two. If P_{live} covers the nonempty side, the total is at most

$$1 + \big(|P_{\text{live}}| + c(P_{\text{live}}) - 1\big) \le k + c(P) - 2 \le k + c(P) - 1.$$

Here induction bounds the live side, while the unused side's charge of at least two bounds its complement in $k + c(P)$: at least one point remains on that side since v lies strictly between the extremes, and one nonempty run contributes one to each term. If neither remains, one query suffices, and $k \ge 2$ in this interior case ensures $1 \le k + c(P) - 1$. □

Claims 3.6 and 3.7 complete the induction. Finally $c(P) \le k$, because each run contains a distinct cover point, giving $2k - 1$ without adjacency information.

For circular intervals, make the first greedy choice and cut the cycle there. Every unhit interval becomes an ordinary interval. The original k cover candidates still cover these demands, so at most $2k - 1$ further choices are needed. □

Proof of Lemma 3.5. First exploit a cached negative answer if one exists. Otherwise find a safe cut geometrically. After either construction, verify that the resulting endpoint searches ask only supplied hull queries.

Claim 3.8 (A cached negative neighbor pins an endpoint)**.** *Only the opposite endpoint remains to be learned.*

Proof. After discovery, one positive hull vertex v is known. First inspect its cached neighbor answers. If a neighbor is known to be negative, cut at that vertex and orient the order so that v is first. The positive interval starts at v, so only its other endpoint needs a search. If both neighbors are negative, the pattern consists of v alone and no endpoint query is needed. □

When the discovery path has no cached negative neighbor, the geometry supplies the cut. The origin's known-negative position ensures that a compatible positive interval cannot cross the opposite edge.

Claim 3.9 (The opposite-edge cut costs no query)**.** *Without a cached cut, the normalized hull supplies a cut that no compatible positive interval can wrap around.*

Proof. Otherwise use a geometric cut. If the origin lies on the hull boundary, cut at its free negative marker. If it is interior, the ray from v through 0 meets an opposite hull edge with consecutive endpoints a, b. The origin is a convex combination of v, a, b, with positive coefficient on v. Explicitly, the intersection point is $y = -tv$ for some $t > 0$ and $y = \theta a + (1 - \theta)b$, so

$$0 = \frac{t}{1+t}v + \frac{\theta}{1+t}a + \frac{1-\theta}{1+t}b.$$

The coefficients are nonnegative and sum to one. If a, b were both positive, then $v \cdot x_h, a \cdot x_h, b \cdot x_h > 1$, giving

$$\begin{aligned} 0 = 0 \cdot x_h &= \frac{t}{1+t}(v \cdot x_h) + \frac{\theta}{1+t}(a \cdot x_h) + \frac{1-\theta}{1+t}(b \cdot x_h) \\ &> \frac{t + \theta + (1-\theta)}{1+t} = 1, \end{aligned}$$

a contradiction. The strict step multiplies the positive test values by nonnegative coefficients; the coefficient of v is strictly positive, so the sum is strictly greater than one even if an opposite-edge

coefficient vanishes. Cut the cyclic order between a and b. A circular positive interval could wrap around this cut only if both endpoints were positive, which was just excluded. If the ray meets a vertex y, then $y \cdot x_h = -t(v \cdot x_h) < 0 < 1$ for every compatible hypothesis, so it is a free negative cut point. □

The cut has removed wrap-around. We can now verify that learning the two endpoints amounts to asking existing hull tests, which is essential for keeping the construction within the supplied-query model.

Claim 3.10 (Endpoint comparisons are supplied hull queries)**.** *Both endpoint searches, including their degenerate cases, are legal.*

Proof. By Claims 3.8 and 3.9, the positive vertices now form an ordinary interval containing v, obtained without an anchor query. To its left, querying a vertex asks whether the left endpoint is at or before that vertex. To its right, querying a vertex asks whether the right endpoint is at or after it. Thus supplied hull queries give the ordered comparisons needed to locate the two endpoints.

When neither endpoint is already fixed, learn the left endpoint, then the right endpoint conditioned on the left endpoint and the earlier transcript. For consecutive possible left endpoints, querying the earlier endpoint separates them; for consecutive possible right endpoints, querying the later endpoint separates them. These are real hull vertices, so all comparisons used by the alphabetic codes of Lemma 2.10 are supplied queries. The conditional endpoint distributions come from the hypothesis table.

If K has dimension at most one, it has at most two real vertices. With two real endpoints, one positive endpoint forces the other negative because 0 lies between them. With one real vertex, its positive bit is the complete nonzero pattern. Thus no further query is needed in either degenerate case. □

□

4 Maximum-depth scheduling ingredients

The maximum-depth algorithm of Section 8 constructs its pattern children first and then schedules their discovery. This section isolates its local tools. The priority scheduler extends the construction beyond the exact scheduler's logarithmic rank cap by supplying a passing height trial. All branch decisions use hypothesis counts and supplied-line geometry.

Let $\delta \in (0, 1/3)$ be the count-balance threshold and let $\gamma > 2$ be the coefficient of the count potential $\Phi(s) = \lfloor \gamma \log_2 s \rfloor$. Assume

$$\gamma \log_2 \frac{1}{1-\delta} \geq 1, \qquad \gamma \log_2 \frac{1}{\delta} \geq 5. \tag{2}$$

The first inequality pays for a balanced query; the second pays for endpoint recovery at a flat state. Section 8.3 chooses the constants at the point where these two credits balance.

For a state S, write $C_W(S)$ for the height of the tree returned by the hybrid recursive algorithm. When a proof refers to a stored child code, its height is the actual height of that code.

4.1 Local credits and the priority fallback

Proof strategy.

1. Restrict an optimal tree to a pattern class: its common witness prefix disappears and gives that class a discovery deadline.
2. Recover the pattern with supplied endpoint comparisons; the count-potential drop pays for this decoder.
3. Use priorities to meet all deadline prefixes in one ordering. Every accepted ordering is checked against actual stored child heights.

In a count-flat state S of size s, use the common witness and hull patterns of Lemmas 3.1 and 3.2. Every nonzero class A has $a = |A| < \delta s$; the zero class A_0 is empty or a singleton. Write

$$z_A = \lfloor \log_2(\delta s/a) \rfloor.$$

All child heights below come from already constructed codes; the optimal tree appears only in the comparison argument.

The recursion needs integer query allowances, while its potential uses logarithms. The following credit converts size reduction into a whole query at a balanced split and enough whole queries for a flat decoder.

Proposition 4.1 (Integer count credit)**.** *A balanced child of size $a \le (1-\delta)s$ satisfies $\Phi(s)-\Phi(a) \ge 1$. A nonzero flat-state class satisfies*

$$\Phi(s) - \Phi(a) \ge \lfloor \gamma \log_2(s/a) \rfloor \ge z_A + 5. \tag{3}$$

For a flat state, the optimal tree supplies an analytical comparison schedule. Its initial witness answers are constant on a class until that class meets a support; restriction turns this delay into saved height inside the class.

Proposition 4.2 (Supporting-prefix credit)**.** *Follow a common witness in a proper optimal tree of height $D = \mathrm{OPT}_W(S)$ along a path of length r. Let t_A be the first path query whose real hull supports meet the positive pattern of A; put $t_A = r + 1$ if none does. Then*

$$\mathrm{OPT}_W(A) \le D - (t_A - 1).$$

The supporting singleton or adjacent-pair blocks cover every nonzero class by its index t_A, after appending one positive support for an uncovered terminal nonzero singleton when necessary.

A first positive answer locates one vertex of the pattern. The endpoint decoder then recovers the full interval, with a pointwise cost bound so maximum depth controls each class separately.

Proposition 4.3 (Legal endpoint allowances)**.** *After a first positive hull answer, the pointwise alphabetic endpoint codes recover class A with at most*

$$e_A^{(4)} = z_A + 4$$

additional supplied queries. If a neighboring hull vertex is already known negative, one endpoint is fixed and the allowance is $e_A^{(2)} = z_A + 2$. A hull of dimension at most one needs no further query after discovery.

Scheduling can now use the codes already built for the children. Adding their actual heights to discovery and endpoint recovery gives a directly checkable route certificate, including the all-negative route.

Proposition 4.4 (Local route-height certificate)**.** *Let h_A be a stored class code's actual height. Discover a positive vertex in a fixed ordering truncated when all nonzero patterns are covered. If its first positive position for A is g_A, the whole route through A has height at most $g_A + h_A + e_A^{(4)}$, or $g_A + h_A + e_A^{(2)}$ on a pinned exit. If every such bound is at most H, the whole tree has height at most H, including its zero route when that class exists.*

The classes have different deadlines, yet the algorithm must choose one query order. Integer priorities make later deadlines unable to interfere with greedy progress on any earlier deadline prefix.

Proposition 4.5 (One ordering for all block deadlines)**.** *Force an oriented adjacent real pair (v, w) first. Consider only classes negative on both, with integer block deadlines $d_A \geq 2$. Suppose a comparison sequence begins with $\{v, w\}$ and hits A within d_A singleton or adjacent-pair blocks. Sort the distinct budgets as $d^{(1)} < \cdots < d^{(r)}$, give a deadline-$d^{(j)}$ class weight $(s+1)^{r-j}$, and complete by maximum-remaining-weight greedy. This polynomial-time ordering satisfies $g_A \leq 3d_A - 2$ for every surviving class simultaneously.*

The endpoint and prefix credits turn the optimal witness path into deadlines, and the priority ordering meets those deadlines. It therefore provides a passing trial even when the capped exact scheduler cannot certify the stronger bound.

Proposition 4.6 (Priority fallback height)**.** *Suppose stored pattern children satisfy the fallback bound*

$$h_A \leq 3\,\mathrm{OPT}_W(A) + \Phi(|A|).$$

At a flat state, Algorithm 5 has a passing trial at

$$H^* = 3\,\mathrm{OPT}_W(S) + \Phi(s).$$

A balanced query also fits this height if its two stored children obey the same bound. Consequently the hybrid recursion can always retain $C_W(S) \leq 3\,\mathrm{OPT}_W(S) + \Phi(s)$ as its fallback invariant.

For a trial H and a forced pair, first-positive exits must pass $1 + h_A + e_A^{(4)} \leq H$, and second-query exits must pass $2 + h_A + e_A^{(2)} \leq H$. Classes negative on both receive

$$u_A = h_A + e_A^{(4)}, \qquad B_A = H - u_A, \qquad d_A = \lfloor (B_A + 2)/3 \rfloor.$$

Reject any $B_A < 4$ and accept only after checking the resulting actual first-hit positions against $g_A + u_A \leq H$. These tests describe an executable height certificate; the optimal comparison blocks are used only to prove that some trial passes.

For a fixed target height H, each oriented adjacent pair determines which patterns are hit at its first endpoint, second endpoint, or neither. The first two groups have immediate local height tests, so a pair that already exceeds H can be discarded. For patterns left on the miss route, the remaining height becomes a deadline; the priority weights make the greedy order favor the tightest deadlines. The final check uses the actual hit ranks and child allowances, and accepts the order only when every miss-route pattern fits.

Algorithm 5: PRIORITYTRIAL: internal fallback for one height trial

Input: Flat state's hull and classes; child heights h_A; allowances $e_A^{(4)}, e_A^{(2)}$; target H.
Output: A passing discovery order, or infeasible.

foreach *oriented adjacent real pair* (v, w) **do**
  Form $\mathcal{F}, \mathcal{S}, \mathcal{N}$ by their first hit on (v, w)
  if *some* $A \in \mathcal{F}$ *has* $1 + h_A + e_A^{(4)} > H$, *or some* $A \in \mathcal{S}$ *has* $2 + h_A + e_A^{(2)} > H$ **then**
    continue
  foreach $A \in \mathcal{N}$ **do**
    $u_A \leftarrow h_A + e_A^{(4)}$; $B_A \leftarrow H - u_A$; $d_A \leftarrow \lfloor (B_A + 2)/3 \rfloor$
  if *some* $B_A < 4$ **then**
    continue
  Sort distinct budgets as $d^{(1)} < \cdots < d^{(r)}$
  Give each deadline-$d^{(j)}$ class weight $(|S| + 1)^{r-j}$
  Force v, w and complete by maximum-remaining-weight greedy; truncate when all nonzero classes are hit
  if *every* $A \in \mathcal{N}$ *has actual* $g_A + u_A \le H$ **then**
    return this order
return infeasible

4.2 Technical proofs of the local ingredients

Proof of Proposition 4.1. For real x, y, write $x - y = k + \theta$ with integer $k = \lfloor x - y \rfloor$ and $0 \le \theta < 1$. Then $\lfloor x \rfloor = \lfloor y + k + \theta \rfloor \ge \lfloor y \rfloor + k$, proving the floor-difference inequality. Apply it to the two count potentials:

$$\Phi(s) - \Phi(a) \ge \lfloor \gamma \log_2(s/a) \rfloor .$$

Indeed, the difference of the unfloored logarithms is $\gamma \log_2 s - \gamma \log_2 a = \gamma \log_2(s/a)$, which is the argument to the floor-difference inequality. For a balanced child, $a \le (1-\delta)s$, hence $s/a \ge 1/(1-\delta)$. The first condition in (2) therefore gives

$$\gamma \log_2(s/a) \ge \gamma \log_2 \frac{1}{1-\delta} \ge 1.$$

Since every real number at least one has floor at least one, the balanced-child claim follows, including the equality case. For a flat class put $x = \log_2(s/a)$ and $\ell = \log_2(1/\delta)$. Since $a < \delta s$, we have $x > \ell$. The calibration gives $\gamma\ell = 5$, so, using $\gamma > 1$,

$$\gamma x = \gamma\ell + \gamma(x - \ell) = 5 + \gamma(x - \ell) > 5 + (x - \ell).$$

Monotonicity of the floor therefore gives

$$\lfloor \gamma x \rfloor \ge \lfloor 5 + x - \ell \rfloor = 5 + \lfloor \log_2(\delta s/a) \rfloor = 5 + z_A,$$

which proves the flat credit including its integer rounding. □

Proof of Proposition 4.2. There are two distinct steps. Common answers give a restriction credit; supporting blocks then cover the positive classes with deadlines measured from the same witness path. The terminal leaf needs separate attention because the witness may end at a real hypothesis with a nonzero pattern. Claims 4.1 and 4.2 establish these two steps, respectively.

Claim 4.1 (The common prefix disappears under restriction)**.** *Every member of A follows the witness's first $t_A - 1$ answers.*

Proof. For every index $i < t_A$, no support of the ith query meets the positive pattern of A. Lemma 3.2 then makes its answer constant on A, with the same value as on the witness. This includes a query whose support uses the free origin marker. Thus every member of A traverses the same first $t_A - 1$ nodes of the optimal tree.

Restrict that tree to A. Since A is nonempty and each of those answers is constant, each corresponding node contracts to its witness-side child. The remaining tree identifies A and has height at most $D - (t_A - 1)$: each original path had length at most D, and the shared prefix has been removed from every such path. Minimizing over identifying trees on A proves the inequality. □

The restriction credit must come with a cover that can be compared against a scheduler. Queries along the path cover every class except, possibly, the hypothesis at its terminal leaf.

Claim 4.2 (The supporting blocks form a comparison cover)**.** *Only an uncovered terminal nonzero singleton needs an appended block.*

Proof. Each class with $t_A \le r$ is hit by that path query's supports. By Claim 4.1, an unhit class follows the entire original path and is therefore the single leaf hypothesis reached by the witness. If its pattern is nonzero, append a positive hull vertex at index $r + 1$; if it is zero, the witness leaf already completes the cover. Retaining the first occurrence of a repeated support preserves every first-hit deadline. □

□

Proof of Proposition 4.3. The query-free cut of Lemma 3.5 makes the positive pattern an ordinary interval. Let R be the hypotheses compatible with the first positive transcript, so $|R| < \delta s$. Use uniform counts, conditioned on that transcript, as code-design probabilities, while the algorithm itself follows the deterministic construction. The two conditional endpoint probabilities multiply to the probability of the full class. Their logarithmic query bounds therefore telescope to one bound depending on the class size. Claims 4.3 and 4.4 establish the two allowances, according to whether discovery has already fixed a boundary.

Claim 4.3 (Two endpoint searches telescope their probabilities)**.** *If a_L compatible hypotheses have the target's left endpoint, its full pattern is learned in fewer than $\log_2(\delta s/a) + 4$ queries.*

Proof. Lemma 3.5 makes both searches legal. Applying the pointwise bound of Lemma 2.10 to the first endpoint and then to the conditional second endpoint gives

$$\log_2 \frac{|R|}{a_L} + 2 + \log_2 \frac{a_L}{a} + 2 = \log_2 \frac{|R|}{a} + 4 < \log_2 \frac{\delta s}{a} + 4.$$

The logarithms add because $(|R|/a_L)(a_L/a) = |R|/a$: the second distribution is conditioned on the learned first endpoint. The endpoint pair uniquely specifies the pattern, while the strict inequality uses $|R| < \delta s$: the first positive transcript confines R to one minority cap. If q_A is the integer number of endpoint queries, the displayed strict bound gives $q_A < \log_2(\delta s/a) + 4 < z_A + 5$. Thus $q_A \le z_A + 4$, including when the logarithm itself is an integer. □

A negative neighbor supplies more information than ordinary discovery: it fixes an endpoint. This removes an entire endpoint search and gives the extra saving needed on the second exit of the initial pair.

Claim 4.4 (A cached negative neighbor removes one search)**.** *A pinned positive vertex fixes one interval endpoint.*

Proof. Cut at the negative neighbor and orient the order to place the positive vertex first. Lemma 3.5 fixes that boundary; hence Lemma 2.10 requires only one search to learn the other endpoint in fewer than $\log_2(\delta s/a) + 2$ queries. Writing its integer query count as q_A, we have $q_A < \log_2(\delta s/a) + 2 < z_A + 3$, hence $q_A \le z_A + 2$. The negative neighbor is a real cached answer, so no anchor query is added. In dimension at most one the positive pattern is already determined; inference and cached answers can only lower cost. □

□

Proof of Proposition 4.4. A route through A makes at most g_A discovery queries, then uses the endpoint allowance of Proposition 4.3, and finally follows a stored code of height h_A. Its height is their sum, bounded by H by hypothesis. The ordering is truncated as soon as all nonzero classes are covered, namely after at most $\max_{A \neq A_0} g_A$ queries. Since h_A and both endpoint allowances are nonnegative, the route tests imply $g_A \le H$ for each nonzero class. Thus the all-negative route also has length at most H and ends at the zero singleton. If that class is absent its leaf is omitted. A nonsingleton state has a nonzero pattern because Lemma 3.2 gives $|A_0| \le 1$. □

Proof of Proposition 4.5. The proof separates priorities from geometry. First, the priorities make the global order greedy for every nested deadline prefix. Then an interval-cover bound limits the number of queries for each prefix.

Claim 4.5 (Later deadlines cannot change a prefix greedy choice)**.** *The global ordering is weighted greedy for every surviving deadline prefix until that prefix is covered.*

Proof. Put $U = (s+1)^{r-j}$. A class of earlier or equal deadline has weight $U(s+1)^{j-i}$ for some $i \le j$, so every candidate's prefix score is an integer multiple of U. For $j < r$, all later classes together have weight at most

$$s(s+1)^{r-j-1} = \frac{s}{s+1}U < U;$$

for $j = r$ their total is zero. Deleting already hit classes can only lower these totals. If one candidate's prefix score exceeds another's, it exceeds it by at least U; no difference in their later-class scores can reverse that comparison. Therefore a global score maximizer also maximizes the prefix score. A nonzero prefix score is at least U and beats every zero prefix score while demands remain. Global tie-breaking chooses among legitimate prefix greedy ties. This applies to every deadline prefix at every step of the same order. □

The prefix argument determines which demands each greedy choice serves. To bound the number of choices, use the forced adjacent pair to cut the line of blocks and remove its first block; the remaining cover's size and runs then pay for every later greedy choice.

Claim 4.6 (The first pair leaves at most $3d-4$ later greedy choices)**.** *A nonempty deadline-d prefix finishes by actual position* $3d-2$.

Proof. Cut at v and delete its adjacent vertex w. Surviving demands are ordinary intervals. Remove these two vertices from the union P of the first d comparison blocks. Each of the other $d-1$ blocks contributes at most three to $|P| + c(P)$, where $c(P)$ counts runs: a pair contributes at most two vertices and one run, and a singleton at most one vertex and one run. Deleting the two adjacent cut vertices cannot split any surviving adjacent-pair block. For finite sets U, V on a line, $|U \cup V| \le |U| + |V|$ and $c(U \cup V) \le c(U) + c(V)$: joining runs cannot create a new run. Applying these inequalities to the remaining $d-1$ blocks gives $|P| + c(P) \le 3(d-1)$. Every prefix demand misses the forced pair and was hit by one of these remaining blocks, so P still covers it. By Claim 4.5, the actual later choices form a greedy sequence for this prefix. Consequently Lemma 3.4 bounds their number by

$$|P| + c(P) - 1 \le 3(d-1) - 1 = 3d - 4.$$

The inequality uses the size-plus-run bound just established; the equality simplifies the resulting integer allowance. Adding the two forced queries gives position at most $3d-2$. An empty prefix needs no further choice. □

The auxiliary weights have $O(s \log s)$ bits, and their incidence scores are computed from the explicit class-to-vertex table. Thus the ordering and actual first-hit times have polynomial bit complexity. □

Proof of Proposition 4.6. Put $D = \mathrm{OPT}_W(S)$. For a balanced child A, restriction and count credit, specifically Lemma 2.12 and Proposition 4.1, give

$$\mathrm{OPT}_W(A) \le D, \qquad \Phi(|A|) \le \Phi(s) - 1.$$

Thus its complete route satisfies

$$1 + h_A \le 1 + 3\,\mathrm{OPT}_W(A) + \Phi(|A|) \le 3D + \Phi(s) = H^*.$$

The first inequality is the stored-child invariant. The second substitutes the two preceding bounds, so the potential drop pays for the new root query. At a flat state, enumerate an initial pair containing the first optimal-witness query's supports; pad a singleton support by an adjacent real neighbor. The first two exits must be checked directly: the deadline scheduler handles only classes that miss both initial queries. The cached negative answer makes the second positive exit cheaper to decode.

Claim 4.7 (Both initial positive exits fit H^*)**.** *The first-query exit has at least one query of discovery credit; the pinned second-query exit has at least three.*

Proof. For either class, Lemma 2.12 gives $\mathrm{OPT}_W(A) \le D$. Using the stored-child invariant and then Proposition 4.1,

$$H^* - h_A \ge 3D + \Phi(s) - 3\,\mathrm{OPT}_W(A) - \Phi(|A|) \ge \Phi(s) - \Phi(|A|) \overset{\text{Prop. 4.1}}{\ge} z_A + 5.$$

The middle inequality drops the nonnegative difference $3(D - \mathrm{OPT}_W(A))$. Thus the first positive exit satisfies $1 + h_A + (z_A + 4) \le H^*$. By Proposition 4.3, the second positive exit has a cached negative neighbor and only the smaller allowance, so $2 + h_A + (z_A + 2) \le H^* - 1 \le H^*$. Neither calculation assumes that the class meets the original support; restriction and flat count credit also cover classes hit only by the padding vertex. □

For the remaining classes, the supporting-prefix credit becomes an integer block deadline. The floor below preserves that credit exactly, so the comparison schedule meets the deadlines computed from actual child heights.

Claim 4.8 (Every later class meets its computed deadline)**.** *For a class negative on the initial pair,* $d_A \geq t_A$.

Proof. Here $t_A \geq 2$, because this class misses the first query's supports. Proposition 4.2 and the actual child invariant give $h_A \leq 3[D - (t_A - 1)] + \Phi(|A|)$. Substituting this into the budget and then using count credit yields

$$\begin{aligned} B_A &= 3D + \Phi(s) - h_A - (z_A + 4) \\ &\geq 3(t_A - 1) + \Phi(s) - \Phi(|A|) - (z_A + 4) \\ &\overset{\text{Prop. 4.1}}{\geq} 3(t_A - 1) + (z_A + 5) - (z_A + 4) = 3t_A - 2 \geq 4. \end{aligned}$$

Hence $(B_A + 2)/3 \geq t_A$; since t_A is an integer, $d_A = \lfloor (B_A + 2)/3 \rfloor \geq t_A$. The padded first block and the remaining comparison blocks, including the terminal singleton block if needed, satisfy Proposition 4.5. Its ordering has

$$g_A \leq 3d_A - 2 = 3\lfloor (B_A + 2)/3 \rfloor - 2 \leq B_A.$$

The first inequality is Proposition 4.5; the equality substitutes the computed deadline. To justify the last inequality, write $B_A + 2 = 3d_A + r_A$ with $r_A \in \{0, 1, 2\}$. Then $3d_A - 2 = B_A - r_A \leq B_A$, so adding $u_A = h_A + e_A^{(4)}$ gives $g_A + u_A \leq H^*$. □

The deadline argument bounds every route that uses block scheduling, including the truncated zero route, by Proposition 4.4 and Claims 4.7–4.8. When the hull has dimension at most one, the scheduler is unnecessary: Lemma 3.5 shows that two extreme tests determine the pattern and leave no endpoints to decode. A nonzero class then costs at most $2 + h_A$. The stored-child, restriction, and count-credit calculation from Claim 4.7 gives

$$2 + h_A \leq 3D + \Phi(s) - z_A - 3 \leq H^*.$$

The second inequality uses $z_A \geq 0$ and the definition of H^*. The zero route also ends after at most the two extreme tests. Thus the local fallback bound holds in every dimension, and induction gives the stated invariant for the hybrid tree. □

5 Scheduling the first positive answer

The maximum-depth algorithm must coordinate discovery with the heights of its child codes. Expected depth poses a different scheduling need: caps containing likely targets should be tested early, which leads to weighted min-sum set cover (MSSC). We use the supplied caps as the covering sets, with a singleton cap included whenever a supplied test realizes it.

In the decision-tree application, a positive hull answer exits the all-negative route and reveals that the target lies in a small cap. The all-zero pattern follows that route through every test; once the schedule has covered every positive pattern, the all-zero class is the remaining outcome. An earlier singleton inference shortens its route. Its expected-cost contribution is therefore bounded by its weight times the schedule length. Accordingly, our ordering bound tracks both the sum of positive-pattern first-cover times and the length paid on the all-negative route.

Symbol	Meaning
$U,\ \pi_h,\ p_0$	Coverable positive-pattern demands, their positive weights, and the weight of a target found only after the schedule finishes.
$t_Q(h),\ \mathrm{MSSC}(Q)$	First-cover position of demand h in ordering Q and the sum $\sum_h \pi_h t_Q(h)$.
k_0	Minimum number of candidate tests needed to cover all demands in U.
$\beta,\ \Phi_\beta,\ Z_\beta$	Geometric budget factor and the first-cover and total-length accounting functions.
$r_Q(x),\ G_u$	Comparison-order mass with first-cover time greater than x, and the constructed schedule for shift u.

Weighted MSSC has a classical greedy four-approximation [1]. For circular intervals, we obtain a stronger guarantee by first solving the maximum-coverage problem exactly for each cardinality budget. Ordering these budget-optimal sets in geometric batches then controls both first-cover cost and total length.

This batch method extends minimum cover by optimizing covered weight at every integer budget. The interval structure makes each optimization tractable: once the previous selected position is known, the next position determines the newly covered demands.

Lemma 5.1 (Exact budgeted interval coverage)**.** *For positively weighted interval or circular-interval demands on a finite candidate set, a set of at most k candidates hitting maximum demand weight is computable in polynomial time.*

Proof. Algorithm 6 gives both interval oracles. For ordinary intervals, number the candidates $1, \ldots, M$. If the previous selected candidate is p and the next is $q > p$, the newly hit demands are the intervals containing q whose left endpoint is greater than p. An interval containing q and an earlier selected point must also contain p. Conversely, any interval containing q with left endpoint greater than p contains no earlier selected candidate. Thus the added weight $g(p, q)$ is determined by the two positions p and q.

To maximize covered weight among selections of t candidates ending at q, use the recurrence

$$D(t, q) = \max_{p<q}\{D(t-1, p) + g(p, q)\}, \qquad D(0, 0) = 0.$$

Here $D(t-1, p)$ records the largest weight covered by a selection of $t-1$ candidates whose last position is p. The preceding characterization shows that appending q adds $g(p, q)$, so the recurrence considers every possible penultimate position and loses no feasible selection. Taking the maximum over final positions and all $t \le k$ therefore gives the best cover using at most k candidates. We reserve position 0 as a *sentinel*, an artificial initial marker before every interval and outside the selectable candidates. It supplies the empty selection as the base case; other nonexistent states have value $-\infty$, so they cannot enter a maximizing transition. Since there are $O(kM)$ states and each maximization checks at most M predecessors, this is a polynomial dynamic program.

For circular intervals and $k \ge 1$, enumerate a selected candidate v. Credit the demands it hits and delete them. Cutting at v turns the remaining demands into ordinary intervals, on which the dynamic program selects at most $k-1$ more candidates. One enumeration contains a candidate of an optimal nonempty selection, so taking the best is exact. The $k = 0$ case is empty. □

Algorithm 6: IntervalCover: exact minimum cover or budgeted coverage

Input: Ordered interval demands with weights; either *minimum* or budget k; ordinary or circular order.

Output: A minimum hitting set, or a maximum-weight set of size at most k.

if *the demand family is empty* **then**
  return $\varnothing$
if *mode is* minimum *and an empty demand exists* **then**
  return infeasible
if *mode is budget coverage and* $k = 0$ **then**
  return $\varnothing$
if *order is circular* **then**
  Enumerate a forced selected candidate v; delete its hit demands and cut there
  Solve each residual ordinary instance, with budget $k - 1$ in budget mode
  return the best residual solution plus v, also considering the empty budget solution
if *mode is* minimum **then**
  $P \leftarrow \varnothing$
  while *a demand remains* **do**
    Choose the right endpoint q of an earliest-ending remaining demand
    $P \leftarrow P \cup \{q\}$; delete all demands containing q
  return P
else
  Set $D(0, 0) = 0$ and all other unreachable states to $-\infty$
  Precompute $g(p, q)$: weight of intervals containing q with left endpoint $> p$
  for $t = 1, \ldots, \min\{k, M\}$ **do**
    for $q = 1, \ldots, M$ **do**
      $D(t, q) \leftarrow \max_{p<q}(D(t - 1, p) + g(p, q))$; store a maximizing p
  return the reconstruction of the best state with $t \le k$, including $(0, 0)$

For minimum circular cover the forced candidate need only range over one nonempty demand. For budget coverage, empty demands contribute zero covered weight. The empty family returns the empty set in both modes. Lemmas 3.3 and 5.1 establish exactness.

We now turn these exact budgeted covers into an ordering. Geometrically increasing budgets let small batches find likely targets early, while the minimum cover at the final budget ensures that every positive pattern is eventually found. A shift changes when the sequence crosses integer budgets; averaging over shifts bounds both positive first-cover times and the wait on the all-negative route. Since only finitely many distinct schedules arise, we can then select one deterministically.

5.1 The batch-ordering guarantee

Fix a nonempty demand family U, its minimum cover size k_0, and the waiting weight p_0. The only independent scheduling parameter is the growth factor $\beta > 1$. The functions below express the shift-averaged bounds on first-cover cost and schedule length. Once β and the current state are fixed, both are determined.

Put

$$r_\beta = \lfloor \beta \rfloor, \qquad \lambda_\beta = \frac{\beta + 1}{2 \ln \beta}, \qquad \nu_\beta = 1 + \frac{1}{\ln \beta}.$$

For integer $t, k \geq 1$, define the accounting functions

$$\Phi_\beta(1) = \frac{r_\beta + 1}{2} - \frac{\ln(r_\beta!)}{2 \ln \beta},$$
$$\Phi_\beta(t) = \Phi_\beta(1) + \frac{1}{2 \ln \beta} \int_1^t \frac{\lfloor x \rfloor + \lfloor \beta x \rfloor}{x}\, dx,$$
$$Z_\beta(k) = k + \frac{k \ln k - \ln(k!)}{\ln \beta}.$$

We first derive a bound for each shifted schedule. Keeping the integer budgets in this calculation preserves the savings gained by rounding down in the early phases, which improves the final approximation factor.

Proposition 5.2 (A shifted schedule has a discrete cost bound)**.** *For a comparison ordering Q, put $r_Q(x) = \sum_{h:t_Q(h)>x} \pi_h$. The shifted construction G_u using budgets $k^{(j)} = \lfloor \beta^j u \rfloor$ satisfies*

$$\mathrm{MSSC}(G_u) \leq \frac{\lfloor u \rfloor + 1}{2} w(U) + \frac{1}{2} \sum_{j \geq 0} (k^{(j)} + k^{(j+1)}) r_Q(\beta^j u).$$

Each such schedule has length at most $(1 + \beta/(\beta - 1))k_0$.

A shift that finds likely positive targets early can still leave a long all-negative route. We therefore average the first-cover cost and the schedule length under one distribution, so a single shift can balance both contributions.

Proposition 5.3 (One shift distribution bounds both costs)**.** *For the log-uniform shift on $[1, \beta)$,*

$$\mathbb{E}[\mathrm{MSSC}(G_u)] \leq \sum_h \pi_h \Phi_\beta(t_Q(h)), \qquad \mathbb{E}[|G_u|] \leq Z_\beta(k_0).$$

They also satisfy $\Phi_\beta(t) \leq \lambda_\beta(t-1) + \Phi_\beta(1)$ and $Z_\beta(k) \leq \nu_\beta k - 1/\ln \beta$.

The average guarantees a schedule with a suitably small joint cost. Because integer budgets change only at explicit breakpoints, a finite search over the resulting schedules can find one no more costly than the average.

Proposition 5.4 (A finite shift search suffices)**.** *For fixed β, polynomially many shifts represent every distinct G_u. Minimizing the joint cost $\mathrm{MSSC}(G_u) + p_0|G_u|$ over these representatives attains at most its log-uniform average.*

Proof strategy.

1. Exact budget coverage bounds the mass left after each phase.
2. Greedy order within a phase gives a half-batch saving in first-cover cost.
3. Average over the same shift for discovery and waiting, then choose the best of finitely many distinct schedules.

The shift averaging establishes the bound; the finite search returns a deterministic schedule.

Lemma 5.5 (Geometric-batch ordering)**.** *For every fixed $\beta > 1$, the following guarantee holds. Let U be a nonempty family of coverable weighted circular intervals. Let k_0 be its minimum cover size and let $p_0 \geq 0$ be the weight of a target covered only when the entire schedule finishes. A deterministic polynomial-time ordering G satisfies, for every comparison cover ordering Q,*

$$\mathrm{MSSC}(G) + p_0|G| \leq \sum_{h \in U} \pi_h \Phi_\beta(t_Q(h)) + p_0 Z_\beta(k_0).$$

The simpler bounds $\Phi_\beta(t) \leq \lambda_\beta(t-1) + \Phi_\beta(1)$ and $Z_\beta(k) \leq \nu_\beta k - 1/\ln\beta$ also hold. Every candidate schedule considered by the construction also has length at most $\big(1 + \beta/(\beta-1)\big)k_0$.

Proof. Proposition 5.2 bounds each shifted schedule. Proposition 5.3 then bounds their joint average, and Proposition 5.4 selects a schedule whose joint cost is no larger. Every candidate also satisfies the length bound, so the selected schedule inherits it. □

Algorithm 7: BATCHORDERS: deterministic family of geometric-batch schedules

Input: Weighted circular demands and fixed $\beta > 1$.
Output: All schedules needed for deterministic shift minimization.

Compute a minimum cover Q_* of size k_0 using INTERVALCOVER
for $k = 1, \ldots, k_0 - 1$ **do**
  Compute a maximum-weight budget-k set Q_k
Find the first J with $\beta^J \geq k_0$
Form the breakpoints $1, \beta$ and $k/\beta^j \in [1, \beta)$, $0 \leq j \leq J$, $1 \leq k \leq k_0$
foreach *breakpoint below β and each midpoint of consecutive breakpoints* **do**
  $u \leftarrow$ that representative; $Q \leftarrow$ empty order
  for $j = 0, 1, \ldots$ **do**
    $k \leftarrow \lfloor \beta^j u \rfloor$; $P \leftarrow Q_k$ if $k < k_0$, and $P \leftarrow Q_*$ otherwise
    Greedily order P by newly covered weight, append its productive unused candidates to Q
    if $k \geq k_0$ **then**
      stop this shift's phases
  Retain Q
return the retained family, removing duplicate schedules

An empty demand family needs no schedule. Lemma 5.5 bounds the family's minimum joint first-cover and waiting cost. The number of representatives is $O(k_0 \log k_0)$ for fixed β.

5.2 Proofs of the batch-ordering propositions

Proof strategy.

1. Compare each budgeted batch with the first equally sized prefix of an arbitrary cover ordering, and bound the total number of scheduled tests by the geometric budget sum.
2. Use greedy order within each batch to bound its first-cover contribution using the half-batch saving.
3. Sum these batch costs, then replace batch residuals by the comparison ordering's residual mass to obtain the discrete bound.

Proof of Proposition 5.2. First use Lemma 3.3 to compute a minimum cover Q_*, and write $k_0 = |Q_*|$. For each integer budget $k < k_0$, Lemma 5.1 supplies a maximum-weight set Q_k of at most

k candidates. Fixing these sets deterministically lets each phase use its best coverage set for that budget; the final phase uses Q_* and therefore covers every demand.

Fix a comparison cover ordering Q, and write

$$r_Q(x) = \sum_{h:t_Q(h)>x} \pi_h$$

for the weight whose first-cover index in Q exceeds x. For a shift $u \in [1, \beta)$, use budgets $k^{(j)} = \lfloor \beta^j u \rfloor$, $j = 0, 1, \ldots$. In each phase, process $Q_{k^{(j)}}$ when $k^{(j)} < k_0$ and Q_* otherwise. Order that set greedily by the weight it covers among the demands still unhit by earlier queries. Omit repeated queries and those hitting no remaining demand. Stop after the first phase whose budget reaches k_0. Two global safeguards come first: each phase covers at least as much weight as a comparison prefix with the same budget, and geometric growth bounds the total number of tests.

Claim 5.1 (Phase coverage and schedule length)**.** *After a phase with budget $k < k_0$, the weight still unhit is at most $r_Q(k)$. The complete shifted schedule has length at most $\big(1 + \beta/(\beta - 1)\big)k_0$.*

Proof. The union of all queries processed through this phase covers every demand covered by Q_k: a query omitted for hitting no remaining demand has its covered demands already hit by an earlier phase. By Lemma 5.1, the set Q_k covers at least as much total weight as the first k queries of Q, since those queries are one feasible at-most-k candidate set. Hence the schedule has hit at least the weight hit by that comparison prefix, leaving at most $r_Q(k)$. The final Q_* phase covers all demands.

Let J be the first phase whose budget reaches k_0. If $J = 0$, there are no pre-final queries and the full length is at most k_0. Otherwise the pre-final phases use at most

$$\sum_{j<J} \lfloor \beta^j u \rfloor \le \sum_{j<J} \beta^j u < \frac{\beta}{\beta - 1} k_0$$

queries. The first inequality uses $\lfloor x \rfloor \le x$. For the strict step, $\lfloor \beta^{J-1} u \rfloor < k_0$ and integrality of k_0 imply $\beta^{J-1} u < k_0$. Reversing the finite geometric sum and bounding it by the infinite sum gives

$$\sum_{j<J} \beta^j u = \beta^{J-1} u \sum_{r=0}^{J-1} \beta^{-r} < k_0 \sum_{r \ge 0} \beta^{-r} = \frac{\beta}{\beta - 1} k_0.$$

The final minimum cover uses k_0 queries, proving the length bound. □

Phase-end coverage alone would charge every new hit at the end of its batch. Greedy places the largest new gains first, giving the half-batch saving captured by the next claim.

Claim 5.2 (First-cover cost within one batch)**.** *Suppose a phase processes m queries, newly hitting weights $g_1, \ldots, g_m$ in order, and starts and ends with unhit weights r and r'. Its contribution to the first-cover sum is at most*

$$mr' + \sum_{i=1}^{m} i g_i \le mr' + \frac{m+1}{2}(r - r') = \frac{m+1}{2} r + \frac{m-1}{2} r'.$$

Proof. After each greedy choice, the remaining coverage of every unused candidate can only decrease. Therefore the newly hit weights are nonincreasing: $g_1 \ge \cdots \ge g_m$. Pairing ranks i and $m + 1 - i$ shows

$$\sum_{i=1}^{m} i g_i \le \frac{m+1}{2} \sum_{i=1}^{m} g_i.$$

Indeed, in each pair the earlier rank has at least as much weight as the later one, so replacing both ranks by their mean cannot decrease the weighted sum. Explicitly, with $a = (m+1)/2$ and $i < a$,

$$(i-a)g_i + (m+1-i-a)g_{m+1-i} = (i-a)(g_i - g_{m+1-i}) \le 0.$$

The middle term, when m is odd, already has rank a. Summing these pair inequalities proves the displayed bound. A demand first hit at rank i remains unhit for i queries in this batch, while each demand still unhit at the end remains unhit for all m queries. Since $\sum_i g_i = r - r'$, their combined contribution gives the claim. □

Now let r_j be the unhit weight after phase j, and put $r_{-1} = w(U)$. If phase j processes m_j queries, the preceding claim bounds its first-cover contribution by

$$\frac{m_j+1}{2} r_{j-1} + \frac{m_j-1}{2} r_j.$$

Let J be the final phase. Since $r_J = 0$, adding the batch costs of Claim 5.2 and collecting the two appearances of each intermediate residual gives

$$\begin{aligned}\mathrm{MSSC}(G_u) &\overset{\text{Claim 5.2}}{\le} \sum_{j=0}^{J} \left(\frac{m_j+1}{2} r_{j-1} + \frac{m_j-1}{2} r_j \right) \\ &= \frac{m_0+1}{2} w(U) + \frac{1}{2} \sum_{j=0}^{J-1} (m_j + m_{j+1}) r_j.\end{aligned}$$

Here the coefficient of r_j is $(m_j-1)/2+(m_{j+1}+1)/2 = (m_j+m_{j+1})/2$. All residuals are nonnegative, so $m_j \le k^{(j)}$ permits replacing each batch length by its budget. Claim 5.1 then bounds $r_j \le r_Q(k^{(j)})$, giving

$$\mathrm{MSSC}(G_u) \le \frac{\lfloor u \rfloor + 1}{2} w(U) + \frac{1}{2} \sum_{j \ge 0} \big(k^{(j)} + k^{(j+1)}\big) r_Q(\beta^j u).$$

For every pre-final phase, Claim 5.1 gives $r_j \le r_Q(k^{(j)}) = r_Q(\beta^j u)$; the equality uses the integer-valued comparison first-cover indices. The final phase has residual zero. The displayed infinite sum only adds nonnegative upper bounds for the final and later phases.

□

Proof strategy.

1. Average the discrete cost bound under the log-uniform shift; compute its initial floor term and combine the scale intervals into one integral, which yields Φ_β.
2. Apply $\lfloor x \rfloor \le x$ for the simpler linear bound.
3. Average the same shift's earlier batch lengths, evaluate the floor integral by a factorial sum, and use the integral lower bound on $\ln(k!)$ to obtain the stated bound on Z_β.

Proof of Proposition 5.3. A logarithmic shift moves the phase boundaries across every scale with the same density. After substitution, their averaging intervals tile $[1, \infty)$; this yields one integral for discovery and another for waiting. Keeping floors in both integrals preserves the discrete savings.

Claim 5.3 (Shift averaging yields the discrete first-cover function)**.** *The same log-uniform shift satisfies* $\mathbb{E}[\mathrm{MSSC}(G_u)] \le \sum_h \pi_h \Phi_\beta(t_Q(h))$.

Proof. Average the cost bound of Proposition 5.2 over a *log-uniform shift*, with probability density $1/(u \ln \beta)$ on $[1, \beta)$. This density integrates to one because $\int_1^\beta du/u = \ln \beta$. Splitting the first term's integral at its integers gives

$$\begin{aligned}\mathbb{E}[\lfloor u \rfloor] &= \frac{\displaystyle\sum_{k=1}^{r_\beta - 1} k \ln \frac{k+1}{k} + r_\beta \ln \frac{\beta}{r_\beta}}{\ln \beta} \\ &= r_\beta - \frac{\ln(r_\beta!)}{\ln \beta}.\end{aligned}$$

For the last equality, expand the finite sum:

$$\sum_{k=1}^{r_\beta - 1} k(\ln(k+1) - \ln k) = (r_\beta - 1)\ln r_\beta - \ln((r_\beta - 1)!).$$

Adding $r_\beta(\ln \beta - \ln r_\beta)$ leaves $r_\beta \ln \beta - \ln(r_\beta!)$. This also holds for $r_\beta = 1$, when the sum is empty. Hence the initial term averages to $\frac{1}{2}(\mathbb{E}[\lfloor u \rfloor] + 1)w(U) = \Phi_\beta(1)w(U)$. The intervals $[\beta^j, \beta^{j+1})$ partition $[1, \infty)$, and substituting $x = \beta^j u$ in each phase gives

$$\begin{aligned}&\sum_{j \geq 0} \int_1^\beta \frac{\lfloor \beta^j u \rfloor + \lfloor \beta^{j+1} u \rfloor}{u} r_Q(\beta^j u)\, du \\ &\quad = \sum_{j \geq 0} \int_{\beta^j}^{\beta^{j+1}} \frac{\lfloor x \rfloor + \lfloor \beta x \rfloor}{x} r_Q(x)\, dx \\ &\quad = \int_1^\infty \frac{\lfloor x \rfloor + \lfloor \beta x \rfloor}{x} r_Q(x)\, dx.\end{aligned}$$

Only finitely many terms have positive residual, since $r_Q(x) = 0$ for $x \geq |Q|$. Dividing by the shift density's normalizing factor $2 \ln \beta$ now yields

$$\begin{aligned}\mathbb{E}[\mathrm{MSSC}(G_u)] &\leq (\Phi_\beta(1))w(U) \\ &\quad + \frac{1}{2\ln \beta} \int_1^\infty \frac{\lfloor x \rfloor + \lfloor \beta x \rfloor}{x} r_Q(x)\, dx \\ &= \sum_{h \in U} \pi_h \Phi_\beta(t_Q(h)).\end{aligned}$$

For each real x, write $r_Q(x) = \sum_{h : t_Q(h) > x} \pi_h$. Interchanging this finite sum with the integral makes each hypothesis contribute the integral from 1 to its integer first-cover time. Adding the initial $(\Phi_\beta(1))w(U)$ term therefore gives $\sum_{h \in U} \pi_h \Phi_\beta(t_Q(h))$. Finally $\lfloor x \rfloor + \lfloor \beta x \rfloor \leq (1 + \beta)x$, so the definition of Φ_β gives

$$\Phi_\beta(t) - \Phi_\beta(1) \leq \frac{1}{2\ln \beta} \int_1^t (1 + \beta)\, dx = \lambda_\beta(t - 1).$$

The inequality replaces each floor by its argument, and the equality uses $\lambda_\beta = (1 + \beta)/(2 \ln \beta)$. Using this simpler bound discards the savings from later integer budgets.

□

The first-cover sum measures the cost for targets that receive a positive answer. To include the all-negative route as well, we average the schedule length under the same shift; the two bounds can then be combined.

Claim 5.4 (Shift-averaged schedule length)**.** *Under the same shift distribution,*

$$\mathbb{E}[|G_u|] \le Z_\beta(k_0) \le \nu_\beta k_0 - \frac{1}{\ln \beta}.$$

Proof. The final cover contributes k_0 queries. The earlier phases contribute at most $\sum_{j:\,\beta^j u < k_0} \lfloor \beta^j u \rfloor$. Under the density $1/(u \ln \beta)$, their expected length is

$$\begin{aligned}
&\frac{1}{\ln \beta} \sum_{j \ge 0} \int_1^\beta \mathbf{1}[\beta^j u < k_0] \frac{\lfloor \beta^j u \rfloor}{u}\, du \\
&\quad = \frac{1}{\ln \beta} \sum_{j \ge 0} \int_{\beta^j}^{\beta^{j+1}} \mathbf{1}[x < k_0] \frac{\lfloor x \rfloor}{x}\, dx \\
&\quad = \frac{1}{\ln \beta} \int_1^{k_0} \frac{\lfloor x \rfloor}{x}\, dx.
\end{aligned}$$

As in Claim 5.3, the first equality uses $x = \beta^j u$, so $du/u = dx/x$; the second joins the disjoint scale intervals and removes the part where the indicator is zero. On each interval $[j, j+1)$, $\lfloor x \rfloor = j$, so summing the integrals gives

$$\begin{aligned}
\int_1^{k_0} \frac{\lfloor x \rfloor}{x}\, dx &= \sum_{j=1}^{k_0 - 1} j(\ln(j+1) - \ln j) \\
&= (k_0 - 1) \ln k_0 - \ln((k_0 - 1)!) \\
&= k_0 \ln k_0 - \ln(k_0!).
\end{aligned}$$

The middle equality cancels consecutive logarithmic terms; the last uses $\ln(k_0!) = \ln k_0 + \ln((k_0-1)!)$. Consequently $\mathbb{E}[|G_u|] \le Z_\beta(k_0)$. Since $\ln x$ is increasing,

$$\ln(k_0!) = \sum_{j=2}^{k_0} \ln j \ge \int_1^{k_0} \ln x\, dx = k_0 \ln k_0 - k_0 + 1.$$

Indeed, on $[j-1, j]$ the increasing function $\ln x$ is at most $\ln j$, so summing those integral comparisons gives the inequality. Evaluating the antiderivative $x \ln x - x$ gives the equality. Consequently

$$Z_\beta(k_0) \le k_0 + \frac{k_0 - 1}{\ln \beta} = \nu_\beta k_0 - \frac{1}{\ln \beta}.$$

The formula also gives $Z_\beta(1) = 1$. □

Claims 5.3 and 5.4 hold under the same shift distribution. Therefore linearity of expectation and $p_0 \ge 0$ give

$$\mathbb{E}[\mathrm{MSSC}(G_u) + p_0 |G_u|] \le \sum_{h \in U} \pi_h \Phi_\beta(t_Q(h)) + p_0 Z_\beta(k_0).$$

□

Proof of Proposition 5.4. No random shift is used by the algorithm. Let J be the first integer with $\beta^J \geq k_0$. All budget changes occur at the breakpoints $u = k/\beta^j$ for $0 \leq j \leq J$, $1 \leq k \leq k_0$, inside $[1, \beta)$. Enumerate these breakpoints and one interior point from each consecutive cell, construct the resulting orderings, and take the one minimizing the rational quantity $\mathrm{MSSC}(G_u) + p_0|G_u|$. There are $O(k_0)$ distinct intervals: for a fixed integer k, a breakpoint k/β^j belongs to $[1, \beta)$ exactly when $\beta^j \leq k < \beta^{j+1}$. These half-open ranges partition $[1, \infty)$, so at most one exponent qualifies for each k. Adding the boundary shifts and one midpoint per gap still leaves $O(k_0)$ representatives. Also $J = O_\beta(1+\log k_0)$. Every constructed ordering uses distinct candidates and hence has length at most $|V|$. Evaluate the joint score $\mathrm{MSSC}(G_u) + p_0|G_u|$ for each representative and return the schedule with minimum score. The breakpoint and midpoint construction leaves only $O(k_0)$ schedules to compare. The budgets, selected sets, and their greedy internal orders are constant on each interval because their only dependence on u is through the integer budgets, with fixed tie-breaking thereafter. Hence the finite family represents every schedule occurring in the average of Proposition 5.3: within each cell, the budget vector, selected sets, and deterministic orders agree with the representative's. Thus every schedule in the average has one of the listed joint scores, and the minimum listed score is at most their weighted average. The waiting-time weight p_0 is used only to choose the schedule; all queries remain supplied candidates. □

6 A polynomial approximation for expected depth

We give one algorithm for arbitrary positive rational priors. The prior is fixed throughout the construction: a query's expected cost is the mass of hypotheses that reach it. At a state admitting a balanced supplied line, the algorithm queries that line. At a flat state, it tests hull blocks until a positive answer is found or a full negative cover identifies the zero-pattern singleton. After a positive answer it learns the complete hull pattern and recurses within that pattern class. Its polynomial-time factor is below 5.182976.

Two costs need to be controlled at a flat state: finding a positive block and recovering the rest of its pattern. The useful comparison is an entire pattern stage. Queries that become constant after the pattern class is known provide credit for discovery. Information already learned during discovery reduces the entropy still needed for endpoint recovery. Charging the stage to these two decreases lets us sum the bound over the recursive tree. The geometric and interval-coverage primitives were proved in Sections 3 and 5.

6.1 The algorithm and its supporting propositions

Use fixed calibration parameters β, δ, p_*, whose values are deferred to Section 6.3. Here $3 < \beta < 4$, $0 < \delta < 1/3$, and $0 < p_* < 1/2$. A line is mass-balanced if each side has mass at least $\delta w(S)$. Otherwise the common-witness construction supplies the normalized hull. Keep its full cyclic order, including the constant-negative origin marker when it is a hull vertex. Candidate blocks are its edges: each block uses only its one or two real supplied endpoint tests.

Write Π for the full-pattern partition and A_0 for its all-negative class, which is empty or a singleton. Throughout this section the prior weights retain their original normalization:

$$w = w(S), \qquad p_0 = w(A_0), \qquad b = w - p_0, \qquad I = I(S, \Pi), \qquad \Delta = \Delta_{\mathrm{E}}(S, \Pi).$$

The quantity I is the information needed to learn the pattern; Δ is the optimal cost made unnecessary after its class is known. These are comparison quantities used in the analysis; the algorithm chooses its schedule from observable pattern masses and coverage scores.

Choose the coverage schedule. Small budgets locate common positive patterns quickly; larger budgets eventually cover every positive pattern. Exact budgeted coverage chooses the blocks, while greedy ordering places their largest new masses first. We vary the first two budgets because these early decisions have the greatest effect on expected cost. Compute a maximum-weight block cover Q_k for each budget $k < k_{\min}$ and a minimum full cover Q_* of size $k_{\min}$. To justify these DPs, Lemma 3.2 gives each positive pattern one circular vertex interval. The hull edges incident to that interval form one circular interval in the edge order, so Lemmas 3.3 and 5.1 apply to the block demands. For a geometric shift u, set $k_j = \lfloor \beta^j u \rfloor$. Try the following initial budget sequences, followed by the common geometric tail:

$$\begin{aligned} &q, r, k_2, k_3, \ldots, \quad 1 \le q \le 6, \quad r \in \{k_1 - 1, k_1, k_1 + 1\}, \quad q < r < k_2, \\ &q, k_2, k_3, \ldots, \quad 1 \le q \le 6. \end{aligned}$$

At each budget $k < k_{\min}$, greedily append productive unused blocks of Q_k, ordering them by newly covered weight. At the first budget reaching $k_{\min}$, use Q_* and finish. There are at most twenty-four variants per shift.

Refine the usual geometric shift partition by the constant set

$$\frac{k}{\beta},\ \frac{k}{\beta^2} \in [1, \beta), \qquad 1 \le k \le \lceil \beta^3 \rceil. \tag{4}$$

Represent each open cell by any point in it, and also include each left breakpoint. Thus initial budgets remain represented even when $k_{\min}$ truncates the ordinary geometric partition. In particular, $r = k_1 - 1$ can first complete the cover at $k_1 = k_{\min} + 1$.

Choose the two-endpoint mode. Finding a positive block leaves a choice: stop at its first positive endpoint, or ask the other endpoint as well. The extra answer can save later decoding cost, so comparing only the number of discovery queries would miss its benefit. The following score includes both the immediate query saving and a guaranteed information credit. On an all-negative schedule prefix, let $x \le y$ be the newly hit masses exclusive to the two endpoints of the next block, and let z be their shared mass. Put $h_* = 1 - 2p_*$ and define the rational entropy credit and saving

$$\begin{aligned} E_*(a, z) &= \min\{(h_*/p_*)a,\ h_*(a + z),\ (h_*/p_*)z\}, \\ D(x, y, z) &= \max\{y,\ x + y - z + E_*(y, z)\}. \end{aligned} \tag{5}$$

The first choice stops at the first positive answer, asking the smaller exclusive mass first. The second choice asks both endpoints, asking the larger exclusive mass first. The latter order gives at least as much credited information because each of the three affine terms defining $E_*(a, z)$ is nondecreasing in a, and so is their pointwise minimum. Choose the endpoint with the larger saving. The exact binary-entropy bound behind E_* is checked only in the final calibration.

For a candidate schedule G, let k_A be its first block hitting positive class A, and set

$$F_*(G) = 2 \sum_{A \ne A_0} w(A) k_A + 2p_0|G| - \sum_{\text{blocks}} D. \tag{6}$$

This one score prices positive discovery, the entire zero-pattern waiting route, and the mode savings. Choose its minimum. At each positive exit, build the legal expected-cost endpoint decoder and attach the recursive tree for its full pattern class.

Algorithm 8: Rational choice between stopping and querying both endpoints

Input: New exclusive masses x, y, shared mass z; calibrated p_* and $h_* = 1 - 2p_*$.
Output: Stored endpoint order, mode, and rational saving D.

Label the endpoints so $x \le y$
$E \leftarrow \min\{(h_*/p_*)y,\ h_*(y+z),\ (h_*/p_*)z\}$
if $y \ge x + y - z + E$ **then**
    Record stop mode with the x endpoint first; $D \leftarrow y$
else
    Record query-both mode with the y endpoint first; $D \leftarrow x + y - z + E$
return the recorded order, mode, and D

These new masses are computed on the all-negative prefix before the block. In the stored discovery stage, stop mode ends at the first positive answer; query-both mode learns both bits before endpoint recovery. Their union and first-cover rank are the same.

Algorithm 9: ExpectedTree: the polynomial expected-depth algorithm

Input: State S, original positive rational priors; calibrated β, δ, p_*.
Output: A supplied-query identification tree.

if $|S| \le 1$ **then**
    return its leaf
if *a mass-balanced supplied line ℓ exists* **then**
    return ℓ with children ExpectedTree(S_ℓ^-) and ExpectedTree(S_ℓ^+)
Compute HullData(S), its pattern classes, and zero weight $p_0 = w(A_0)$
if *there are at most two real extreme hull tests* **then**
    Build their direct full-pattern stage
else
    Use edges of the full hull as blocks, retaining its free origin marker
    Select the schedule and stored modes by Algorithm 10
    Build discovery along its all-negative prefix, with those modes on each first hit
    At each positive exit build expected EndpointCode to recover the full pattern
Attach ExpectedTree(A) to each nonzero pattern class A
Use a zero-query leaf for the zero singleton; omit it when absent
return the tree after contracting constant and cached questions

The cases in the algorithm have a direct purpose: singleton states already identify their hypothesis, balanced lines split immediately, and flat states use the hull schedule to discover a full pattern before recursing within its class. Each recursive call reuses the fixed calibration and full supplied-query family. Since the incidence table and prior weights are known, schedule scores, endpoint modes, and child codes can all be determined before the hidden hypothesis is queried; the routine therefore returns a concrete tree for it to follow.

Algorithm 10: Two-budget schedule selection for ExpectedTree

Input: Positive-pattern block demands, original weights, zero weight p_0; calibrated β, p_*.
Output: A schedule and stored modes minimizing the rational score F_*.

Compute Q_* of size $k_{\min}$ and every Q_k for $k < k_{\min}$
Refine the geometric shift cells by (4)
foreach *breakpoint or one interior representative* $u \in [1, \beta)$ **do**
    $k_1 \leftarrow \lfloor \beta u \rfloor$; $k_2 \leftarrow \lfloor \beta^2 u \rfloor$
    for $q = 1, \ldots, 6$ **do**
        $\mathcal{V} \leftarrow \{(q, 0)\} \cup \{(q, r) : r \in \{k_1 - 1, k_1, k_1 + 1\},\ q < r < k_2\}$
        foreach $(q, r) \in \mathcal{V}$ **do**
            $G \leftarrow$ empty order
            Process q, then r if $r > 0$, then $k_2, k_3, \ldots$
            At each budget k, greedily append productive unused blocks from Q_k if $k < k_{\min}$, and from Q_* otherwise
            Stop these phases at the first budget reaching $k_{\min}$
            Compute the new masses and stored mode of each block by Algorithm 8
            Retain (G, modes) with its rational score $F_*(G)$
return a retained pair of minimum score

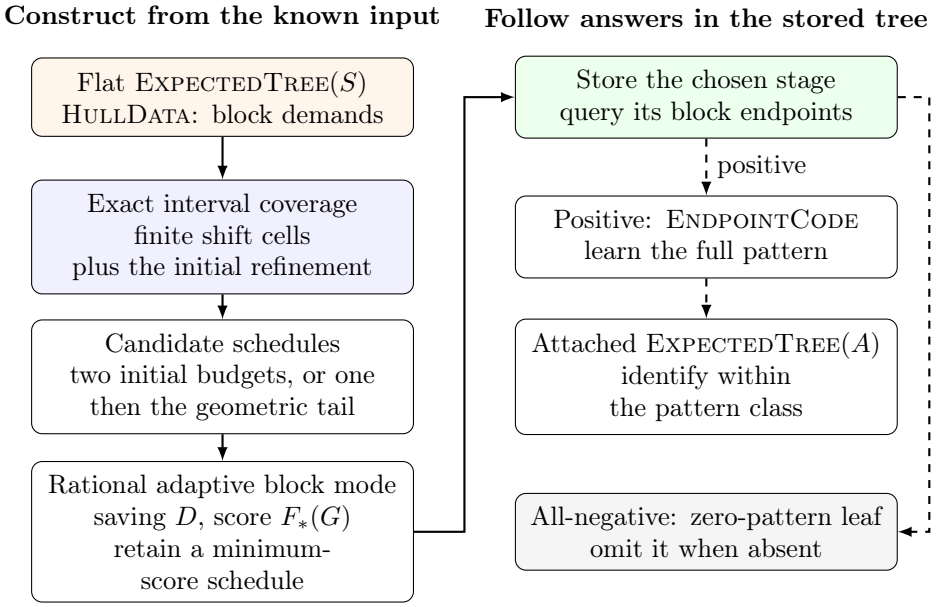


Figure 4: The polynomial expected-depth algorithm first computes hull patterns and their block demands, then uses interval coverage to build candidate schedules and selects one by its rational score. Solid arrows show this construction-time data flow, using the supplied incidence table and prior. The chosen stage is then stored as a query tree: dashed arrows trace the unknown hypothesis through discovery, endpoint decoding, and its already constructed recursive child. The adaptive mode decides whether discovery stops at the first positive endpoint or queries both endpoints. The algorithm makes this schedule choice deterministically; the certificate's mixture is used to prove its cost bound.

The analysis follows the same separation between discovery and pattern recovery. First we extract a benchmark from an optimal tree. Then we bound the cost of a stage with a given schedule, compare the chosen schedule with that benchmark, and sum stage bounds over the recursion. The

next four propositions state these steps; their proofs follow the theorem. Recall the binary entropy h from Definition 2.7, and define

$$\sigma(\delta) = -k_\delta \delta \ln \delta - r_\delta \ln r_\delta, \quad k_\delta = \lfloor 1/\delta \rfloor, \quad r_\delta = 1 - k_\delta \delta,$$

with $0 \ln 0 = 0$. The latter is the minimum entropy of a fully capped mass distribution, including its possible remainder.

For analysis, follow the common witness through an optimal tree. Each pattern stays on that path until a supporting hull test hits it; once the pattern is known, its earlier queries can be omitted. The first proposition quantifies this restriction credit and translates the witness path into adjacent hull blocks for comparison with our schedule.

Proposition 6.1 (Witness-path restriction credit)**.** *Follow the common witness through an optimal tree, along a path of length ℓ. For each pattern class A, let t_A be the first path query whose hull supports meet its positive pattern; put $t_A = \ell + 1$ if none do. Then*

$$\Delta \geq \sum_{A \neq A_0} w(A)(t_A - 1) + p_0 \ell. \tag{7}$$

The corresponding positive witness masses $m_t = \sum_{A \neq A_0 : t_A = t} w(A)$ sum to b and satisfy $m_t \leq 2\delta w$. An edge comparison ordering hits every positive class by rank t_A. If A_0 exists, it covers all positive classes with at most ℓ blocks, so $k_{\min} \leq \ell$.

Next we price an actual stage. The rational score accounts for discovery and endpoint-mode savings, while the information term accounts for the remaining decoding. We deliberately retain only a coarsened transcript: every positive cell then lies inside one small cap, which gives a uniform entropy lower bound.

Write $W_{\mathrm{pat}}(S)$ for the original-prior expected number of queries used by this stage to discover the full pattern and decode its endpoints. It excludes the recursive child trees and includes the all-negative route when the zero-pattern class is present.

Proposition 6.2 (Pattern-stage cost and discovery information)**.** *For every candidate schedule and its selected modes, the full-pattern stage uses only supplied queries and satisfies*

$$W_{\mathrm{pat}}(S) \leq F_*(G) + 3b + \frac{I - J_*}{\ln 2}, \qquad 0 \leq J_* \leq I, \tag{8}$$

where J_* is the information of a coarsened discovery transcript. Every positive coarsened exit is contained in one hull cap, and hence

$$J_* \geq b\sigma(\delta). \tag{9}$$

The zero transcript is the zero-pattern singleton when it exists.

Exact coverage connects the optimal witness ranks to the actual schedule. Proposition 5.3 bounds the average joint score of shifted schedules, and Proposition 5.4 shows that the finite search returns one whose score is no larger than that average. The next proposition converts this schedule comparison into a local stage bound; its coefficient B is calibrated after the proof.

Proposition 6.3 (Polynomial schedule and the local bound)**.** *With the final calibration, the two-budget schedule algorithm is polynomial in the input and rational bit length. At every flat state, including a degenerate hull, its full-pattern stage satisfies*

$$W_{\mathrm{pat}}(S) \leq A\Delta + Bb + \frac{I - J_*}{\ln 2}, \qquad A = \frac{\beta + 1}{\ln \beta}, \tag{10}$$

where B is the positive-mass coefficient certified in Section 6.3.

The local bound still contains positive mass and residual information. The capped-transcript entropy absorbs the mass term, leaving a charge to restriction and information decreases alone. Those decreases telescope over the recursive pattern classes.

Proposition 6.4 (Stage-to-tree conversion)**.** *Suppose flat stages satisfy* $W \le A\Delta + Bb + (I-J)/\ln 2$ *and* $0 \le J \le I$, $J \ge b\sigma(\delta)$, *with nonnegative* A, B. *A recursion using those stages and mass-balanced queries has*

$$c_{\mathrm{E}}(T(S)) \le A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S), \qquad c = \max\{1/h(\delta), B/\sigma(\delta)\}.$$

Consequently its ratio is at most $A + c\ln 2$.

Proof strategy.

Expected-depth proof roadmap.

1. Replace an optimal witness query by its adjacent hull supports. Restriction pays for the resulting first-hit ranks.
2. Choose whether the first positive block stops early or learns both endpoint bits. Its recorded information reduces the endpoint-decoding cost.
3. Exact coverage transfers witness prefix mass to actual batches. A fixed certificate pays for changed initial ranks and for the complete zero waiting route.
4. Bound every stage by its restriction and entropy decreases, then telescope. Numerical tuning is deferred until after these proofs.

These propositions reduce the overall guarantee to a local schedule bound and a final choice of constants. The theorem's proof only combines those ingredients; the geometric, information, and schedule arguments are established separately below.

Theorem 6.5 (Polynomial expected-depth approximation)**.** *For a separating supplied family of binary unit-cost planar line queries and arbitrary strictly positive rational priors,* ExpectedTree *runs in deterministic polynomial time and returns*

$$c_{\mathrm{E}}(T(S)) \le A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S), \qquad \boxed{A + c\ln 2 < 5.182976.}$$

Here $c = \max\{1/h(\delta), B/\sigma(\delta)\}$. *Its running-time exponent is independent of the smallest prior.*

Proof. Proposition 6.3 gives the local stage bound; Proposition 6.2 gives its discovery-information lower bound. Proposition 6.4 sums them over the recursion. The calibration in Section 6.3 gives the stated factor. Each nonsingleton stage partitions its state into proper disjoint classes, so the polynomial stage constructions form at most $n-1$ recursive stages. □

6.2 Proofs of the supporting propositions

Proof of Proposition 6.1. We first show why an unhit pattern shares the witness's optimal prefix. This yields restriction credit directly. We then enlarge each query's supports to an adjacent hull block, obtaining the coverage benchmark needed by the actual scheduling algorithm.

Claim 6.1 (An unhit support forces an optimal-prefix query to be constant)**.** *Before the first supporting hit* t_A, *every member of* A *follows the common witness through its first* $t_A - 1$ *queries.*

Proof. All supporting hull bits of each earlier query are negative on A. The cap-cover implication of Lemma 3.2 therefore forces that supplied query to have the witness's answer throughout A. Induct along the path. Restricting the optimal tree to A contracts these constant queries, saving at least

$w(A)(t_A - 1)$. To see the weighted comparison explicitly, let T° be the optimal tree and let π_h be the original weight of hypothesis h. For every positive class A,

$$\begin{aligned}\mathrm{OPT}_{\mathrm{E}}(A) \le c_{\mathrm{E}}(T^\circ|_A) &= \sum_{h\in A} \pi_h d_{T^\circ|_A}(h) \\ &\le \sum_{h\in A} \pi_h d_{T^\circ}(h) - w(A)(t_A - 1).\end{aligned}$$

The first inequality uses feasibility of the restricted tree. The second uses the removal of at least $t_A - 1$ common queries from every surviving path; summing their original weights gives the credit $w(A)(t_A-1)$. A class unhit on the entire path reaches its leaf z, so it is $\{z\}$. When A_0 exists, this is exactly A_0 and its credit is $p_0\ell$; its restricted tree has cost zero. Summing the displayed inequality over all positive classes and including this singleton gives

$$\begin{aligned}\sum_{A\in\Pi} \mathrm{OPT}_{\mathrm{E}}(A) &\le \sum_{h\in S} \pi_h d_{T^\circ}(h) - \sum_{A\neq A_0} w(A)(t_A - 1) - p_0\ell \\ &= \mathrm{OPT}_{\mathrm{E}}(S) - \sum_{A\neq A_0} w(A)(t_A - 1) - p_0\ell.\end{aligned}$$

The classes partition S, so their original depth contributions sum to the full tree's contribution; its optimality then gives the final equality. Subtracting the left side from $\mathrm{OPT}_{\mathrm{E}}(S)$ proves (7). When A_0 is empty, its term is zero and the same sum applies. In particular, $\sum_{A\neq A_0} w(A)(t_A - 1) = \sum_t m_t(t-1)$. □

The prefix credit comes from supplied queries on an optimal-tree path. Replace each query by a full-hull edge containing its supports. This enlargement preserves or improves its first-hit rank, so the prefix gives a feasible comparison schedule on adjacent hull blocks.

Claim 6.2 (Adjacent comparison blocks cover witness prefixes)**.** *Replace each witness query by a full-hull edge containing its one or two real supports. A singleton support may use either incident edge. This comparison hits every class by its index t_A.*

Proof. By Lemma 3.2, the one or two real supports fit in one hull edge. Enlarging the support set to that edge cannot delay a hit. Removing repeated or unproductive edges improves ranks further. If the terminal singleton has a nonzero pattern and remains unhit, append an edge incident to one of its positive vertices; its rank is at most $\ell + 1 = t_A$. If A_0 exists, all positive classes were already hit within the ℓ witness blocks, by Claim 6.1. Each ordinary first-hit mass lies in the union of the supports of that path query. Writing these caps as $C_v, C_{v'}$, and allowing one to be absent,

$$m_t \le w((C_v \cup C_{v'}) \cap S) \le w(C_v \cap S) + w(C_{v'} \cap S) \le 2\delta w.$$

The first inequality puts each rank-t class in at least one support cap. The second is the union bound, and the last uses mass-flatness, since both hull vertices are supplied tests. An appended terminal singleton lies in one such cap, so its mass is at most δw. The classes partition $S \setminus A_0$, hence $\sum_t m_t = b$. If A_0 exists, every positive class leaves its optimal path before the zero leaf; the at-most-ℓ comparison blocks therefore give a full cover and $k_{\min} \le \ell$. This proves all remaining statements of the proposition. The algorithm constructs its own order; we use the witness order only to compare costs. □

Proof of Proposition 6.2. There are three parts to the stage accounting. Count the physical queries in each endpoint mode, credit the additional information without counting it twice, and lower-bound the information retained in capped discovery exits. Throughout, first-hit ranks refer to the actual candidate schedule.

Claim 6.3 (Two endpoint modes have explicit physical costs)**.** *A positive class first hit by block k has nonentropy allowance $2k + 3$. Stop mode saves y weighted queries; query-both mode saves $x + y - z$ before accounting for its additional information.*

Proof. At most $2(k - 1)$ tests precede the hitting block. In stop mode the first-positive route uses one discovery query and the two legal endpoint codes of Lemmas 3.5 and 2.10, with overhead four. The second-positive route uses two discovery queries, but its adjacent negative answer pins one boundary, leaving only overhead two. Relative to $2k + 3$, exactly the second endpoint's exclusive mass y saves one query. In query-both mode each exclusive route uses two discovery queries and one endpoint code, for allowance $2k + 2$; the shared route uses two queries and two codes, for allowance $2k + 4$. Their weighted saving is $x + y - z$. A free origin endpoint or a cached answer can only lower these costs. Finish the recorded mode before singleton inference in the analysis, then contract constant questions in the constructed tree. Here are the allowances before the conditional entropy term is added; each endpoint code contributes overhead at most two:

mode and exit	discovery	codes	allowance
stop: first positive	1	2	$2(k-1)+1+4 = 2k+3$
stop: second positive	2	1	$2(k-1)+2+2 = 2k+2$
both: exclusive	2	1	$2(k-1)+2+2 = 2k+2$
both: shared	2	2	$2(k-1)+2+4 = 2k+4$

For $m = x + y + z$, the stop allowance is consequently

$$(x + z)(2k + 3) + y(2k + 2) = (2k + 3)m - y,$$

whereas the query-both allowance is

$$(x + y)(2k + 2) + z(2k + 4) = (2k + 3)m - (x + y - z).$$

The zero route uses at most two tests per block, contributing $2p_0|G|$. Summing the common positive allowance gives $\sum_{A\neq A_0} w(A)(2k_A + 3) = 2\sum_{A\neq A_0} w(A)k_A + 3b$. □

The second endpoint can reduce the information the decoder must learn, so the stage measure accounts for both query count and information. Temporarily forgetting that answer creates a coarsened transcript whose entropy loss measures the benefit exactly and whose positive cells remain inside individual caps.

Claim 6.4 (Coarsening accounts for the additional information once)**.** *In a query-both block, forget the second answer on routes whose first answer was positive. The resulting positive events lie in individual caps, and the forgotten information pays for $E_*(a, z)$, where a is the first endpoint's exclusive mass.*

Proof strategy.

1. Compute the entropy loss from merging the two events and use the calibrated entropy tent to lower-bound it by $E_*(a, z)$.
2. Express endpoint-code entropy as the information left after discovery, then compare the actual and coarsened transcripts.
3. Combine that information difference with the physical mode savings, so each block's saving is charged once in (8).

Proof. The coarsening merges masses a, z inside the first endpoint's cap; the other exclusive event lies in the second endpoint's cap. Using the same total state mass w before and after the merge, the weighted information loss is

$$\begin{aligned}\mathcal{L}(a, z) &= -a \ln(a/w) - z \ln(z/w) + (a + z) \ln((a + z)/w) \\ &= a \ln \frac{a + z}{a} + z \ln \frac{a + z}{z} = (a + z) h\left(\frac{a}{a + z}\right).\end{aligned}$$

The first equality cancels the common normalization terms in $\ln w$. For the second, substitute the two conditional probabilities $a/(a + z)$ and $z/(a + z)$ into binary entropy. Terms of zero mass have value zero, and $\mathcal{L}(0, 0) = 0$. To justify the entropy tent, first take $0 \le v \le p_*$. Concavity between 0 and p_* gives $H_2(v) \ge (v/p_*)H_2(p_*) \ge (h_*/p_*)v$. On $[p_*, 1 - p_*]$, concavity between the two equal endpoint entropies gives $H_2(v) \ge H_2(p_*) \ge h_*$. On $[1 - p_*, 1]$, symmetry gives $H_2(v) \ge (h_*/p_*)(1 - v)$. These three inequalities prove

$$H_2(v) \ge \min\{(h_*/p_*)v,\ h_*,\ (h_*/p_*)(1 - v)\},$$

provided the calibration satisfies $H_2(p_*) \ge h_*$. For $v = a/(a + z)$, multiplying by $a + z$ therefore gives

$$\frac{\mathcal{L}(a, z)}{\ln 2} = (a + z) H_2\left(\frac{a}{a + z}\right) \ge \min\{(h_*/p_*)a, h_*(a + z), (h_*/p_*)z\} = E_*(a, z).$$

Let $\mathcal{D}_{\text{act}}$ be the actual discovery partition and J_{act} its weighted information. On an exit B, learn the left endpoint U and then the right endpoint V conditioned on U and B. The endpoint pair and the full pattern determine each other. Thus Lemma 2.9 makes the entropy part of the expected code lengths supplied by Lemma 2.10 equal to

$$\frac{w(B)}{\ln 2}\big[H(U \mid B) + H(V \mid U, B)\big] = \frac{w(B)}{\ln 2} H(\Pi \mid B).$$

A pinned endpoint is constant and has entropy zero; its code and overhead are omitted. Summing over exits, including the zero exit whose pattern is already known, Lemma 2.9 gives

$$\sum_{B \in \mathcal{D}_{\text{act}}} w(B) H(\Pi \mid B) = wH(\Pi) - wH(\mathcal{D}_{\text{act}}) = I - J_{\text{act}}.$$

Here the conditional entropies use the normalized weights within each exit, while the prefactors preserve the original weights. The discovery label is determined by the full pattern, so its entropy is exactly the information already removed from the remaining pattern entropy. The final equality uses the definitions of I and J_{act}.

Let $\mathcal{Q}$ be the blocks using query-both mode, with physical weighted savings $P_g = x_g + y_g - z_g$; for stop blocks put $P_g = y_g$. Claim 6.3 and this conditional-entropy sum give

$$W_{\mathrm{pat}}(S) \le 2 \sum_{A \neq A_0} w(A) k_A + 2p_0|G| + 3b - \sum_g P_g + \frac{I - J_{\mathrm{act}}}{\ln 2}.$$

Write $\mathcal{D}_*$ for the resulting coarsened partition. The merged exits are disjoint across blocks, so $J_{\mathrm{act}} - J_* = \sum_{g \in \mathcal{Q}} \mathcal{L}(a_g, z_g)$. Indeed, each merge changes only its two positive exit cells, so the separate entropy losses add. Substituting this identity gives

$$\begin{aligned} -\sum_g P_g + \frac{I - J_{\mathrm{act}}}{\ln 2} &= \frac{I - J_*}{\ln 2} - \sum_{g \notin \mathcal{Q}} y_g \\ &\quad - \sum_{g \in \mathcal{Q}} \left(x_g + y_g - z_g + \frac{\mathcal{L}(a_g, z_g)}{\ln 2} \right) \\ &\le \frac{I - J_*}{\ln 2} - \sum_g D(x_g, y_g, z_g). \end{aligned}$$

The last inequality uses the chosen mode and $\mathcal{L}(a_g, z_g)/\ln 2 \ge E_*(a_g, z_g)$, with $a_g = y_g$. Thus the lost information is subtracted exactly once. Because each of the three linear forms in $E_*(a, z)$ is nondecreasing in a, asking the larger exclusive mass first maximizes this credit. In stop mode, ask the smaller exclusive mass first. The displayed cost inequality is now precisely (8), after inserting the definition of $F_*(G)$. □

We now need a lower bound on the retained discovery information that works even when the zero-pattern singleton has large mass. All positive exits are capped; the zero exit is the sole unrestricted one. Concentrating the capped masses as much as possible identifies the least informative such transcript.

Claim 6.5 (Capped discovery exits have entropy proportional to positive mass)**.** *A probability vector with one unrestricted entry p_0 and all remaining entries at most δ has entropy at least $(1 - p_0)\sigma(\delta)$.*

Proof strategy.

1. Use entropy-decreasing mass transfers to reduce a minimizer to entries packed at the cap, with at most one remainder.
2. On each interval with a fixed number of capped entries, show the packed entropy minus $b\sigma(\delta)$ is concave.
3. Check the interval endpoints using the packed-mass ratios; the chord bound then gives the claim throughout each interval.

Proof. Put $f(a) = -a \ln a$ with $f(0) = 0$, and let $b = 1 - p_0$. Moving mass from a smaller capped entry to a larger one cannot increase entropy: if $a \ge c > 0$ and $0 \le \epsilon \le \min\{\delta - a, c\}$, then

$$\frac{d}{d\epsilon}\big[f(a + \epsilon) + f(c - \epsilon)\big] = \ln \frac{c - \epsilon}{a + \epsilon} \le 0.$$

Transfer until one entry is zero or the other reaches its cap. Repeating leaves at most one positive entry below δ, giving the minimum shape: $j = \lfloor b/\delta \rfloor$ entries of mass δ and one possible remainder. Its total entropy is

$$T(b) = f(1 - b) + jf(\delta) + f(b - j\delta).$$

On each interval with fixed j, the only varying terms are $f(1-b)+f(b-j\delta)$, and

$$T''(b) = -\frac{1}{1-b} - \frac{1}{b-j\delta} < 0$$

in its interior. Thus $T(b) - b\sigma(\delta)$ is concave there. It suffices to check $T(b) \geq b\sigma(\delta)$ at those multiples and at $0, 1$. At $b = j\delta > 0$,

$$T(b)/b = L + g(1-b), \quad L = \ln(1/\delta), \quad g(a) = \frac{-a\ln a}{1-a}.$$

Differentiating the quotient gives

$$g'(a) = \frac{(-\ln a - 1)(1-a) - a\ln a}{(1-a)^2} = \frac{-\ln a - 1 + a}{(1-a)^2} \geq 0,$$

where $\ln a \leq a - 1$ proves the sign. These ratios decrease as b increases. If the final remainder $r_\delta = 1 - \lfloor 1/\delta \rfloor \delta$ is zero, the last ratio is $L = \sigma(\delta)$. Otherwise its excess over $T(1) = \sigma(\delta)$ is, writing $r = r_\delta$,

$$\left[L + \frac{f(r)}{1-r}\right] - \big[(1-r)L + f(r)\big] = rL + \frac{rf(r)}{1-r} \geq 0.$$

The last sign uses $r, L, f(r) \geq 0$ and $r < 1$. Consequently every multiple has nonnegative excess, while $T(0) = 0$ and $T(1) = \sigma(\delta)$. A concave function lies above the chord between its endpoints, so the required inequality holds on every intervening interval. Also $\sigma(\delta) = \ln(1/\delta) + r_\delta \ln(\delta/r_\delta) \geq \ln(1/\delta)$, because $0 < r_\delta < \delta$ when the remainder is positive; the second term is zero at $r_\delta = 0$. □

By Claim 6.4, every nonzero coarsened discovery cell lies in a cap of mass below δw. Its normalized zero mass is p_0/w, so the claim gives

$$J_* = wH(\mathcal{D}_*) \overset{\text{Claim 6.5}}{\geq} w\left(1 - \frac{p_0}{w}\right)\sigma(\delta) = b\sigma(\delta).$$

This is (9). The transcript is a function of the full pattern, so Lemma 2.9 gives $I = J_* + wH(\Pi \mid \mathcal{D}_*) \geq J_*$; entropy nonnegativity also gives $J_* \geq 0$. Endpoint recovery identifies the complete pattern, while a negative full cover identifies A_0. This finishes Proposition 6.2.

Proof of Proposition 6.3. The next claims separate actual coverage and mode savings from their optimal-witness comparison. Actual first-hit ranks price the constructed schedule; witness ranks provide restriction credit. Exact coverage is the connection between them.

The exact mode saving depends on how a block's mass is split between its endpoints. For a batch comparison we need bounds depending only on its total newly hit mass and the cap size. Two affine minorants of the saving provide that simpler form.

Claim 6.6 (Block mode savings have two affine lower bounds)**.** *For newly hit mass $m = x + y + z$, with endpoint caps bounded by δw, the selected saving satisfies*

$$D \geq 0, \qquad D \geq m - \delta w, \qquad D \geq c_* m - d_* \delta w, \quad c_* = \frac{2 - p_*}{1 - p_*}, \quad d_* = \frac{2}{1 - p_*}.$$

Proof. Stop mode gives $D \geq y \geq 0$ and $m - y = x + z \leq \delta w$. These are the first two bounds, by the definition (5) and the first endpoint's cap bound. For the last inequality normalize $\delta w = 1$; the saving is homogeneous because every term in E_* and D is linear before taking minima or maxima.

It is immediate if $m \le 1$, because $c_* m - d_* \le c_* - d_* = -p_*/(1-p_*) < 0$. The two cap bounds imply $m + z = (x+z) + (y+z) \le 2$, so no mass $m > 2$ is feasible. For $1 \le m \le 2$, substitute $z = m - x - y$. The cap inequalities and the chosen endpoint order become

$$x \ge m-1, \qquad y \ge m-1, \qquad x \le y, \qquad x+y \le m.$$

Specifically, $y + z \le 1$ gives $x \ge m-1$, $x + z \le 1$ gives $y \ge m-1$, and $z \ge 0$ gives $x + y \le m$. Thus the feasible (x, y) triangle has vertices $(m-1, m-1)$, $(m-1, 1)$, and $(m/2, m/2)$. The concave expression $x + y - z + E_*(x, z)$ is a lower bound on the selected query-both saving, since $E_*(y, z) \ge E_*(x, z)$. Concavity follows because, after substituting z, it is a linear term plus the minimum of three affine functions. The selected saving D dominates it by (5). At the two vertices with $z = 0$ it equals m; at the other it is

$$f(m) = 3m - 4 + \min\{(h_*/p_*)(m-1),\ h_*,\ (h_*/p_*)(2-m)\}.$$

For $m \le 1 + p_*$, $c_* m - d_* \le m - 1$, already covered by stop mode. Indeed,

$$c_* m - d_* - (m-1) = \frac{m - 1 - p_*}{1 - p_*} \le 0.$$

On $[1 + p_*, 2]$, the minimum of the three affine terms makes f concave. Since $h_* = 1 - 2p_*$ and $p_* < 1/2$,

$$f(1+p_*) = -1 + 3p_* + \min\{h_*, h_*, h_*(1-p_*)/p_*\} = p_*,$$
$$f(2) = 2 + \min\{h_*/p_*, h_*, 0\} = 2.$$

The chord between these endpoints is

$$p_* + \frac{2-p_*}{1-p_*}(m - 1 - p_*) = \frac{(2-p_*)m - 2}{1-p_*} = c_* m - d_*.$$

At the other two triangle vertices the value m also dominates this chord, because $m - (c_* m - d_*) = (2-m)/(1-p_*) \ge 0$. For fixed m, every feasible point is a convex combination of the triangle vertices. Concavity therefore bounds its query-both saving below by the same chord. Restoring the scale δw multiplies both m and the saving by that scale, proving the last affine bound. □

The witness's first q blocks cover a known amount of comparison mass. Exact budgeted coverage must cover at least that amount, although it may choose entirely different blocks. Summing either affine minorant over the chosen batch therefore transfers witness prefix mass into an actual mode-saving guarantee.

Claim 6.7 (Exact first-batch coverage transfers witness prefix mass)**.** *Let $M_q = \sum_{t \le q} m_t$. A first budget-q batch gives total schedule savings at least each of*

$$\mathsf{A}_q = M_q - \delta w q, \qquad \mathsf{C}_q = c_* M_q - d_* \delta w q, \qquad 0.$$

Proof. By Claim 6.2, the witness comparison covers M_q using at most q blocks. Thus Lemma 5.1 selects a set covering mass at least M_q. Let its productive blocks have newly hit masses $\widehat{m}_1, \ldots, \widehat{m}_h$, where $h \le q$ and $\sum_i \widehat{m}_i \ge M_q$. The two sums are explicitly

$$\sum_{i=1}^{h} D_i \overset{\text{Claim 6.6}}{\ge} \sum_{i=1}^{h} (\widehat{m}_i - \delta w) = \sum_i \widehat{m}_i - h\delta w \ge M_q - q\delta w,$$
$$\sum_{i=1}^{h} D_i \overset{\text{Claim 6.6}}{\ge} \sum_{i=1}^{h} (c_* \widehat{m}_i - d_* \delta w) = c_* \sum_i \widehat{m}_i - d_* h \delta w \ge c_* M_q - d_* q \delta w.$$

Here $c_*, d_* > 0$, so replacing the covered mass by its lower bound and the number of blocks by its upper bound has the stated signs. All subsequent mode savings are nonnegative by Claim 6.6, so both batch bounds also bound the complete schedule's saving. If $q \geq k_{\min}$, the minimum cover hits all positive mass with at most q blocks, so $\sum_i \widehat{m}_i = b \geq M_q$ and the same calculation holds. If coefficients $\theta_A, \theta_C, \theta_0 \geq 0$ sum to one, then $\theta_A \mathsf{A}_q + \theta_C \mathsf{C}_q + \theta_0 \cdot 0$ is also at most the one total saving, because each of its three terms is a lower bound on that same saving. □

Changing the first budgets can help early hits and hurt later ones. It can also change how long the zero hypothesis waits before a full cover finishes. We keep both changes with their signs; this prevents an improvement for common positive patterns from hiding an increased cost on the all-negative route.

Claim 6.8 (Changing the initial budgets has signed first-hit and zero-route costs)**.** *Relative to the geometric sequence $k_0, k_1, k_2, \ldots$, the exact doubled first-hit improvement is*

$$\begin{aligned} d_{q,r}(u,t) &= k_0 - q + (k_0 + k_1)\mathbf{1}_{\{t>k_0\}} + (k_1 + k_2)\mathbf{1}_{\{t>k_1\}} \\ &\quad - (q + r)\mathbf{1}_{\{t>q\}} - (r + k_2)\mathbf{1}_{\{t>r\}}, \quad r > 0, \\ d_{q,0}(u,t) &= k_0 - q + (k_0 + k_1)\mathbf{1}_{\{t>k_0\}} + (k_1 + k_2)\mathbf{1}_{\{t>k_1\}} \\ &\quad - (q + k_2)\mathbf{1}_{\{t>q\}}. \end{aligned} \tag{11}$$

The full all-negative length invoice changes by

$$z_{q,r}(u,k) = q\mathbf{1}_{\{q<k\}} + r\mathbf{1}_{\{r<k\}} - k_0\mathbf{1}_{\{k_0<k\}} - k_1\mathbf{1}_{\{k_1<k\}}, \qquad k = k_{\min}. \tag{12}$$

Proof. The within-batch bound of Claim 5.2, together with phase coverage from Claim 5.1, bounds first-hit cost for any increasing budgets $a_0, a_1, \ldots$ by

$$\frac{a_0 + 1}{2} + \frac{1}{2}\sum_{j \geq 0}(a_j + a_{j+1})\mathbf{1}_{\{t>a_j\}}.$$

Write $\mathcal{B}_{\text{old}}(u,t)$ and $\mathcal{B}_{q,r}(u,t)$ for these invoices. If $\mathcal{T}(u,t) = \sum_{j\geq 2}(k_j + k_{j+1})\mathbf{1}_{\{t>k_j\}}$, their doubled values are

$$\begin{aligned} 2\mathcal{B}_{\text{old}} &= k_0 + 1 + (k_0 + k_1)\mathbf{1}_{\{t>k_0\}} + (k_1 + k_2)\mathbf{1}_{\{t>k_1\}} + \mathcal{T}, \\ 2\mathcal{B}_{q,r} &= q + 1 + (q + r)\mathbf{1}_{\{t>q\}} + (r + k_2)\mathbf{1}_{\{t>r\}} + \mathcal{T} \quad (r > 0), \\ 2\mathcal{B}_{q,0} &= q + 1 + (q + k_2)\mathbf{1}_{\{t>q\}} + \mathcal{T}. \end{aligned}$$

Subtracting cancels the common 1 and $\mathcal{T}$, giving $2(\mathcal{B}_{\text{old}} - \mathcal{B}_{q,r}) = d_{q,r}(u,t)$. Every coefficient in $\mathcal{B}_{q,r}$ is nonnegative. Hence this invoice is nondecreasing in t, and comparison edges that hit earlier than t_A, as allowed by Claim 6.2, only improve the bound. Early completion removes nonnegative continuation terms. For the negative route and $k = k_{\min}$, the full budget invoices are

$$\begin{aligned} \mathcal{L}_{\text{old}}(u,k) &= k + k_0\mathbf{1}_{\{k_0<k\}} + k_1\mathbf{1}_{\{k_1<k\}} + \sum_{j\geq 2} k_j\mathbf{1}_{\{k_j<k\}}, \\ \mathcal{L}_{q,r}(u,k) &= k + q\mathbf{1}_{\{q<k\}} + r\mathbf{1}_{\{r<k\}} + \sum_{j\geq 2} k_j\mathbf{1}_{\{k_j<k\}}. \end{aligned}$$

The final minimum cover contributes the same k; a budget below k contributes at most its full size. Subtracting now cancels both the final cover and the common tail, so $\mathcal{L}_{q,r} - \mathcal{L}_{\text{old}} = z_{q,r}(u,k)$. The r term is zero for $r = 0$. If an initial budget already reaches k, all later budgets are larger and their indicators vanish, so this formula also includes early minimum-cover completion. Neither change is assumed nonnegative. □

After averaging schedules, each witness rank will have a credit coefficient. Most coefficients meet a common threshold, and the cap bound limits the mass at any exceptional rank. The next calculation combines these facts without assuming that all mass is positive: its final term explicitly retains the zero prior.

Claim 6.9 (One capped-mass calculation includes the zero prior)**.** *Suppose* $\sum_t m_t = b$, $m_t \le 2\delta w$, *and coefficients* a_t *are at least* τ *except on a finite set* E. *Define*

$$\chi = \tau - 2\delta \sum_{t\in E} (\tau - a_t)_+ - \delta\mu.$$

If $\chi + \zeta \ge \tau$, *then*

$$\sum_t m_t a_t - \delta w\mu + \zeta p_0 \ge \chi b. \tag{13}$$

Proof. Only coefficients below τ need their individual mass caps. Let $\varepsilon_t = (\tau - a_t)_+$ for $t \in E$, and put it equal to zero elsewhere. Then $a_t \ge \tau - \varepsilon_t$, so

$$\begin{aligned}\sum_t m_t a_t &\ge \tau \sum_t m_t - \sum_{t\in E} m_t\varepsilon_t \\ &\ge \tau b - 2\delta w \sum_{t\in E} \varepsilon_t.\end{aligned}$$

Subtract $\delta w\mu$ and add ζp_0. By the definition of χ, $2\delta\sum_{t\in E}\varepsilon_t + \delta\mu = \tau - \chi$, so the result is at least

$$\tau b - (\tau - \chi)w + \zeta p_0 = \chi b + p_0(\chi + \zeta - \tau) \ge \chi b.$$

The equality uses $w = b + p_0$; the final sign uses $p_0 \ge 0$ and $\chi + \zeta \ge \tau$. This argument permits arbitrary zero-pattern mass and uses no identification of witness ranks with actual discovery ranks. □

Assembling the local bound. We now combine three credits against the discovery score: the geometric schedule's discount relative to its linear restriction bound, the signed benefit of changing its initial budgets, and the first batch's endpoint-mode saving. The certificate records only their coefficients; it is not used to choose a schedule at runtime. We describe this combination symbolically here and defer every numerical check to Section 6.3.

Retain $\Phi_\beta, Z_\beta, \lambda_\beta, \nu_\beta$ from the geometric-batch lemma. Define the nondecreasing discrete discount

$$e_\beta(t) = 2[\lambda_\beta(t-1) + \Phi_\beta(1) - \Phi_\beta(t)] = \frac{1}{\ln\beta}\int_1^t \left[\beta + 1 - \frac{\lfloor x\rfloor + \lfloor \beta x\rfloor}{x}\right] dx.$$

The integral identity follows from the definition of Φ_β and $2\lambda_\beta = (\beta+1)/\ln\beta$. Its integrand is nonnegative because each floor is at most its argument, so e_β is nondecreasing. Proposition 6.1 provides the comparison ranks. Log-uniform averaging of the old first-hit bound gives $\Phi_\beta(t)$ by Proposition 5.3; Claim 6.8 subtracts $d_{q,r}(u,t)/2$ from that invoice. The same claim makes the zero length bound at most $Z_\beta(k_{\min}) + \mathbb{E}[z_{q,r}(u, k_{\min})]$.

The first credit is $e_\beta(t)$ above. The other two credits depend on the selected initial budgets and minorant, so average them using the fixed distribution in Section A. This distribution compares only valid two-budget variants and first-batch minorants. Its coefficient definition is

$$a_t = e_\beta(t) + \mathbb{E}[d_{q,r}(u,t) + s\mathbf{1}_{\{t\le q\}}], \qquad \mu = \mathbb{E}[\rho], \tag{14}$$

where the selected minorant is $sM_q - \rho\delta w$. Claim 6.7 bounds the saving of each selected schedule by its selected minorant. Consequently

$$\begin{aligned}
&\sum_t m_t e_\beta(t) + \mathbb{E}\left[\sum_t m_t d_{q,r}(u,t) + \sum_{\text{blocks of } G} D\right] \\
&\quad \geq \sum_t m_t e_\beta(t) + \mathbb{E}\left[\sum_t m_t d_{q,r}(u,t) + s\sum_{t\leq q} m_t - \rho\delta w\right] \\
&\quad = \sum_t m_t \left[e_\beta(t) + \mathbb{E}\big(d_{q,r}(u,t) + s\mathbf{1}_{\{t\leq q\}}\big)\right] - \delta w \mathbb{E}[\rho] \\
&\quad = \sum_t m_t a_t - \delta w \mu.
\end{aligned}$$

The first inequality substitutes that minorant. The next equality uses $M_q = \sum_t m_t \mathbf{1}_{\{t\leq q\}}$ and moves the fixed witness masses through expectation; the last uses (14). The witness masses m_t are fixed independently of the shift. Thus the schedule's saving is credited once before taking expectations. It remains to check that this combined credit is sufficient at every rank and that the zero waiting route has not become longer on average. The final calibration proves three conditions: all a_t except a_1 exceed τ while $a_1 < \tau$; every mean zero-length change is nonpositive; and $\chi + 2/\ln\beta \geq \tau$, where

$$\chi = \tau - 2\delta(\tau - a_1) - \delta\mu, \qquad B = 3 + 2\Phi_\beta(1) - \chi. \tag{15}$$

It also gives $\lambda_\beta \geq \nu_\beta$. To make the score comparison explicit, write $S(G) = \sum_{\text{blocks of } G} D$ and $D_0(k) = \mathbb{E}[z_{q,r}(u,k)]$. The modified first-hit and length invoices give

$$\mathbb{E}\left[\sum_{A\neq A_0} w(A)k_A\right] \leq \sum_t m_t \Phi_\beta(t) - \frac{1}{2}\mathbb{E}\left[\sum_t m_t d_{q,r}(u,t)\right],$$
$$\mathbb{E}[|G|] \leq Z_\beta(k_{\min}) + D_0(k_{\min}).$$

The marginal shift law is log-uniform, so averaging the old invoice gives Φ_β and Z_β even though the conditional choice of initial budgets depends on the shift. Insert these inequalities in $F_*(G)$, use $D_0(k_{\min}) \leq 0$ and $p_0 \geq 0$, and expand $2\Phi_\beta(t) = 2\lambda_\beta(t-1) + 2\Phi_\beta(1) - e_\beta(t)$:

$$\begin{aligned}
\mathbb{E}[F_*(G)] &\leq 2\sum_t m_t \Phi_\beta(t) + 2p_0 Z_\beta(k_{\min}) - \mathbb{E}\left[\sum_t m_t d_{q,r}(u,t) + S(G)\right] \\
&= 2\lambda_\beta \sum_t m_t(t-1) + 2\Phi_\beta(1)b + 2p_0 Z_\beta(k_{\min}) \\
&\qquad - \sum_t m_t e_\beta(t) - \mathbb{E}\left[\sum_t m_t d_{q,r}(u,t) + S(G)\right] \\
&\overset{\text{Eq. 14}}{\leq} 2\lambda_\beta \sum_t m_t(t-1) + 2\Phi_\beta(1)b + 2p_0 Z_\beta(k_{\min}) - \sum_t m_t a_t + \delta w \mu.
\end{aligned}$$

If $p_0 > 0$, Proposition 6.1 gives $k_{\min} \leq \ell$. Since $\nu_\beta \leq \lambda_\beta$,

$$2p_0 Z_\beta(k_{\min}) \overset{\text{Prop. 5.3}}{\leq} 2p_0\left(\nu_\beta k_{\min} - \frac{1}{\ln\beta}\right) \leq 2\lambda_\beta p_0 \ell - \frac{2p_0}{\ln\beta}.$$

The second inequality uses $k_{\min} \le \ell$, $\nu_\beta \le \lambda_\beta$, and the nonnegative factors $p_0, \nu_\beta, \lambda_\beta$. When $p_0 = 0$ these terms vanish, so no relation between $k_{\min}$ and ℓ is needed. Finally,

$$2\lambda_\beta \left[\sum_t m_t(t-1) + p_0 \ell \right] \overset{\text{Prop. 6.1}}{\le} 2\lambda_\beta \Delta = A\Delta.$$

Substitution gives

$$\mathbb{E}[F_*(G)] \le A\Delta + 2\Phi_\beta(1) b - \sum_t m_t a_t + \delta w \mu - \frac{2p_0}{\ln \beta}.$$

The zero credit is retained in the last term of this inequality. The capped-mass claim with $E = \{1\}$ and $\zeta = 2/\ln \beta$ gives

$$\sum_t m_t a_t - \delta w \mu + \frac{2p_0}{\ln \beta} \overset{\text{Claim 6.9}}{\ge} \chi b, \qquad \mathbb{E}[F_*(G)] \le A\Delta + (2\Phi_\beta(1) - \chi) b.$$

For the selected schedule $\widehat{G}$, minimality gives $F_*(\widehat{G}) \le \mathbb{E}[F_*(G)]$. The expectation averages scores of valid candidates, so their minimum is no greater than that average. Applying the pattern-stage proposition to its own transcript,

$$\begin{aligned} W_{\text{pat}}(S) &\overset{\text{Prop. 6.2}}{\le} F_*(\widehat{G}) + 3b + \frac{I - J_*}{\ln 2} \\ &\le A\Delta + \underbrace{(3 + 2\Phi_\beta(1) - \chi)}_{B} b + \frac{I - J_*}{\ln 2}, \end{aligned}$$

which proves (10).

Finally, the averaging argument must describe schedules the algorithm actually tries. Shift cells give a finite family of representatives, including when a small initial budget completes the cover early. Hulls with too few tests for the block argument are handled directly and satisfy the same local bound.

Claim 6.10 (Degenerate hulls and finite shift enumeration preserve the bound)**.** *The direct stage and the finite shift family are polynomial, including all terminal cases.*

Proof. With at most two real extreme hull tests, query them directly to learn the full pattern, at cost at most $2w$. In a nonsingleton state with $p_0 > 0$, the witness path has $\ell \ge 1$, so Proposition 6.1 gives $\Delta \ge p_0$; otherwise $b = w$. The calibrated $A, B > 2$ imply

$$2w = 2b + 2p_0 \le Bb + Ap_0 \le Bb + A\Delta,$$

where $\Delta \ge 0 = p_0$ if the zero class is absent. Here $J_* = I$, so the residual-information term is zero.

For a nondegenerate hull, all coverage calls are polynomial interval DPs by Lemmas 3.3 and 5.1. Proposition 5.4 gives polynomially many geometric shift cells; (4) adds only a constant number. More explicitly, for any fixed $\beta > 1$, there are $O(\log k_{\min})$ geometric scales, and for each integer k at most one exponent places k/β^j in $[1, \beta)$. The initial refinement adds at most $2\lceil \beta^3 \rceil$ breakpoints, a constant for fixed β. The algorithm enumerates each breakpoint and one interior representative per cell; all such representatives give the same integer budgets within the cell. Within a refined cell every executed budget, candidate order, mode, and score is fixed. Thus every schedule in the proof average has an actual enumerated representative. There are at most twenty-four variants per cell. Prior weights enter only sums and comparisons: for rational input priors, a sum of at most

n input values has a denominator dividing the product of their denominators and therefore has polynomial encoding length. The fixed rational coefficients in E_*, D, and F_* preserve that bound. No logarithm or proof distribution is computed. Empty positive demands occur only at a terminal singleton. Early inferences, cached questions, free origin markers, and empty branches can only reduce the constructed cost. □

This completes Proposition 6.3.

Proof of Proposition 6.4. We use one charge for both kinds of recursive step. A balanced query already earns enough information to pay for itself. A flat stage earns restriction credit as well, and its capped discovery information absorbs the positive-mass term. Once both steps have this form, summing over the recursion requires no further geometric or scheduling argument.

Claim 6.11 (Each stage is paid for by two potential decreases)**.** *Every balanced query or flat stage costs at most $A\Delta + cI$.*

Proof. A balanced query has information at least $w(S)h(\delta)$, because its normalized side mass lies in $[\delta, 1-\delta]$ and the concave, symmetric binary entropy is minimized there at an endpoint. Thus

$$W = w(S) \overset{I \geq w(S)h(\delta)}{\leq} \frac{I}{h(\delta)} \leq cI \overset{\text{Lem. 2.12}}{\leq} A\Delta + cI;$$

the final step uses $A \geq 0$ and the nonnegative restriction credit. Also Lemma 2.9 gives $h(\delta) \leq \ln 2$, so $c \geq 1/h(\delta) \geq 1/\ln 2$. For a flat stage, $J \geq b\sigma(\delta)$ and $c \geq B/\sigma(\delta)$ give $Bb \leq cJ$. Since $I - J \geq 0$,

$$\begin{aligned} W &\overset{\text{assumed flat-stage bound}}{\leq} A\Delta + Bb + \frac{I-J}{\ln 2} \\ &\leq A\Delta + cJ + c(I-J) = A\Delta + cI. \end{aligned}$$

The second inequality substitutes $Bb \leq cJ$ and $1/\ln 2 \leq c$; the latter can multiply $I - J$ without changing the inequality because $I - J \geq 0$. The last equality then recombines the information already learned and the information still needed. Thus the argument uses only this proposition's assumptions, and applies to any stages satisfying them. □

Each stage partitions its current state into the classes on which the recursion continues. Thus the two local credits are decreases of the same potentials before and after that partition. Summing them cancels all internal-state terms and leaves only the root's optimal expected cost and entropy.

Claim 6.12 (Restriction and information telescope over pattern classes)**.** *The sum of stage costs is at most $A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S)$.*

Proof. By their definitions,

$$\Delta = \mathrm{OPT}_{\mathrm{E}}(S) - \sum_{A \in \Pi} \mathrm{OPT}_{\mathrm{E}}(A), \qquad I = \mathcal{E}(S) - \sum_{A \in \Pi} \mathcal{E}(A).$$

Let $\mathcal{V}$ be the nonsingleton recursive states and $\mathcal{L}$ their singleton leaves. As in Proposition 2.13, every nonroot state occurs once as a child, so the internal terms cancel exactly:

$$\begin{aligned} \sum_{V \in \mathcal{V}} \Delta(V) &= \sum_{V \in \mathcal{V}} \mathrm{OPT}_{\mathrm{E}}(V) - \sum_{V \in \mathcal{V}} \sum_{A \in \Pi_V} \mathrm{OPT}_{\mathrm{E}}(A) \\ &= \mathrm{OPT}_{\mathrm{E}}(S) - \sum_{A \in \mathcal{L}} \mathrm{OPT}_{\mathrm{E}}(A), \\ \sum_{V \in \mathcal{V}} I(V) &= \mathcal{E}(S) - \sum_{A \in \mathcal{L}} \mathcal{E}(A). \end{aligned}$$

Both leaf sums vanish. Each query belongs to one stage, with its original reach mass, so summing the previous claim gives

$$c_{\mathrm{E}}(T(S)) \le \sum_{V \in \mathcal{V}} W(V) \overset{\text{Claim 6.11}}{\le} A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S).$$

Finally Lemma 2.12, proved by Claim 2.5, gives the binary-tree entropy lower bound $\mathcal{E}(S) \le \ln 2\ \mathrm{OPT}_{\mathrm{E}}(S)$; substituting it yields

$$c_{\mathrm{E}}(T(S)) \overset{\text{Claim 2.5}}{\le} (A + c\ln 2)\,\mathrm{OPT}_{\mathrm{E}}(S).$$

This completes the proposition and its multiplicative guarantee. □

6.3 Calibration of the expected-depth bound

The preceding proof separates the algorithm from its numerical tuning. The calibration has two tasks: make every symbolic credit valid, then show that the resulting restriction and entropy coefficients give the announced approximation factor. The checks below follow that order. The calibration uses three fixed values:

$$\beta = \frac{3512863}{10^6}, \qquad \delta = \frac{393931}{2500000}, \qquad p_* = \frac{21317}{125000}. \tag{16}$$

All other coefficients are derived from these and from the fixed comparison distribution in Section A. That distribution is used to prove the score bound; the algorithm minimizes its rational score over the entire candidate family.

Each parameter has a separate role: the growth rate sets the batch budgets, the balance threshold bounds flat-state cap masses, and the tent parameter determines the rational credit for asking both endpoints. The remaining numbers describe their proof analysis.

What must be checked. Claim 6.6 requires the entropy tent $H_2(p_*) \ge 1 - 2p_*$. The geometric comparison in Proposition 6.3 also requires $\lambda_\beta \ge \nu_\beta$. The fixed distribution must then pass three certificate checks. First, every rank coefficient except one must stay above the target threshold. Next, the mean waiting length of the zero route must remain within its comparison bound. Finally, the retained credit must cover the zero-route term. Set $\tau = 2190079/10^6$. The exact coefficient construction (14) gives

$$1.276476788627 < a_1 < 1.276476788629 < \tau, \qquad \min_{2 \le t \le 14} a_t > 2.1900797 > \tau.$$

All first-batch cutoffs are at most six, $k_0 \le 3$, $k_1 \le 12$, and $r \le 13$. Thus the signed corrections are constant for $t \ge 14$; $a_t = e_\beta(t) + \kappa$ on that entire tail. Its monotonicity reduces all later coefficients to the single a_{14} check. Indeed, when $t \ge 14$ all signed-correction indicators equal one and all first-batch credit indicators equal zero, so

$$d_{q,r}(u,t) = 2(k_0 + k_1 - q - r), \qquad \kappa = 2\mathbb{E}[k_0 + k_1 - q - r].$$

Since $\lfloor x \rfloor \le x$ and $\lfloor \beta x \rfloor \le \beta x$, the integrand defining e_β is nonnegative. Thus $a_t - a_{14} = e_\beta(t) - e_\beta(14) \ge 0$ for every $t \ge 14$: (14) cancels the common tail correction κ, and the integral over $[14, t]$ has nonnegative integrand.

The all-negative route adds a separate waiting cost. We check its entire length because its mass can be arbitrarily large. Likewise, writing $D_0(k) = \mathbb{E}[z_{q,r}(u,k)]$,

$$D_0(1) = 0, \qquad \max_{2\le k\le 14} D_0(k) < -0.06992, \qquad D_0(k) = D_0(14) \quad (k \ge 14).$$

By the zero-route correction (12), these comparisons prove the required mean waiting inequality for every possible minimum-cover size, including small ones. For $k = 1$, every indicator in $z_{q,r}(u,k)$ vanishes. For $k \ge 14$, all indicators for nonzero budgets equal one, and consequently

$$D_0(k) = \mathbb{E}[q + r - k_0 - k_1] = -\kappa/2 = D_0(14).$$

The capped-rank calculation now bounds the loss at the exceptional first rank. Its net credit must still cover the zero-prior term; Claim 6.9 makes this the condition $\chi + 2/\ln\beta \ge \tau$; the direct treatment of degenerate hulls also needs both local coefficients to exceed two. The derived coefficients satisfy $A, B > 2$ and

$$\chi > 1.199202587061, \qquad B < 4.374727111314, \qquad \chi + 2/\ln\beta > \tau.$$

Here $3 + 2\Phi_\beta(1) = 7 - \ln 6/\ln\beta$. The strict inequality $\beta - 1 > 2\ln\beta$ gives $\lambda_\beta > \nu_\beta$. More explicitly, $\lfloor\beta\rfloor = 3$ gives $\Phi_\beta(1) = 2 - \ln 6/(2\ln\beta)$, while

$$\lambda_\beta - \nu_\beta = \frac{\beta - 1 - 2\ln\beta}{2\ln\beta} > 0.$$

How the checks are certified. All finite checks reduce to rational coefficients and logarithms. We enclose the logarithms from above and below, then evaluate the coefficient formulas with outward bounds. The rank and waiting-length tail arguments above complete the checks beyond the finite range. For rational $x \ge 1$, write $z = (x-1)/(x+1)$ and $L_N(x) = 2\sum_{j=0}^{N-1} z^{2j+1}/(2j+1)$. The positive logarithm series has the rational enclosure

$$L_N(x) \le \ln x \le L_N(x) + \frac{2z^{2N+1}}{(2N+1)(1-z^2)}. \tag{17}$$

To derive the enclosure, write $x = (1+z)/(1-z)$. Integrating the geometric series for $1/(1-t^2)$ on $[0,z]$ gives

$$\ln x = 2\int_0^z \frac{dt}{1-t^2} = 2\sum_{j\ge 0} \frac{z^{2j+1}}{2j+1}.$$

Every term is nonnegative, and its omitted tail satisfies

$$0 \le \ln x - L_N(x) \le \frac{2z^{2N+1}}{2N+1}\sum_{j\ge 0} z^{2j} = \frac{2z^{2N+1}}{(2N+1)(1-z^2)}.$$

For the inequality, the jth omitted denominator is $2N + 2j + 1 \ge 2N + 1$, so replacing it by $2N + 1$ increases that term. The equality then sums the geometric series, valid because $0 \le z < 1$. Range reduction to $[1,2]$ makes $N = 100$ terms sufficient for the comparisons above. In detail, write any positive rational argument as $x = 2^s y$, with integer s and $1 \le y \le 2$; then $\ln x = s\ln 2 + \ln y$. Evaluate the displayed enclosure for y and for 2. If $s < 0$, multiplying the interval for $\ln 2$ by s reverses its endpoints. This also handles the arguments below one in the entropy formulas. For $y \in [1,2]$, $z \le 1/3$, so the remainder at $N = 100$ is at most $2 \cdot 3^{-201}/(201(1 - 1/9))$.

For integer $t \geq 2$, put $n_t = \lfloor \beta t \rfloor$. The first floor integral telescopes as

$$\int_1^t \frac{\lfloor x \rfloor}{x}\, dx = \sum_{j=1}^{t-1} j \ln \frac{j+1}{j} = t \ln t - \ln(t!).$$

On $[j, j+1)$ the floor is j, giving the first equality. In the resulting sum, the coefficient of $\ln j$ is -1 for $2 \leq j < t$ and the coefficient of $\ln t$ is $t-1$; collecting these terms gives the factorial expression. For the second, substitute $y = \beta x$ and subtract the same floor-integral primitive at the two endpoints. Since $\lfloor \beta \rfloor = 3$,

$$\begin{aligned}\int_1^t \frac{\lfloor \beta x \rfloor}{x}\, dx &= \int_\beta^{\beta t} \frac{\lfloor y \rfloor}{y}\, dy \\ &= n_t \ln(\beta t) - \ln(n_t!) - \bigl(3 \ln \beta - \ln 6\bigr) \\ &= (n_t - 3) \ln \beta + n_t \ln t - \ln(n_t!/6).\end{aligned}$$

Subtract these two integrals from $(\beta+1)(t-1)$ and divide by $\ln \beta$, as prescribed by the definition of e_β in Proposition 6.3, to obtain

$$\begin{aligned}e_\beta(t) = \frac{(\beta+1)(t-1) - (t+n_t) \ln t + \ln(t!)}{\ln \beta} \\ - (n_t - 3) + \frac{\ln(n_t!/6)}{\ln \beta}.\end{aligned}$$

Together with the complete fixed rational table, these formulas reproduce every finite coefficient and waiting-length check. They also verify $h(p_*) > (1 - 2p_*) \ln 2$, which supplies the entropy tent. The fixed table and outward interval bounds provide a reproducible certificate for these numerical claims.

For clarity, the outward rational checks of the finite ranks are listed below. Each decimal is an exact rational endpoint, rounded outward from the $N = 100$ enclosure. Only rank one needs an upper bound, namely $a_1 < 1.276476788629 < \tau$ as stated above.

t or k	Lower bound for a_t	Upper bound for $D_0(k)$
1	1.276476788627	0
2	2.190079716489	−0.246077347242
3	2.190079716462	−0.308016911051
4	2.190079716311	−0.339177879272
5	2.190079716316	−0.313280064542
6	2.190079716371	−0.261411526812
7	2.190079716472	−0.219499207512
8	2.190079716131	−0.186529025533
9	2.190079716691	−0.158717650632
10	2.190079716422	−0.135101669420
11	2.190079716565	−0.115461022418
12	2.190079716480	−0.097796497729
13	2.190079716692	−0.083683085595
14	2.190079716520	−0.069926303070

Every listed a_t with $t \geq 2$ exceeds 2.1900797, and every listed $D_0(k)$ with $k \geq 2$ is below -0.06992.

The remaining intermediate enclosures include

$$\begin{aligned} 4.461183727319 &< \mu < 4.461183727320, \\ 1.256431374515 &< \ln \beta < 1.256431374516, \\ 0.435622350180 &< h(\delta) < 0.435622350181, \\ 1.905736047002 &< \sigma(\delta) < 1.905736047003, \\ \beta - 1 - 2\ln\beta &> 0.000000250968, \\ h(p_*) - (1-2p_*)\ln 2 &> 0.000001006640. \end{aligned}$$

The script checks the displayed rational inequalities directly; their decimal renderings serve only as readable summaries.

With the local inequalities verified, only the stage-to-tree conversion of Proposition 6.4 remains to be calibrated. It chooses the larger of the balanced-query entropy coefficient and the capped-discovery coefficient; the following bound checks their resulting total. Finally $6 < 1/\delta < 7$, so $\sigma(\delta) = -6\delta\ln\delta - (1-6\delta)\ln(1-6\delta)$. The same rational enclosures certify the final factor as follows. If an expression has enclosure $[\underline{x}, \overline{x}]$, sums add endpoints, products take the minimum and maximum of the four endpoint products, and a division by a positive interval uses the reciprocal interval $[1/\overline{x}, 1/\underline{x}]$. These rules preserve every sign in the preceding sums. In particular,

$$\begin{aligned} \underline{\chi} &= \tau - 2\delta(\tau - \underline{a}_1) - \delta\overline{\mu}, \\ \overline{B} &= 7 - \frac{\ln 6}{\overline{\ln\beta}} - \underline{\chi}, \\ \underline{h} &= -\delta\,\overline{\ln\delta} - (1-\delta)\,\overline{\ln(1-\delta)}, \\ \underline{\sigma} &= -6\delta\,\overline{\ln\delta} - (1-6\delta)\,\overline{\ln(1-6\delta)}, \\ A + c\ln 2 &\le \frac{\beta+1}{\underline{\ln\beta}} + \max\left\{\frac{\overline{\ln 2}}{\underline{h}}, \frac{\overline{B}\,\overline{\ln 2}}{\underline{\sigma}}\right\}. \end{aligned}$$

Both entropy denominators and B have strictly positive enclosures. The sign of the subtracted quotient in B explains why its numerator uses a lower bound and its denominator an upper bound. The negative entropy coefficients explain the upper logarithm endpoints in $\underline{h}$ and $\underline{\sigma}$. Evaluating the last line with the full rational series bounds, before their decimal display rounding, proves

$$\boxed{A + \max\{\ln 2/h(\delta), B\ln 2/\sigma(\delta)\} < 5.182975508794 < 5.182976.}$$

The file `expected-two-phase-check.py` checks these enclosures with exact fractions. Its supplementary tests verify 9,900 signed-phase identities and 1,650 modified-batch comparisons, including early minimum-cover completion and full negative waiting invoices. They support the implementation audit; the stated claims and the exact fixed-table certificate establish the theorem.

7 Quasipolynomial approximations approaching two

The same construction works for both objectives. A graded schedule orders adjacent hull blocks with only a small loss in their ranks, giving a leading factor close to two plus an additive penalty. We then optimize a short initial part of the tree exactly; this lookahead absorbs the remaining penalty into the multiplicative factor. The algorithm returns a completed tree, with its graded recursion supplying each internal continuation.

7.1 Algorithm and the three supporting propositions

Given $0 < \varepsilon \le 1$, use grade spacing $\lambda = 1 + \varepsilon/2$ and write $a = 2\lambda$. The only user-selected parameter is ε. At the end of the proof, we choose the balance threshold and continuation coefficients to satisfy the inductive inequalities; these feasible values yield the stated factor. GradeDP stores a small number of cumulative rank budgets. Rollout compares each legal root query with stopping lookahead and attaching the graded continuation, using the costs of the constructed trees.

A nearly exact schedule must satisfy several early-rank requirements at once. The optima for separate coverage budgets can have incompatible prefixes, so the graded dynamic program retains their compatibility explicitly.

Proposition 7.1 (Graded interval scheduling)**.** *For ordinary interval demands with r nested rank budgets at most B,* GradeDP *decides simultaneous feasibility and returns a schedule in $(M+1)^{O(r)}(B+1)^{O(r)}$ time, where M is the number of candidates. Circular demands add enumeration of their first candidate.*

For positive demands of weights $w(I)$ and zero-pattern mass p_0, the same DP, with delay levels 0 and the distinct $\lfloor \lambda^j \rfloor$, returns an ordering whose joint delay cost

$$F(Q) = 2\sum_I w(I)(\mathrm{first}_Q(I) - 1) + 2p_0|Q|$$

is at most λ times the minimum. It uses $r = O(\varepsilon^{-1}\log n)$ grades. For rational input weights, the data-dependent DP entries are sums of weights times integer ranks, so their bit lengths remain polynomial. The subtraction of one prices delay before the hitting block; the zero term prices the complete negative route.

Scheduling ranks is only part of identification: endpoint recovery and later recursive work still cost queries. The next proposition combines them with restriction credit, leaving a small state potential as the only additive penalty.

Let $B_W(S)$ be the height and $B_E(S)$ the original-prior expected depth of the corresponding graded continuation on state S. Both quantities include discovery, endpoint queries, and all recursive child trees; the continuations return a leaf at singleton states.

Proposition 7.2 (Graded continuation bounds)**.** *There are fixed balance parameters for which the two graded continuations, zero on singletons, satisfy*

$$B_W(S) \le a\,\mathrm{OPT}_W(S) + 3\log_2|S|, \tag{18}$$

$$B_E(S) \le a\,\mathrm{OPT}_E(S) + 4\mathcal{E}(S). \tag{19}$$

They use only supplied lines and run in $(mn)^{O(\varepsilon^{-1}\log n)}$ time, times a polynomial in the input bit length. The expected-depth bound has no dependence on the smallest prior.

To turn the continuation bound into a multiplicative guarantee, we optimize an initial prefix. We charge those queries at their realized cost, and the resulting saving absorbs the remaining penalty.

Proposition 7.3 (Lookahead removes a bounded continuation penalty)**.** *Suppose a legal continuation on every restricted state S' obeys either $B_W(S') \le a\,\mathrm{OPT}_W(S') + L_W$, or $B_E(S') \le a\,\mathrm{OPT}_E(S') + L_E w(S')$, with $a > 1$. Exact k-level lookahead is an a-approximation if, respectively, $k \ge L_W/(a-1)$ or $k \ge L_E/(a-1)$. The continuation costs at singletons are zero.*

Algorithm 11: ROLLOUT: exact initial-tree optimization with a baseline

Input: State S, objective o, lookahead j, and legal baseline routine B.
Output: The cheapest initial tree of at most j levels, completed by B.

if $|S| \le 1$ **then**
  return its leaf
if (S, o, j) *is memoized* **then**
  return the stored tree and cost
Compute the actual baseline tree $B(S)$ and retain it as the initial best choice
if $j > 0$ **then**
  foreach *supplied line* ℓ *splitting* S **do**
    Recurse on S_ℓ^-, S_ℓ^+ with lookahead $j - 1$
    Compare $1 + \max\{v_-, v_+\}$ if $o = \mathrm{W}$, or $w(S) + v_- + v_+$ if $o = \mathrm{E}$, with the best cost
    Store a lower-cost choice and its two child trees
Memoize and return the best choice

For Theorem 7.4, use Algorithm 13 with $\lambda = 1 + \varepsilon/2$ and $k = \min\{n - 1, \lceil 3\lceil \log_2 n \rceil / (1 + \varepsilon) \rceil\}$. This choice sets the lookahead cap for both objectives; the baseline completes every search branch that reaches the cap.

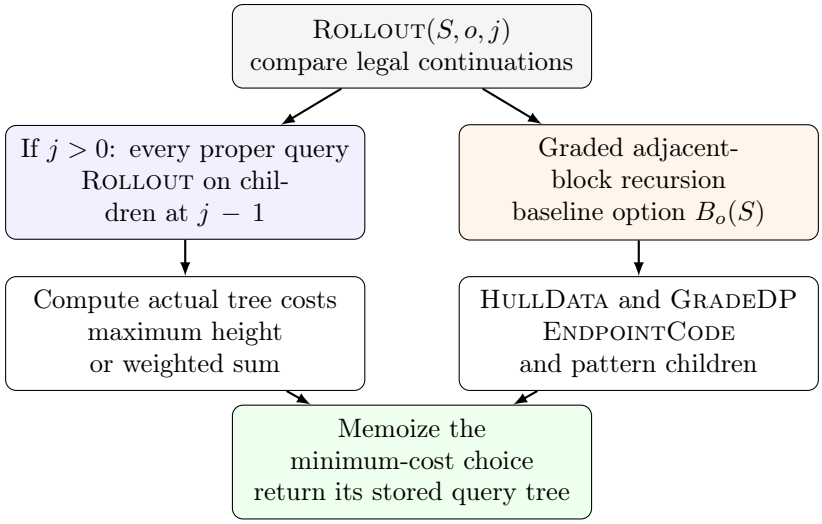


Figure 5: Exact lookahead wrapped around the quasipolynomial baseline. The arrows denote calls and the returned trees or costs. The baseline completes any branch that reaches its stopping depth. Thus the logarithmic cap on j bounds the number of search transcripts while each returned tree still completes identification. The proof compares this finite search with an optimal tree; the algorithm evaluates only trees it has constructed.

Proof strategy.

1. Retain several cumulative budgets together. Exact feasibility requires nested prefixes, so the graded DP stores their compatibility explicitly.
2. Map an optimal witness path to adjacent hull blocks. A query already removed by restriction pays for its block delay; a count or entropy potential pays for endpoint recovery.
3. Compare lookahead with the first levels of an optimal tree. Each paid prefix query replaces a query previously charged at factor a, which absorbs the continuation penalty.

Theorem 7.4. *For every fixed $\varepsilon > 0$, maximum depth and arbitrary-prior expected depth have supplied-query quasipolynomial $(2+\varepsilon)$ approximations. For $0 < \varepsilon \le 1$, their time is*

$$(mn)^{O(\varepsilon^{-1}\log n)}$$

times a polynomial in the input bit length.

Proof. Use the continuations of Proposition 7.2 with $a = 2+\varepsilon$. Their penalties on any restricted state are at most $3\log_2 n$ and $4w(S')\ln n$, respectively: $|S'| \le n$ bounds the count potential, and entropy on $|S'|$ hypotheses is at most $\ln|S'|$ per unit mass. Proposition 7.3 therefore removes both penalties after $O(\log n)$ levels. Concretely, the final calibration permits

$$q_n = \lceil \log_2 n \rceil, \qquad k = \min\{n-1, \lceil 3q_n/(1+\varepsilon)\rceil\}.$$

When the cap is $n-1$, every proper identifying tree is available, so the result is exact. Otherwise the lookahead guarantee applies. At depth $i \le k$, there are at most 2^i answer transcripts, and each transcript is generated by at most m^i choices of supplied queries along its path. Summing these bounds over $i \le k$ gives $(2m)^{O(k)}$ lookahead states. Multiplying by the graded-continuation cost preserves the stated time. For $\varepsilon > 1$, use the construction with $\varepsilon = 1$. □

7.2 Proofs of the supporting propositions

7.2.1 Graded schedules retain compatible prefixes

Each grade prefix has two obligations: stay within its rank budget and hit every interval whose deadline it represents. Counts enforce the first; the last selected position suffices to check the second.

Claim 7.1 (Counts and last positions decide every interval deadline)**.** *For each grade prefix j, store its selected count c_j and last selected position p_j. When a demand $[l,q]$ ends, it has been hit within that grade exactly when $p_j \ge l$.*

Proof. Scan the candidates in spatial order. Selecting position q at grade i increments c_j and replaces p_j by q for every $j \ge i$; skipping changes neither. These are exactly the cumulative counts and last positions. No selected position exceeds the scanned endpoint q, so $p_j \ge l$ is equivalent to a hit in $[l,q]$. Reject any count exceeding its budget. Sorting the selected candidates by grade therefore meets every deadline.

Conversely, write the cumulative budgets as b_j and take any feasible comparison ordering. Drop candidates whose ranks exceed the last budget; no required deadline hit is lost. Assign each remaining candidate v the grade

$$\mathrm{grade}(v) = \min\{j : \mathrm{rank}(v) \le b_j\}.$$

The candidates with grade at most j are exactly those with original rank at most b_j, so their count is at most b_j. Every demand with deadline j has one such candidate in its interval. At its right endpoint the last selected position of that prefix is therefore at least its left endpoint. Thus all count and hit checks pass, proving the converse. The recurrence is exact, with at most $(M+1)^r(B+1)^r$ states and $r+1$ transitions per position. For circular demands, force the first candidate, delete its hit demands and cut there. Its absolute rank consumes one unit of *every* cumulative capacity, including the total-count cap. □

Independent prefix coverage would be insufficient. For example, $[1, 2]$ and $[2, 3]$ can both be hit by rank one, while $\{1\}$ and $\{3\}$ can both be hit by rank two. The first requirement forces candidate 2 first, leaving too few ranks for the second. The DP stores exactly this compatibility information.

Exact deadline feasibility must now become an approximation for delay. Charging the first qualifying grade bounds each positive delay, while the final selected count also prices the complete negative route.

Claim 7.2 (Rounded delays bound the joint objective)**.** *Charging a demand at its earliest selected grade gives a λ approximation for F, including zero-pattern waiting.*

Proof. Let $d_0 = 0$ and let the positive d_j be the distinct $\lfloor \lambda^j \rfloor$ up to a level covering n. Grade j has capacity $d_j + 1$. At the right endpoint of demand I, charge $2w(I)d_j$ for the least j whose last selected position lies in I. Its actual first-cover delay is at most d_j. Keep the least accumulated charge for each state, and add $2p_0c_{\text{last}}$ at completion.

For a comparison order, rank one uses grade zero. At rank $t \geq 2$, the first positive level at least $t - 1$ is smaller than $\lambda(t - 1)$: the preceding unrounded power is less than the integer $t - 1$. Duplicate-floor plateaus do not change this argument. These assigned grades respect every cumulative capacity, so Claim 7.1 makes this graded comparison feasible. For each positive delay, its rounded charge is at most λ times its original charge; the final selected count, and therefore the zero waiting charge, is unchanged. Since that waiting charge is nonnegative and $\lambda \geq 1$, the minimum surrogate is consequently at most λF. Conversely, the actual delay of each returned demand is at most its charged grade level, so its actual objective is no larger than the minimized surrogate. An optimum uses at most the number of demands, after unproductive candidates are removed. For a circular order the forced first candidate has zero delay, all capacities decrease by one, and the final waiting charge is $2p_0(1 + c_{\text{last}})$. □

Algorithm 12: GRADEDP: cumulative-grade interval feasibility or weighted delay

Input: Ordinary intervals; r capacities $b_0 < \cdots < b_{r-1}$; feasibility deadlines, or weights, delay levels $d_j = b_j - 1$, and zero-pattern mass p_0.
Output: A feasible graded selection, or a minimum surrogate-delay selection.

```
Initialize the state (c_0, ..., c_{r-1}; s_0, ..., s_{r-1}) to all zeros, with cost zero; all other states are unreachable
for q = 1, ..., M do
    From each reachable state, try skipping q or selecting it at grade i
    A grade-i selection increments c_j and sets s_j = q for all j ≥ i
    Reject any transition with c_j > b_j
    foreach interval I = [l, q] ending here do
        if mode is feasibility then
            Reject if s_j < l for its deadline grade j
        else
            Find the least j with s_j ≥ l; reject if none
            Add 2w(I)d_j to this transition's cost
    Keep one feasible witness, or the minimum cost, for each resulting state
In weighted mode add 2p_0 c_{r-1} to each final state's cost
Reconstruct a winning selection and order it by grade
return the selection and schedule, or infeasible
```

For circular demands enumerate the forced first candidate, remove its demands, cut there, and reduce *every* capacity by one. Residual expected costs still use the original delay levels and add $2p_0(1+c_{r-1})$ at completion. In maximum-depth mode append a final grade with the total-count cap

if it is not already a capacity. Section 7 proves the state invariant and the $(M+1)^{O(r)}(B+1)^{O(r)}$ bound.

7.2.2 Restriction pays for a graded pattern stage

For the expected-depth calculations, use the pattern partition Π and abbreviate $\Delta = \Delta_{\mathrm{E}}(S,\Pi)$, $I = I(S,\Pi)$, $w = w(S)$, and $b = w - w(A_0)$. Write J for the state-mass-weighted entropy of the actual discovery transcript, and W for the expected cost of this stage's queries, before attaching the recursive child trees.

A block rank must be translated back into real queries. The next claim combines the two possible discovery exits with endpoint recovery; the second exit benefits from the cached negative first endpoint.

Claim 7.3 (An adjacent block has a uniform endpoint allowance)**.** *If a class A first meets block k, its discovery and endpoint queries use at most*

$$2(k-1) + z_A + 5, \qquad z_A = \lfloor \log_2(\delta|S|/|A|) \rfloor \tag{20}$$

on each route. For expected depth, the analogous overhead is at most five times the nonzero-pattern mass, plus the pattern information remaining after discovery.

Proof. At most $2(k-1)$ endpoint queries precede the hitting block. A first endpoint positive costs one query and has endpoint allowance $z_A + 4$ by Proposition 4.3. A second endpoint positive costs two queries but has a cached negative neighbor, reducing its allowance to $z_A + 2$. Thus the two total allowances are respectively

$$2(k-1) + 1 + (z_A + 4) = 2(k-1) + z_A + 5,$$
$$2(k-1) + 2 + (z_A + 2) = 2(k-1) + z_A + 4.$$

Their maximum is the stated common allowance. A block containing the free origin has one real endpoint and obeys the first bound. For expected depth, the two endpoint-code overheads are four on the first exit and two on the pinned second exit. Together with discovery this gives respectively five and four queries beyond the block delay and the conditional entropy cost. Summing over positive exits bounds this overhead by $5b$. If P is the pattern label and D_{disc} the discovery transcript, the latter is a function of P: at this fixed state every discovery query is a hull test, and all choices and stopping rules depend only on earlier hull answers. Hence $H(D_{\mathrm{disc}} \mid P) = 0$, and the entropy chain rule of Lemma 2.9 gives

$$wH(P \mid D_{\mathrm{disc}}) = wH(P) - wH(D_{\mathrm{disc}}) = I - J.$$

Thus the remaining entropy invoice is $(I-J)/\ln 2$. The second exit's smaller allowance is precisely the negative-neighbor saving. □

For maximum depth, build all pattern-child codes first. For a trial height H, set

$$B_A = \left\lfloor \frac{H - B_W(A) - z_A - 5}{2} \right\rfloor.$$

Reject negative budgets. Round each budget down to a delay level, use block deadline $\widehat{B}_A + 1$, and cap the total selected blocks at $\lfloor H/2 \rfloor$. Enumerate the first block and apply GradeDP. The trial range ends at $\lceil (a+3)(|S|-1) \rceil$. Low-dimensional hulls use their direct extreme tests.

The computed deadlines use actual child heights, so their feasibility requires care with integer rounding. Restriction supplies the needed rank credit before the deadline is rounded down to a grade.

Claim 7.4 (Downward rounding retains every required integer rank)**.** *At $H^* = \lfloor aD + \gamma \log_2 s \rfloor$, where $D = \mathrm{OPT}_W(S)$ and $s = |S|$, every comparison class is hit by its computed deadline, provided*

$$\gamma \log_2(1/(1-\delta)) \geq 1, \qquad \gamma \log_2(1/\delta) \geq 5, \qquad \gamma \geq 1. \tag{21}$$

Proof. Write $t = t_A$, $x = \log_2(s/|A|)$ and $u_\delta = \log_2(1/\delta)$. Proposition 4.2 gives $\mathrm{OPT}_W(A) \leq D - (t-1)$. Set $R_A = a\,\mathrm{OPT}_W(A) + \gamma \log_2 |A|$. Induction and the integrality of the constructed child height imply $B_W(A) \leq \lfloor R_A \rfloor$, while $z_A = \lfloor x - u_\delta \rfloor$. Applying $\lfloor u \rfloor - \lfloor v \rfloor \geq \lfloor u - v \rfloor$ twice gives the following chain. The first step uses the inductive child bound; the second takes the difference of the parent and child potentials inside one floor; and the third uses the supporting-prefix credit before taking the second floor difference:

$$\begin{aligned} H^* - B_W(A) - z_A - 5 &\geq \lfloor aD + \gamma \log_2 s \rfloor - \lfloor R_A \rfloor - \lfloor x - u_\delta \rfloor - 5 \\ &\geq \lfloor a(D - \mathrm{OPT}_W(A)) + \gamma x \rfloor - \lfloor x - u_\delta \rfloor - 5 \\ &\geq \lfloor a(t-1) + (\gamma - 1)x + u_\delta \rfloor - 5. \end{aligned}$$

Because $|A| < \delta s$, we have $x > u_\delta$. The conditions $\gamma \geq 1$ and $\gamma u_\delta \geq 5$ therefore give $(\gamma - 1)x + u_\delta \geq 5$. Hence the last line is at least $\lfloor a(t-1) + 5 \rfloor - 5 = \lfloor a(t-1) \rfloor$. Dividing the integer credit by two yields

$$B_A \geq \left\lfloor \frac{\lfloor a(t-1) \rfloor}{2} \right\rfloor = \lfloor \lambda(t-1) \rfloor.$$

The equality uses $a = 2\lambda$ and $\lfloor \lfloor v \rfloor / 2 \rfloor = \lfloor v/2 \rfloor$. For $t = 1$, level zero suffices. Otherwise choose the last exponent j with $\lfloor \lambda^j \rfloor \leq B_A$, including duplicate floors. Maximality implies $\lambda^{j+1} \geq B_A + 1 > \lambda(t-1)$, so dividing by $\lambda > 0$ gives $\lambda^j > t - 1$. Since $t - 1$ is an integer, $\widehat{B}_A = \lfloor \lambda^j \rfloor \geq t - 1$. Rounding a real expression before using this integer credit would not give that conclusion.

Map the optimal witness path to incident hull edges. Each class is hit by rank at most t_A; append an incident edge for an uncovered terminal nonzero singleton. The comparison uses at most D blocks when the zero pattern exists, and at most $D + 1$ otherwise. The final calibration ensures $\gamma \log_2 s \geq 2$ for $s \geq 2$. Since $a \geq 2$ and D is an integer,

$$H^* = \lfloor aD + \gamma \log_2 s \rfloor \geq 2D + 2, \qquad \lfloor H^*/2 \rfloor \geq D + 1.$$

The comparison therefore meets the total cap. Proposition 7.1 now supplies a passing trial, and Claim 7.3 bounds every positive route by H^*. The cap also bounds the full negative route, which uses at most $2\lfloor H^*/2 \rfloor \leq H^*$ actual tests. At a balanced state, each child A has $|A| \leq (1-\delta)s$ and $\mathrm{OPT}_W(A) \leq D$ by Lemma 2.12. Substituting these two inequalities into the inductive child bound gives

$$1 + B_W(A) \leq aD + \gamma \log_2 s + 1 - \gamma \log_2 \frac{1}{1-\delta} \leq aD + \gamma \log_2 s.$$

The last inequality is exactly the first condition in (21). For a low-dimensional hull, at most two extreme tests identify the pattern. Each nonzero class satisfies $|A| < \delta s$, giving

$$2 + B_W(A) \leq aD + \gamma \log_2 s + 2 - \gamma u_\delta \leq aD + \gamma \log_2 s.$$

Here the final step uses $\gamma u_\delta \geq 5$, so the count-potential drop pays for both extreme tests. The zero route ends at its singleton after at most two tests and also meets the bound because $D \geq 1$ and $a \geq 2$. This proves the maximum-depth continuation bound. □

For expected depth, the same witness comparison is summed with the original hypothesis weights. Discovery information then pays for the endpoint allowance, giving a local bound that cancels through recursion.

Claim 7.5 (Weighted delay and discovery information bound the expected stage)**.** *The graded expected-depth stage satisfies*

$$W \le 2\lambda\Delta + 5b + (I - J)/\ln 2, \qquad J \ge b\sigma(\delta).$$

Proof. The witness-edge order hits class A by rank t_A. When the zero class exists, its length is at most the witness-path length. Thus the restriction-credit proposition gives

$$\begin{aligned} F(Q_{\mathrm{wit}}) &= 2 \sum_{A \neq A_0} w(A)(\mathrm{first}_{Q_{\mathrm{wit}}}(A) - 1) + 2p_0|Q_{\mathrm{wit}}| \\ &\le 2 \sum_{A \neq A_0} w(A)(t_A - 1) + 2p_0\ell \\ &\overset{\text{Prop. 6.1}}{\le} 2\Delta. \end{aligned}$$

Without the zero class, an appended terminal singleton has its index $t_A = \ell + 1$ and no waiting term, so the same positive-delay inequality applies. Claim 7.2 gives $F(Q_{\mathrm{alg}}) \le \lambda F(Q_{\mathrm{wit}}) \le 2\lambda\Delta$.

Adding the allowance of Claim 7.3 yields the first asserted stage inequality. To bound its information credit, the actual discovery transcript's positive cells each lie in one vertex cap, even though a comparison block can meet two caps. Claim 6.5 therefore gives $J \ge b\sigma(\delta)$. Apply Proposition 6.4 with $A = 2\lambda$ and $B = 5$. To see the absorption explicitly, $ch(\delta) \ge 1$ and $h(\delta) \le \ln 2$ imply $c \ge 1/\ln 2$. Since $0 \le J \le I$,

$$cI - \frac{I - J}{\ln 2} = cJ + \left(c - \frac{1}{\ln 2}\right)(I - J) \ge cJ \ge cb\sigma(\delta) \ge 5b.$$

In this chain, the first inequality discards the nonnegative term $(c - 1/\ln 2)(I - J)$, the second uses the capped-discovery entropy bound, and the last uses $c\sigma(\delta) \ge 5$. Consequently $W \le a\Delta + cI$. At a mass-balanced split with relative child mass $v \in [\delta, 1 - \delta]$, the query costs w and $I = wh(v) \ge wh(\delta)$. Thus $cI \ge w$; the nonnegative restriction decrement permits $w \le a\Delta + cI$ there as well. Proposition 2.13 now adds these local inequalities over the recursion and cancels every nonroot optimal-cost and entropy term, giving $B_E(S) \le a\,\mathrm{OPT}_E(S) + c\mathcal{E}(S)$. The final calibration permits $c = 4$. For a hull of dimension at most one, direct extreme testing costs at most $2w$. If $p_0 > 0$, its nonempty zero witness path gives $\Delta \ge p_0$, so $2w \le 2\lambda\Delta + 5b$. Indeed $2w = 2p_0 + 2b \le 2\lambda p_0 + 5b \le 2\lambda\Delta + 5b$, since $\lambda \ge 1$. If $p_0 = 0$, $2w = 2b \le 5b$. Thus the same stage bound covers the degenerate branch without invoking the graded DP. □

Algorithm 13: Internal graded continuation for the quasipolynomial algorithm

Input: State S; objective o; rational $\lambda > 1$; fixed balance threshold $1/4$.
Output: The baseline tree of Section 7.

Stop at a singleton; at a balanced state query a balanced line and recurse
At a flat state compute HullData(S) and use full-hull edges as blocks
Handle a hull of dimension at most one by its direct extreme tests
if $o = \mathrm{W}$ **then**
    Build the pattern children once and record their heights $C_W(A)$
    for *trial heights H in increasing order* **do**
        Compute $B_A = \lfloor (H - C_W(A) - z_A - 5)/2 \rfloor$; reject negative budgets
        Round each down to the last level in $\{0\} \cup \{\lfloor \lambda^j \rfloor\}$ not exceeding it
        Use deadlines $\widehat{B}_A + 1$ and total cap $\lfloor H/2 \rfloor$ in circular GradeDP; stop at the first feasible trial
else
    Use distinct levels $d_0 = 0$, $d_j = \lfloor \lambda^j \rfloor$ through a level covering n
    Minimize the circular weighted-delay surrogate with capacities $d_j + 1$ by GradeDP and zero-pattern mass $w(A_0)$
Execute the selected blocks, use pointwise endpoint codes for W or expected endpoint codes for E, and attach recursive pattern children
Identify the zero singleton after the negative cover if present; omit it otherwise
return the contracted tree

The maximum-depth trial range is $1 \le H \le \lceil (2\lambda + 3)(|S| - 1) \rceil$. The baseline selects its trial from these explicit tests, while the optimum appears in the comparison argument.

There are $O(\varepsilon^{-1} \log n)$ grades, with polynomially many counts and spatial positions. To compute z_A, compare $2^z|A|$ with $\delta|S|$ and choose the largest admissible integer z; this locates the grade directly, without evaluating a logarithm. All objectives are rational sums of actual costs. Proper disjoint-child recursion has at most $n - 1$ stages. This proves the time and legality parts of Proposition 7.2.

7.2.3 Paid prefix queries absorb continuation penalties

To analyze lookahead, compare with a prefix of an optimal tree. On every unfinished maximum-depth route, the prefix has already consumed exactly the queries that create the saving against factor a.

Claim 7.6 (The maximum-depth penalty is paid on every frontier path)**.** *An optimal height-D tree, truncated at depth k, has continuation height at most $D - k$ at each surviving frontier state.*

Proof. If $D < k$, the whole optimal tree is a lookahead candidate. Otherwise each surviving frontier state has optimal height at most $D - k$: the original optimal subtree is a feasible tree of that height. The assumed continuation bound therefore completes its path at height at most

$$k + a(D - k) + L_W = aD - (a - 1)k + L_W \le aD.$$

The equality isolates the saving of $a - 1$ per paid prefix query, and the inequality uses $(a-1)k \ge L_W$. Earlier leaves are already within height D. Exact lookahead can only improve this candidate. □

The expected-depth comparison needs no common leaf depth. Only the mass still unfinished matters: each such hypothesis has paid all prefix queries, so their total payment absorbs its continuation penalty.

Claim 7.7 (Expected prefix cost pays for every surviving frontier mass)**.** *Let P be an optimal tree's expected cost through depth k, and M_k the mass still requiring continuation there. Then $P \geq kM_k$.*

Proof. Every surviving hypothesis has paid for all k prefix queries. If R is the sum of the optimal continuation costs, then $P + R = \mathrm{OPT}_E(S)$ by partitioning the optimal tree's queries at depth k. At each frontier state its optimal restricted cost is no larger than that of the original subtree, by Lemma 2.12. Thus completing the frontier with the baseline costs at most

$$P + aR + L_E M_k = a\,\mathrm{OPT}_E(S) - (a-1)P + L_E M_k \overset{P \geq kM_k}{\leq} a\,\mathrm{OPT}_E(S) + [L_E - (a-1)k]M_k.$$

This is at most $a\,\mathrm{OPT}_E(S)$ under the stated condition. It assumes neither a bound on the optimal tree's height nor a lower bound on its prior masses. These two claims prove the lookahead proposition. □

7.3 A simple calibration; no optimization is needed

Parameter calibration.
Fix the balance threshold at $\delta = 1/4$, the count coefficient at $\gamma = 3$, and the entropy coefficient at $c = 4$. These values affect only the continuation penalty and lookahead length, which disappear from the final ratio. Thus no table of optimized constants is needed for this scheme.

Indeed, $(4/3)^3 > 2$ and $3\log_2 4 = 6 > 5$ verify (21). Here $\sigma(1/4) = \ln 4$ and $h(1/4) \geq \frac{1}{2}\ln 2$. The elementary bounds $2/3 \leq \ln 2 < 3/4$ give $4h(1/4) > 1$ and $4\sigma(1/4) > 5$. The lower bound follows from the first term of the logarithm series in (17); the upper bound is the strict trapezoidal bound for $\int_1^2 dx/x$. Finally $4\ln n < 3\log_2 n$, so the single rational lookahead cap in the theorem covers both objectives. For singleton inputs, all procedures return the zero-query leaf.

8 Polynomial maximum depth with rank-subset scheduling

The algorithm combines exact scheduling inside a logarithmic rank cap with the priority fallback of Section 4. It constructs one code for each pattern child and compares scheduling alternatives against those codes' actual heights. In the proof, an optimal tree certifies that at least one trial is feasible.

8.1 The algorithm and its guarantee

Use the count threshold δ and potential coefficient γ of (2). For the original input size n, fix once

$$q = \lceil \log_2 n \rceil, \qquad R = \left\lfloor \frac{\gamma q}{\gamma - 1} \right\rfloor, \qquad L = R + 1. \tag{22}$$

We fix the cap L from the original size n and pass it unchanged to every recursive child; consequently, the rank-state bound stays uniform throughout the tree. When a state contains only one hypothesis, its identity is already known and recursion stops. At every larger state, we first use a supplied count-balanced query whenever one is available. Its two answer classes become the next recursive states, each with the same cap L. Once no count-balanced query remains, the state is flat, and the algorithm proceeds to the scheduling step below.

At a flat state S, construct every full hull-pattern child once and record its actual height $h_A = C_W(A)$. Then try increasing integer heights H through $\lfloor (3+\gamma)(|S|-1) \rfloor$; because the children

stay fixed, each trial reuses their codes and the first passing trial gives the smallest certified height. Compare the priority fallback with exact adjacent-block scheduling. For the latter, candidates are edges of the full normalized hull, with the origin kept as a free negative marker. Each positive vertex arc therefore becomes a circular interval of edge blocks. Write

$$z_A = \lfloor \log_2(\delta|S|/|A|) \rfloor, \qquad B_A = \left\lfloor \frac{H - h_A - z_A - 5}{2} \right\rfloor.$$

Proposition 4.3 bounds the endpoint continuation by $z_A + 4$ queries after the first positive answer; the shared allowance $z_A + 5$ includes the first positive query and also covers a pinned exit. Thus, after reserving the child height and endpoint allowance, at most B_A full blocks can precede the hit block, since each uses at most two queries. This gives deadline $B_A + 1$. The total cap $\lfloor H/2 \rfloor$ follows for the same reason: every block costs at most two queries. Reject negative budgets; otherwise assign block deadline $B_A + 1$ and use the total cap

$$\min\{L, \lfloor H/2 \rfloor\}. \tag{23}$$

The scheduler receives these integer deadlines directly, so its feasibility checks match the route-height bounds. An optional exact whole-exit decoder, proved safe below, may be included in the same trial. Return a passing alternative at the least height, attach its endpoint decoders and stored child codes, and contract constant queries. If the hull has dimension at most one, its two real extreme tests already determine every pattern, so query them directly.

Algorithm 14: HybridTree: general polynomial maximum depth

Input: State S, original size n, fixed global cap $L = R + 1$, where $q = \lceil \log_2 n \rceil$ and $R = \lfloor \gamma q/(\gamma - 1) \rfloor$, and the count parameters calibrated in Section 8.3.
Output: A tree satisfying Theorem 8.1.

```
Stop at a singleton; at a count-balanced state query a balanced line and recurse
Compute HullData(S) and build each child's hybrid code once
Handle a low-dimensional hull directly; otherwise record child heights h_A
for H = 1, ..., ⌊(3 + γ)(|S| − 1)⌋ do
    Try PriorityTrial with these stored children
    Set z_A = ⌊log_2(δ|S|/|A|)⌋ and B_A = ⌊(H − h_A − z_A − 5)/2⌋
    if all B_A ≥ 0 then
        Try circular SlotDP on full-hull-edge demands with deadlines B_A + 1 and slot cap
          min{L, ⌊H/2⌋}
    Optionally try whole-exit SlotDP using actual endpoint-continuation heights and the same
      total cap; test feasibility with these heights
    if any alternative passes then
        Attach its endpoint decoders and the stored child trees
        Terminate the negative cover at A_0 when present; omit that leaf otherwise
        return a passing tree, contracting constant queries
```

The global cap is inherited unchanged by every child. The priority fallback proves that a trial always passes; the algorithm never tests the unknown condition $\mathrm{OPT}_W(S) \le R$.

Theorem 8.1 (General polynomial maximum-depth approximation)**.** *For separating supplied binary unit-cost planar line queries, Algorithm 14 runs in deterministic polynomial time and returns a tree satisfying*

$$\boxed{C_W(\mathcal{H}) \le (2 + \gamma)\,\mathrm{OPT}_W(\mathcal{H}) < 4.464966\ \mathrm{OPT}_W(\mathcal{H})}$$

with the calibrated parameters of Section 8.3. A singleton has cost zero, and every query is a supplied line.

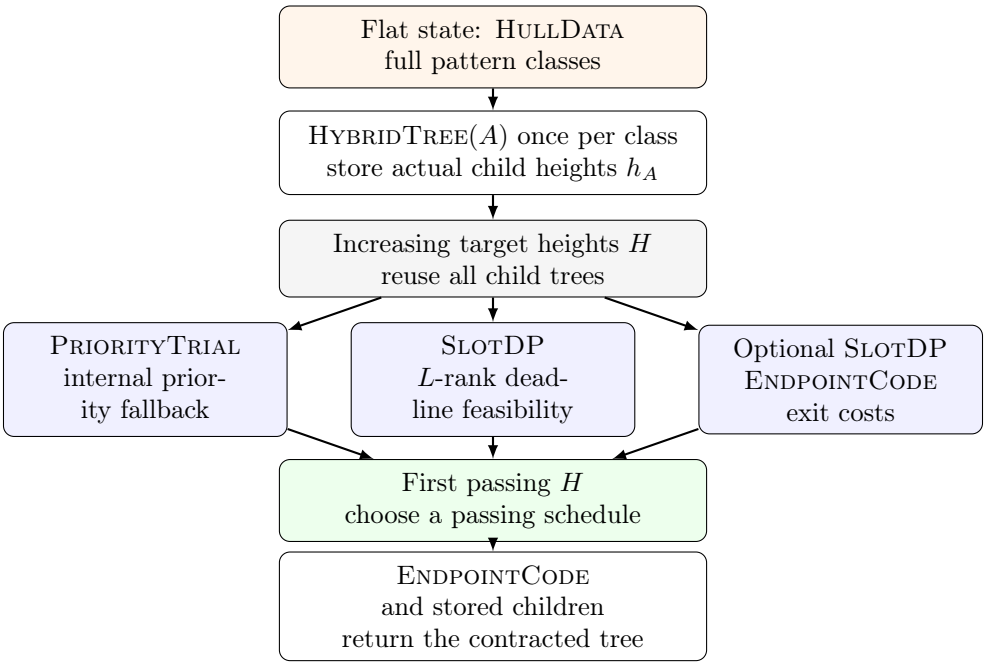


Figure 6: At a flat state, the algorithm first computes the hull and children, then tests increasing target heights using three schedule checks. The arrows carry the geometry, child heights, and candidate schedules needed by those checks. The logarithmic cap L is fixed at the original root. The priority fallback guarantees a passing trial by the stated height bound; the exact and whole-exit checks can certify an earlier trial, and the algorithm selects the first trial accepted by its explicit checks.

Proof strategy.

1. Solve interval deadlines exactly by partitioning a small set of global query ranks (the positions assigned in the full schedule) between spatial left and right regions.
2. Prove two recursive bounds simultaneously: the priority fallback works for every optimum; exact scheduling improves it when the optimum fits the global cap.
3. At the root, a small optimum uses the stronger bound. A large optimum absorbs the fallback's count potential. Calibrate the two count-credit inequalities only after these structural arguments.

Spatial left and right regions may be queried in an interleaved order. The exact scheduler assigns each region a subset of global query ranks; these positions preserve its deadlines while allowing the two searches to interleave.

Proposition 8.2 (Exact rank-subset scheduler)**.** *For N interval demands on M ordered candidates, with integer first-hit deadlines and at most L distinct selected candidates, feasibility and a schedule can be found in $O(M^3 3^L + NM^2)$ time with $O(M^2 2^L)$ states. Circular intervals add a factor M by enumerating the first candidate.*

The exact scheduler's short global cap makes its stronger bound available on small-optimum states, while the priority fallback covers the remaining states. Proving both bounds together lets the recursion use the strongest child estimate whose threshold survives restriction.

Proposition 8.3 (Two simultaneous recursive guarantees)**.** *At every recursive state S, with $s = |S|$ and $D = \mathrm{OPT}_W(S)$,*

$$C_W(S) \le 3D + \Phi(s), \tag{24}$$

$$D \le R \implies C_W(S) \le 2D + \Phi(s). \tag{25}$$

The threshold R is global, so restriction preserves the second invariant's inductive hypothesis in every child.

A feasible abstract schedule must still account for actual endpoint queries, stored child continuations, and the negative route. The next statement connects those certificates to a finite legal construction.

Proposition 8.4 (Executable trials and polynomial reconstruction)**.** *Every accepted trial has complete height at most H, including its zero route when present. A passing trial is reached within the stated range; geometry, scheduling, endpoint decoding, and reconstruction have polynomial bit complexity. The optional whole-exit alternative preserves both guarantees in Proposition 8.3.*

Proof of Theorem 8.1. At the root, if $D \le R$, (25) gives $C_W \le 2D + \Phi(n) \le 2D + \gamma \log_2 n \le (2+\gamma)D$. The middle step removes the floor from $\Phi(n)$, while Claim 2.6 gives $D \ge \log_2 n$; together these facts bound the logarithmic potential by γD. If $D > R$, integrality gives $D \ge R + 1 > \gamma q/(\gamma - 1)$. Since $\log_2 n \le q$, this implies $\gamma \log_2 n / D < \gamma - 1$. Substituting this bound into (24) therefore gives

$$\frac{C_W}{D} \le 3 + \frac{\gamma \log_2 n}{D} < 3 + (\gamma - 1) = 2 + \gamma.$$

Proposition 8.4 supplies the legal polynomial construction, and the calibration below gives its numerical factor. Trial acceptance is determined by the constructed code; the proof uses D to show that one of the checked trials meets the bound. □

8.2 Technical proofs of the propositions

Proof of Proposition 8.2. The scheduler's state separates spatial information from temporal information. A spatial split removes the intervals it hits; absolute ranks preserve every deadline while allowing the left and right searches to interleave in any order.

Candidates are $1, \ldots, M$. A demand $[a, b]$ of deadline d must contain a selected candidate assigned a rank at most d. For region $[l, r]$, let $\mathcal{I}(l, r)$ contain the original demands wholly inside it. Define $F(l, r, T)$ for an available subset $T \subseteq [L]$ of absolute ranks; unused ranks are allowed. The key sufficiency question is which demands can still matter inside a region. Interval geometry makes ancestor hits recognizable from the region alone, while the rank subset records its remaining time slots.

Claim 8.1 (Contained intervals are exactly the unhit demands)**.** *The spatial region and the absolute rank subset are sufficient state.*

Proof. Each region is a connected component of the candidate line after its ancestor separator points have been deleted. An original interval not contained in one such component must contain a deleted separator: intervals contain every candidate between their endpoints. It was therefore hit at an ancestor. Conversely, an interval contained in a remaining region contains none of those separators and is unhit. Queries in other remaining regions cannot hit it either. Thus exactly $\mathcal{I}(l, r)$ remains relevant in $[l, r]$. The subset T records when its future candidates may be queried in the global order; two regions with the same number of queries can have different available ranks and hence different deadline feasibility. □

Once that state is sufficient, its earliest available rank can be used for the next separator. Partitioning the later ranks permits every interleaving of the left and right searches.

Claim 8.2 (The earliest available rank gives an exact recurrence)**.** *For nonempty $\mathcal{I}(l, r)$ and T, put $t = \min T$. Reject if a contained deadline is less than t; otherwise accept precisely when some $p \in [l, r]$ and partition $T \setminus \{t\} = T_L \dot{\cup} T_R$ satisfy*

$$F(l, p-1, T_L) \quad \textit{and} \quad F(p+1, r, T_R).$$

Proof. By Claim 8.1, it suffices to check the contained demands. For sufficiency, place p at rank t. Every contained demand containing p is hit at rank t, no later than its deadline because the preliminary test checked every contained deadline. An interval not containing p is wholly in $[l, p-1]$ or $[p+1, r]$. The two accepted subproblems give distinct candidates with disjoint absolute rank sets, so merging their selections by rank preserves all their deadlines and completes the state.

For necessity, let a feasible state have first selected point p at rank $t' \in T$, and put $t = \min T$. If $t < t'$, then t is unused: every other selected point has rank greater than t'. Assign p to t and keep all other ranks fixed. If $t = t'$, leave the schedule unchanged. Thus selected ranks remain distinct, and advancing the first point cannot delay any first hit. No contained deadline can be smaller than t, since no available rank would then meet it. Partition the other selected ranks by whether their candidate lies left or right of p, and distribute unused ranks arbitrarily between the two parts. Each unhit interval lies wholly on one side, so the corresponding subproblem is feasible. This gives a partition of all $T \setminus \{t\}$ as required. When the demand family is empty, the empty selection is feasible; a nonempty family with $T = \varnothing$ is infeasible. □

The DP may leave rank gaps, which only postpone hits. Compressing those gaps produces an executable linear schedule; choosing its first point similarly turns a circular instance into a linear one.

Claim 8.3 (Reconstruction and circular cuts preserve feasibility)**.** *The recurrence has the stated time and state bounds.*

Proof. Sort selected points by their assigned ranks and remove unused slots; all hits accelerate. Precompute existence and minimum deadline of contained demands in $O(NM^2)$ time. There are $O(M^2 2^L)$ states, and for a mask of size k there are at most 2^{k-1} partitions after its minimum is removed. The coarser total bound is

$$\sum_{T \subseteq [L]} 2^{|T|} = \sum_{k=0}^{L} \binom{L}{k} 2^k = (1+2)^L = 3^L.$$

The first equality groups masks by cardinality, and the second is the binomial theorem. Hence all masks contribute a total partition factor 3^L. With $O(M^2)$ spatial regions and at most M candidate roots per region, the transitions take $O(M^3 3^L)$ time, in addition to the $O(NM^2)$ preprocessing. Removing unused ranks sends every assigned rank to an earlier or equal schedule position, so no deadline is missed.

An empty circular demand family accepts immediately. Otherwise enumerate its first selected point at rank one. Delete the intervals hit there and cut the candidate circle at that point; every survivor becomes an ordinary interval. A surviving original deadline at most one is impossible. For all other survivors subtract one from their deadline and use $L-1$ remaining slots. Appending the first point to any accepted linear schedule restores the original deadlines, and every feasible circular schedule has one of the enumerated first points. The enumeration adds a factor M. □

□

Within an interval state, the earliest available rank must be assigned to one candidate p. Choosing p divides the remaining candidates into left and right intervals, and the unused ranks must split between those two subproblems. The recursion enumerates these partitions; memoization reuses each interval-and-rank state.

Algorithm 15: SlotDP: exact interval deadlines with a rank subset

Input: Ordinary interval demands and deadlines; region $[l, r]$; available absolute ranks $T \subseteq [L]$.
Output: A feasible rank assignment, or infeasible.

```
if no original demand is wholly contained in [l, r] then
    return the empty assignment
if T = ∅ then
    return infeasible
if this state is memoized then
    return its stored result
t ← min T
if a wholly contained demand has deadline < t then
    return infeasible
foreach p ∈ [l, r] and T_L ⊆ T \ {t} do
    T_R ← T \ (T_L ∪ {t})
    Recurse on ([l, p − 1], T_L) and ([p + 1, r], T_R)
    if both assignments are feasible then
        Memoize their union with (p, t) and return it
Memoize and return infeasible
```

After the recursion succeeds, sort the selected candidates by rank and suppress unused ranks to obtain the schedule. For circular demands, enumerate the first candidate and cut the circle there; this converts the remaining demands to intervals, with one subtracted from their deadlines and the slot cap to account for the forced first rank. In the optional complete-exit variant, first compute both oriented decoder costs using Algorithm 4. Those costs replace the ordinary deadline precheck by the local height tests of Section 8; the same rank partitions then handle the remaining scheduling choices.

Proof of Proposition 8.3. Induct on s, with zero-cost singleton states, proving both bounds simultaneously. A balanced child A has $\mathrm{OPT}_W(A) \le D$ and $\Phi(|A|) \le \Phi(s) - 1$, by restriction and Proposition 4.1. For either coefficient $c \in \{2, 3\}$, its inductive route bound gives

$$1 + h_A \le 1 + c\,\mathrm{OPT}_W(A) + \Phi(|A|) \le cD + \Phi(s).$$

The first inequality is the appropriate inductive hypothesis; in the second, restriction bounds the optimal child height by D and the unit potential drop pays for the root query. When $c = 2$, $D \le R$ also gives $\mathrm{OPT}_W(A) \le R$, so the stronger child hypothesis is available. Thus a balanced query preserves either bound. The priority alternative preserves the universal bound by Proposition 4.6; smaller actual children only improve its budgets.

For the stronger flat-state bound, assume $D \le R$ and put $H^* = 2D + \Phi(s)$. All children meet the same global threshold, so their stronger inductive bound is available. The stronger flat-state proof has three parts: convert child-height credit into deadlines, fit a comparison cover inside both caps, and translate its blocks into physical routes. The optimal witness is used only to establish existence of a passing trial.

Claim 8.4 (Actual child heights give feasible comparison deadlines)**.** *Every nonzero class satisfies $B_A + 1 \ge t_A$ at H^*.*

Proof. Proposition 4.2 gives $\mathrm{OPT}_W(A) \le D - (t_A - 1)$. The actual stored height therefore satisfies $h_A \le 2[D - (t_A - 1)] + \Phi(|A|)$. Substitution gives

$$\begin{aligned} H^* - h_A - z_A - 5 &\ge 2(t_A - 1) + \Phi(s) - \Phi(|A|) - z_A - 5 \\ &\ge 2(t_A - 1), \end{aligned}$$

The first inequality substitutes the stronger inductive child bound; the second uses the nonzero-class count credit of Proposition 4.1, namely $\Phi(s) - \Phi(|A|) \ge z_A + 5$. Thus

$$H^* - h_A - z_A - 5 \ge 2(t_A - 1). \tag{26}$$

Dividing by two gives a lower bound by the integer $t_A - 1$; taking a floor consequently preserves it, so $B_A \ge t_A - 1 \ge 0$. Thus this comparison trial passes the nonnegative-budget check and has deadline $B_A + 1 \ge t_A$. This links an unknown optimal witness to deadlines computed from actual stored codes. □

The comparison must satisfy each block deadline, the global rank cap, and the cap on the negative route. A terminal nonzero singleton may require one appended block, which accounts for the extra slot in the global cap.

Claim 8.5 (The comparison fits both total caps)**.** *Every class meets its deadline in a cover of at most $D + 1$ edge blocks.*

Proof. Map each witness-path query to a full-hull edge containing its supports. Enlarging a singleton support only accelerates hits; delete repeated and unproductive blocks. If the path ends at an uncovered nonzero singleton, append an incident edge at its supporting index $r+1$, with $r \le D$. If the zero class exists it is the terminal witness leaf and no append is needed. The comparison has at most D blocks with a zero class and at most $D+1$ otherwise, and it hits A by rank t_A. Since $D \le R$, this fits $L = R+1$. Also $s \ge 2$ gives $\log_2 s \ge 1$, and $\gamma > 2$ gives $\Phi(s) = \lfloor \gamma \log_2 s \rfloor \ge 2$. As D is an integer,

$$\left\lfloor \frac{H^*}{2} \right\rfloor = \left\lfloor D + \frac{\Phi(s)}{2} \right\rfloor = D + \left\lfloor \frac{\Phi(s)}{2} \right\rfloor \ge D+1.$$

Pulling the integer D through the floor gives the second equality, and $\Phi(s) \ge 2$ gives the final inequality. Thus the comparison fits both the L-block rank limit and the cap $2\lfloor H^*/2 \rfloor$ on the all-negative route, even when a terminal singleton block has been appended. □

An abstract edge block becomes at most two supplied queries. The second first-positive exit has a cached negative neighbor and a cheaper decoder; accounting for this asymmetry yields one safe block allowance.

Claim 8.6 (A feasible block trial bounds every physical route)**.** *Class A first hit at block k has complete route height at most*

$$2(k-1) + z_A + 5 + h_A. \tag{27}$$

Proof. The preceding $k-1$ blocks use at most $2(k-1)$ supplied queries. If the first endpoint of block k is positive, discovery uses at most $2k-1$ queries. Proposition 4.3 then adds its z_A+4 endpoint allowance and the stored height h_A, so the complete route is bounded by

$$(2k-1) + (z_A+4) + h_A = 2(k-1) + z_A + 5 + h_A.$$

If the second endpoint gives the first positive answer, the first endpoint is a cached negative neighbor. The pinned allowance in Proposition 4.3 therefore gives

$$2k + (z_A+2) + h_A = 2(k-1) + z_A + 4 + h_A,$$

which is one query smaller than the stated common allowance. A block with one real endpoint obeys the first bound; cached answers and the origin marker cost nothing. Put $X = H^* - h_A - z_A - 5$. Since $B_A = \lfloor X/2 \rfloor$ and $k - 1 \le B_A$,

$$2(k-1) + z_A + 5 + h_A \le 2\lfloor X/2 \rfloor + z_A + 5 + h_A \le X + z_A + 5 + h_A = H^*.$$

The last inequality is $2\lfloor X/2 \rfloor \le X$. Thus either positive route fits the trial. The all-negative route has at most $2\lfloor H^*/2 \rfloor \le H^*$ queries and identifies A_0 if present; an absent zero leaf is omitted. □

Claims 8.4 and 8.5 provide a feasible comparison schedule. Hence Proposition 8.2 finds a passing trial at H^*, and Claim 8.6 converts it into the required route-height bound. If the hull has dimension at most one, its at most two extreme tests learn the pattern directly. For a nonzero class the stronger child bound and Proposition 4.1 give $2 + h_A \le 2D + \Phi(s) - z_A - 3 \le H^*$, since $z_A \ge 0$. The all-negative route also uses at most two tests, and $H^* \ge 2$. The universal calculation is identical with coefficient three. Both invariants follow by induction. □

Proof of Proposition 8.4. The local route certificate and (27) prove the accepted-height tests. The universal invariant supplies a passing integer trial. Indeed $D \le s-1$ follows from a proper identifying tree, and $\log_2 s \le s-1$ for integer $s \ge 1$, so

$$3D + \Phi(s) \le 3(s-1) + \lfloor \gamma(s-1) \rfloor = \lfloor (3+\gamma)(s-1) \rfloor,$$

where the equality uses the integrality of $3(s-1)$. The remaining obligations are running time and reconstruction. The rank search is exponential only in the cap fixed at the root, while all recursive stages retain explicit supplied queries and child codes.

Claim 8.7 (The global cap makes exact scheduling polynomial)**.** *All arithmetic and the complete reconstructed tree have polynomial size.*

Proof. For fixed γ, put $\alpha = \gamma/(\gamma-1)$. For $n \ge 2$, $q = \lceil \log_2 n \rceil < \log_2 n + 1$ and $L = \lfloor \alpha q \rfloor + 1 < \alpha \log_2 n + \alpha + 1$. For either $b = 2$ or $b = 3$, exponentiating gives

$$b^L < b^{\alpha+1} b^{\alpha \log_2 n} = b^{\alpha+1} n^{\alpha \log_2 b}.$$

The inequality uses monotonicity of b^x for $b > 1$, and the equality follows from $b^{\log_2 n} = n^{\log_2 b}$. The constants are fixed, so both rank-mask costs are polynomial. The singleton input returns immediately. There are polynomially many trial heights and first blocks, and at most $n-1$ nonterminal recursive stages with disjoint child sets. To see this count, the recursive states form a rooted partition tree whose leaves are individual hypotheses. Each nonterminal state has at least two nonempty children: balanced queries split properly, and a flat nonsingleton state cannot be one class because every nonzero class has size less than δs and the zero class has size at most one. In a rooted tree with n leaves and at least two children at each internal vertex, counting edges gives at most $n-1$ internal vertices. The witness LP, hull incidence, and endpoint codes have polynomial bit complexity. Every recursive child is smaller: balanced queries split properly, nonzero patterns have size below δs, and A_0 is a singleton. Compute q and z_A from their defining power-of-two thresholds, then set $R = \lfloor \gamma q/(\gamma-1) \rfloor$. Priority weights have $O(n \log n)$ bits. Contracting constant queries gives a proper tree with at most n leaves and $n-1$ internal nodes. □

Fixed endpoint allowances make the deadline instance simple, but may overestimate a particular exit. An exact rectangle decoder can inspect the realized patterns and their stored continuations. It uses the same legal endpoint comparisons and therefore can only improve a trial.

Claim 8.8 (Exact whole-exit decoding cannot weaken a trial)**.** *The optional alternative is polynomial under the same cap and accepts every fixed-allowance schedule.*

Proof strategy.

1. Use dynamic programming on endpoint-coordinate rectangles, and prove by induction on the realized classes that the recurrence gives the minimum continuation height.
2. Show that unhit interval demands are exactly those contained in the current region. This makes each rectangle decoder independent of queries already resolved outside it.
3. Compare the minimizing decoder with the explicit endpoint code used by a fixed-allowance schedule; its route bounds imply that the same schedule passes the more precise height test.
4. Count the rectangles and threshold transitions, then include the rank-subset factor 3^L.

Proof. After the legal cut, each class has one realized endpoint pair; let M be the number of hull candidate positions. For a coordinate rectangle U, give a one-class leaf value h_A and compute

$$E(U) = \min_{\text{proper supplied coordinate thresholds } \ell} \big(1 + \max\{E(U \cap \ell^-), E(U \cap \ell^+)\}\big),$$

ignoring empty outcomes. For an empty rectangle no decoder is needed. A rectangle containing only class A has value h_A; otherwise two distinct endpoint pairs differ in some coordinate, and a legal hull comparison between those coordinate values gives a proper threshold. Each proper threshold sends its realized pairs to strictly smaller nonempty sets. Induction on the number of realized pairs shows exactness: every permissible endpoint tree has such a first threshold and costs one plus the larger continuation height, while attaching the two minimizing continuations attains that value. A rectangle is specified by two boundaries in each coordinate, giving $O(M^4)$ rectangles, with $O(M)$ thresholds to try in each. The leaf values include the stored class continuations. A pinned endpoint is the same recurrence in one coordinate.

At state $([l, r], T)$, Claim 8.1 identifies the unhit demands as precisely the original intervals contained in $[l, r]$. For candidate block p in either orientation, compute the whole decoder heights $E_1(l, r, p)$ and $E_2(l, r, p)$ for its first-positive and second-first-positive cells. These cells depend only on the region and block: an outside query cannot hit a contained interval, and a crossing interval was already hit by an ancestor. At $k = \min T$, replace the fixed-deadline check by

$$2(k-1) + 1 + E_1(l, r, p) \le H, \qquad 2k + E_2(l, r, p) \le H,$$

omitting empty cells. Keep the rank partitions, circular first-block enumeration, and negative-route cap. These prefixes conservatively include cached endpoints and the origin. For a cell containing classes A, the explicit endpoint code with their stored continuations is a feasible rectangle decoder tree. Its whole height is at most $\max_A(e_A^{(4)} + h_A)$ on a first-positive cell and at most $\max_A(e_A^{(2)} + h_A)$ on a pinned cell. Therefore the minimizing values E_1, E_2 are no larger than these bounds. If each fixed-allowance class route passes at block rank k, then respectively $2(k-1) + 1 + E_1 \le H$ and $2k + E_2 \le H$. Keeping the same candidate schedule and negative-route cap thus passes the whole-exit trial as well. Precomputation and the rank-subset transitions are polynomial times 3^L. □

□

8.3 Parameter calibration

Parameter calibration.

The balanced-query and flat-decoder credits require

$$\gamma(\delta) = \max\left\{\frac{1}{\log_2(1/(1-\delta))},\ \frac{5}{\log_2(1/\delta)}\right\}.$$

The first term decreases with δ, while the second increases, so their maximum is minimized where they meet. Equating them gives $\delta = (1-\delta)^5$. Let δ_0 be the unique root of this equation in $(0, 1/3)$, and set

$$\gamma_0 = \frac{1}{\log_2(1/(1-\delta_0))} = \frac{5}{\log_2(1/\delta_0)}.$$

In the algorithm, take $\delta = \delta_0$ and $\gamma = \gamma_0$. The root is unique because $x - (1-x)^5$ is strictly increasing on this interval: its first term increases and its second decreases. Its values at 0 and $1/3$ are -1 and $49/243$, respectively, so it crosses zero once. Thus $\gamma_0 \log_2(1/(1-\delta_0)) = 1$ and $\gamma_0 \log_2(1/\delta_0) = 5$, so both count-credit conditions hold at equality. To bound the resulting factor, put $a = 245122333/10^9$ and $b = 245122334/10^9$. Exact substitution gives $a - (1-a)^5 < 0 < b - (1-b)^5$, so $a < \delta_0 < b$. The first function in $\gamma(\delta)$ is decreasing and the second is increasing; hence

$$\gamma_0 \le \frac{1}{\log_2(1/(1-a))}, \qquad \gamma_0 \le \frac{5}{\log_2(1/b)}.$$

Applying the rational logarithm enclosure (17) with $N = 100$ to these endpoint expressions certifies that both upper bounds are less than $G = 2464966/10^6$. More explicitly, if $U_{100}(2)$ is the upper endpoint for $\ln 2$ in that enclosure, the exact rational comparisons are

$$\frac{L_{100}(1/(1-a))}{U_{100}(2)} > \frac{10^6}{2464966}, \qquad \frac{L_{100}(1/b)}{U_{100}(2)} > \frac{5 \cdot 10^6}{2464966}.$$

Each numerator is the lower endpoint for its logarithm, so these comparisons imply the claimed upper bounds on γ_0. Also $\delta_0 < b < 1/4$, so $1/(1-\delta_0) < 4/3 < \sqrt{2}$ (since $16/9 < 2$) and therefore $\gamma_0 > 2$. Thus

$$\boxed{2 + \gamma_0 < 4.464966.}$$

The same calibration gives $\alpha = \gamma/(\gamma-1)$. Since $q < \log_2 n + 1$, the rank-mask contributions obey

$$3^L < 3^{\alpha+1} n^{\alpha \log_2 3}, \qquad 2^L < 2^{\alpha+1} n^{\alpha},$$

Thus each is polynomial in n, as required for the scheduling component. The full algorithm's exponent also includes its other stages. The ceiling $q = \lceil \log_2 n \rceil$ is found by comparing integer powers of two with n.

9 Stronger algorithms for small caps

Consider instances with a distinguished hypothesis p and orient each supplied query so that its *cap* is the answer side missing p. Let P be the other N hypotheses. These assumptions concern the supplied queries themselves. At a recursive flat state, the common external witness serves as an analysis aid, while p remains the distinguished hypothesis. The restricted algorithms below clarify which small-cap constructions can support approximation hardness.

9.1 Exact maximum depth with bounded caps

Theorem 9.1. *If every cap has size at most a fixed integer b, maximum depth is exactly solvable in polynomial time. A running-time bound is $m^{b-1}2^{O(b^2)}\,\mathrm{poly}(n, m, L)$, where L is the input bit length.*

Proof strategy.

1. A minimum cap cover gives a lower bound K from the witness spine and a tree of height at most $K + b - 1$.
2. Only the last $b - 1$ witness queries need to be guessed. Their caps contain at most $b(b - 1)$ points, so guess their residual state and find the earlier cover exactly.
3. Exact local subset tables verify every suffix exit. A feasible optimal tree supplies one of these guesses, proving exactness.

9.1.1 Supporting propositions and the algorithm

Let K be the minimum number of supplied caps covering P. For a target height $D \in [K, K + b - 1]$, let $d_0 = \max\{0, D - b + 1\}$. A suffix guess consists of at most $D - d_0$ ordered witness queries and the set $R \subseteq P$ alive immediately before them.

The witness path must eliminate every other hypothesis, while each positive exit contains only a few hypotheses. Comparing these two requirements confines the optimum to a short interval.

Proposition 9.2 (The optimum lies in a short height interval)**.** *A minimum supplied-halfplane cover is computable in polynomial time, and the optimal height lies in $[K, K + b - 1]$.*

Early exits have enough remaining depth for any proper local code. The only delicate exits occur near the end of the witness path; preserving their guessed residual set makes the earlier cover independent.

Proposition 9.3 (Each suffix guess is exactly checkable)**.** *Guess R inside the suffix caps' union. A prefix cover of $P \setminus R$ using caps disjoint from R, followed by the suffix, gives height at most D if its cover size is at most d_0 and every suffix first-hit group A_j satisfies $j + h(A_j) \le D - d_0$, where j is its first-hit position within the suffix and h is exact local identification height. These conditions are decidable in polynomial time for fixed b.*

Exact checking is useful only if an optimum appears among the guesses. Cutting an optimal witness path at the prescribed depth provides precisely the small suffix and residual set being enumerated.

Proposition 9.4 (Enumeration is complete and polynomial)**.** *Every height-D tree provides an accepted suffix guess. There are $O(m^{b-1})$ ordered suffixes and $2^{O(b^2)}$ candidate residual sets per suffix. For $b = 1$, a minimum cover itself is exact.*

Algorithm 16: Exact maximum depth with a distinguished miss hypothesis and bounded caps

Input: Distinguished hypothesis p; other hypotheses P; supplied caps of size at most fixed b.
Output: An exact minimum-height supplied-query tree.

```
if P = ∅ then
    return the zero-query tree
Compute the minimum supplied-halfplane cover size K of P
for D = K, ..., K + b − 1 do
    d_0 ← max{0, D − b + 1}
    foreach ordered supplied-query suffix of length q ≤ D − d_0 do
        foreach R contained in the union of its caps do
            if the suffix does not cover R then
                continue
            Find a minimum cover Q of P \ R using only caps disjoint from R
            if |Q| > d_0 or the cover is infeasible then
                continue
            Compute the suffix's first-hit groups A_j on R ∪ {p}
            if some j + c_W(ExactTree(A_j, W)) > D − d_0 then
                continue
            Execute Q followed by the suffix on the all-miss spine
            Attach exact local supplied-query trees to all positive first-hit groups
            return this tree
```

The theorem's height interval means the loop needs only the listed values of D. For each value, the algorithm searches suffixes and residual sets; since a suffix has at most $b-1$ queries and each cap contains at most b points, its union contains at most $b(b-1)$ points. An empty suffix therefore permits only $R = \varnothing$. The prefix step is a minimum supplied-halfplane cover, computed by the polynomial algorithm cited in Theorem 9.1; when the instance has the special hull-interval representation, its dedicated interval oracle performs the same step.

Proof of Theorem 9.1. For $P = \varnothing$, return the zero-query tree. Otherwise, Proposition 9.2 limits the target heights. Proposition 9.3 proves that every accepted guess constructs a legal tree, and Proposition 9.4 ensures acceptance at the optimum. Returning the first accepted height is exact; the stated enumeration and polynomial cover and local optimizations give the running time. □

9.1.2 Technical proofs of the height propositions

Proof of Proposition 9.2. Let K be the minimum number of supplied caps covering P. Minimum planar halfplane cover is exactly solvable in polynomial time [14]. Separability ensures that the cover exists. The path identifying p in any tree must cover P, so its height is at least K. Conversely, query a minimum cover along the all-miss spine and identify each positive hit group locally. A hit group has at most b members and therefore admits a proper identification tree of height at most $b-1$. This gives height at most $K+b-1$. Only these b target heights need to be tested.

□

Proof of Proposition 9.3. For a target $D \in [K, K+b-1]$, put $d_0 = \max\{0, D-b+1\}$. Guess an ordered suffix of $q \le D-d_0 \le b-1$ supplied queries on the witness spine. Guess also the set $R \subseteq P$ remaining just before that suffix. Require that the suffix caps cover R; thus R is a subset of their

union and has size at most $b(b-1)$. For each guess, compute a minimum cap cover of $P \setminus R$ using only supplied caps disjoint from R, and require size at most d_0. An empty suffix permits $R = \varnothing$.

The suffix must fit on both kinds of path: a positive exit needs its local code, and the all-miss route must also terminate within the budget. Checking both prevents a hidden witness-path overrun.

Claim 9.1 (Local suffix tests include the witness leaf)**.** *The exact checks verify every positive exit and the full miss route.*

Proof. Execute the guessed suffix on $R \cup \{p\}$. For each nonempty first-hit group A_j at position j, compute its exact local identification height $h(A_j)$ by subset Bellman DP on at most b hypotheses. Require $j + h(A_j) \le D - d_0$ and $q \le D - d_0$. These tests verify the entire suffix tree, including the witness leaf. The prefix cover takes at most d_0 queries, preserves R on its all-miss route, and its positive exits cost at most $b-1$ additional queries. Thus its exits have height at most $d_0 + b - 1 = D$ when $d_0 > 0$; if $d_0 = 0$, the prefix is empty. The equality here uses the positive branch of $d_0 = \max\{0, D - b + 1\}$; no such equality is needed when the prefix has length zero. The suffix starts no later than depth d_0, so an accepted guess constructs a tree of height at most D. □

□

Proof of Proposition 9.4. For completeness, take an identifying tree of height at most D. If its witness path ends before depth d_0, use an empty suffix and $R = \varnothing$; its witness queries cover P within d_0. Otherwise take the remaining witness queries after depth d_0 as the suffix, and the actual nonwitness state there as R. The prefix caps are disjoint from R and cover $P \setminus R$. The suffix has at most $b-1$ queries and covers R; its positive subtrees satisfy the exact local-height tests. This guess is accepted. There are $O(m^{b-1})$ suffix descriptions and $2^{O(b^2)}$ candidate subsets per suffix, and all remaining computations are polynomial for fixed b. For $b = 1$, a minimum cover itself is exact. □

9.2 An EPTAS for weighted singleton and pair caps

Assume every cap has size one or two, all singleton caps for P are supplied, and the graph G of two-point caps is a disjoint union of paths and cycles. The latter condition follows for planar instances with all singleton caps: every nonwitness point is a convex-hull vertex, and a two-point cap joins consecutive vertices. Let the positive raw weights be w_0 for p and w_v for $v \in P$, and put $\Omega = w_0 + \sum_v w_v$.

For a matching M, replace each matched pair by a group of mass $w_u + w_v$ and leave unmatched vertices as singleton groups. Let $J(M)$ be the group multiset, $k = N - |M|$, and $P(M) = \sum_{uv \in M}(w_u + w_v)$. With group masses $a_1 \ge \cdots \ge a_k$, write $C(J) = \sum_j j a_j$. The matching cost is

$$G(M) = w_0 k + C(J(M)) + P(M). \tag{28}$$

Theorem 9.5. *For this weighted singleton/pair path-and-cycle class, expected depth has a deterministic $(1+\varepsilon)$ approximation in time $2^{O(\varepsilon^{-3})}\,\mathrm{poly}(n, L, \log(1/\varepsilon))$ for $0 < \varepsilon \le 1$. The polynomial exponent is independent of ε; this is an EPTAS.*

Proof strategy.

1. Delete one residue class of path and cycle edges. Averaging finds a near-optimal matching on bounded-size components.
2. Enumerate components containing the heaviest vertices. Their complement has little mass, so only light components need rounding.
3. Express the exact matching cost as a convex quadratic program on component configurations. Its fractional optimum lower-bounds the best integral invoice; the computed feasible solution has only an additive numerical optimization error.
4. Independent configuration rounding adds only a controlled variance term. Conditional expectations make it deterministic.

9.2.1 Supporting propositions and the algorithm

Work with raw weights and put $F^* = \min_M G(M) = \Omega \operatorname{OPT}_E$. The graph, weights, group counts, and count functions below describe the instance. The tuning controls are integers $K \geq 1$, $H \geq 0$ and an additive accuracy $0 < \eta \leq 1$, chosen only in the final calibration. Put $B = 2K$, the resulting component-size bound. Take the H heaviest nonwitness vertices, or all of them if fewer than H exist, and let Ω_{tail} be the mass of the others.

Before approximating matchings, we need to justify that they describe the full adaptive problem. The witness spine supplies this link; the supplied singleton queries realize every resulting matching.

Proposition 9.6 (The matching cost formula is exact)**.** *Minimizing* (28) *is exactly the minimum raw weighted depth, including arbitrary adaptive trees. Every matching is executable with supplied singleton and pair queries.*

Two reductions make the remaining optimization manageable. Edge deletion bounds component size at a small relative cost, while fixing heavy components leaves little mass exposed to rounding.

Proposition 9.7 (Residue deletion and heavy enumeration)**.** *Deleting edge numbers in one residue modulo K leaves components of at most B vertices. Some residue has a surviving matching of cost at most $(1 + 1/K)F^*$. Moreover, $\Omega_{\text{tail}} \leq 2F^*/(H + 1)$, and all matching choices on components meeting a heavy vertex can be enumerated in at most 2^{BH} trials.*

For a fixed heavy trial, enumerate the matching configurations c of each remaining, light component i. Let k_{ic} be its group count, P_{ic} its matched-pair mass, and $z_{ic}(t)$ its number of groups of mass at least t. Let $Z_f(t), k_f, P_f$ be the corresponding totals of the fixed components. Fractional configuration weights satisfy $x_{ic} \geq 0$ and $\sum_c x_{ic} = 1$. Set $\mu(t) = Z_f(t) + \sum_{ic} x_{ic} z_{ic}(t)$ and minimize

$$Q(x) = w_0 \left(k_f + \sum_{ic} x_{ic} k_{ic} \right) + P_f + \sum_{ic} x_{ic} P_{ic} + \frac{1}{2} \int_0^\infty (\mu(t)^2 + \mu(t))\, dt. \tag{29}$$

Group order couples choices made in different components. Counting groups above each mass threshold captures this coupling in one quadratic objective, without guessing the final order.

Proposition 9.8 (An exact convex quadratic model)**.** *The displayed objective equals the matching cost on integral configurations. Its minimum over the product of configuration simplices lower-bounds the best integral invoice. It is a rational convex quadratic program with $N2^{O(B)}$ variables and $O(N)$ mass breakpoints. A feasible rational solution within raw additive error $\eta\Omega$ is computable with polynomial bit complexity and time polynomial in the model size, L, and $\log(1/\eta)$.*

Rounding combines the component choices into one matching. Only the light components fluctuate, and their mass bounds the extra cost of turning the relaxation into a tree.

Proposition 9.9 (Deterministic rounding costs only light mass)**.** *From a feasible rational x, conditional expectations produce a matching of cost at most $Q(x) + (B/2)\Omega_{\text{tail}}$ in polynomial time in the model size and bit length. If every trial uses a feasible rational x within $\eta\Omega$ of its fractional optimum, the cheapest tree over all residues and heavy trials has cost at most*

$$\left(1 + \frac{1}{K} + \eta + \frac{B}{H+1}\right) F^*.$$

Algorithm 17: Expected-depth EPTAS for weighted singleton/pair paths and cycles

Input: Distinguished miss hypothesis, supplied singleton tests and pair graph G; positive raw weights; $0 < \varepsilon \leq 1$.
Output: A deterministic $(1+\varepsilon)$ approximation.

```
Use the controls K, H, η from Section 9.2.3; set B ← 2K
if n = 1 then
    return the zero-query tree
Find the H heaviest nonwitness vertices (or all of them if fewer exist)
for r = 0, ..., K − 1 do
    Delete edges numbered r modulo K on each path or cycle
    Enumerate each remaining component's matching configurations
    foreach joint configuration on components meeting the heavy vertices do
        Fix these heavy configurations
        Solve the light-component convex program (29) to raw additive accuracy ηΩ
        foreach unfixed light component do
            Fix a configuration minimizing the conditional expected invoice, using rational first and
              second count moments
        Form the matched-pair and singleton groups; sort them by decreasing mass
        Query the supplied group tests on the miss spine, using one supplied singleton test to
          identify each pair hit
        Retain the resulting tree and its exact expected cost
return the cheapest retained tree
```

Deleting one edge residue bounds the component size, and enumerating the heavy components leaves a light-instance relaxation. The conditional-expectation step then fixes each remaining component while preserving the invoice bound, so every retained candidate is an actual matching and hence yields a legal tree. Propositions 9.7–9.9 prove the component bound, heavy enumeration, convex relaxation, and rounding guarantee. Proposition 9.6 then converts the rounded matching into its exact tree cost. The fractional solution guides the rounding, while the returned tree comes from an actual matching.

Proof of Theorem 9.5. For $n = 1$, return the exact zero-query tree. For $n \geq 2$, choose K, H, η by Section 9.2.3. Proposition 9.7 provides a good residue and an enumerated heavy part of its near-optimal matching. Propositions 9.8 and 9.9 return cost at most $(1+\varepsilon)F^*$. Proposition 9.6 turns the matching into a legal supplied-query tree. Divide by Ω to obtain the stated expected-depth ratio. The calibration gives $BH = O(\varepsilon^{-3})$; the other stages have polynomial exponent independent of ε. □

9.2.2 Technical proofs of the EPTAS propositions

Proof of Proposition 9.6. Follow the witness in an arbitrary proper tree. Its first-hit groups are disjoint singletons or supplied pairs, so the pair groups form a matching M and the number of groups is $k = N - |M|$. A supplied pair query first hitting only one vertex can be replaced here by that vertex's supplied singleton test. The witness contributes $w_0 k$, group positions contribute $\sum_j j a_j$, and each pair exit needs exactly one further supplied singleton query, contributing its full pair mass. This is the displayed invoice before sorting.

For adjacent group masses $a < b$, putting b first reduces their contribution by $[ja + (j+1)b] - [jb + (j+1)a] = b - a > 0$. Thus nonincreasing order minimizes the position term, giving $C(J(M))$. Conversely, query any matching's disjoint pairs and unmatched singletons in that order, and attach one supplied singleton query at each pair exit. This attains $w_0 k + C(J(M)) + P(M)$, proving exactness for arbitrary trees.

□

Proof of Proposition 9.7. We first control the cost of simplifying the graph, then control the mass left for rounding. For graph simplification, compare each residue with the same optimal matching by splitting its deleted pairs.

Claim 9.2 (One residue loses at most a $1/K$ fraction)**.** *Splitting the removed matched edges leaves a legal near-optimal matching.*

Proof. Number edges consecutively on each path or cycle. For each common residue $r \in \{0, \dots, K-1\}$, delete edges numbered r modulo K. A path component between consecutive deleted edges has at most K vertices, as do its end components. On a cycle, all components except possibly the one crossing the numbering cut have the same bound. The crossing component joins two end portions, each with fewer than K edges, and has at most $2K = B$ vertices. If a residue deletes no edge of a cycle, that cycle has fewer than K edges, so is already smaller than B.

Take an optimal matching, with sorted group masses $a_1, \dots, a_k$. For a matched pair of weights $a \geq b$ at position j, put $R_j = w_0 + \sum_{l>j} a_l$. Its original weighted depth contribution is $(j+1)(a+b)$, including the local pair query. After replacing it by two consecutive singleton queries the contribution is $ja + (j+1)b$, a change of $-a$. Every later group and the witness are delayed by one query, a change of R_j. Thus the net change is exactly $R_j - a$.

For simultaneous splits, each earlier split delays all points of a later group by one regardless of whether that group is also split. The local change $-a$ is likewise independent of the group's shifted position. The net changes therefore add. Sorting the new singleton and pair groups can only improve the temporary schedule. Since the R_j are positive,

$$\sum_{j \text{ matched}} R_j \leq \sum_{j=1}^{k} R_j = k w_0 + \sum_{l=1}^{k} (l-1) a_l \leq k w_0 + C(J) \leq F^*.$$

The first inequality adds the remaining nonnegative R_j terms. For the equality, group l contributes to exactly $l-1$ earlier tails. Replacing $l-1$ by l gives the next inequality, and the last discards the nonnegative pair-continuation term from the exact invoice of Proposition 9.6. Each matched edge belongs to exactly one residue trial. Averaging the temporary schedules over the K residues gives cost at most

$$F^* + \frac{1}{K} \sum_{j \text{ matched}} (R_j - a) \leq F^* + \frac{F^*}{K}.$$

This inequality first discards each nonpositive local saving $-a$ and then applies the preceding bound on the summed tails. At least one trial meets this average bound, and sorting its groups produces a surviving matching of no larger cost.

□

A component can be small in vertex count yet carry a heavy vertex, raising the rounding cost. Enumerating the heavy part removes this risk, while the rank lower bound controls the remaining mass.

Claim 9.3 (Heavy components leave a low-mass rounding problem)**.** *The rank lower bound controls all unfixed vertex mass.*

Proof. Order individual nonwitness weights as $w_{(1)} \geq \cdots \geq w_{(N)}$. A first-hit group contains at most two vertices, so at most $2j$ vertices can leave by spine position j. Consequently the sorted list of first-hit positions is componentwise at least $\lceil 1/2 \rceil, \ldots, \lceil N/2 \rceil$. Pairing larger weights with smaller positions minimizes their weighted sum by adjacent exchanges. Ignoring local continuation and witness costs therefore gives

$$F^* \geq \sum_{i=1}^{N} \lceil i/2 \rceil w_{(i)}.$$

For every $i > H$, $\lceil i/2 \rceil \geq (H+1)/2$, hence

$$F^* \geq \sum_{i>H} \lceil i/2 \rceil w_{(i)} \geq \frac{H+1}{2} \sum_{i>H} w_{(i)} = \frac{H+1}{2} \Omega_{\mathrm{tail}}.$$

The first inequality drops the nonnegative contributions of the heaviest H vertices, and the second applies the rank lower bound uniformly to every remaining index. Thus the tail mass is bounded without assuming any lower bound on an individual weight. For each residue, enumerate all matching configurations in components containing one of the H heaviest vertices. There are at most H such components. A path or cycle on at most B vertices has at most B edges, so has at most 2^B edge subsets and hence at most that many matching configurations. The joint number of choices is at most $(2^B)^H = 2^{BH}$. This enumeration is independent of n in its exponent. The remaining components' total vertex mass is at most Ω_{tail}. If $H \geq N$, all components are fixed and there is no rounding stage.

□

□

Proof of Proposition 9.8. For sorted group masses $a_1 \geq \cdots \geq a_k$, the groups above a threshold occupy ranks 1 through $Z(t)$. Finite summation and integration therefore give

$$C(J) = \sum_{j=1}^{k} j a_j = \int_0^\infty \sum_{j : a_j \geq t} j \, dt = \int_0^\infty \frac{Z(t)(Z(t)+1)}{2} \, dt.$$

For the first integral, integrate the indicator of $t \leq a_j$ over $t \geq 0$ and sum over j. The second integral then sums the consecutive ranks $1, \ldots, Z(t)$. Consequently the formula prices all cross-component ordering effects exactly. On integral configurations, $\mu(t) = Z(t)$ and the linear terms are exactly the witness and pair-continuation terms of the invoice. Allowing fractional configurations enlarges the feasible set, so its minimum cannot exceed the integral minimum. The only breakpoints are singleton and available-pair masses, at most $N + |E(G)| = O(N)$ rational values. Include zero

among the ordered breakpoints τ_j. On each interval between consecutive breakpoints, $\mu(t)$ is a fixed affine function $\mu_j(x)$, so the integral is

$$\frac{1}{2}\sum_j(\tau_{j+1}-\tau_j)\big(\mu_j(x)^2+\mu_j(x)\big).$$

The coefficients are rational and the interval lengths are nonnegative; thus its quadratic part is a nonnegative sum of squares of affine functions and is convex. Each component has at most 2^B matching configurations and there are at most N components, giving $N2^{O(B)}$ variables over a product of simplices. After normalizing by Ω, solve it to additive accuracy η using polynomial-time convex quadratic optimization [15]. Equivalently the raw error is $\eta\Omega$. The simplex constraints have rational coefficients and a feasible integral point obtained by choosing one configuration per component. For any feasible fractional point, $0 \le \mu(t) \le N$ and $\int \mu(t)\,dt = \sum_v w_v$, since every configuration partitions its component's vertices. Its witness term is at most Nw_0, its pair-continuation term is at most $\sum_v w_v$, and

$$\frac{1}{2}\int(\mu(t)^2+\mu(t))\,dt \le \frac{N+1}{2}\int\mu(t)\,dt = \frac{N+1}{2}\sum_v w_v.$$

Here $\mu(t)^2 \le N\mu(t)$ follows pointwise from $0 \le \mu(t) \le N$, and the equality uses the invariant total group mass across configurations. Thus $0 \le Q(x) \le (N+3)\Omega$, giving a normalized objective bound $O(n)$. Together with bounded simplices and rational coefficients of polynomial bit length, the cited convex-QP algorithm provides the stated feasible rational approximation and bit complexity.

□

Proof of Proposition 9.9. We separate the rounding analysis from its deterministic implementation. The fractional objective records the mean counts; independent rounding changes only their quadratic contribution, which can be charged to light vertex mass.

Claim 9.4 (Only the quadratic count term incurs variance)**.** *The expected rounded cost is at most* $Q(x) + (B/2)\Omega_{\text{tail}}$.

Proof. Independently round each light component according to its fractional configuration distribution, and write $Z(t) = Z_f(t) + \sum_i X_i(t)$. Then $\mathbb{E}Z(t) = \mu(t)$. For each component $0 \le X_i(t) \le B$, so

$$\operatorname{Var} X_i(t) = \mathbb{E}[X_i(t)^2] - (\mathbb{E}X_i(t))^2 \le \mathbb{E}[X_i(t)^2] \le B\mathbb{E}X_i(t).$$

The first inequality removes a nonnegative square; the second uses $X_i(t)^2 \le BX_i(t)$ on $[0, B]$. Hence the variance bound depends on component size and expected mass, regardless of how many configurations the component has. Independence gives $\mathbb{E}[Z(t)^2] = \mu(t)^2 + \sum_i \operatorname{Var} X_i(t)$. The witness and pair-continuation terms are linear, so their expectations are exactly their terms in $Q(x)$. Applying the rank integral from Proposition 9.8 and then the variance bound yields

$$\begin{aligned}\mathbb{E}G(M) &= Q(x) + \frac{1}{2}\int_0^\infty \sum_i \operatorname{Var} X_i(t)\,dt \\ &\le Q(x) + \frac{B}{2}\sum_i \int_0^\infty \mathbb{E}X_i(t)\,dt.\end{aligned}$$

For every configuration, the integral of its count function is the sum of its group masses. These groups partition the component's vertices, so this sum is its fixed vertex mass, independent of the

configuration. The displayed sum of integrals is therefore the total light-component mass, at most Ω_{tail}. Hence

$$\mathbb{E}G(M) \le Q(x) + \frac{B}{2}\Omega_{\text{tail}} \overset{\text{Prop. 9.7}}{\le} Q(x) + \frac{B}{H+1}F^*,$$

using the tail bound proved above. □

An expected-cost bound still needs an individual matching that attains it. Fixing one component at a time preserves that bound and keeps all intermediate computations rational and explicit.

Claim 9.5 (Exact conditional expectations give a supplied tree)**.** *One component at a time can be fixed without increasing the expected invoice.*

Proof. Suppose some components have already been fixed. For the next component, the current conditional expected invoice is the weighted average of the conditional expected invoices obtained by fixing each configuration, with weights x_{ic} summing to one. At least one configuration of positive probability therefore has value no larger than the current expectation. Choose a minimum such value and repeat.

Compute each value exactly from the fixed counts and the remaining independent first and second count moments at the same $O(n)$ breakpoints. The identity for $\mathbb{E}[Z(t)^2]$ remains valid with the fixed counts incorporated into $Z_f(t)$. All interval lengths, probabilities, moments, and linear terms are rational, with polynomial bit length; the number of configurations and breakpoints is the model size. At the end no randomness remains, so the actual matching cost is at most the initial expectation. Sorting its final groups yields a supplied-query tree by Proposition 9.6.

□

For the good residue, enumeration contains the heavy part of an optimum surviving matching. The fractional optimum is no greater than that matching's invoice. In a nontrivial instance every hypothesis has depth at least one, so $F^* \ge \Omega$. For the good residue and heavy trial, the computed feasible point thus satisfies

$$Q(x) \overset{\text{Prop. 9.8}}{\le} Q^* + \eta\Omega \overset{\text{Prop. 9.7}}{\le} (1 + 1/K)F^* + \eta F^*.$$

Claim 9.2 bounds the surviving comparison invoice, while $F^* \ge \Omega$ converts the numerical error to a relative one. Adding Claim 9.4, with the tail estimate of Claim 9.3, gives cost at most

$$(1 + 1/K)F^* + \eta F^* + \frac{B}{H+1}F^* = \left(1 + \frac{1}{K} + \eta + \frac{B}{H+1}\right)F^*.$$

The algorithm takes the cheapest trial, so it meets this same bound. □

9.2.3 Calibration of the EPTAS controls

Parameter calibration.
For $0 < \varepsilon \le 1$, choose

$$K = \lceil 3/\varepsilon \rceil, \qquad B = 2K, \qquad H = \lceil 3B/\varepsilon \rceil, \qquad \eta = \varepsilon/3.$$

Then $1/K + \eta + B/(H+1) \le \varepsilon$. Also $B = O(\varepsilon^{-1})$ and $H = O(\varepsilon^{-2})$, so heavy enumeration costs $2^{O(\varepsilon^{-3})}$. Configuration enumeration, convex optimization, and conditional expectations keep a polynomial exponent independent of ε.

The ceiling choices give $K \geq 3/\varepsilon$ and $H + 1 \geq 3B/\varepsilon$, hence $1/K \leq \varepsilon/3$ and $B/(H+1) \leq \varepsilon/3$. Together with $\eta = \varepsilon/3$ this proves the calibrated ratio. For $0 < \varepsilon \leq 1$, $K = O(\varepsilon^{-1})$, so $B = O(\varepsilon^{-1})$ and $H = O(B/\varepsilon) = O(\varepsilon^{-2})$; consequently $BH = O(\varepsilon^{-3})$ as used in the enumeration bound.

All-singleton availability is essential both for the invoice's converse and for splitting removed pairs. The EPTAS includes the weighted pair-path hardness instances below, and its exponential dependence on $1/\varepsilon$ is compatible with strong NP-completeness. This subclass therefore cannot establish APX-hardness unless P = NP.

10 A PTAS for uniform shallow caps in convex position

The bounded-cap restriction also permits an approximation scheme for uniform expected depth when the hypotheses are in convex position. The construction first clears short gaps in their cyclic order. Every remaining query then acts inside one bounded-size block. A count-vector dynamic program retains all choices of how to interleave queries from these blocks. Uniform weights make blocks with the same local test family interchangeable.

Theorem 10.1. *Suppose $\mathcal{H} = P \cup \{p\}$ is in strictly convex position, p is a distinguished hypothesis, and every supplied side missing p contains at most a fixed number $b \geq 1$ of hypotheses. Under the uniform prior, expected depth admits a polynomial-time approximation scheme. Every query in the returned tree is supplied, and the result permits input families with no singleton caps.*

For $0 < \varepsilon \leq 1$, the returned cost is at most $(1+\varepsilon)\,\mathrm{OPT}_E$. For the block length ℓ chosen in the final calibration, put $d_{b,\ell} = 2^{(b+1)\ell}$. One running-time bound is $f(b,\ell)(n+1)^{d_{b,\ell}}\,\mathrm{poly}(n,m,L)$, where L is the input bit length and f depends only on b, ℓ. The polynomial exponent depends on b and ε; the guarantee is a PTAS.

Proof strategy.

1. Clear short gaps in the convex cyclic order using supplied caps containing marked points. A cap cannot reach two remaining blocks.
2. Describe each small block by its supplied interval masks. Equal point weights make blocks with the same type and live mask interchangeable, so only their multiplicities need to be stored.
3. Optimize all block interleavings by an exact count-vector DP, using exact supplied-query codes on positive branches.
4. Charge the clearing prefix against the witness-spine rank lower bound. Large enough blocks make that charge at most ε times the optimum.

10.1 Supporting propositions and the algorithm

Write $N = |P| = n - 1$, $G(T) = \sum_{h \in \mathcal{H}} d_T(h)$, and $\mathrm{OPT}_G = \min_T G(T) = n\,\mathrm{OPT}_E$. We use a positive integer block length $\ell \geq \max\{2b, b(b-1)\}$ and choose its value in the final calibration. When $N < \ell$, exact subset Bellman optimization solves the instance. Assume henceforth that $N \geq \ell$.

The purpose of clearing is to make every surviving query local. Short gaps suffice because a shallow cap cannot cross a fully cleared gap and still reach points on both sides.

Proposition 10.2 (Clearing gaps isolates small residual blocks)**.** *Partition the convex cyclic order into pieces of length ℓ and mark the first b points of each piece, or the whole piece if shorter. The number r of marked points satisfies*

$$r \leq b\lceil N/\ell \rceil \leq \frac{2bN}{\ell}. \tag{30}$$

A legal supplied-query prefix clears all marked points in at most r spine queries and has total raw contribution

$$G_{\text{prefix}} \le r(N+1) + b(b-1)r \le 2r(N+1). \tag{31}$$

Every remaining cap acts inside one residual block of size at most ℓ.

The global state records each block's available splits and live positions. Uniform weights then let us forget a block's identity.

Proposition 10.3 (Block types retain all continuation information)**.** *A residual block's type is its size and its family $\mathcal{F}_T$ of nonempty supplied interval masks. There are at most $2^{b\ell}$ types and at most $d_{b,\ell}$ pairs (T, R) of a type and a nonempty live mask. Under uniform weights, blocks with the same (T, R) can be exchanged without changing the optimum continuation cost. Each mask retains a realizing query for every physical block of its type.*

Let $c_{T,R}$ count blocks in local state (T, R), and put $W(c) = 1 + \sum_{T,R} c_{T,R}|R|$, including the distinguished hypothesis. For each type, use Algorithm 1 to compute the exact raw local identification cost $g_T(A)$ for every subset A using its supplied masks; $g_T(A) = 0$ for $|A| \le 1$. The positive branch has no distinguished witness. Let $e_{T,R}$ be a unit count vector, with $e_{T,\varnothing} = 0$.

The algorithm must still choose which block to query next. The count vector preserves these choices, while the local tables handle the positive exits that no longer interact with other blocks.

Proposition 10.4 (The residual count-vector recurrence is exact)**.** *The optimum residual raw cost is*

$$\begin{aligned} &F(0) = 0, \\ &F(c) = W(c) + \min_{\substack{c_{T,R}>0,\ q\in\mathcal{F}_T \\ A=R\cap q\neq\varnothing}} \left\{ g_T(A) + F\big(c - e_{T,R} + e_{T,R\setminus A}\big) \right\}. \end{aligned} \tag{32}$$

The DP has at most $(N+1)^{d_{b,\ell}}$ states, polynomial bit complexity, and a supplied-query reconstruction realizing its value. The recurrence includes every adaptive interleaving of the blocks.

Clearing buys exact residual optimization at an extra prefix cost. The witness path already incurs substantial rank cost when caps are small, so this prefix can be charged against the full optimum.

Proposition 10.5 (The clearing charge is small relative to the optimum)**.** *The exact residual optimum is at most OPT_G, and*

$$\mathrm{OPT}_G \ge \sum_{i=1}^{N} \lceil i/b \rceil \ge \frac{N(N+1)}{2b}. \tag{33}$$

The prefix followed by the exact residual tree therefore has $G_{\text{alg}} \le (1 + 8b^2/\ell)\,\mathrm{OPT}_G$.

Algorithm 18: Uniform expected-depth PTAS for bounded convex-position caps

Input: Uniform hypotheses $P \cup \{p\}$ in convex position; supplied caps missing p, each of size at most fixed b; $0 < \varepsilon \leq 1$.

Output: A supplied-query tree of expected cost at most $(1+\varepsilon)\,\mathrm{OPT}_E$.

$N \leftarrow |P|$; use ℓ from Section 10.3
if $N < \ell$ **then**
  return ExactTree$(P \cup \{p\}, \mathrm{E})$ with raw unit weights
Partition the convex cyclic order of P into pieces of length ℓ; mark their first $\min\{b, \text{piece size}\}$ points as D
$U \leftarrow P$; start an empty miss-spine prefix
while $D \cap U \neq \varnothing$ **do**
  Choose $x \in D \cap U$ and a supplied cap q containing x
  $A \leftarrow U \cap q$
  Append q to the prefix; attach a legal local code for A of height at most $|A| - 1$ on its hit side
  $U \leftarrow U \setminus A$; continue on the miss side
Intersect the unmarked pieces with U to obtain the residual blocks
Store each physical block's map from its masks to realizing supplied queries; group identical mask families as types T and compute every local cost $g_T(A)$ by ExactTree
Compute the initial count vector c of the full residual block masks
Evaluate (32), memoizing minimizing transitions
Backtrack from c: select a physical block in the chosen state, query that block's stored realizing line, and translate the local positive code using the same block's query map
Attach the reconstructed continuation to the prefix's miss leaf
return the resulting tree

Proof of Theorem 10.1. Choose ℓ by Section 10.3. If $N < \ell$, exact subset optimization takes time depending only on b, ε, apart from polynomial input processing. Otherwise, Proposition 10.2 constructs the legal prefix, and Propositions 10.3 and 10.4 give an exact supplied-query residual tree with the stated time bound. Proposition 10.5 and the calibration yield $G_{\mathrm{alg}} \leq (1+\varepsilon)\,\mathrm{OPT}_G$. Divide by n to obtain the expected-depth ratio. No step introduces an unsupplied singleton test. □

10.2 Technical proofs of the supporting propositions

Proof of Proposition 10.2. There are two separate obligations: show that the marked gaps isolate the surviving blocks, and clear those gaps using only supplied queries. We begin with the geometric separation property.

Claim 10.1 (Marked gaps prevent caps from crossing blocks)**.** *Every residual cap meets at most one unmarked block.*

Proof. Each cap is a cyclic interval in the convex order of P: a line crosses the boundary of its convex polygon at most twice. Partition that order into consecutive pieces of length ℓ, with a possibly shorter last piece. Mark the first b points of each piece, or the whole piece when its size is at most b. Let D be the marked set and $r = |D|$. There are $\lceil N/\ell \rceil$ pieces and each contributes at most b marked points. Since $N/\ell \geq 1$,

$$r \leq b\lceil N/\ell \rceil \leq b(N/\ell + 1) \leq 2bN/\ell,$$

which proves (30). The middle inequality uses $\lceil x \rceil \leq x + 1$, and the last uses $N/\ell \geq 1$ to absorb the additional piece. The unmarked portions are blocks of at most ℓ points. Between neighboring nonempty blocks, including across the cyclic wraparound, there are at least b marked points. A

cyclic cap of size at most b cannot meet two such blocks, because it would also contain an entire intervening gap and hence at least $b+2$ points.

□

Supplied caps containing the remaining marked points clear them legally; removing additional unmarked points can only shrink the residual blocks.

Claim 10.2 (Supplied queries clear the marked set with controlled cost)**.** *No private or arbitrary singleton query is required by the prefix.*

Proof. While a marked point x remains on the miss spine, query any supplied cap containing x. Such a cap exists because the supplied tests distinguish x from p. Identify its first-hit group by a proper local supplied-query tree, and continue on the miss side. Separation gives a local height at most $|A|-1 \le b-1$ for each hit group A: repeatedly query a line separating two remaining hypotheses. Each spine query clears a new marked point, so at most r are used. Additional unmarked points may leave through these exits. The residual portions of the blocks still have size at most ℓ, and Claim 10.1 therefore ensures that every residual cap still lies in one block: clearing extra points cannot create a new hit across a marked gap.

Each spine node queries at most $N+1$ live hypotheses, so at most r such nodes contribute $r(N+1)$ to the raw sum of depths. Its disjoint positive exits contain at most br points in total, each with at most $b-1$ additional local queries. Therefore

$$G_{\text{prefix}} \le r(N+1) + br(b-1).$$

Since $b(b-1) \le \ell \le N$, the second term is at most $rN \le r(N+1)$. This gives $G_{\text{prefix}} \le 2r(N+1)$, as required.

□

□

Proof of Proposition 10.3. Relabel the points of each residual block by their positions. Its type T is its size together with the family $\mathcal{F}_T$ of all distinct nonempty supplied-query masks on that block. No cap can wrap around a block's ends, since it would contain an entire marked gap. Deleting additional points preserves interval order. Thus every mask is an ordinary interval of at most b positions. For each possible left endpoint there are at most b possible interval lengths. Padding to ℓ positions therefore gives at most $b\ell$ masks, hence at most $2^{b\ell}$ subsets of masks that could form a family. The union of a family's masks is its full block, since every point is separated from p, so the family also determines the block size. A local state (T, R) records a nonempty live mask R in a block of type T. For each type there are at most $2^\ell - 1$ nonempty live masks, so the total number of local states is at most $2^{b\ell}(2^\ell - 1) \le 2^{(b+1)\ell} = d_{b,\ell}$, using $2^\ell - 1 \le 2^\ell$.

Two blocks of the same type have identical available splits after relabelling positions. Store a realizing supplied query separately for each physical block and mask, so every chosen split uses a line valid on that block. Each realizing query misses all other residual blocks. Uniform costs depend only on their live cardinalities, so exchanging blocks of the same (T, R) preserves every continuation cost. The count vector therefore retains all needed information.

□

Proof of Proposition 10.4. Exactness requires both directions: every adaptive tree must obey the recurrence, and each recurrence choice must have a legal supplied query realization. Locality supplies the common reason for both. Any query that is constant on the current live hypotheses can be

deleted: every hypothesis continues to the same child, and removing the query shortens each of their depths by one. Thus we may suppress such queries and analyze a nonconstant root query.

Claim 10.3 (Every nonconstant adaptive query gives one recurrence transition)**.** *A positive branch is local; the miss branch changes only one block state.*

Proof. By Proposition 10.2, a nonconstant query hits only one residual block. A query at state c contributes one unit of depth for each live hypothesis, hence raw cost $W(c)$. Its hit group lies in one block with local state (T, R) and equals $A = R \cap q \neq \varnothing$. The miss branch retains p and all other blocks, and changes this block to $R \setminus A$. The two branch costs are bounded below by $g_T(A)$ and $F(c - e_{T,R} + e_{T,R\setminus A})$ respectively. Thus every adaptive tree has cost at least the recurrence value, by induction on the number of live nonwitness points.

Conversely, for a minimizing transition choose a physical block in state (T, R), which exists because $c_{T,R} > 0$, and its stored supplied query realizing q. The query misses every other residual block and is proper because its hit group is nonempty and p is on the miss side. Attach the exact local code of cost $g_T(A)$ and the inductively reconstructed miss continuation. Their total cost is $W(c) + g_T(A) + F(c - e_{T,R} + e_{T,R\setminus A})$, attaining the recurrence. Its live nonwitness count decreases by $|A| \geq 1$, so the induction and the DP are acyclic. The choice of block at every transition remains unrestricted, covering all adaptive interleavings. When no live block remains, only p remains and $F(0) = 0$.

□

Once these transitions are exact, the remaining issue is their representation. Counting interchangeable block states gives the size bound, while backtracking retains the physical queries needed for one output tree.

Claim 10.4 (The compressed state space has the stated bit complexity)**.** *Only counts, supplied masks, and integral costs are needed.*

Proof. There are at most $d_{b,\ell}$ count coordinates, each between zero and N, so the enclosing count cube has at most $(N+1)^{d_{b,\ell}}$ states. For a coordinate with positive count, there are at most $b\ell$ supplied masks to try. Each local Bellman table concerns at most 2^ℓ subsets; these bounds depend only on b, ℓ. Constructing the type and physical-query maps requires polynomial input processing.

Every reachable state contains at most n actual hypotheses. A proper identifying tree for it has height at most $n - 1$, giving raw cost at most $n(n-1)$. All weights are one, so the DP costs are integers with $O(\log n)$ bits on reachable states. Storing a minimizing transition and a physical realizing query at each used state suffices for backtracking; no list of all adaptive trees is stored.

□

□

Proof of Proposition 10.5. Restrict an optimal full tree to the residual hypotheses, suppressing constant-answer nodes. Lemma 2.12 shows that this can only decrease depths; Proposition 10.4 therefore makes the exact residual cost at most OPT_G. For the lower bound, order the nonwitness hypotheses by their first-hit position on the witness spine. The first j queries can remove at most bj of them, so the ith hypothesis in this order has first-hit position at least $\lceil i/b \rceil$ and leaf depth at least that large. Ignoring the nonnegative witness and continuation costs yields

$$\mathrm{OPT}_G \geq \sum_{i=1}^{N} \lceil i/b \rceil \geq \sum_{i=1}^{N} i/b = \frac{N(N+1)}{2b}.$$

The second inequality uses $\lceil i/b \rceil \geq i/b$ term by term, and the equality sums the first N integers. Write G_{res} for the exact residual cost. The restriction argument gives $G_{\mathrm{res}} \leq \mathrm{OPT}_G$, while the prefix estimate gives $G_{\mathrm{alg}} \leq 2r(N+1) + G_{\mathrm{res}}$. Consequently

$$\begin{aligned}\frac{G_{\mathrm{alg}} - \mathrm{OPT}_G}{\mathrm{OPT}_G} &\overset{\text{Prop. 10.2}}{\leq} \frac{2r(N+1)}{\mathrm{OPT}_G} \\ &\leq \frac{2r(N+1)}{N(N+1)/(2b)} = \frac{4br}{N} \\ &\overset{\text{Eq. 30}}{\leq} \frac{4b}{N}\frac{2bN}{\ell} = \frac{8b^2}{\ell}.\end{aligned}$$

The first inequality uses residual optimality, the second the rank lower bound, and the last the marked-point count. Dividing by n proves the expected-depth guarantee. □

10.3 Calibration of the block length

Parameter calibration.
For $0 < \varepsilon \leq 1$, choose

$$\ell = \max\left\{2b,\ b(b-1),\ \left\lceil \frac{8b^2}{\varepsilon} \right\rceil \right\}.$$

The first two bounds ensure gap separation and the prefix estimate; the last gives $8b^2/\ell \leq \varepsilon$. For fixed b, ε, ℓ and $d_{b,\ell}$ are constants, yielding the claimed PTAS time.

The proof uses only the cyclic-interval property of the caps. In particular it also applies when the nonwitness points alone are in convex position. For fixed b, a constant-factor hardness reduction through uniform one-layer shallow-cap instances would contradict this PTAS unless P = NP. A construction based on bounded-size caps must therefore prove interactions beyond this cyclic-interval subclass to establish such a gap.

11 Randomized per-hypothesis depth and optimal mixing

The previous algorithms return one tree. For a fixed prior, expected depth averages that tree's depths over hypotheses; deterministic maximum depth protects the slowest hypothesis in that same tree. We now consider a separate extension: preprocessing returns a small collection of legal trees and probabilities for choosing among them. At the start of each complete identification run, sample *one* tree and execute it to identify the hidden hypothesis. Each run follows that tree from start to finish, and every sampled tree identifies every hypothesis correctly.

The hidden hypothesis is fixed before this random choice. Different trees may favor different hypotheses, so we ask that *each* hypothesis have small expected depth over the choice of tree. For a distribution μ over legal supplied-query trees, define

$$R(\mu) = \max_{h \in \mathcal{H}} \mathbb{E}_{T \sim \mu} d_T(h), \qquad R^* = \min_{\mu} R(\mu). \tag{34}$$

We call this the Las Vegas per-hypothesis depth objective: correctness is certain, while the query count depends on the random tree. For each fixed hypothesis, the objective first averages over trees and then takes the maximum across hypotheses. A hypothesis may have a long path in one sampled tree and still meet the bound when other trees give it shorter paths.

For fixed-prior expected cost and expected maximum depth, randomization leaves the optimum unchanged: each mixture averages a single-tree objective. Explicitly, for a fixed prior π, linearity gives

$$\min_{\mu} \sum_{h} \pi_h \mathbb{E}_\mu d_T(h) = \min_{T} \sum_{h} \pi_h d_T(h), \qquad \min_{\mu} \mathbb{E}_\mu \big[\max_{h} d_T(h)\big] = \min_{T} \max_{h} d_T(h).$$

Each mixture cost is a convex combination of the corresponding single-tree costs, so it is at least their minimum; choosing a minimizing tree with probability one attains equality in both cases. The different order of expectation and maximum in (34) is what makes mixing useful here.

A strict planar separation. Take the three hypotheses $h_t = (t, t^2)$, $t = 0, 1, 2$, and supply the three lines

$$4y - 8t_0 x + 4t_0^2 - 1 = 0, \qquad t_0 = 0, 1, 2.$$

Each line's negative side contains only its own point: its expression there is -1, and its value at another point is at least 3. Asking a hypothesis's singleton line first identifies it in one query; the other two hypotheses require a second query. Thus choosing the first line trades one hypothesis's depth against the others:

hypothesis isolated by the first line	h_0	h_1	h_2
h_0	1	2	2
h_1	2	1	2
h_2	2	2	1
uniform mixture's expected depth	5/3	5/3	5/3

These are all possible proper depth vectors. Every individual tree has maximum depth 2, but sampling the favored hypothesis uniformly gives every fixed hypothesis expected depth 5/3. Every tree has uniform-prior expected depth at least 5/3, so $R^* = 5/3$ exactly. All coordinates and line coefficients are integers, and every hypothesis lies strictly off the supplied lines. The gain comes from distributing the short paths across runs while each run follows one complete path.

11.1 One algorithm and its supporting propositions

Let $n = |\mathcal{H}| \geq 2$ and $L = n - 1$. Contracting queries constant on their current state makes every legal identification tree proper: at each internal node, its query separates the hypotheses reaching that node into two nonempty sets. Every such tree has n leaves and depths at most L. Write $\mathcal{T}$ for the finite family of these trees and

$$c_\pi(T) = \sum_{h \in \mathcal{H}} \pi_h d_T(h), \qquad \mathrm{OPT}_E(\pi) = \min_{T \in \mathcal{T}} c_\pi(T).$$

We use the full family $\mathcal{T}$ to define the comparison optimum, while the algorithm stores the trees it constructs. A comparison prior may assign zero mass to some hypotheses; every tree in $\mathcal{T}$ still identifies all of $\mathcal{H}$. Write $\Delta(\mathcal{H}) = \{\pi : \pi_h \geq 0,\ \sum_{h \in \mathcal{H}} \pi_h = 1\}$ for the probability simplex.

The connection to the previous sections is constructive. An expected-depth algorithm can build a tree favoring a chosen prior. By repeatedly emphasizing hypotheses that earlier trees treated slowly, we use that algorithm to build a collection that protects all hypotheses. We represent the available expected-depth algorithm by an oracle $\mathcal{A}_E$ with a fixed rational guarantee $\alpha \geq 1$:

$$c_\pi(\mathcal{A}_E(\pi)) \leq \alpha\, \mathrm{OPT}_E(\pi) \quad \text{for every strictly positive rational prior } \pi. \tag{35}$$

The polynomial or quasipolynomial expected-depth construction supplies this oracle. A rational learning rate $0 < \eta \le 1$ and a round count K will be calibrated at the end for the desired tolerance.

Initialize equal positive weights. In each round normalize them to a prior, construct its oracle tree, and increase a hypothesis's weight according to its depth in that tree. This directs later trees toward hypotheses that earlier trees identify slowly. In the three-point example, after a tree with depths $(1, 2, 2)$ the last two hypotheses receive larger multipliers, encouraging later trees to give them the short path. Any tree meeting the oracle's weighted-cost guarantee yields the bound, regardless of which hypothesis it favors. If $\eta = p/q$ in lowest terms, use the integer update

$$w_h \leftarrow w_h\big(qL + p\, d_{T_t}(h)\big). \tag{36}$$

Its common factor qL cancels on normalization. This is the usual multiplicative-weights update: hypotheses that are deeper in the current tree receive larger weight and therefore receive more emphasis in later oracle calls. The update and its game interpretation use the multiplicative-weights framework of Freund and Schapire and of Arora, Hazan, and Kale [17, 16]. Here the losses are hypothesis depths scaled by L, and each oracle tree becomes a column of the restricted mixture LP. The next proposition derives the regret bound for this particular choice of losses.

The weight updates decide which trees to construct. A second step decides how often to use each constructed tree. After constructing $T_1, \dots, T_K$, optimize their actual mixture by the *restricted master LP*

$$\begin{aligned} \text{minimize} \quad & r, \\ \text{subject to} \quad & \sum_{t=1}^{K} \mu_t d_{T_t}(h) \le r \qquad (h \in \mathcal{H}), \\ & \sum_{t=1}^{K} \mu_t = 1, \quad \mu_t \ge 0 \ (1 \le t \le K), \quad r \ge 0. \end{aligned} \tag{37}$$

Each inequality is the expected depth of one fixed hypothesis, and r is their common upper bound. The LP has one variable for each constructed tree and represents the full family $\mathcal{T}$ implicitly. Retain an optimal basic mixture, which will use at most n trees. For each identification run, sample one of those trees from its exact rational probabilities and execute it. The proof first bounds the uniform mixture of all K generated trees. Since that mixture is a feasible LP solution, optimization preserves its bound and may remove most stored trees.

Algorithm 19: LP-compressed Las Vegas per-hypothesis identification

Input: Identifiable $\mathcal{H}$, supplied lines $\mathcal{L}$; expected-depth oracle $\mathcal{A}_E$ of fixed rational factor α; rational tolerance $0 < \varepsilon \le 1$.

Output: A rational mixture of at most $|\mathcal{H}|$ legal identifying trees.

if $|\mathcal{H}| = 1$ **then**
  return its leaf with probability one
Set $n = |\mathcal{H}|$, $L = n - 1$, and choose η, K by (40); write $\eta = p/q$ in lowest terms
Set $w_h = 1$ for every $h \in \mathcal{H}$
for $t = 1, \dots, K$ **do**
  Set $\pi_{h,t} = w_h / \sum_g w_g$
  Construct $T_t = \mathcal{A}_E(\pi_t)$ and contract constant queries
  Record every depth $d_{T_t}(h)$
  foreach $h \in \mathcal{H}$ **do**
    Set $w_h \leftarrow w_h(qL + p\, d_{T_t}(h))$
Solve (37), retaining an optimal basic solution (μ, r)
Discard trees whose probability is zero
return the retained rational mixture; sample it exactly and execute the selected tree on each identification run

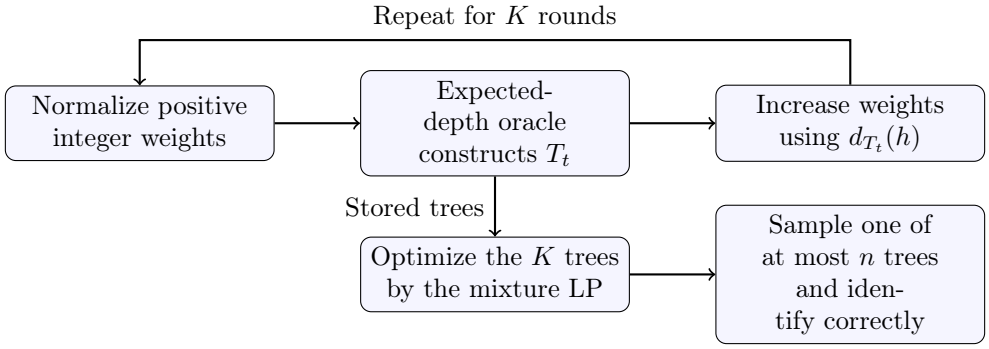


Figure 7: Preprocessing uses the oracle to build trees and the master LP to choose their probabilities. Each later identification run samples one complete tree and follows it. Weight updates and the LP run during preprocessing, before the hidden hypothesis is queried.

To compare this construction with the best possible randomized strategy, we first identify its benchmark. The hardest prior is the dual of the tree-mixture problem: even an ideal mixture must pay the optimal expected cost of that prior. Since the dual benchmark is an expected-depth optimum for some prior, an expected-depth approximation oracle can bound its value in every round.

Proposition 11.1 (The comparison game and its entropy lower bound)**.** *The optimum in* (34) *also covers adaptive zero-error randomized algorithms that terminate almost surely on every hypothesis. It satisfies*

$$R^* = \max_{\pi \in \Delta(\mathcal{H})} \mathrm{OPT}_E(\pi) \ge \log_2 n. \tag{38}$$

The dual maximum ranges over the full simplex and may assign zero mass; Algorithm 19 supplies strictly positive priors to the expected-depth oracle.

The benchmark bounds the oracle's cost on every round's prior. The regret proposition turns these changing weighted averages into a bound for *each* hypothesis: a repeatedly slow hypothesis

gains influence until the oracle's weighted cost reflects its depth. The resulting average-depth bound is for the uniform distribution over generated trees. Since the restricted master LP also considers this mixture, its optimized maximum per-hypothesis cost is no greater.

Proposition 11.2 (Depth-scaled multiplicative weights)**.** *For any sequence of proper oracle trees, let π_t be the normalized weights before round t. The uniform mixture of the generated trees satisfies*

$$R(\mu^{\mathrm{unif}}) \le (1+\eta)\frac{1}{K}\sum_{t=1}^{K} c_{\pi_t}(T_t) + \frac{2L\ln n}{\eta K}. \tag{39}$$

The uniform mixture in the regret bound may contain many trees. The master LP keeps its guarantee while producing a concise strategy. The next proposition also ensures that the repeated priors and the final random choice can be implemented exactly.

Proposition 11.3 (Exact rational construction and sparse optimal mixing)**.** *Every iterative prior is strictly positive and rational, with encoding length polynomial in n, K and the encoding length of η. The LP (37) is polynomial in size and has an optimal basic solution supported on at most n trees. Its value is at most $R(\mu^{\mathrm{unif}})$. Its probabilities have polynomial encoding length and admit exact sampling with an expected polynomial number of fair random bits. Every sampled query run is correct and uses at most $n-1$ supplied tests.*

Proof strategy.

1. View hypotheses and legal trees as the two sides of a finite game. Its hardest prior lower-bounds every randomized strategy.
2. Increase the weight of hypotheses with large current depths. The potential comparison bounds each hypothesis's average depth by the oracle's average weighted cost.
3. Optimize the generated mixture by a small LP. The entropy lower bound makes the regret term a multiplicative tolerance; a basic solution leaves at most n trees to sample.

Together, these statements transfer the oracle's approximation coefficient to the new objective, with a controllable tolerance. The extra work constructs and mixes trees; the query procedures inside each tree remain those of the expected-depth oracle.

Theorem 11.4 (Transfer to randomized per-hypothesis depth)**.** *For every rational $0 < \varepsilon \le 1$, Algorithm 19, with the calibration at the end of this section, satisfies*

$$R(\mu) < (\alpha+\varepsilon)R^* \qquad (n \ge 2).$$

It makes $O(n/\varepsilon^2)$ calls to the expected-depth oracle, uses only positive rational priors, and returns a mixture of at most n legal trees. If the oracle is polynomial-time in its rational input, preprocessing is polynomial in the input size and $1/\varepsilon$. For a singleton, return its leaf with zero queries.

Proof. Proposition 11.1 and (35) give $c_{\pi_t}(T_t) \le \alpha R^*$ in every round. Apply Proposition 11.2 and use $\ln n \le (\ln 2)R^* < R^*$ to obtain

$$R(\mu^{\mathrm{unif}}) < \left[(1+\eta)\alpha + \frac{2L}{\eta K}\right] R^*.$$

The final calibration makes the bracket at most $\alpha+\varepsilon$. Proposition 11.3 transfers this bound to the optimized mixture and supplies positivity, legality, and bit complexity. The chosen K gives the stated number of calls. □

11.2 Proofs of the supporting propositions

11.2.1 The hardest prior is the dual comparison object

The game interpretation rests on two facts. First, fixing the random seed turns the policy into a proper tree, and contracting constant queries can only shorten its paths. Averaging over seeds therefore gives a tree mixture whose expected depth on each hypothesis is at most the algorithm's. Second, the mixture optimum equals the expected-depth cost of its hardest prior. We prove these in order, then use the resulting lower bound to absorb the weight-update error.

Claim 11.1 (Random seeds reduce to a finite proper-tree mixture)**.** *Any zero-error randomized algorithm terminating almost surely on each hypothesis induces a mixture over $\mathcal{T}$ whose expected depth on each hypothesis is at most that of the algorithm.*

Proof. Condition on a complete random seed. Since $\mathcal{H}$ is finite, outside a probability-zero union of events the resulting deterministic policy terminates correctly on every hypothesis. The union of its n finite paths is a finite identification tree. Delete unreachable branches and contract queries constant on their current states. Every retained query splits a state properly, so the resulting tree has n leaves, $n-1$ internal nodes, and maximum depth at most L. The finite supplied query family admits only finitely many such labeled trees. Pushing the seed distribution onto these trees gives the required mixture. Conversely, sampling any such mixture gives a legal zero-error strategy. □

After fixing the seed, every strategy is a proper legal tree. The finite family of these trees defines a mixture LP for the analysis; its dual assigns mass to hypotheses, giving precisely the prior needed by the expected-depth oracle. The algorithm constructs a polynomial-size subset of this finite family, then optimizes the mixture over those trees.

Claim 11.2 (Finite LP duality identifies the hardest prior)**.** *The finite-tree mixture optimum equals* $\max_{\pi\in\Delta(\mathcal{H})}\min_{T\in\mathcal{T}} c_\pi(T)$.

Proof strategy.
We derive the dual by assigning one multiplier to each hypothesis constraint and one to mixture normalization. The Lagrangian gives the dual constraints directly; weak duality is then explicit, and finite LP strong duality completes the equality.

Proof. Write the full mixture LP with one variable μ_T for each $T \in \mathcal{T}$, the depth inequalities, and normalization $\sum_T \mu_T = 1$. Its dual is

$$\begin{aligned} \text{maximize} \quad & \lambda, \\ \text{subject to} \quad & \lambda \le \sum_h \pi_h d_T(h) \qquad (T \in \mathcal{T}), \\ & \pi_h \ge 0, \qquad \sum_h \pi_h = 1. \end{aligned}$$

Here one may leave the primal variable r unrestricted: for $n \ge 2$, every depth is at least one, so its inequalities already force $r \ge 1$. To derive the dual, assign nonnegative multipliers π_h to $\sum_T \mu_T d_T(h) - r \le 0$ and a free multiplier λ to $1 - \sum_T \mu_T = 0$. The Lagrangian is

$$\begin{aligned} \mathcal{L}(r,\mu;\pi,\lambda) &= r + \sum_h \pi_h \left(\sum_T \mu_T d_T(h) - r \right) + \lambda \left(1 - \sum_T \mu_T \right) \\ &= r \left(1 - \sum_h \pi_h \right) + \lambda + \sum_T \mu_T \big(c_\pi(T) - \lambda \big). \end{aligned}$$

Its infimum over free r and nonnegative μ_T is finite exactly when $\sum_h \pi_h = 1$ and $c_\pi(T) \geq \lambda$ for every T. Under those conditions the infimum is λ, giving the stated dual. In particular, weak duality is visible directly:

$$\lambda \leq \sum_T \mu_T c_\pi(T) = \sum_h \pi_h \sum_T \mu_T d_T(h) \leq \sum_h \pi_h r = r.$$

The first inequality averages the dual constraint over a normalized mixture. The equality exchanges two finite sums, and the last inequality weights the primal depth constraints by the nonnegative π_h. Their sum is one, which gives the final equality. The finite primal is feasible and bounded: any proper tree gives a feasible pure mixture with $1 \leq r \leq L$. Thus finite LP duality gives equality of its optimum and this dual optimum. For fixed π, the largest feasible λ is exactly $\min_T c_\pi(T)$. The dual permits zero-mass hypotheses because it defines the comparison benchmark. The algorithm itself uses strictly positive priors, as required by the oracle. □

The dual identity already gives the comparison benchmark. We also need its scale: identifying n possibilities requires logarithmic expected depth under the uniform prior. This ensures that the additive regret term becomes a multiplicative tolerance.

Claim 11.3 (The uniform prior provides a logarithmic lower bound)**.** *Every identifying binary tree has uniform-prior expected depth at least* $\log_2 n$*.*

Proof. Its leaf depths satisfy Theorem 2.6. By convexity of 2^{-x},

$$2^{-\frac{1}{n}\sum_h d_T(h)} \leq \frac{1}{n} \sum_h 2^{-d_T(h)} \leq \frac{1}{n}.$$

The first inequality is Jensen's inequality and the second is Kraft's bound divided by n. Taking $-\log_2$ reverses both inequalities and gives the claim. Applying Claim 11.2 to the uniform prior proves (38). □

11.2.2 A weight potential bounds every hypothesis's depth

Consider a hypothesis that remains slow in many generated trees. Its weight must grow quickly. The sum of all weights, however, grows according to the oracle's cost on the current prior. Comparing these two growth rates bounds every hypothesis's average depth by the oracle's average weighted cost. Scaling depths by L lets the same calculation apply to every proper tree.

Claim 11.4 (Individual weight growth is paid by total weight growth)**.** *For the scaled multiplicative-weights update in* (36) *and* $g_{h,t} = d_{T_t}(h)/L \in [0,1]$*,*

$$\ln(1+\eta) \sum_{t=1}^{K} g_{h,t} \leq \ln n + \eta \sum_{t=1}^{K} \sum_g \pi_{g,t} g_{g,t} \quad \textit{for every } h.$$

Proof strategy.
Remove the common integer factor so the normalized priors are unchanged. A single hypothesis's weight grows at a rate set by its depth, while the sum of all weights grows at a rate set by the current prior's mean depth. Comparing these bounds gives the claim.

Proof. Write $\widehat{w}_{h,t} = w_{h,t}/(qL)^{t-1}$ for the integer weights with their common factor removed. This leaves their normalized prior unchanged and makes their update $1 + \eta g_{h,t}$. Concavity of $x \mapsto \ln(1+\eta x)$ places its graph above the chord between $x = 0$ and $x = 1$, giving $\ln(1+\eta x) \geq x \ln(1+\eta)$, so each final weight obeys

$$\ln \widehat{w}_{h,K+1} \geq \ln(1+\eta) \sum_t g_{h,t}.$$

For their sum $\widehat{W}_t$, initially $\widehat{W}_1 = n$ and

$$\frac{\widehat{W}_{t+1}}{\widehat{W}_t} = 1 + \eta \sum_h \pi_{h,t} g_{h,t}.$$

Taking logarithms and summing the successive total-weight ratios gives

$$\begin{aligned} \ln \widehat{W}_{K+1} &= \ln n + \sum_{t=1}^{K} \ln \left(1 + \eta \sum_g \pi_{g,t} g_{g,t} \right) \\ &\leq \ln n + \eta \sum_{t=1}^{K} \sum_g \pi_{g,t} g_{g,t}. \end{aligned}$$

The inequality uses $\ln(1+x) \leq x$ with nonnegative x. For each h, positivity and $\widehat{w}_{h,K+1} \leq \widehat{W}_{K+1}$ imply

$$\ln(1+\eta) \sum_t g_{h,t} \leq \ln \widehat{w}_{h,K+1} \leq \ln \widehat{W}_{K+1},$$

which is the required comparison. □

The growth comparison is expressed in normalized depths and logarithms. For the uniform mixture, $R(\mu^{\mathrm{unif}}) = \max_h K^{-1} \sum_t d_{T_t}(h)$; thus rescaling the claim by L/K converts its per-hypothesis sums to the objective in (34). Bounding the two logarithmic coefficients then gives the explicit regret bound.

Claim 11.5 (The potential comparison gives the stated regret)**.** *Claim 11.4 implies* (39).

Proof. Multiply the bound in Claim 11.4 by $L/[K \ln(1+\eta)]$ and maximize over h. Since $\sum_g \pi_{g,t} g_{g,t} = c_{\pi_t}(T_t)/L$, the weighted-depth term becomes the average oracle cost, and we obtain

$$R(\mu^{\mathrm{unif}}) \leq \frac{\eta}{\ln(1+\eta)} \frac{1}{K} \sum_t c_{\pi_t}(T_t) + \frac{L \ln n}{K \ln(1+\eta)}.$$

The integral for $\ln(1+\eta)$ gives

$$\ln(1+\eta) = \int_0^\eta \frac{dx}{1+x} \geq \frac{\eta}{1+\eta} \geq \frac{\eta}{2} \qquad (0 < \eta \leq 1).$$

The first inequality bounds the integrand below by its endpoint value $1/(1+\eta)$; the second uses $1 + \eta \leq 2$. Therefore $\eta/\ln(1+\eta) \leq 1+\eta$ and $1/\ln(1+\eta) \leq 2/\eta$. Substituting separately into the nonnegative mean-cost and error terms gives

$$R(\mu^{\mathrm{unif}}) \leq (1+\eta) \frac{1}{K} \sum_t c_{\pi_t}(T_t) + \frac{2L \ln n}{\eta K},$$

as claimed. □

11.2.3 Integer updates and a basic LP solution give a finite strategy

We analyze the updates with logarithmic potentials and implement them using positive integer weights; normalization gives the exact rational priors supplied to the oracle. A basic solution of the master LP bounds the number of stored trees and the bit length of their probabilities. Exact sampling then completes the construction and preserves the query guarantee.

Claim 11.6 (All oracle inputs have strictly positive rational weights)**.** *The updates* (36) *preserve positivity and have polynomial encoding length.*

Proof. The initial weights are one. Every update factor is a positive integer at most $(p+q)L$. After K rounds, each weight therefore has $O(K\log((p+q)L)+1)$ bits. Normalization adds at most $\lceil \log_2 n \rceil$ bits to the denominator. All priors are strictly positive, so every call meets the oracle's hypothesis. The generated proper trees have $O(nK)$ total nodes; their depth matrix consists of integers at most L. □

All generated trees can therefore be stored in a polynomial-size depth matrix. The LP has one column per generated tree and n hypothesis-depth constraints; these constraints determine the worst expected depth. An optimal basic solution uses this structure to retain a small support.

Claim 11.7 (An optimal basic mixture uses at most n trees)**.** *The LP* (37) *has an optimal basic solution with at most n positive tree probabilities and polynomial-bit rational coefficients.*

Proof strategy.
Put the LP in standard form and use positivity of the objective variable to bound the number of tree probabilities in a basic solution. Then bound the basis determinants to control the bit length of those probabilities.

Proof. Its uniform mixture is feasible. Replacing every depth inequality by

$$\sum_t \mu_t d_{T_t}(h) + s_h - r = 0, \qquad s_h \geq 0,$$

gives standard form with $n+1$ equalities and nonnegative variables μ_t, s_h, r. Because $n \geq 2$, every proper tree has depth at least one on every hypothesis. Since the probabilities sum to one, each depth inequality therefore forces $r \geq 1$, so the variable r is positive in every feasible solution. A basic solution has at most $n+1$ positive variables, leaving at most n for tree probabilities. If some equalities are dependent, the basis dimension decreases and the same support bound remains valid. A finite optimal basic solution exists, and its integer basis matrix has entries of absolute value at most L. For completeness, let $b \leq n+1$ be the basis size. The basis matrix B is nonsingular and integer, so $|\det B| \geq 1$. Each of its columns has Euclidean norm at most $\sqrt{b}\,L$. Hadamard's bound and Cramer's rule therefore give

$$1 \leq |\det B| \leq (\sqrt{b}\,L)^b, \qquad |\det B_i| \leq (\sqrt{b}\,L)^b, \qquad x_i = \frac{\det B_i}{\det B}.$$

The bound on B_i also holds because the right-hand side has entries zero or one, and $L \geq 1$. Each numerator and denominator thus needs $O(b(\log b + \log L)+1)$ bits. In particular the tree probabilities have polynomial encoding length. Rational LP algorithms, followed by exact basis extraction on the optimal face, find such an optimal basic solution in polynomial time [12]. The uniform mixture is feasible, so the optimized value is at most $R(\mu^{\mathrm{unif}})$. □

The sparse LP solution assigns probabilities to the retained trees. Each run samples one tree and executes it from root to leaf. Its rational probabilities allow an exact draw using fair bits, so the execution preserves the zero-error query guarantee.

Claim 11.8 (The compressed mixture is exactly sampleable)**.** *The optimized rational mixture gives a legal zero-error execution with at most L queries and expected polynomial sampling cost.*

Proof strategy.
Clear the rational probabilities to obtain integer interval lengths. Rejection sampling produces a uniform integer in their common range, which selects a tree with exactly its LP probability and finite expected bit cost.

Proof. By Claim 11.7, take a common denominator D for its at most n polynomial-bit probabilities; its encoding length is polynomial. Explicitly, if their denominators are $b_1, \dots, b_s$, take $D = \prod_{i=1}^{s} b_i$; then $\log_2 D = \sum_i \log_2 b_i$ is polynomial. Writing the probabilities as a_i/D gives nonnegative integer interval lengths with $\sum_i a_i = D$. Draw a uniform integer from $\{0, \dots, D-1\}$ by fair bits and rejection, then use the cumulative integer probability intervals. Rejection makes every accepted integer equiprobable, so an interval of length a_i has probability a_i/D. If $D = 1$, select the sole supported tree deterministically. Otherwise each attempt uses $\lceil \log_2 D \rceil$ bits and is accepted with probability greater than $1/2$, so the expected number of attempts is less than two: with $b = \lceil \log_2 D \rceil$, each b-bit draw is uniform on $\{0, \dots, 2^b - 1\}$, and $D/2^b > 1/2$. Conditioning on a draw below D makes it uniform on the desired range. The sampled tree identifies every hypothesis and has depth at most L. This completes the compression proposition. □

The two optimization viewpoints also give a useful stopping certificate. The restricted master LP supplies an upper bound on R^* because its mixture is a feasible strategy, while

$$\max_t \frac{c_{\pi_t}(T_t)}{\alpha} \le R^*$$

supplies a lower bound by (35) and (38). Their ratio is an instance-specific certificate. Stop early when it meets the desired factor; otherwise, the fixed-round guarantee still applies.

11.3 Calibration and the strongest resulting guarantees

The preceding proof separates the oracle's approximation loss from the loss due to finitely many weight-update rounds. Split the desired tolerance equally between these two terms; the following choices make that split explicit.

Parameter calibration.
For a fixed rational oracle coefficient $\alpha \ge 1$ and target $0 < \varepsilon \le 1$, choose

$$\eta = \frac{\varepsilon}{2\alpha}, \qquad K = \left\lceil \frac{8\alpha(n-1)}{\varepsilon^2} \right\rceil. \tag{40}$$

The definition of η gives $\alpha\eta = \varepsilon/2$. For the second term, use $L = n - 1$ and the lower bound on the ceiling-defined K:

$$\frac{2L}{\eta K} \le \frac{2L}{(\varepsilon/(2\alpha))(8\alpha L/\varepsilon^2)} = \frac{\varepsilon}{2}.$$

Thus each error term is at most $\varepsilon/2$, and their sum is at most ε.

The oracle and duality give $c_{\pi_t}(T_t) \le \alpha \operatorname{OPT}_E(\pi_t) \le \alpha R^*$ in each round. Substitution into the regret inequality yields

$$\begin{aligned} R(\mu^{\mathrm{unif}}) &\overset{\text{Prop. 11.2}}{\le} (1+\eta)\alpha R^* + \frac{2L\ln n}{\eta K} \\ &\overset{\text{Eq. 40}}{\le} \left(\alpha + \frac{\varepsilon}{2}\right) R^* + \frac{\varepsilon}{2}\ln n \\ &\overset{\text{Prop. 11.1}}{<} (\alpha+\varepsilon)R^*. \end{aligned}$$

The final inequality is strict because $n \ge 2$ and $\ln n = (\ln 2)\log_2 n < \log_2 n \le R^*$. The optimized mixture has cost at most that of the uniform one.

The current expected-depth certificate in Section 6.3 gives the rational valid oracle coefficient $\alpha = 5.18297551$. Choosing $\varepsilon = 1/4000000$ in the single mixture algorithm yields

$$R(\mu) < 5.18297576\, R^* < 5.182976\, R^*.$$

This is the best polynomial factor established here for the randomized per-hypothesis objective. The small tolerance makes the iteration constant large; the parameterized theorem and the optimized master LP allow more practical tolerance choices.

Using a stronger expected-depth oracle preserves the prior updates and mixture LP while improving the guarantee for this randomized objective. The quasipolynomial oracle gives the stronger factor, and its running time carries over to preprocessing.

Corollary 11.5 (Quasipolynomial randomized per-hypothesis approximation)**.** *For every fixed rational $\varepsilon > 0$, the same LP-compressed mixture algorithm has a supplied-query quasipolynomial $(2+\varepsilon)$ approximation for R^*. For $0 < \varepsilon \le 1$, its time is $(mn)^{O(\varepsilon^{-1}\log n)}$ times a polynomial in input bit length, and its mixture is supported on at most n trees.*

Proof. Use the expected-depth oracle of Theorem 7.4 with coefficient $\alpha = 2 + \varepsilon/2$, and apply the transfer theorem with tolerance $\varepsilon/2$. The factor is strictly below $2 + \varepsilon$ for $n \ge 2$. There are $\lceil 32\alpha(n-1)/\varepsilon^2 \rceil$ oracle calls, whose positive rational priors have polynomial encoding length. These calls and the master LP preserve the oracle's stated quasipolynomial running-time form. For $\varepsilon > 1$, use $\varepsilon = 1$; a singleton again returns its leaf. □

12 Allowing hypotheses on supplied lines

Allowing a hypothesis to lie on a queried line adds a third outcome. To compare with the binary algorithms, encode each line answer using at most two bits: the first separates zero from nonzero, and the second separates negative from positive on the nonzero branch. Thus each ternary query expands to at most two levels in a binary comparison tree, which accounts for the factor of two in the guarantee.

Algorithm 20 implements the reduction below. In this section a supplied line returns one of three answers: negative, zero, or positive. The *zero*, or *on-line*, answer means the hypothesis lies on the queried line, and its answer class is $S_\ell^0 = \{h \in S : f_\ell(h) = 0\}$. A *ternary decision tree* has an outgoing edge for each possible answer; a proper such tree retains only nonempty outcomes. Leaf depths and both cost objectives use the same conventions as in the binary model. Use the two virtual binary predicates

$$b_1(h) = \mathbf{1}[f_\ell(h) < 0], \qquad b_2(h) = \mathbf{1}[f_\ell(h) \le 0].$$

Their pair of bits distinguishes the three outcomes. In the order (b_1, b_2), negative, zero, and positive signs map to $(1,1)$, $(0,1)$, and $(0,0)$, respectively; the unused pair $(1,0)$ never occurs. Thus no outcome is merged, and the virtual binary tree can omit the unused branch at each query. Requesting either bit is implemented by querying the original line once and storing its full sign. Both bits are then known. For analysis we count each requested virtual bit as a separate binary query; the actual number of supplied-line queries can only be smaller.

For preprocessing, we also represent these bit labelings by planar queries whose lines avoid every hypothesis. On the finite hypothesis set, sufficiently small shifts preserve both bit values while moving the virtual boundaries off the points. During execution, the algorithm asks the original supplied line and reads its full ternary sign, so the virtual representation changes only the analysis and preprocessing. Any query whose answer is constant on the hypothesis set can be discarded. For every remaining query, choose ϵ_ℓ to be half the minimum nonzero $|f_\ell(h)|$ on the finite hypothesis set. At least one such nonzero value exists because the query is not constant (a nonconstant sign labeling must contain a nonzero value). For a rational input instance, this minimum is computable by scanning the hypotheses and has polynomial encoding length: evaluating an input affine form and taking the minimum of finitely many rational absolute values preserves polynomial bit length. Then b_1 is represented by $f_\ell(x) < -\epsilon_\ell$ and b_2 by $f_\ell(x) < \epsilon_\ell$. These strict inequalities agree with the definitions on every hypothesis: a negative nonzero value is at most $-2\epsilon_\ell$, and a positive value is at least $2\epsilon_\ell$, while zero lies between the two thresholds. In particular, neither shifted boundary contains a hypothesis. The shifts are used only for preprocessing; their answers are obtained from the original supplied line.

Symbol	Meaning
$f_\ell(h)$	Affine value whose sign is the answer for hypothesis h.
$b_1(h),\ b_2(h)$	Virtual bits for $f_\ell(h) < 0$ and $f_\ell(h) \le 0$; together they encode the ternary sign.
ϵ_ℓ	Positive shift smaller than every nonzero $\lvert f_\ell(h)\rvert$ on the finite hypothesis set.

Algorithm 20: Transfer a binary algorithm to on-line outcomes

Input: Finite hypotheses, supplied ternary lines, binary tree-construction routine B.
Output: An executable tree using the original supplied lines.

foreach *nonconstant supplied line* ℓ **do**
  $\epsilon_\ell \leftarrow \frac{1}{2} \min_{h: f_\ell(h) \neq 0} |f_\ell(h)|$
  Represent virtual bits $b_1 = \mathbf{1}[f_\ell(h) < 0]$ and $b_2 = \mathbf{1}[f_\ell(h) \le 0]$ by the two shifted off-boundary lines
Construct a virtual binary tree with B
Execute a requested virtual bit by querying its original line, caching the entire ternary sign; answer further requests from that cache
return the contracted original-line tree

Lines constant on the whole hypothesis set are discarded before taking the minimum nonzero affine value. Proposition 12.1 proves that the approximation ratio increases by at most two.

There are two comparisons to make. An optimal ternary tree can be simulated using at most two virtual bits per query. A constructed virtual tree can then be executed using original line queries, with the cached sign supplying both bits whenever either is asked. Because the first comparison bounds each hypothesis's virtual depth by twice its original depth, and the second bounds each executed depth by that hypothesis's virtual depth, both comparisons survive either taking a maximum or summing depths with nonnegative prior weights.

Proposition 12.1 (Boundary reduction)**.** *An α-approximation for either binary objective gives a 2α-approximation for the corresponding three-answer objective.*

Proof. Simulate every query of an optimal original tree by its two virtual bits. Each hypothesis uses at most twice as many binary queries, so the virtual optimum is at most twice the original optimum for either objective. Conversely, execute a constructed virtual tree by original line queries and cached signs. Its actual depth is no larger than its virtual depth on every hypothesis. To combine these comparisons explicitly, write C_{orig} for the executed cost and $\mathrm{OPT}_{\mathrm{virt}}, \mathrm{OPT}_{\mathrm{orig}}$ for the two optima. Then, for either depth objective,

$$C_{\mathrm{orig}} \le C_{\mathrm{virt}} \le \alpha\,\mathrm{OPT}_{\mathrm{virt}} \le 2\alpha\,\mathrm{OPT}_{\mathrm{orig}}\,.$$

The first step uses cached-sign execution, the second is the assumed binary approximation guarantee, and the last uses the two-bit simulation of the optimal ternary tree. Thus the comparison applies to both maximum depth and any weighted sum of leaf depths. □

The reduction is independent of the binary algorithm's internal geometry or schedule. Applying it to the strongest deterministic algorithms established above gives the following bounds for the model with on-line outcomes.

Corollary 12.2 (Constant approximations with on-line outcomes)**.** *With on-line answers, expected depth has a polynomial-time approximation factor below* 10.365952 *for arbitrary positive priors, and deterministic maximum depth has a factor below* 8.929931*.*

Proof. Apply Proposition 12.1 to Theorems 6.5 and 8.1. □

13 Strong NP-completeness

The approximation algorithms exploit planar structure, but exact optimization remains hard. We first state the decision and source problems and the graph terminology used by both reductions.

Definition 13.1 (Complexity and reductions)**.** A *polynomial-time many-one reduction* maps instances of one decision problem to another in polynomial time, so an input is a yes-instance if and only if its image is a yes-instance.

A decision problem is in *NP* if every yes-instance has a certificate of polynomial encoding length whose validity is checkable in polynomial time, and no certificate is accepted on a no-instance. It is *NP-hard* if every problem in NP admits such a reduction to it, and *NP-complete* if it is both NP-hard and in NP. A numerical problem is *strongly NP-hard* if hardness persists when its integer data are bounded by a fixed polynomial in the combinatorial instance size; it is *strongly NP-complete* if it is also in NP. For rational data, our restricted instances explicitly bound both numerators and denominators.

Definition 13.2 (3-Partition)**.** The input consists of positive integers $a_1, \dots, a_m, B$, where $m = 3p$ for a positive integer p, satisfying

$$s = \sum_{i=1}^{m} a_i = pB, \qquad B/4 < a_i < B/2.$$

The question is whether $[m]$ can be partitioned into p groups $I_1, \dots, I_p$ with $|I_b| = 3$ and $\sum_{i \in I_b} a_i = B$ for every b.

Symbol	Meaning
$m = 3p,\ [m]$	Number of source items and their index set.
$a_i,\ B$	Item size i and target sum of each bin.
$s = pB = \sum_i a_i$	Total item size and total capacity of the p bins.
I_b	Index set of the three items assigned to source bin b.

Both reductions use the strong NP-completeness of this problem [13]. We use the NP-hard restriction in which B is polynomially bounded in m. The case $p = 1$ is a trivial yes-instance, so excluding it preserves NP-hardness. In this restriction each a_i is polynomially bounded, so the explicit repetitions in our constructions remain polynomial in the source size. A partition into p groups of sum B then necessarily has three items per group: two items sum to less than B, while four items sum to more than B.

Definition 13.3 (Paths and matchings)**.** A *path* is an undirected graph with distinct vertices $v_0, \dots, v_k$ and edges exactly $\{v_{j-1}, v_j\}$ for $1 \le j \le k$. A *linear forest* is a disjoint union of paths, allowing isolated vertices. Throughout, an *odd path* has an odd number of vertices; when its edge count matters, we state that count explicitly. Positions are numbered locally from zero along each item path, separately from global hypothesis indices.

A *matching* is a set of edges with no shared endpoint. A vertex is *unmatched* if it is incident to no chosen edge. A *maximum matching* has the largest possible number of edges. A *perfect matching* leaves no vertex unmatched.

Both reductions turn the same source decision into a choice of one bin for each item. To represent those choices, we place an item on an odd path and supply tests for each vertex and each adjacent pair. The distinguished hypothesis forces an optimal tree to use a maximum matching of these pair tests; by Lemma 13.5, the one unmatched even-position vertex then specifies the item's bin. This Each unmatched-position choice changes a different statistic. For maximum depth, clouds change the number of branches at each height; for expected depth, weights change the pair-mass histogram. Summing these statistics across the items recovers the source bin loads. Finally, the parabola realizes every singleton and adjacent-pair test as a supplied line, and the cloud replacement in Section 14 transfers the weighted expected-depth instance to a uniform prior while adding the same fixed internal cost to every maximum matching.

13.1 The geometric realization and the matching choice

For integer abscissae, the line

$$I_{a,b}: \quad 4y - 4(a+b)x + (2a-1)(2b+1) = 0$$

has value $4(x-a+\frac{1}{2})(x-b-\frac{1}{2})$ on (x, x^2). Its negative answer selects exactly the integer abscissae $a, \dots, b$. This realizes interval membership using one line. The line

$$P_t: \quad 2y - 4tx + 2t^2 - 1 = 0 \tag{41}$$

has value $2(x-t)^2 - 1$, and its negative answer isolates exactly the point of abscissa t. Both formulas are nonzero at all integer abscissae, so every answer is binary. The tangent $y = 2tx - t^2$ exposes the point (t, t^2), since every other parabola point lies strictly above it. The constructed hypotheses are therefore in strictly convex position.

We use the negative answer class of each displayed line as its membership set. Consequently, a hit in Definition 2.2 is precisely a negative affine sign, while the distinguished hypothesis is

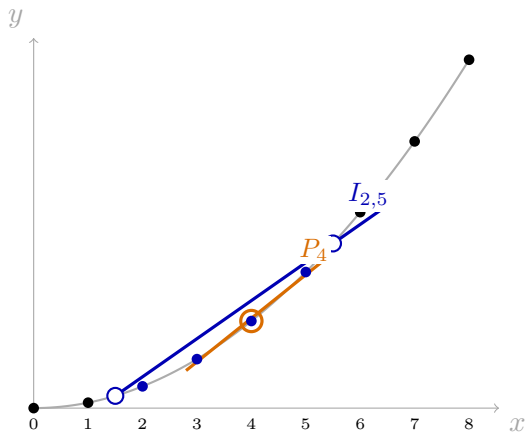


Figure 8: The integer points lie on $y = x^2$. The blue line $I_{2,5}$ has negative answer exactly on the points with $x \in \{2, 3, 4, 5\}$; the orange line P_4 has negative answer only at $x = 4$, marked with an orange ring. The interval endpoints are the half-integer intersections $x = 1.5$ and $x = 5.5$.

placed outside every such set. This convention lets the combinatorial path tests above enter the membership-spine framework without changing the supplied queries.

With this membership convention, the two formulas realize every singleton and adjacent-pair test used by the matching construction. The parabola keeps all hypotheses exposed on the convex hull, and the integer abscissae give explicit integer coefficients for each line. Figure 8 illustrates $I_{2,5}$ and P_4 on the points with integer abscissae from 0 to 8.

For maximum depth we also use *clouds* of ordinary points. They make a local identification problem take a chosen number of queries without changing the objective. A cloud's size w_i counts its hypotheses in this maximum-depth construction. Given positive integer cloud sizes w_i, put $Q = 4 \max_i w_i$ and represent cloud i by

$$\{(Qi + j, (Qi + j)^2) : 0 \le j < w_i\}.$$

Its point with $j = 0$ is its *anchor*. Supply the cloud-interval line

$$4y - 4Q(a + b)x + Q^2(2a - 1)(2b + 1) = 0. \tag{42}$$

For $a \ge 1$ its negative answer selects exactly the clouds $a, \dots, b$, while excluding cloud 0. This follows by factoring its value as $4(x - Q(a - \frac{1}{2}))(x - Q(b + \frac{1}{2}))$. Each cloud lies within $Q/4$ of its anchor, so neither boundary, halfway between consecutive anchors, contains a hypothesis. Each nonanchor point receives a private line (41). The anchors use only cloud tests: an anchor and a point in its own cloud agree on every cloud-interval test and on every other private test. Their separation therefore requires the point's own private query. Thus a cloud of size w forces $w - 1$ queries on the anchor's local path, while the anchor remains available to represent the cloud in the matching construction.

Clouds are ordered so that consecutive vertices of each path used below have consecutive cloud indices. The only cloud-interval tests are whole-cloud singleton tests and adjacent-pair tests specified by those paths. Every point, including every anchor, must be identified.

The matching construction uses clouds as exits with prescribed local heights. The following exact values let the later histogram account for those exits without making assumptions about their internal adaptive trees.

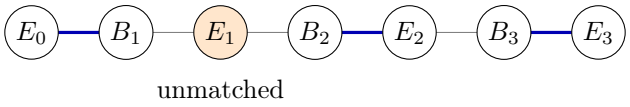


Figure 9: A maximum matching of $E_0, B_1, E_1, B_2, E_2, B_3, E_3$ with E_1 left unmatched. Blue edges are the selected adjacent pairs. In the reduction, the unmatched even-position cloud selects the item's bin; the uniquely forced pairs lie to its left and right.

Lemma 13.4 (Identifying one or two clouds)**.** *For this supplied family, the minimum height on one cloud of size w is $w - 1$. On two clouds of sizes u, v, when their whole-cloud singleton tests are supplied, the minimum height is* $\max\{u, v\}$.

Proof. On a single cloud, old cloud-interval tests are constant and each private test isolates one nonanchor point. The anchor answers the same way as every nonanchor on all the other private tests, so its path must ask all $w - 1$ of them. Querying them sequentially achieves that height.

On two clouds, the anchor of the larger cloud must use all its private tests: each is the only query separating that cloud's anchor from its corresponding nonanchor. It must also use at least one cloud-interval test, since private tests give the same answer at both anchors. The larger cloud therefore forces depth at least $\max\{u, v\}$. First separating the clouds and then performing their private searches attains the bound. □

Each odd item path encodes one bin choice. Its single unmatched vertex fixes the entire remaining matching, so that position identifies the chosen bin.

Lemma 13.5 (One unmatched position)**.** *A maximum matching of the path*

$$E_0, B_1, E_1, \ldots, B_t, E_t$$

leaves exactly one vertex E_k unmatched. It then uses pairs $\{E_{j-1}, B_j\}$ for $j \le k$ and $\{B_j, E_j\}$ for $j > k$.

Proof. There are $2t+1$ vertices, so a matching contains at most t edges. Taking the pairs $\{E_{j-1}, B_j\}$ for $j = 1, \ldots, t$ attains this bound; hence every maximum matching covers $2t$ vertices and leaves one vertex unmatched. If that vertex is at position r along the path, the r vertices before it and the $2t - r$ vertices after it are both covered by matching edges wholly within their respective segments. Both segment sizes are therefore even, so r is even; write $r = 2k$, making the unmatched vertex E_k.

The left segment now begins at endpoint E_0, whose only available neighbor is B_1. Thus its first pair is forced to be $\{E_0, B_1\}$; after removing that pair, the same endpoint argument forces $\{E_1, B_2\}$, and so on through $\{E_{k-1}, B_k\}$. Reading the right segment from endpoint E_t forces, in turn, $\{B_t, E_t\}, \ldots, \{B_{k+1}, E_{k+1}\}$. These are precisely the pairs in the statement, and each was forced by the endpoint of its remaining segment. □

Figure 9 shows the smallest nontrivial example of the matching pattern. Moving the unmatched even-position cloud changes which segment represents the item's bin choice.

The item paths only organize the supplied membership queries. The reduction uses singleton and adjacent-pair tests directly; no underlying edge-query search tree is part of the instance.

13.2 Maximum depth: bin loads become branch-height counts

Theorem 13.6 (Strong NP-completeness of maximum depth)**.** *Deciding whether* $\mathrm{OPT}_W(\mathcal{H}) \le K$ *is strongly NP-complete, even for integer-coordinate points on a parabola, polynomially bounded integer line coefficients, and* $O(n)$ *supplied binary line queries.*

Proof strategy.

1. Force the distinguished anchor's path to attain its minimum length. Its first-hit pairs must then be a maximum matching, with one even-position cloud unmatched in each item path.
2. Use local cloud heights and a sorted exit histogram to exclude internal unmatched positions. Each remaining boundary is one bin choice.
3. Moving that boundary transfers the item's size between disjoint histogram windows. The height threshold requires every bin load to be at most B, hence exactly B.

13.2.1 Construction and supporting propositions

Use the restricted 3-Partition instance of Definition 13.2. Let $q = (p-1)(4s+2m)$ be the total matching size and $T = s - B$ the width of a bin window. Choose a cloud offset M, bin levels $\lambda_1, \dots, \lambda_p$, and high cloud sizes H, C by the calibration in Section 13.2.3; the decision threshold is $K = q + M$. The source information is a_i, B, p; these additional quantities only separate branch-height levels and make the distinguished path tight.

For item i, construct an odd path whose vertices represent clouds. Starting with an even-position cloud of size 2, append $p-1$ segments, indexed by $b = 1, \dots, p-1$. Each row below appends an odd-position cloud and then the next even-position cloud. Segment b consists of one entry row, a_i repetitions of the four block rows, and one exit row:

part	odd-position size	next even size
entry	C	M
block step 1	λ_b	H
block step 2	$\lambda_b + T$	M
block step 3	$\lambda_{b+1} + T$	H
block step 4	λ_{b+1}	M
exit	C	2

The repeated block is the only part depending on a_i. It transfers a_i units between neighboring histogram windows. There are $4a_i + 2$ odd-position vertices per segment, so the whole item path has $2s_i + 1$ vertices, where $s_i = (p-1)(4a_i+2)$, and its maximum matching has size s_i. Its p even-position clouds of size 2 are the segment boundaries.

Add one auxiliary cloud of size M and cloud 0 of size 2. Supply each nonzero cloud's whole-cloud singleton test, each adjacent-pair test within an item path, and each nonanchor point's private singleton test. Realize them by (42) and (41).

For boundary choices, let L_b be the total item size assigned to bin b. Put

$$Q_{\mathrm{hi}} = 2(p-1)(s+m), \qquad \beta_1 = q - s, \qquad \beta_b = Q_{\mathrm{hi}} + 2s(p-b) \quad (2 \le b \le p).$$

Here Q_{hi} counts the always-high pairs, and β_b is the reference tail count in bin window b after its bin load is removed. These counts are combinatorial; their levels are calibrated later.

Soundness starts with an arbitrary adaptive tree. A tight lower bound on the distinguished path forces its cloud groups to form maximum matchings; only then can their heights be sorted and counted.

Proposition 13.7 (Tight-spine matching normalization)**.** *Every identifying tree has height at least K. A tree of height at most K can be normalized without increasing height so its first-hit pairs form a maximum matching in every item path, all clouds are intact at their first hit, and the distinguished cloud's private query is last. If $N(t)$ counts exits of local height at least t, sorting those exits by nonincreasing height gives the exact criterion*

$$t + N(t) \leq K \tag{43}$$

at every exit-height level.

The odd-path lemma allows every even position to be unmatched, but only the designated boundaries should represent bins. One height level detects and excludes all internal positions at once.

Proposition 13.8 (Only boundary positions are feasible)**.** *Every matching whose normalized tree fits height K leaves a boundary cloud unmatched in each item path. Its p possible boundaries choose the item's bin.*

With internal positions excluded, moving an unmatched boundary is a genuine bin choice. Its effect on the exit histogram adds across items, turning the height constraints into bin-capacity constraints.

Proposition 13.9 (Histogram windows encode bin loads)**.** *At the upper level $t_b = \lambda_b + T$ of each bin window,*

$$N(t_b) = \beta_b + L_b, \qquad t_b + \beta_b = K - B.$$

Consequently the height criterion requires $L_b \leq B$ for every bin.

The bin windows establish the necessary conditions. For the reverse direction, balanced loads must also respect every remaining height level and identify all cloud points, including the distinguished cloud.

Proposition 13.10 (Balanced loads give a complete tree)**.** *Boundary matchings with $L_b = B$ satisfy every height-profile constraint and yield a complete identifying tree of height K.*

The combinatorial equivalence must be accompanied by a size bound. Explicit clouds and integer lines preserve strong hardness only if their replication and numerical values stay polynomial.

Proposition 13.11 (Polynomial geometric encoding)**.** *The construction has polynomial size on the restricted source input, integer coordinates and coefficients bounded by $O(n^4)$, and $O(n)$ supplied strict binary lines. A height-K identifying tree is a polynomially checkable NP certificate.*

Proof of Theorem 13.6. A source partition yields a height-K tree by Proposition 13.10. Conversely, normalize any height-K tree by Proposition 13.7; Propositions 13.8 and 13.9 give bin loads at most B. Since their sum is $s = pB$ across p bins, every load must equal B. Each such bin contains three items: two items sum to less than B, while four sum to more than B, by the source promise $B/4 < a_i < B/2$. These groups form a 3-partition. Proposition 13.11 supplies strong polynomial encoding and NP membership. Thus the reduction is exact for arbitrary adaptive trees. □

13.2.2 Technical proofs of the height propositions

Proof of Proposition 13.7. We normalize in two stages. First, counting the queries unavoidable on the distinguished anchor's path forces maximum matchings and intact cloud exits. Then sorting those exits gives the complete height test.

Claim 13.1 (The distinguished anchor forces maximum matchings)**.** *Every identifying tree has height at least K. If its height is at most K, the first-covered pairs form a maximum matching in every item path, and no nonzero-cloud private test is asked on the anchor path.*

Proof. Follow the anchor of cloud 0 through any complete tree. It needs the private test for the other point of cloud 0. Private tests of other clouds do not distinguish their anchors from this anchor. It therefore also needs cloud queries covering the $2q + m$ item-path anchors and the auxiliary anchor.

Classify each anchor by its first hitting cloud query along this all-miss path. A query first covers one anchor or two adjacent anchors. The first-covered pairs are disjoint, hence form a matching of size at most $\sum_i s_i = q$. There are $2q+m+1$ nonzero anchors to cover. If g first-covered groups are pairs, the remaining groups are singletons, so the path uses $(2q + m + 1) - g$ cloud queries. Since $g \le q$, this is at least $q + m + 1$. It must also ask the private query separating the two points of cloud 0, giving the lower bound $q+m+2 = K$, where the equality uses $M = m+2$ and $K = q+M$ from Section 13.2.3.

If a tree has height at most K, equality holds. No private test of a nonzero cloud can occur on this path, and its pairs form a maximum matching in every item path. Lemma 13.5 therefore leaves exactly one even-position cloud unmatched per item. □

Claim 13.1 gives equality on the distinguished path. Before a cloud's first whole-cloud query, each of its points answers every earlier whole-cloud query just as its anchor does. The only supplied queries that can separate a nonanchor from its anchor are private tests, and Claim 13.1 excludes those from this path. Thus each first-hit exit contains one whole cloud or, for a pair query, two whole clouds. Replace any query first covering just one cloud by its supplied whole-cloud singleton test. Put cloud 0's private query last; this advances the other exits and leaves its two points within depth K. Attach the optimal local codes of Lemma 13.4.

After normalization, each exit has an independent local height. Its only interaction with the other exits is its spine position, so sorting reduces all depth constraints to one tail-count criterion.

Claim 13.2 (Sorting exit heights gives the complete depth test)**.** *After the cloud-query groups are sorted by nonincreasing local height, every target path has depth at most K exactly when $t+N(t) \le K$ at every exit-height level t.*

Proof. The cloud queries now act on disjoint singletons and pairs. By Lemma 13.4, their exit heights are $w - 1$ for a singleton and $\max\{u, v\}$ for a pair. Include the final private query as an exit of height zero. For consecutive heights $r < s$, the old maximum is $\max\{j + r, j + 1 + s\} = j + 1 + s$. After exchanging them, both $j + s$ and $j + 1 + r$ are at most $j + 1 + s$, so the new maximum cannot be larger. Adjacent exchanges therefore sort the heights nonincreasingly without increasing any overall height bound.

Write h_j for the local height of the exit at spine position j. The *tail histogram* of exit heights is $N(t) = |\{j : h_j \ge t\}|$, the number of exits of height at least t. In sorted order, for every realized exit-height level t, the last exit counted by $N(t)$ has height exactly t and position $N(t)$. Thus $t + N(t)$ is an actual route depth. Conversely, for any exit at position j, $j \le N(h_j)$, so $j + h_j \le N(h_j) + h_j$. These two inequalities prove

$$\max_j (j + h_j) = \max_{t \text{ an exit height}} (t + N(t)).$$

Thus all target paths fit within K exactly when (43) holds at every exit-height level t. Thus the depth test reduces to the largest sum of spine position and remaining local depth over all exits. □

□

Proof of Proposition 13.8. Every matched pair has exit height at least M, because each odd cloud has size at least $\lambda_1 = M$. The auxiliary singleton has exit height $M-1$. An unmatched internal even vertex has size M or H, and hence exit height at least $M-1$. At $t = M-1$, these give $N(t) \geq q+1+$ number of such internal unmatched vertices. If there are c internal unmatched vertices, then $(M-1)+N(M-1) \geq (M-1)+(q+1+c) = K+c$. Claim 13.2 permits at most K at this level, so $c = 0$. The unmatched vertex in each item path therefore chooses one of its p boundary positions, interpreted as a bin. □

Proof of Proposition 13.9. To read bin loads from the histogram, first compute the effect of crossing one segment. These local transfers telescope across segments; a reference matching then fixes the baseline counts.

Claim 13.3 (A segment transfers load only between its neighboring windows)**.** *Its two boundary matchings differ by the window transfer of item i.*

Proof. By Lemma 13.5, moving the unmatched vertex across segment b reverses all its left/right pairing choices. Applying Lemma 13.4 to those two matchings gives their exit heights as follows. When its unmatched vertex is before the segment, a repeated block has heights

$$H,\ \lambda_b + T,\ H,\ \lambda_{b+1}.$$

When that vertex is after the segment, the same block has heights

$$\lambda_b,\ H,\ \lambda_{b+1} + T,\ H.$$

The two connector pairs have height C in either case. With a_i repetitions, the change in $N(t)$ is exactly

$$-a_i \mathbf{1}_{(\lambda_b, \lambda_b+T]}(t) + a_i \mathbf{1}_{(\lambda_{b+1}, \lambda_{b+1}+T]}(t).$$

The windows are disjoint because $T < s$ and consecutive λ_b differ by at least s. Write $W_j = (\lambda_j, \lambda_j + T]$. Moving from boundary 1 to boundary b adds the changes of segments $1, \ldots, b-1$:

$$a_i \sum_{j=1}^{b-1} (-\mathbf{1}_{W_j} + \mathbf{1}_{W_{j+1}}) = -a_i \mathbf{1}_{W_1} + a_i \mathbf{1}_{W_b}.$$

Each intermediate window occurs once with each sign and cancels. For $b = 1$ the sum is empty and the displayed difference is zero. Thus the item removes a_i from window 1 and adds it to its chosen window b. Figure 10 illustrates this change relative to the reference matching. □

The transfer calculation gives each item's change in the tail count. Taking the all-first-bin matching as a reference and summing these changes yields the tail count needed by the height criterion.

Claim 13.4 (Reference tail counts turn transfers into bin loads)**.** *At $t_b = \lambda_b + T$, the resulting count is $N(t_b) = \beta_b + L_b$.*

Proof. In the reference matching assigning all items to bin 1, Q_{hi} pairs have height H or C. For each segment $c = 1, \dots, p-1$, the remaining pairs comprise s of height $\lambda_c + T$ and s of height λ_{c+1}. Consider a threshold t in window b, so $\lambda_b < t \le \lambda_b + T$. If $c < b$, then both heights in segment c are below t: in particular, $\lambda_{c+1} \le \lambda_b < t$. If $c \ge b$, both heights are at least t, since $\lambda_c + T \ge \lambda_b + T \ge t$ and $\lambda_{c+1} > \lambda_b + T$. Thus, for $b \ge 2$, precisely the $p - b$ segments $c = b, \dots, p-1$ contribute their two batches of s pairs, giving the reference count $Q_{\mathrm{hi}} + 2s(p-b) = \beta_b$. In the first window all $p-1$ segments contribute, so the count is $Q_{\mathrm{hi}} + 2s(p-1) = q = \beta_1 + s$. Summing Claim 13.3 over items changes the first reference count by $-(s - L_1)$ and changes each later reference count by L_b. Thus throughout each window, and at its upper level, $N(t_b) = \beta_b + L_b$. For the offset identity, first use $\lambda_1 = M$, $T = s - B$, and $\beta_1 = q - s$ to get

$$t_1 + \beta_1 = M + (s - B) + (q - s) = K - B.$$

For $b \ge 2$, substitute the calibrated levels and reference counts:

$$\begin{aligned} t_b + \beta_b &= M + (2b-3)s + (s - B) + Q_{\mathrm{hi}} + 2s(p-b) \\ &= M - B + Q_{\mathrm{hi}} + 2s(p-1) \\ &= M - B + 2(p-1)(s+m) + 2s(p-1) = K - B. \end{aligned}$$

The second line collects the s terms, whose coefficient is $(2b-3) + 1 + 2(p-b) = 2(p-1)$. The third substitutes $Q_{\mathrm{hi}} = 2(p-1)(s+m)$; the last equality uses $q = (p-1)(4s+2m)$ and $K = M + q$. The profile condition at this level is $t_b + N(t_b) \le K$, or equivalently $L_b \le B$. Thus (43) requires $L_b \le B$ for every bin. Since $\sum_b L_b = s = pB$, all these inequalities are equalities and yield a source 3-partition. □

□

Proof strategy.

List every possible exit height and compute its tail count for balanced loads. The table checks the profile at each such level; between levels the tail count is constant, so the profile expression can only increase up to the next listed height. Sorting the exits and attaching their local codes then constructs the required tree.

Proof of Proposition 13.10. Choose the boundary matchings of a source partition, so $L_b = B$. Proposition 13.9 shows that the window upper levels satisfy (43) with equality. For completeness, the remaining levels have the following counts. Lemma 13.4 and the construction table enumerate the possible local heights. Cloud 0's final private exit has height 0, each unmatched boundary cloud has size 2 and singleton height 1, and the auxiliary cloud of size M has singleton height $M - 1$. The repeated-block pairs have heights among $H, \lambda_b, \lambda_b + T, \lambda_{b+1}, \lambda_{b+1} + T$, while the connector pairs have height C. Hence the only possible levels are $0, 1, M-1, H, C, \lambda_b$, and $\lambda_b + T$; the last levels $\lambda_b + T$ were already checked by Proposition 13.9.

t	$N(t)$	$t + N(t)$
0	K	K
1	$K - 1$	K
$M - 1$	$q + 1$	K
λ_b	$\beta_b + s$	K
H	Q_{hi}	K
C	$2m(p-1)$	$K - 2s(p-1) + 1 \le K$

To verify the table, there are $q + m + 1 = K - 1$ nonzero cloud groups and the final height-zero private exit. This gives the first two rows. At $M - 1$, precisely the q pairs and the auxiliary cloud count; unmatched boundary clouds have height one. At λ_b, the transfer windows exclude their lower endpoint, so the count is its reference value $\beta_b + s$. For $b = 1$ this gives $\lambda_1 + \beta_1 + s = M + q = K$; for $b \geq 2$ it gives

$$\lambda_b + \beta_b + s = M + Q_{\mathrm{hi}} + 2s(p-1) = M + q = K.$$

There are two height-H pairs per repeated block and two height-C connectors per segment, totaling Q_{hi}. Hence $H + Q_{\mathrm{hi}} = M + q = K$, whereas

$$C + 2m(p-1) = K - 2s(p-1) + 1 \leq K$$

because $p \geq 2$ and $s \geq 1$. These are all possible exit heights, together with the already checked $\lambda_b + T$. The count $N(t)$ is constant between successive exit heights, so $t + N(t)$ increases there and its maximum occurs at a listed exit height. Sort the corresponding queries, attach their exact one- or two-cloud codes, and finish with cloud 0's private query. This constructs a complete tree of height K. □

Proof strategy.
Bound the number and size of clouds first, then use those bounds to control point coordinates and line coefficients. Count the supplied queries separately, and finish by bounding the size and verification time of a proper identifying-tree certificate.

Proof of Proposition 13.11. There are $2q+m+2$ clouds, each of size at most $C = O(ps+m)$. Their total number of hypotheses is polynomial in the restricted source input. For the explicit geometric bounds, each cloud size and its index are at most the total point count n, and $Q = 4\max_i w_i \leq 4n$. Thus every abscissa satisfies $0 \leq Qi+j \leq 4n^2+n = O(n^2)$, and every ordinate is $O(n^4)$. The cloud-query coefficient $4Q(a+b)$ is $O(n^2)$ and its constant $Q^2(2a-1)(2b+1)$ is $O(n^4)$. The private-query coefficients $4t$ and $2t^2-1$ have the same bounds. There is one private query per nonanchor and at most twice the number of clouds in singleton and path-edge queries, so there are $O(n)$ supplied lines. Every proper identifying tree has $n-1$ internal nodes; its answers and height can be checked in polynomial time. This proves membership in NP and strong NP-hardness with polynomially bounded numerical data. □

13.2.3 Calibration of the height construction

Parameter calibration.
Choose

$$M = m + 2, \qquad \lambda_1 = M,$$
$$\lambda_b = M + (2b-3)s \quad (2 \leq b \leq p), \qquad H = M + 2s(p-1), \qquad C = H + 1.$$

The width $T = s - B < s$ makes the windows disjoint. All block levels are below H, and the connectors have height C. These choices give $t_b + \beta_b = K - B$ and make the minimum distinguished-path length $q + m + 2$ exactly $K = q + M$.

13.3 Expected depth: an exact matching formula

The expected-depth reduction needs no clouds. Each nonzero hypothesis is one path vertex or one isolated vertex, and every query isolates either a singleton or an adjacent pair. The combinatorial choice is still a matching, but its objective depends on how the resulting pairs are ordered.

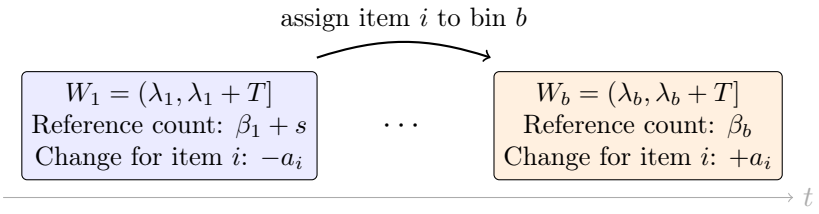


Figure 10: Relative to the reference matching, assigning one item of size a_i from bin 1 to bin b removes a_i from the first histogram window and adds a_i to window b; intermediate windows are unchanged. Superposing the item transfers gives $N(t) = \beta_j + L_j$ throughout window $W_j = (\lambda_j, \lambda_j + T]$.

For a finite positive weight vector z, keeping repeated values as distinct entries, define the *weighted completion sum* by

$$\mathcal{C}(z) = \sum_{j=1}^{|z|} j z_{(j)}, \qquad z_{(1)} \geq \cdots \geq z_{(|z|)}.$$

It equals the sum of the individual weights plus the smaller weight for each unordered pair of entries: in decreasing order, an earlier entry delays a later entry by one. Thus concatenating vectors u, v adds the cross contribution $\sum_{i,j} \min\{u_i, v_j\}$. Equivalently, if $h_z(t) = |\{j : z_j \geq t\}|$, then

$$\mathcal{C}(z) = \int_0^\infty \frac{h_z(t)(h_z(t)+1)}{2}\, dt. \tag{44}$$

At height t, the first $h_z(t)$ ranks contribute $1 + \cdots + h_z(t)$; integrating those contributions proves the identity.

The completion sum becomes a tree objective once first-hit groups are identified with a matching. This correspondence must hold for arbitrary adaptive trees as well as for the sorted constructions.

Lemma 13.12 (Exact singleton/pair formula)**.** *Consider a* linear forest*, meaning a disjoint union of paths (isolated vertices are allowed), with vertices* $1, \ldots, N$ *ordered component by component. Represent them and a distinguished hypothesis by* (i, i^2)*,* $0 \leq i \leq N$*. Supply* $I_{i,i}$ *for each nonzero vertex and* $I_{i,i+1}$ *for each forest edge. Give the distinguished hypothesis weight* $\omega_0 > 0$ *and the others positive weights* ω_v*. For a matching* $\mathcal{M}$*, let* $J(\mathcal{M})$ *be the vector containing mass* $\omega_u + \omega_v$ *for each matched pair and* ω_v *for each unmatched vertex, retaining multiplicities when masses coincide. The optimal unnormalized expected depth is*

$$\min_{\mathcal{M}} \left\{ \omega_0(N - |\mathcal{M}|) + \mathcal{C}(J(\mathcal{M})) + \sum_{\{u,v\} \in \mathcal{M}} (\omega_u + \omega_v) \right\}. \tag{45}$$

If $\omega_0 > N \sum_{v>0} \omega_v$*, an optimal matching in this formula has maximum cardinality.*

Proof strategy.
Read any tree from the distinguished all-miss spine: its disjoint first-hit pairs form a matching. Then sort the group masses; adjacent exchanges minimize first-cover cost. A sufficiently heavy distinguished point forces maximum cardinality before any other cost matters.

Proof. We first establish the invoice in both directions: extract its terms from any adaptive tree, and realize them for any matching. The distinguished-weight condition is a separate restriction applied afterward.

Claim 13.5 (First-hit groups attain the matching cost)**.** *The spine contributes one query per singleton or matched pair, and only a pair requires an extra local query.*

Proof. Follow hypothesis 0, which misses every supplied membership query. The hypotheses first covered together are disjoint singletons or adjacent pairs. The pairs form a matching $\mathcal{M}$, and the spine has $N - |\mathcal{M}|$ queries. A pair's hit branch requires exactly one additional query; a singleton branch requires none. This gives the first and third terms of (45).

For the first-cover term, suppose masses $a < b$ occupy consecutive positions $j, j+1$. Their contribution is $ja + (j+1)b$ in that order and $jb + (j+1)a$ after swapping, a decrease of $b - a$. Repeated exchanges show that nonincreasing mass is optimal. Every such ordering is executable: query the selected pairs and the singleton tests for the unmatched vertices in that order. The groups are disjoint, so each query's hit branch exits only to its own group. This proves the formula both as a lower bound on arbitrary trees and as an attainable value for each matching. □

The exact invoice still permits matchings of different sizes. Giving the distinguished point enough weight makes its spine length decisive, so later gadget calculations need consider only maximum matchings.

Claim 13.6 (The distinguished weight forces maximum cardinality)**.** *Under the stated dominance condition, minimizing its depth is the first objective.*

Proof. Every proper complete tree has depth at most N, so the total weighted depth of all nonzero hypotheses is at most $N \sum_{v>0} \omega_v$. One additional query on hypothesis 0's path outweighs this entire budget under the stated inequality. An optimal tree must consequently minimize that path's length, which means choosing a maximum-cardinality matching. □

□

13.4 Expected depth: bin loads become a quadratic penalty

Theorem 13.13 (Strong NP-completeness of expected depth)**.** *Given supplied binary lines, strictly positive integer hypothesis weights bounded by a fixed polynomial in n, and a rational threshold, deciding the optimal expected depth is strongly NP-complete. This holds for $\mathcal{H} = \{(i, i^2) : 0 \le i < n\}$, $O(n)$ integer-coefficient lines, and negative answer classes of size at most two.*

Proof strategy.

1. Use the exact matching cost formula and a heavy distinguished point to normalize the optimum to one unmatched even vertex per item.
2. Pair-mass histograms encode bin loads. Integrating their tail counts produces a quadratic load penalty and an unwanted linear term.
3. Tiny, ordered boundary-weight adjustments cancel that linear term. Medium-weight isolated vertices exclude internal choices, and scale separation makes rounding smaller than a unit imbalance.

13.4.1 Construction and supporting propositions

Let $q = 2s(p-1)$ be the total matching size and define the reference bin counts

$$\beta_1 = q - s, \qquad \beta_b = 2s(p-b) \quad (2 \le b \le p).$$

Take an isolated-vertex count r, an even window width V, and bin levels λ_b from Section 13.4.3. The r isolated vertices have weight $Z = V/2$. To cancel the bin linear term, define

$$t_i = q + r + m - i, \quad \delta_{ib} = \left\lfloor \frac{V(\beta_1 - \beta_b)a_i}{t_i} \right\rfloor,$$
$$D = \max_{i,b} \delta_{ib}, \qquad A_i = 1 + (2D+1)i.$$

Here t_i will be the exact coefficient of unmatched item i; A_i fixes the unmatched-weight order, and δ_{ib} is its small bin-dependent correction. Since $\beta_1 - \beta_b = (2b-3)s > 0$ for $b \ge 2$, all corrections are nonnegative and $\delta_{i1} = 0$.

For item i, form an odd path

$$E_{i0},\, B_{i1},\, E_{i1},\, \ldots,\, B_{is_i},\, E_{is_i}, \qquad s_i = 2a_i(p-1),$$

where each displayed symbol also denotes that vertex's weight. First define unperturbed even weights

$$E^0_{ij} = \begin{cases} A_i, & j \equiv 0 \pmod{2a_i}, \\ A_i + V, & j \text{ odd}, \\ A_i + 2V, & \text{otherwise}. \end{cases}$$

Each consecutive difference is $+V$ or $-V$. Between two boundary positions there are $2a_i$ steps, of which a_i go up and a_i go down. For $1 \le j \le s_i$ let $b = \lceil j/(2a_i) \rceil$ and put

$$B_{ij} = \begin{cases} \lambda_b - E^0_{i,j-1}, & E^0_{ij} - E^0_{i,j-1} = V, \\ \lambda_{b+1} - E^0_{ij}, & E^0_{ij} - E^0_{i,j-1} = -V. \end{cases}$$

Finally set $E_{i,2a_i(b-1)} = A_i + \delta_{ib}$ at each of the p boundaries and leave all other even weights unperturbed. The odd weights do not change when this adjustment is made.

Add r isolated forest vertices of weight Z. The nonzero forest has $N = 2q + m + r$ vertices, with maximum matching size q. Let P be the total weight of its item-path vertices, $W = P + rZ$, and give hypothesis 0 weight $\omega_0 = (N+1)W + 1$. Use precisely the lines of Lemma 13.12. Normalize the weights only after the construction.

Use $E_+ = q \sum_{i,b} \delta_{ib}$ for the possible pair-mass perturbation and $E_- = \sum_i (t_i - 1)$ for the rounding loss. Write G for the minimum raw weighted cost for a fixed maximum matching in Lemma 13.12, $U = (U_1, \ldots, U_m)$ for its unmatched item weights, and z for its q pair masses. Boundary choice b_i assigns item i to that bin, whose load is $L_b = \sum_{i:b_i=b} a_i$.

The three weight classes serve different purposes: pair masses carry the histogram, isolated vertices penalize internal choices, and small boundary weights permit controlled corrections. Their separation also fixes the orders used in the invoice.

Proposition 13.14 (Separated scales and maximum-matching normalization)**.** *All weights are positive. Every pair mass exceeds Z and every unmatched item weight; boundary weights are below Z and ordered by item index. The optimum uses a maximum-cardinality matching, and the calibration ensures $E_+ + E_- < V$.*

Once matching cardinality is fixed, the distinguished contribution and several cross terms are constant. The remaining completion and unmatched-vertex costs vary with the bin choices, so we record their sum in the matching cost formula below.

Proposition 13.15 (The matching-dependent invoice)**.**

$$G = G_{\mathrm{fix}} + \mathcal{C}(z) + \mathcal{C}(U) + (q-1)\sum_i U_i + r\sum_i \min\{U_i, Z\}, \tag{46}$$

where

$$G_{\mathrm{fix}} = \omega_0(q+m+r) + \frac{Zr(r+1)}{2} + qrZ + P.$$

For boundary matchings, the same transfer idea now changes pair masses. The completion integral converts their tail counts into a quadratic load term and a linear term.

Proposition 13.16 (Pair windows give a quadratic load identity)**.** *Let b_i be the bin chosen by item i and $L_b = \sum_{i:b_i=b} a_i$. Let z^0 be the unperturbed pair-mass vector for these choices, and let C_{ref} be the completion sum of the reference vector in which every item chooses bin* 1*. Then*

$$\mathcal{C}(z^0) = C_{\mathrm{ref}} + V\left[\sum_b \beta_b L_b - \beta_1 s + \frac{\sum_b L_b^2 - s^2}{2}\right]. \tag{47}$$

A quadratic penalty favors balanced loads only after the bin-dependent linear term is removed. The boundary corrections perform that cancellation, and the error bounds preserve a strict threshold gap.

Proposition 13.17 (Boundary corrections cancel the linear term)**.** *For every boundary matching, there is an error e with $-E_- \le e \le E_+$ such that*

$$G = G_{\mathrm{fix}} + \sum_i t_i A_i + C_{\mathrm{ref}} + \frac{V}{2}\left(\sum_b L_b^2 - s^2\right) + e. \tag{48}$$

Define the raw threshold

$$k_0 = G_{\mathrm{fix}} + \sum_i t_i A_i + C_{\mathrm{ref}} + \frac{V}{2}(pB^2 - s^2), \qquad k = k_0 + E_+. \tag{49}$$

A boundary matching has $G \le k$ exactly when every bin load is B.

The quadratic calculation concerns boundary choices. Full soundness also requires excluding internal unmatched vertices; the isolated vertices provide that penalty without introducing non-polynomial data.

Proposition 13.18 (Internal choices are excluded; encoding is strong)**.** *Every maximum matching with an internal unmatched even vertex has $G > k$. The construction and its normalized threshold k/Ω, where $\Omega = \omega_0 + W$, have polynomially bounded numerical data and $O(n)$ supplied lines; a proper tree is a polynomial NP certificate.*

Proof of Theorem 13.13. A source partition selects boundary choices of raw cost at most k by Proposition 13.17. Conversely, Proposition 13.14 reduces the optimum to maximum matchings. Proposition 13.18 excludes internal unmatched vertices, so every unmatched item position chooses a boundary and hence a bin. Proposition 13.17 shows that a qualifying matching has every bin load equal to B. Because two source items sum to less than B and four sum to more than B, each such bin contains exactly three items; the chosen bins therefore give a 3-partition. Normalizing divides both objective and threshold by Ω, while the encoding proposition supplies strong hardness and NP membership, including arbitrary adaptive trees. □

13.4.2 Technical proofs of the expected-depth propositions

Proof strategy.
Compare the weight classes in turn: pair masses must dominate unmatched and isolated weights, while boundary weights must stay below the isolation threshold and retain their item order. These inequalities activate the invoice lemma's maximum-cardinality criterion.

Proof of Proposition 13.14. The scale estimates verified in Section 13.4.3 are used here only to compare the three weight classes. Every even weight is positive. By the first bound in (50), $A_i + \delta_{ib} < V/100$, so $A_i + 2V < 2.01V$ and every unmatched even weight is below $3V$. For an upward step, the two possible pair masses are λ_b and $\lambda_b + V$; for a downward step they are $\lambda_{b+1} + V$ and λ_{b+1}. Since $\lambda_b \geq 12V$ and $\lambda_{b+1} \geq 14V$, all these masses are at least $12V$. Boundary adjustments are nonnegative and can only increase a matched pair's mass. The odd weights are also positive: each is either $\lambda_b - E^0_{i,j-1}$ or $\lambda_{b+1} - E^0_{ij}$, and every unperturbed even weight is less than $2.01V$. Hence every pair mass exceeds $Z = V/2$ and every unmatched even weight. Boundary weights are below Z, and they increase strictly with the item index: $A_{i+1} - (A_i + D) = D + 1 > 0$. Finally, $E_+ + E_- < V$ follows from the last two calibration bounds. Here $\omega_0 = (N+1)W + 1 > NW$, so the dominance condition of Lemma 13.12 restricts the optimum to maximum matchings. By Lemma 13.5, each item then leaves one even vertex unmatched. □

Proof of Proposition 13.15. Split the completion sum into z, U, and the vector of r isolated weights Z. Its within-vector terms are $\mathcal{C}(z)$, $\mathcal{C}(U)$, and $Zr(r+1)/2$. Proposition 13.14 puts every pair mass above every unmatched and isolated weight. Thus the minimum-weight cross-term identity preceding (44) gives cross terms $q\sum_i U_i$ and qrZ. The unmatched/isolated cross term is $r\sum_i \min\{U_i, Z\}$. Thus the total completion term is

$$\mathcal{C}(z) + \mathcal{C}(U) + \frac{Zr(r+1)}{2} + q\sum_i U_i + qrZ + r\sum_i \min\{U_i, Z\}.$$

The item paths partition into matched vertices and the unmatched vertices, so the extra local pair cost is $P - \sum_i U_i$. The witness path has $N - q = q + m + r$ queries and costs $\omega_0(q + m + r)$. Adding these terms replaces $q\sum_i U_i$ by $(q-1)\sum_i U_i$ and gives the displayed G_{fix}, proving the invoice. □

Proof of Proposition 13.16. We first determine the pair-mass histogram for each boundary choice, then integrate it. This separates the local matching calculation from the algebra that produces the quadratic objective.

Claim 13.7 (Boundary movement transfers pair masses between windows)**.** *The pair-mass tail count in window b is $\beta_b + L_b$, and it is unchanged outside the windows.*

Proof. By Lemma 13.5, unmatching boundary $2a_i(b-1)$ fixes all left and right pairs and assigns item i to bin b. Temporarily omit the boundary perturbations and denote the resulting pair-mass vector by z^0. For an upward step in segment b, pairing its odd vertex to the even vertex on the left gives mass λ_b, while pairing right gives $\lambda_b + V$. For a downward step, those masses are $\lambda_{b+1} + V$ and λ_{b+1}. Moving the unmatched vertex across a whole segment consequently changes the pair-mass histogram by

$$-a_i \mathbf{1}_{(\lambda_b, \lambda_b + V]} + a_i \mathbf{1}_{(\lambda_{b+1}, \lambda_{b+1} + V]}.$$

There are exactly a_i steps of each type. Write $W_j = (\lambda_j, \lambda_j + V]$. Moving an item from boundary 1 to boundary b_i adds

$$a_i \sum_{j=1}^{b_i-1} (-\mathbf{1}_{W_j} + \mathbf{1}_{W_{j+1}}) = -a_i \mathbf{1}_{W_1} + a_i \mathbf{1}_{W_{b_i}}.$$

For $b_i = 1$ this is zero; otherwise each intermediate indicator cancels. This is the same telescoping mechanism as in the height construction. In the reference matching, all odd vertices are paired to their right. Segment b contributes s masses $\lambda_b + V$ and s masses λ_{b+1} after summing over items. For a threshold t in window b, where $\lambda_b < t \le \lambda_b + V$, a segment $c < b$ contributes no mass: its larger mass is $\lambda_{c+1} \le \lambda_b < t$. Every segment $c \ge b$ contributes both batches, since $\lambda_c + V \ge \lambda_b + V \ge t$ and $\lambda_{c+1} > \lambda_b + V$. Therefore, when $b \ge 2$, the $p - b$ segments $c = b, \dots, p-1$ contribute $2s(p-b) = \beta_b$ masses in total. In the first window all $p-1$ segments contribute, so the reference count is $q = 2s(p-1) = \beta_1 + s$. Assigning items according to loads L_b changes the tail count throughout window b to $\beta_b + L_b$: the first window loses $s - L_1$ pair masses from its reference count q, while each later window gains L_b. Outside these disjoint windows the tail histogram is unchanged. □

The histogram now depends on the matching only through bin loads. Integrating the triangular count polynomial reveals both the desired imbalance penalty and the linear term that the next proposition must cancel.

Claim 13.8 (Integrating the tail counts yields the quadratic term)**.** *The triangular histogram formula gives* (47)*.*

Proof. Let C_{ref} be the completion sum of the unperturbed pair masses when every first boundary is unmatched. It is explicitly computable by sorting those q masses. Let $f(h) = h(h+1)/2$. Direct expansion gives

$$f(\beta_b + L_b) - f(\beta_b) = \frac{2\beta_b L_b + L_b^2 + L_b}{2} = \beta_b L_b + \frac{L_b^2 + L_b}{2}.$$

By Claim 13.7, all windows have width V and the histogram outside them is unchanged. Therefore (44) reduces the difference of completion sums to those windows. Subtracting the reference loads $(s, 0, \dots, 0)$ therefore gives

$$\begin{aligned}
\mathcal{C}(z^0) - C_{\mathrm{ref}} &= V \left[\sum_b \left(\beta_b L_b + \frac{L_b^2 + L_b}{2} \right) - \beta_1 s - \frac{s^2 + s}{2} \right] \\
&= V \left[\sum_b \beta_b L_b - \beta_1 s + \frac{\sum_b L_b^2 - s^2}{2} + \frac{\sum_b L_b - s}{2} \right] \\
&= V \left[\sum_b \beta_b L_b - \beta_1 s + \frac{\sum_b L_b^2 - s^2}{2} \right].
\end{aligned}$$

The second equality separates the quadratic and linear load terms; the last uses $\sum_b L_b = s$ to cancel the remaining unweighted linear term. This proves the stated identity and identifies the unwanted linear term that remains alongside the useful quadratic term. □

□

Proof of Proposition 13.17. The corrections affect two parts of the invoice. We bound their incidental effect on pair completion cost, compute their intended unmatched contribution, and finally compare the error with an integral load imbalance.

Claim 13.9 (Pair perturbations change completion cost by at most E_+)**.** *The increase is nonnegative and at most* $q \sum_{i,b} \delta_{ib}$.

Proof. Boundary perturbations increase pair masses by at most the total added boundary weight. For a vector z of q pair masses, $\mathcal{C}(z) = \min_\sigma \sum_{j=1}^{q} j z_{\sigma(j)}$, by the adjacent-swap ordering argument in Lemma 13.12. Write the new vector as $z + d$ with $d_j \geq 0$ and $\Delta = \sum_j d_j$. Every ordering's weighted sum is nondecreasing, so taking the minimum gives $\mathcal{C}(z + d) \geq \mathcal{C}(z)$. If σ minimizes the old vector, then

$$\mathcal{C}(z + d) \leq \sum_{j=1}^{q} j(z_{\sigma(j)} + d_{\sigma(j)}) \leq \mathcal{C}(z) + q \sum_j d_j = \mathcal{C}(z) + q\Delta.$$

The first inequality evaluates the new optimum in the old minimizing order, while the second bounds each added mass's rank by q. Thus a change in the sorted order cannot invalidate the perturbation estimate. A perturbed boundary vertex belongs to at most one matched pair, so the total pair-mass increase is at most the sum of all boundary corrections, $\Delta \leq \sum_{i,b} \delta_{ib}$. Denote the pair completion increase by e_{pair}; the inequalities give $0 \leq e_{\text{pair}} \leq E_+$. □

Pair completion may reorder after perturbation, which is why its change was bounded separately. The unmatched weights keep their fixed order, giving an exact coefficient for each correction and the intended cancellation.

Claim 13.10 (The unmatched ranks cancel the unwanted bin linear term)**.** *The boundary cost is* (48) *with* $-E_- \leq e \leq E_+$.

Proof. Proposition 13.14 puts all boundary unmatched weights below Z and fixes their decreasing item order. Indeed, $A_{i+1} - (A_i + D) = D + 1 > 0$, so even the largest perturbation of item i leaves it below the unperturbed weight of item $i + 1$. Thus their terms in (46) are $\sum_i t_i(A_i + \delta_{ib_i})$. Indeed, item i has rank $m - i + 1$ in $\mathcal{C}(U)$, plus coefficient $q - 1 + r$ from the other terms, giving $t_i = q + r + m - i$. The floor definition gives

$$-E_- \leq \sum_i t_i \delta_{ib_i} - V \sum_i (\beta_1 - \beta_{b_i}) a_i \leq 0.$$

For each item, the difference is minus the remainder when the integer $V(\beta_1 - \beta_{b_i})a_i$ is divided by t_i; its magnitude is therefore at most $t_i - 1$. Write $e_{\text{floor}} = \sum_i t_i \delta_{ib_i} - V \sum_i (\beta_1 - \beta_{b_i}) a_i$. The remainder calculation gives $-E_- \leq e_{\text{floor}} \leq 0$. Grouping the intended correction by chosen bin gives

$$V \sum_i (\beta_1 - \beta_{b_i}) a_i = V \beta_1 s - V \sum_b \beta_b L_b.$$

In the invoice the unmatched terms are $\sum_i t_i A_i + \sum_i t_i \delta_{ib_i}$, while the pair completion term equals the window expression plus e_{pair}. Substituting (47) for the unperturbed pair completion term and

Claim 13.9 for its change gives

$$\begin{aligned} G &= G_{\text{fix}} + \sum_i t_i A_i + C_{\text{ref}} \\ &\quad + V\left[\sum_b \beta_b L_b - \beta_1 s + \frac{\sum_b L_b^2 - s^2}{2}\right] \\ &\quad + V\beta_1 s - V\sum_b \beta_b L_b + e_{\text{floor}} + e_{\text{pair}} \\ &= G_{\text{fix}} + \sum_i t_i A_i + C_{\text{ref}} + \frac{V}{2}(\sum_b L_b^2 - s^2) + e. \end{aligned}$$

Here $e = e_{\text{floor}} + e_{\text{pair}}$ satisfies $-E_- \le e \le E_+$, since the first summand is nonpositive and the second is nonnegative with the respective bounds. □

After cancellation, the only matching-dependent term of substantial size is the quadratic load penalty. Integral loads give a minimum positive imbalance, and scale separation prevents the bounded errors from hiding it.

Claim 13.11 (An unbalanced integral load costs more than the error budget)**.** *The threshold k is met exactly when all loads equal B.*

Proof strategy.
Center each load at B and expand the sum of squared deviations; the fixed total load removes the linear term. A valid partition then has zero penalty, whereas any other integral zero-sum deviation has penalty at least V. Finally compare that gap with the two calibrated error bounds.

Proof. Write $\Delta_b = L_b - B$ for the deviation of bin b from its target load. Since the loads total pB, these deviations sum to zero. Expanding each square, then using that total-load identity, gives

$$\begin{aligned} \sum_b (L_b - B)^2 &= \sum_b L_b^2 - 2B\sum_b L_b + pB^2 && \text{by termwise expansion and summation} \\ &= \sum_b L_b^2 - pB^2 && \text{since } \sum_b L_b = pB. \end{aligned}$$

Substituting this identity into Claim 13.10 and the definition of k_0 rewrites the corrected invoice as

$$G = k_0 + \frac{V}{2}\sum_b \Delta_b^2 + e, \qquad -E_- \le e \le E_+.$$

If the source instance has a valid partition, every load equals B, so each $\Delta_b = 0$. The invoice is then $G = k_0 + e \le k_0 + E_+ = k$.

Conversely, suppose the loads are not all B. The deviations are integers, sum to zero, and are not all zero. They must therefore include both a positive and a negative entry: if all were nonnegative or all nonpositive, a zero sum would force every entry to vanish. Each nonzero integer has square at least one, so

$$\sum_b \Delta_b^2 \ge 1^2 + (-1)^2 = 2.$$

It follows that the quadratic penalty is at least $(V/2) \cdot 2 = V$. Together with $e \geq -E_-$, this gives $G \geq k_0 + V - E_-$. Finally, the two bounds in (50) imply

$$V - E_- > \frac{1999}{2000} V > \frac{1}{1000} V \geq E_+.$$

The first strict inequality uses $E_- < V/2000$; the middle inequality follows from $V > 0$ and $1999/2000 > 1/1000$; the last is the stated bound on E_+. Hence $G > k_0 + E_+ = k$, so an unbalanced load cannot meet the threshold. □

□

Proof of Proposition 13.18. We complete soundness before checking encoding. An internal unmatched vertex is heavier than the isolated vertices, so its repeated cross interactions create a penalty exceeding the entire boundary threshold allowance.

Claim 13.12 (The isolated-vertex penalty excludes every internal choice)**.** *An internal unmatched even vertex makes the raw invoice exceed k.*

Proof strategy.
Lower-bound the isolated-vertex cross term when an internal vertex is unmatched, and upper-bound the threshold using the balanced-load formula. The calibration makes the former penalty larger than all remaining threshold terms.

Proof. The preceding calculation proves soundness for boundary choices. It remains to rule out a maximum matching that leaves an internal even vertex unmatched. By Lemma 13.5, every maximum matching leaves one even vertex unmatched on each item path. If item i uses a boundary, its unmatched weight is $U_i = A_i + \delta_{ib_i}$ for some bin b_i, with $\delta_{ib_i} \geq 0$. An internal choice instead has weight $A_i + V$ or $A_i + 2V$. Thus every unmatched weight satisfies $U_i \geq A_i$, while the internal choice at item i_* has $U_{i_*} > Z$ because $V = 2Z$. Since $\delta_{i1} = 0$, the scale bound (50) gives $A_i = A_i + \delta_{i1} < V/100 < V/2 = Z$. Consequently $\min\{U_i, Z\} \geq A_i$ for every item, and $\min\{U_{i_*}, Z\} = Z$. Therefore

$$\sum_i \min\{U_i, Z\} = Z + \sum_{i \neq i_*} \min\{U_i, Z\} \geq Z + \sum_{i \neq i_*} A_i = \sum_i A_i + Z - A_{i_*} \geq \sum_i A_i + Z - \max_i A_i.$$

The other terms in the matching-dependent invoice (46) are nonnegative. Indeed, $\mathcal{C}(z)$ and $\mathcal{C}(U)$ are positive completion sums, and $q = 2s(p-1) \geq 2$ because $p \geq 2$ and $s = pB \geq 1$; hence $(q-1)\sum_i U_i \geq 0$ by positivity of the unmatched weights. Since $r > 0$ as well, substituting the preceding lower bound into the isolated-vertex term and retaining the other nonnegative terms gives

$$G \geq G_{\text{fix}} + r \sum_i A_i + r\big(Z - \max_i A_i\big).$$

We compare this with the boundary threshold from Proposition 13.17, whose threshold definition is recorded in (49). Since $s = pB$ and $p \geq 2$,

$$pB^2 - s^2 = pB^2 - p^2B^2 = -p(p-1)B^2 \leq 0.$$

Thus the quadratic reference term in k_0 is nonpositive and may be dropped for an upper bound. Moreover, from $t_i = q + r + m - i$ we obtain

$$\sum_i t_i A_i = r \sum_i A_i + \sum_i (q + m - i) A_i \leq r \sum_i A_i + (q+m) \sum_i A_i,$$

because $q + m - i \le q + m$ and every A_i is positive. Substituting this bound into the threshold formula gives

$$k \le G_{\text{fix}} + r \sum_i A_i + C_{\text{ref}} + (q+m) \sum_i A_i + E_+.$$

Each reference pair mass is one of λ_b or $\lambda_b + V$. Because $\lambda_b = (10 + 2b)V$ increases with b, every such mass is at most $\lambda_p + V = (11 + 2p)V$. By the definition of the weighted completion sum, these q masses receive ranks $1, \dots, q$; hence

$$C_{\text{ref}} \le (11 + 2p)V \frac{q(q+1)}{2} < (p+6)(q+1)^2 V.$$

The first inequality bounds every reference mass by its largest level and sums the ranks $1, \dots, q$. For the strict step, $(11 + 2p)/2 = p + 11/2 < p + 6$, and $q(q+1) < (q+1)^2$ because $q + 1 > 0$. Also, (50) with $\delta_{i1} = 0$ gives $A_i < V/100$ for every i, so $\sum_i A_i < mV/100$; the same scale bounds give $E_+ \le V/1000$. Set $X = (q + m + 1)^2 V$. Since $q + 1 \le q + m + 1$ and $m \le q + m + 1$, we have $(q+1)^2 V \le X$ and $(q+m)mV \le X$. Also $q + m + 1 \ge 1$, so $V/1000 \le X/1000$. Therefore

$$\begin{aligned} C_{\text{ref}} + (q+m) \sum_i A_i + E_+ &< (p+6)X + \frac{X}{100} + \frac{X}{1000} \\ &< (p+8)X. \end{aligned}$$

The last strict inequality follows from $p + 6 + 1/100 + 1/1000 < p + 8$. The calibration also gives the factors in the penalty explicitly: $Z = V/2$ and $\max_i A_i < V/100$, so $Z - \max_i A_i > V/2 - V/100 = 49V/100 > V/3$. Its definition of r gives $r > 1000(p+10)(q+m+1)^2$, since the remaining summands in the brackets are positive. Multiplying these positive bounds gives

$$r(Z - \max_i A_i) > \frac{1000}{3}(p+10)(q+m+1)^2 V > (p+8)(q+m+1)^2 V,$$

where the final inequality follows from $1000(p+10) > 3(p+8)$. Thus the internal-choice lower bound exceeds the threshold upper bound. These bounds share the term $G_{\text{fix}} + r \sum_i A_i$; after subtracting it, the penalty remainder is strictly larger than the threshold remainder, so $G > k$. □

The penalty and error margins must be attainable with polynomial scales. The final size calculation verifies this and gives the certificate bound needed for strong NP-completeness.

Claim 13.13 (The reduction has polynomial numerical data and certificates)**.** *The normalized instance establishes strong hardness with polynomially bounded numerical data.*

Proof strategy.
Bound the unnormalized weights from the source-size parameters, then bound their sum and the distinguished weight. These bounds control the normalized rational input size; the proper-tree certificate has at most $n - 1$ internal nodes and is checked by simulating its supplied queries.

Proof. Normalize the positive weights by their total Ω and use the expected-depth threshold k/Ω. The source restriction above gives $B = \text{poly}(m)$ and $a_{\max} < B/2$, while $p = m/3$. Hence $s = pB$, $q = 2s(p-1)$, $\beta_1 = q - s$, and the calibrated value of r are all polynomial in m. The same then holds for V and the bin levels λ_b. The floor formula makes each δ_{ib} polynomially bounded, so its maximum D, the boundary weights A_i, and the even and odd path weights are polynomially bounded as well. Consequently W and $\omega_0 = (N+1)W + 1$ are polynomially bounded. Thus the forest, its weights, and

the $O(N)$ supplied lines are constructed in polynomial time in the source input. Write $n = N + 1$. The forest has $N = 2q + m + r$ vertices, so $q, m, r \le N < n$. Moreover, $q = 2s(p-1)$ with $s \ge 1$ gives $p \le q + 1 \le n$ and $s \le q/2 < n$. Since $B = s/p \le s$, $B < n$ as well. In the calibration formula for r, the factor $\beta_1 = q - s = s(2p-3)$ is at least one, and $qmp + 2m + 1 \ge 1$; therefore its second summand alone gives $r \ge a_{\max}$. We have thus bounded every source-size parameter by n. It follows that $V = 2000m(r + q + m + 1) = O(n^2)$ and $\lambda_b = (10 + 2b)V = O(n^3)$. Proposition 13.14 gives $A_i + \delta_{ib} < V/100$ and $E^0_{ij} < 2.01V$. Every odd weight is either $\lambda_b - E^0_{i,j-1}$ or $\lambda_{b+1} - E^0_{ij}$. Since $\lambda_b \ge 12V$, these weights are positive; subtracting a positive even weight also places each below its corresponding bin level, and hence below λ_{b+1}. The same proposition bounds internal even weights by $3V$; isolated weights equal $V/2$. Thus every nonzero weight is $O(n^3)$. There are $O(n)$ vertices, so the total nonzero weight $W = P + rZ$ is $O(n^4)$, and the distinguished weight $\omega_0 = (N+1)W + 1$ is $O(n^5)$.

The parabola abscissae are at most N, their squares are $O(n^2)$, and the singleton and pair lines $I_{a,b}$ have coefficients bounded by $O(n^2)$. There are N singleton queries and at most N path-edge queries. A proper complete tree has $n - 1$ internal nodes, so evaluating its supplied answers and weighted depths takes polynomial time. We next bound the threshold. Its distinguished term in G_{fix} is $\omega_0(q+m+r) = O(n^6)$. The other terms have smaller orders: $Zr(r+1)/2 = O(n^4)$, $qrZ = O(n^4)$, and $P = O(n^4)$. Therefore $G_{\text{fix}} = O(n^6)$. The reference completion sum has $q = O(n)$ pair masses. Each is at most $(11 + 2p)V = O(n^3)$, and its rank is at most $q = O(n)$; summing these $O(n)$ contributions gives $C_{\text{ref}} = O(n^5)$. Also, $t_i = O(n)$ and $A_i < V/100 = O(n^2)$, so $\sum_i t_i A_i = O(n^4)$. Since $s = pB$ and $p \ge 2$,

$$\left|pB^2 - s^2\right| = p(p-1)B^2 = s^2\frac{p-1}{p} \le s^2.$$

Therefore the magnitude of the quadratic threshold term is at most $Vs^2/2 = O(n^4)$. Finally, $E_+ \le V/1000 = O(n^2)$. Applying these estimates to each term in k gives $|k| = O(n^6)$, while $\Omega = W + \omega_0 = O(n^5)$ and $\Omega \ge 1$. Thus every normalized prior has a polynomially bounded integer numerator and denominator, as does k/Ω. This establishes strong NP-completeness with strictly positive, polynomially bounded hypothesis weights. □

□

13.4.3 Calibration of the expected-depth construction

The semantic quantities q, β_b, t_i, L_b describe matchings and bin loads. The following slack choices only separate their weight scales; they are not additional source data.

Parameter calibration.
With $a_{\max} = \max_i a_i$, choose

$$r = 1000\big[(p+10)(q+m+1)^2 + (qmp + 2m + 1)\beta_1 a_{\max} + 1\big],$$
$$V = 2000m(r + q + m + 1), \qquad \lambda_b = (10 + 2b)V \quad (1 \le b \le p).$$

These polynomial integers make V even and give

$$\max_{i,b}(A_i + \delta_{ib}) < V/100, \qquad E_+ \le V/1000, \qquad E_- < V/2000. \tag{50}$$

In particular $E_+ + E_- < V$; a load imbalance cannot be hidden by rounding or pair perturbation.

Since $t_i = q + r + m - i \ge r$ and $0 \le \beta_b \le \beta_1$, the floor definition gives $\delta_{ib} \le V\beta_1 a_{\max}/r$ and hence the same bound for D. The second term in r makes $r > 1000(qmp + 2m + 1)\beta_1 a_{\max}$.

Therefore

$$E_+ = q\sum_{i,b}\delta_{ib} \le qmpD < \frac{qmp}{1000(qmp+2m+1)}V < \frac{V}{1000}.$$

The first inequality bounds each of the mp corrections by D; the next substitutes the strict lower bound on r into the estimate for D, and the last uses $qmp < qmp+2m+1$. Similarly $(2m+1)D < V/1000$. The largest adjusted boundary weight satisfies

$$\begin{aligned} A_i + \delta_{ib} &\le 1 + (2D+1)m + D = (2m+1)D + m + 1 \\ &< \frac{V}{1000} + \frac{V}{2000} < \frac{V}{100}. \end{aligned}$$

Here $m+1 \le V/2000 = m(r+q+m+1)$ because $m \ge 1$ and $r+q+m+1 \ge m+1$. Also $t_i - 1 < r+q+m+1$, so

$$E_- = \sum_i (t_i - 1) < m(r+q+m+1) = \frac{V}{2000}.$$

In particular $E_+ + E_- < 3V/2000 < V$.

The boundary bound gives $Z - \max_i A_i > V/2 - V/100 = 49V/100 > V/3$. The first term of r gives $r/3 \ge (1000/3)(p+10)(q+m+1)^2 > (p+8)(q+m+1)^2$, since $1000(p+10) > 3(p+8)$. Together these bounds exclude internal unmatched vertices: the boundary gap supplies the per-vertex penalty, and the lower bound on r makes that penalty dominate the adjustment error. Both quantities remain polynomial in the restricted source data.

14 Uniform expected-depth hardness

We transfer the weighted singleton/pair hardness instance to a uniform prior by replacing each vertex of weight w with a cloud of w hypotheses. Private queries can occur anywhere in an adaptive tree, so the key step is to move them off the distinguished spine while preserving or reducing cost. This normalization exposes whole clouds as matching groups and yields an exact identity: the cloud optimum is the weighted optimum plus a computable internal-cloud cost. Here raw uniform cost denotes the sum of leaf depths, before division by the number of hypotheses.

14.1 Exact cloud replacement: normalization before accounting

14.1.1 The construction

Consider the weighted singleton/pair linear forest instance of Lemma 13.12, with vertices $0, 1, \dots, N$ and positive integer weights $w_0, \dots, w_N$. Vertex 0 is distinguished. Order the nonzero vertices along each path and component by component, so every edge joins consecutive indices. Here a cloud C_i is the block of w_i hypotheses representing vertex i, its anchor is the point with offset $j = 0$, and a private query is one of the supplied lines that isolates a single nonanchor point. The singleton and pair queries represent, respectively, a vertex and an edge of the weighted forest. The invoice notation $J(\mathcal{M})$ and $\mathcal{C}(J(\mathcal{M}))$ is as in Lemma 13.12; it records the weighted instance left after choosing the matching $\mathcal{M}$ of pair queries. Choose the cloud spacing Q by the final calibration below and define

$$n = \sum_{i=0}^{N} w_i, \qquad C_i = \{(Qi+j, (Qi+j)^2) : 0 \le j < w_i\}.$$

The point with $j = 0$ is the anchor of cloud C_i. For every nonzero vertex supply its whole-cloud singleton query; for every linear forest edge supply its whole-cloud pair query. For consecutive cloud indices $a, \dots, b$, these are the negative answers of

$$4y - 4Q(a+b)x + Q^2(2a-1)(2b+1) = 0. \tag{51}$$

The value on the parabola factors as $4(x - Q(a - \frac{1}{2}))(x - Q(b + \frac{1}{2}))$. The product is negative exactly between its two roots. Cloud c has abscissae in $[Qc, Qc + w_c - 1]$, with $w_c - 1 < Q/4$; hence every point in clouds a through b lies strictly between the roots, while each other cloud lies strictly outside. Since $a > 0$, cloud 0 is excluded. Thus the negative answer selects exactly the designated cloud or adjacent pair, and no point lies on the query line.

For every nonanchor point of abscissa t, including those of cloud 0, supply the private line

$$2y - 4tx + 2t^2 - 1 = 0.$$

Its value on (x, x^2) is $2(x - t)^2 - 1$; it is negative at $x = t$ and positive at every other integer abscissa, since then $|x - t| \geq 1$. It therefore isolates precisely that point, with no point on the line. No private line is supplied for an anchor. All answers are strict and binary. The whole-cloud singleton queries distinguish each nonzero anchor from all other anchors and cloud 0; the private queries then separate every nonanchor from the anchor in its cloud. Consequently the query family identifies every hypothesis. There are $n = \sum_i w_i$ hypotheses, so the uniform prior gives each weight $1/n$.

Put

$$D(w) = \frac{(w-1)(w+2)}{2}, \qquad \Delta = \sum_{i=0}^{N} D(w_i). \tag{52}$$

Algorithm 21 gives the construction and the threshold conversion.

Algorithm 21: Uniform cloud replacement of a weighted linear forest instance

Input: Vertices $0, \dots, N$ with each path in order, component by component, positive integer weights w_i, and a raw weighted threshold k

Output: Supplied-line instance with uniform prior and threshold $(k + \Delta)/n$

```
Q ← 4 max_i w_i; n ← Σ_i w_i; H ← ∅; L ← ∅
for i = 0, ..., N do
    for j = 0, ..., w_i − 1 do
        t ← Qi + j; add (t, t^2) to H
        if j > 0 then
            Add 2y − 4tx + 2t^2 − 1 = 0 to L
For each i > 0, add (51) with (a, b) = (i, i) to L
For each linear forest edge {i, i + 1}, add (51) with (a, b) = (i, i + 1) to L
Δ ← Σ_{i=0}^{N} (w_i − 1)(w_i + 2)/2
return (H, L), prior 1/n, and threshold (k + Δ)/n
```

Theorem 14.1 (Exact uniform cloud transfer)**.** *Let* $\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}}$ *be the minimum raw weighted cost of the singleton/pair linear forest instance. For its uniform cloud replacement,*

$$n\,\mathrm{OPT}_{\mathrm{E}}^{\mathrm{cloud}} = \mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} + \Delta. \tag{53}$$

Equivalently, if $J(\mathcal{M})$ *has the pair and singleton masses from Lemma 13.12, the raw uniform optimum is*

$$\Delta + \min_{\mathcal{M}} \left\{ w_0(N - |\mathcal{M}|) + \mathcal{C}(J(\mathcal{M})) + \sum_{\{u,v\} \in \mathcal{M}} (w_u + w_v) \right\}.$$

Proof strategy.

1. Rotate a private singleton past the next query. Equal point weights make its one-unit depth increase no larger than the savings on the opposite, nonempty answer branch.
2. Push all external private queries off the distinguished anchor's spine and move its own private chain to the end. Every first-hit cloud is then intact, so the first-hit pairs form a matching.
3. Sum exact one- and two-cloud local costs, then sort group sizes. The result is the weighted matching cost plus the fixed offset Δ; the same construction attains it.

14.1.2 Three supporting statements

Uniform weights make one local exchange cost nonincreasing: delaying an isolated point by one query saves a query for a nonempty opposite branch. Since every point has the same weight, the exchange changes the raw cost only through the depths immediately above the attached subtrees; their internal costs cancel. This is the tool for normalizing arbitrary trees.

Lemma 14.2 (Uniform singleton rotation)**.** *Suppose a proper identifying tree on a uniformly weighted state S first asks a supplied query isolating $x \in S$, and the other child first asks a query r that is proper on $S \setminus \{x\}$. These two queries can be exchanged, placing the singleton query in the answer branch of r containing x, without increasing the raw cost.*

Once clouds reach their first hit intact, their internal cost can be accounted for exactly. We need both the single-cloud value and a check that early private queries cannot improve a two-cloud exit.

Lemma 14.3 (Local uniform cloud costs)**.** *The optimal raw local cost on one intact cloud of size w is $D(w)$. On two intact clouds of sizes u, v, it is*

$$D(u) + D(v) + u + v.$$

The rotations must establish a global normal form before costs are summed. External private queries leave the distinguished spine, while its own private chain moves to the end, exposing whole-cloud matching groups.

Proposition 14.4 (A normalized matching spine)**.** *An optimal uniform cloud tree can be chosen so that its distinguished anchor's path first asks only whole-cloud queries, with every newly hit cloud intact, and then asks only cloud-0 private queries. The first-hit pairs form a matching $\mathcal{M}$. For this matching, the best raw cost is*

$$\Delta + w_0(N - |\mathcal{M}|) + \mathcal{C}(J(\mathcal{M})) + \sum_{\{u,v\} \in \mathcal{M}} (w_u + w_v).$$

Every matching has a tree attaining this invoice.

Proof of Theorem 14.1. Lemma 14.2 permits the normalization in Proposition 14.4 without increasing cost. Lemma 14.3 supplies its exact local offset. Minimize the resulting invoice over matchings and apply Lemma 13.12. This gives $\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{cloud}} = \mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} + \Delta$. Dividing by n proves the displayed expected-depth identity. □

14.1.3 Technical proofs of the supporting statements

Proof strategy.
First, compare the two possible query orders by their per-state depth changes; uniform weights make the singleton rotation nonincreasing. Next, compute the forced private-chain cost for one cloud and use induction to show that a whole-cloud separator is optimal for two. Finally, apply those rotations to normalize the distinguished path, then sum each first-hit group's local cost and order the groups by decreasing mass. This yields the matching cost formula and its attaining construction.

Proof of Lemma 14.2. Let A, B be the two nonempty answer classes of r on $S \setminus \{x\}$, and choose their names so that x has the answer of A. In the original tree, x has relative depth one and every hypothesis in $A \cup B$ passes through both queries. Ask r first, put the private singleton query in its $A \cup \{x\}$ branch, and attach the original subtrees on A and B. Originally x has relative depth one and points of A and B have relative depth two before their attached subtrees. Afterwards x has relative depth two, the points of A still have depth two, and the points of B have depth one before their unchanged subtrees. Thus the raw cost change is

$$(2-1) + |A|(2-2) + |B|(1-2) = 1 - |B| \leq 0,$$

because B is nonempty. The resulting queries are proper and are members of the original supplied family. □

If the opposite class B has one point, the rotation has zero cost; if it has more than one, the cost decreases. The lemma concerns equal weights, so we use the rotation only when every point contributes the same unit cost at each level.

Proof of Lemma 14.3. We first compute the forced private-chain cost within one cloud. For two clouds, we then compare separating them immediately with every possible private-first choice; this verifies the offset without assuming a local normal form.

Claim 14.1 (A single cloud forces a private chain)**.** *Its raw local cost is $D(w)$, including zero cost when $w = 1$.*

Proof. Within one cloud, every whole-cloud query is constant, and every nonconstant query isolates one of its nonanchors. The only possible proper code is a chain: every private query has a singleton outcome, and the other outcome retains the anchor and all points not yet isolated. Thus the nonanchors exit at depths $1, \ldots, w-1$, and the anchor exits at depth $w-1$. The sum of depths is $1 + 2 + \cdots + (w-1) + (w-1)$. For $w \geq 2$,

$$\sum_{j=1}^{w-1} j + (w-1) = \frac{w(w-1)}{2} + (w-1) = \frac{(w-1)(w+2)}{2} = D(w).$$

For $w = 1$ there is no query and both the sum and $D(1)$ are zero.

□

Separating the clouds gives the proposed cost directly. The remaining question is whether isolating a private point first could save more; induction accounts for the delay imposed on the other cloud.

Claim 14.2 (A whole-cloud separator is optimal on two clouds)**.** *Private-first choices cannot beat $D(u) + D(v) + u + v$.*

Proof. On two clouds a nonconstant whole-cloud query separates the clouds. Its root query is traversed by all $u + v$ points, after which the clouds cost $D(u) + D(v)$; this attains the displayed formula. Every other nonconstant query isolates one nonanchor. Induct on $u + v$, with the whole-cloud separator available at every induction state. If a private point of the u cloud is queried first, the remaining two-cloud state has sizes $u - 1, v$ and still has its anchors and whole-cloud separator. Using induction, this choice costs

$$u + v + D(u - 1) + D(v) + (u - 1) + v.$$

A private point exists in this cloud only if $u \geq 2$. The single-cloud formula of Claim 14.1 gives

$$D(u) - D(u - 1) = \frac{(u - 1)(u + 2) - (u - 2)(u + 1)}{2} = u.$$

Subtracting the proposed separator cost from the private-first cost therefore gives

$$\begin{aligned}[u + v + D(u - 1) + D(v) + (u - 1) + v] - [D(u) + D(v) + u + v] \\ = D(u - 1) - D(u) + (u - 1) + v = -u + (u - 1) + v = v - 1 \geq 0.\end{aligned}$$

The second equality substitutes $D(u) - D(u - 1) = u$, and the final inequality uses the presence of the other cloud, namely $v \geq 1$. Thus any saving on the queried cloud is paid for by delaying the other cloud's points. For a private point in the other cloud, the identical calculation with u, v interchanged gives $u - 1 \geq 0$. The base case $u = v = 1$ has raw cost two. When one size is one it has no private point, and the same induction applies to the other cloud. Queries selecting other clouds are constant or separate these two whole clouds, so the argument covers the entire supplied family. □

□

Proof of Proposition 14.4. The two normalization steps have different destinations. A nonzero cloud's private query should leave the distinguished path at its whole-cloud hit; a cloud-0 private query stays on that path but moves behind its whole-cloud queries.

Claim 14.3 (External private queries can be pushed off the spine)**.** *No nonzero cloud needs to lose a private point before its first whole-cloud hit.*

Proof. Start with a proper optimal uniform tree and follow the anchor of cloud 0. Suppose a private query for a nonanchor x of a nonzero cloud occurs on this path. Its own cloud anchor is still on the path: x agrees with its anchor on every whole-cloud query, and private queries never remove an anchor. At each step, let r be the next query on the anchor-0 path. The private query isolates x, so the anchor-0 path continues in the other child; the state reaching r is precisely that non-x answer class. Since r is proper on $S \setminus \{x\}$, it remains proper on this state. Apply Lemma 14.2 to move the private query for x past r. This also applies when r is private: it isolates a different point, and the same depth comparison is nonincreasing. Eventually a whole-cloud query separates x's anchor from the anchor of cloud 0: otherwise the tree would not identify both anchors. The point x has the answer of its own anchor to that query. After the rotation its private query is in the hit branch, off the anchor-0 path.

Each completed operation removes one external private query from the path and introduces none. Repeating over the finite set of such queries terminates; each individual push crosses only the finite path prefix up to the separating whole-cloud query. □

External private queries have now been removed from the spine. The remaining private points agree with the distinguished anchor on every whole-cloud query, so the same rotation postpones their chain without moving it to an exit.

Claim 14.4 (The distinguished private chain can be moved to the end)**.** *All whole-cloud queries can precede cloud* 0*'s private queries.*

Proof. Next move each cloud-0 private query past the next whole-cloud query on the anchor path by the rotation lemma. Its point always has the answer of the anchor of cloud 0 to that query, so the path continues through the whole-cloud query and the private query remains on the path behind it. This second process terminates because each rotation decreases by one the finite number of private/whole-cloud inversions (pairs in which a private query precedes a whole-cloud query on this path). Every rotation is cost nonincreasing by Lemma 14.2; together with Claim 14.3, this establishes the required whole-cloud-first spine.

□

With private queries in these positions, each first-hit cloud is intact. We can therefore use the exact local costs and sort group masses; constructing the same arrangement proves that the resulting invoice is attainable.

Claim 14.5 (The normalized groups attain the matching cost)**.** *The minimum normalized raw cost is the expression in Theorem 14.1.*

Proof. Claims 14.3 and 14.4 imply that all newly hit clouds are intact at their first whole-cloud query: no private query for any of their points has occurred on the path earlier. First-hit groups are whole singleton clouds or whole adjacent pairs, and the pairs form a matching $\mathcal{M}$: the only supplied whole-cloud queries are singleton queries and pair queries on forest edges, and a cloud can belong to only one first-hit group. A query first hitting just one cloud (for example, a pair query whose other endpoint has already left the path) can be replaced by its supplied whole-cloud singleton query, which has the same split on the current state. There are $|\mathcal{M}|$ pair groups and $N - 2|\mathcal{M}|$ singleton groups, hence $k = N - |\mathcal{M}|$ whole-cloud spine queries. Every point of cloud 0 traverses all of them, contributing $w_0 k$, followed by its private-chain cost $D(w_0)$. A singleton cloud contributes $D(w_i)$ after its first hit. By Lemma 14.3, a pair contributes $D(w_u) + D(w_v) + w_u + w_v$ after its first hit. Summing these local costs over disjoint groups therefore gives $\Delta + \sum_{\{u,v\}\in\mathcal{M}}(w_u + w_v)$.

If the group's total masses in spine order are $a_1, \ldots, a_k$, their first-hit depths contribute $\sum_j j a_j$: every point in group j passes through exactly the first j whole-cloud queries before it exits the distinguished path. For consecutive $a < b$, exchanging their positions changes their contribution from $ja + (j+1)b$ to $jb + (j+1)a$, a decrease of $b - a > 0$. Thus their minimum is $\mathcal{C}(J(\mathcal{M}))$ in nonincreasing mass order. Adding the witness, first-hit, and local contributions gives the stated matching cost as a lower bound. Every $D(w_i)$ occurs once because the first-hit groups partition the nonzero clouds and the final private chain accounts for cloud 0; this is precisely why their sum is the matching-independent offset Δ of (52).

Conversely, for any matching, query its disjoint pairs and unmatched singleton clouds in decreasing total cloud size. Each selected pair query or singleton query peels exactly its group from the still-unhit clouds on the distinguished path; the branch that exits the path contains exactly that group's cloud or clouds. Attach the optimal local codes of Lemma 14.3 in those branches and identify cloud 0 by its private chain at the end. This realizes the decreasing-mass first-hit order and attains the matching invoice plus Δ. Taking the minimum proves equality. □

□

14.2 Uniform-prior strong NP-completeness

Theorem 14.5 (Strong NP-completeness under a uniform prior)**.** *Deciding whether the optimal expected depth is at most a rational threshold is strongly NP-complete under a uniform prior, even for integer-coordinate points on a parabola, polynomially bounded integer line coefficients, and $O(n)$ supplied strict binary lines.*

Proof strategy.

1. Apply the exact cloud replacement to the hard weighted forest instances of Theorem 13.13.
2. Their polynomial integer weights allow explicit replication in polynomial time. Separated parabola clouds give strict binary queries with polynomial coordinates and coefficients.
3. Convert the raw threshold to $(k+\Delta)/n$ by the transfer identity. A proper tree provides a polynomial certificate.

14.2.1 Supporting size and encoding propositions

Replacing a weight by that many points is polynomial only when the weights themselves are polynomially bounded. The hard weighted family satisfies this stronger requirement, so explicit clouds are feasible.

Proposition 14.6 (Polynomial replication)**.** *For a hard weighted instance with $n_0 = N+1$ hypotheses, the explicit cloud replacement has $n = O(n_0^5)$ points and is constructible in polynomial time on the restricted 3-Partition source instances.*

Replication increases the number of points, and spacing the clouds increases their coordinates. We must bound both effects while retaining strict binary answers and the supplied-query geometry.

Proposition 14.7 (Strong geometric encoding)**.** *The replacement has $O(n)$ supplied strict binary lines. Its integer abscissae are $O(n^2)$; its ordinates and line coefficients are $O(n^4)$. The points remain in strictly convex position on a parabola.*

The transfer identity adds a fixed cost to every matching. Adding the same offset to the threshold preserves the decision answer; polynomial threshold data and tree certificates complete the hardness argument.

Proposition 14.8 (Threshold conversion and NP certificates)**.** *For the hard weighted instances and thresholds of Theorem 13.13, the rational threshold $(k+\Delta)/n$ has polynomially bounded numerator and denominator. Its yes/no answer agrees with that of the weighted raw threshold k for arbitrary adaptive trees, and the uniform decision problem belongs to NP.*

Proof of Theorem 14.5. Apply Algorithm 21 to the weighted hard instances. Proposition 14.8 and Theorem 14.1 preserve the yes/no answer. Propositions 14.6 and 14.7 give a polynomial reduction with polynomial numerical data. The certificate proposition supplies NP membership, proving strong NP-completeness. □

14.2.2 Technical proofs of the encoding propositions

Proof strategy.

The replication bound follows by summing the calibrated weights. For the geometric bound, count the queries and substitute the spacing bound into the point and line formulas. For threshold size and NP membership, use the transfer identity, bound Δ by n^2, and encode a proper identifying tree by its $n-1$ internal nodes.

Proof of Proposition 14.6. Proposition 13.18 and its calibration give nonzero weights $O(n_0^3)$ and distinguished weight $O(n_0^5)$. There are $N = n_0 - 1$ nonzero weights, so their sum is $O(n_0 \cdot n_0^3) = O(n_0^4)$. Consequently $n = w_0 + \sum_{i>0} w_i = O(n_0^5) + O(n_0^4) = O(n_0^5)$. The restricted source makes n_0 polynomial in its combinatorial size, so all points and queries can be listed explicitly in polynomial time. □

Proof of Proposition 14.7. There are $n - (N + 1)$ private lines and at most $2N$ whole-cloud lines, hence $O(n)$ supplied queries. The calibrated spacing bounds all coordinates and coefficients as verified in the calibration below. On every private line the value is $2(x - t)^2 - 1$, which is -1 at its target and at least 1 at every other integer abscissa. The whole-cloud lines have the factored values in the construction, with roots strictly between clouds, so their answers are also strict. Each nonzero cloud is separated from the rest by its singleton query, and each nonanchor is separated from every other point by its private query; hence all hypotheses are distinguishable. Finally, for any point (t, t^2), the tangent $y = 2tx - t^2$ satisfies $s^2 - (2ts - t^2) = (s - t)^2 > 0$ at every other abscissa s. Thus each sampled point is an exposed vertex of the convex hull, proving strict convex position. □

Proof of Proposition 14.8. Theorem 14.1 gives $\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} \le k$ if and only if $\mathrm{OPT}_{\mathrm{E}}^{\mathrm{cloud}} \le (k+\Delta)/n$. For the weighted instance, Proposition 13.18 gives $k = O(n_0^6)$ and total weight $\Omega = O(n_0^5)$, where $n_0 = N + 1$. The cloud construction has $n \ge n_0$, since every weight is a positive integer, so $k = O(n^6)$. For $w \ge 1$,

$$w^2 - D(w) = \frac{w^2 - w + 2}{2} \ge 0.$$

Indeed $w^2 - w = w(w - 1) \ge 0$, so each cloud's offset is at most the square of its size. Thus

$$\Delta \le \sum_i w_i^2 \le \left(\sum_i w_i \right)^2 = n^2,$$

where the second inequality adds the nonnegative cross products. Hence the unreduced numerator satisfies $k + \Delta = O(n^6)$, while the denominator is n. Both integers have $O(\log n)$ binary digits, and reducing the fraction can only decrease their magnitudes. A proper identifying tree has $n - 1$ internal nodes and is a polynomial certificate. Evaluate each supplied answer and sum the leaf depths to verify its rational expected cost in polynomial time. □

14.2.3 Calibration of the cloud spacing

Parameter calibration.

Choose $Q = 4 \max_i w_i$. Every cloud has width less than $Q/4$; the half-anchor boundaries of (51) therefore miss all points and select exactly the stated clouds. Since $Q \le 4n$ and $N \le n$, every abscissa is $O(n^2)$. Squaring it bounds the ordinates by $O(n^4)$, and the formulas for both query types bound all integer coefficients by $O(n^4)$.

Remark 14.9 (Limits of the reduction). The cloud identity adds the same nonnegative internal cost Δ to every matching. Let T_w be the matching returned by the weighted EPTAS and let T_u be its lifted tree on the uniform cloud instance. The exact cost identity gives

$$c(T_u) = c(T_w) + \Delta \le (1 + \varepsilon)\,\mathrm{OPT}_{\mathrm{weighted}} + \Delta \le (1 + \varepsilon)(\mathrm{OPT}_{\mathrm{weighted}} + \Delta) = (1 + \varepsilon)\,\mathrm{OPT}_{\mathrm{uniform}}\,.$$

Thus the lift preserves the EPTAS as well as exact hardness. A constant-factor inapproximability result would need a different construction.

15 Conclusion

The table collects the approximation factors proved for the two depth objectives and the two answer models. Here γ is the calibrated count-potential coefficient from Section 8.3, and ρ_{two} is the calibrated expected-depth factor from Section 6.3.

Objective	Polynomial time	Quasipolynomial time
Maximum depth, binary answers	$2 + \gamma < 4.464966$	$2 + \varepsilon$
Maximum depth, on-line answers	$4 + 2\gamma < 8.929932$	$4 + \varepsilon$
Expected depth, binary answers and arbitrary priors	$\rho_{\text{two}} < 5.182976$	$2 + \varepsilon$
Expected depth, on-line answers and arbitrary priors	$2\rho_{\text{two}} < 10.365952$	$4 + \varepsilon$

The common-witness normalization is the structural step shared by both algorithms. In a flat state, it turns each supplied test into an opposite cap and represents that cap by at most two adjacent vertices of a convex hull. For each hypothesis, the positive hull answers form a circular interval. This representation keeps the query family fixed and gives the supplied tests an interval view.

The two objectives use this shared structure in different ways. To obtain the maximum-depth guarantee, we follow a common witness through an optimal tree. Restricting that tree to a pattern class removes the queries on the witness prefix, so that prefix gives the class a deadline. The adjacent-support lemma reduces these deadlines to singleton and pair blocks, which the scheduler orders. Endpoint decoding then uses cached negative answers to preserve the remaining height credit. When there are logarithmically many discovery slots, an exact Cartesian-tree dynamic program improves the schedule; the global cap and priority fallback handle the other cases. Together these steps give a factor below 4.464966.

For expected depth, the algorithm first learns each full hull pattern and then recurses within its class. Restricting an optimal tree to a class removes queries whose answers are already determined, and that saved cost helps pay for the first positive hull answer. The answers also reveal information about the pattern, which shortens the endpoint code. Exact integer-budget coverage over adjacent hull blocks selects when each pattern is found, while the all-negative waiting charge accounts for the zero-pattern route. The schedule search chooses the first two budgets, geometric shift, and endpoint mode together, because asking for a second endpoint can reduce later decoding work. The signed first-cover correction registers how moving a batch boundary changes each class's first-hit rank, while the endpoint term credits the information a second answer provides. Combined with restriction savings, these credits telescope over the recursive tree, so each stage cost is charged once.

The quasipolynomial algorithms store compatible discovery grades. This lets them handle all deadlines together. Their internal bounds have leading coefficient arbitrarily close to two and an additive count or entropy potential; exact logarithmic-depth lookahead absorbs that potential and gives the final $(2 + \varepsilon)$ ratio. The on-line-answer entries follow from the same factor-two simulation, using the binary guarantee with parameter $\varepsilon/2$. The algorithms run in quasipolynomial time.

For randomized per-hypothesis depth, the adversary chooses the hidden hypothesis before sampling the random tree. Multiplicative weights then constructs trees for a sequence of positive rational priors. A restricted mixture LP combines these trees to minimize the largest per-hypothesis expected depth. Proposition 11.1 identifies the hardest prior as the comparison benchmark, and Theorem 11.4 shows that the weight updates preserve the expected-depth oracle's approximation factor up to an arbitrary tolerance. The LP compresses the resulting strategy to at most n trees.

Since this objective differs from deterministic maximum depth, the LP value and its comparison priors provide a separate, instance-specific approximation certificate.

The small-cap results also delimit possible hardness constructions. With a distinguished all-miss hypothesis and fixed cap size, maximum depth is exactly solvable in polynomial time. For expected depth, the weighted singleton/pair path-and-cycle subclass with all singleton tests has an EPTAS. Consequently, APX-hardness for the current weighted pair-path reduction would imply P = NP. A broader uniform subclass also has a PTAS: fixed-size caps in convex position with an all-miss hypothesis, even when singleton tests are absent. Clearing short gaps permits exact optimization of the remaining block interleavings, and the prefix cost stays a small fraction of the full optimum. A constant-factor hardness construction must therefore use instances outside this uniform fixed-cap class.

The hardness reductions show that strong NP-completeness persists even when all hypotheses lie on a parabola and every query is a supplied line. The expected-depth reduction applies to arbitrary positive priors and to uniform priors: replacing each weight by a uniform cloud adds a fixed internal identification cost to every matching. Several approximation questions remain: whether constant-factor inapproximability holds, whether the polynomial factors can be lowered, and whether the quasipolynomial factors can fall below two.

A Fixed certificate for the two initial batches

The local proof uses a comparison distribution to show that some candidate schedule has enough combined first-hit and mode-saving credit while preserving the all-negative waiting bound. This distribution certifies existence; the algorithm searches candidate schedules and minimizes their rational score. We list the complete fixed distribution here, making the calibration reproducible directly from the stated data.

This table specifies the proof distribution in Theorem 6.5. The distribution is independent of the input. The shift has density $1/(u \ln \beta)$ on $[1, \beta)$. An entry $(q, r; \mathsf{L})$ selects that initial-budget variant and one first-batch saving minorant. Here $r = 0$ means budgets $q, k_2, k_3, \ldots$, and $r > 0$ means $q, r, k_2, k_3, \ldots$. All entries use

$$\mathsf{A}_q = M_q - \delta w q, \qquad \mathsf{C}_q = c_* M_q - d_* \delta w q.$$

A mixture $\theta(q, r; \mathsf{L}) + (1 - \theta)(q', r'; \mathsf{L}')$ is conditional on that row's shift range. It compares two actual variants; it never adds two savings from the same schedule.

Read the certificate in two stages: the small table supplies the conditional mixing weights, and the row tables select which variants to compare on each shift range. A row with one option assigns it all conditional mass. The twelve rational mixing weights are listed as integer numerators with common denominator 10^9:

i	1	2	3	4	5	6
$10^9\theta_i$	363738471	165441165	467444028	619060535	88721951	689586436
i	7	8	9	10	11	12
$10^9\theta_i$	751593847	874972087	599476351	160123414	234739978	835733626

The shift ranges also split at the integer breakpoints of k_2. Thus each $k_j = \lfloor \beta^j u \rfloor$, $j = 0, 1, 2$, is constant inside each listed range, and the signed correction of (11) is constant there.

Row	Shift range	Variant and saving minorant
1	$[1, 13/\beta^2)$	$(3, 0; \mathsf{C}_3)$
2	$[13/\beta^2, 14/\beta^2)$	$\theta_1(3, 0; \mathsf{C}_3) + (1-\theta_1)(4, 0; \mathsf{A}_4)$
3	$[14/\beta^2, 4/\beta)$	$(4, 0; \mathsf{A}_4)$
4	$[4/\beta, 15/\beta^2)$	$(4, 0; \mathsf{A}_4)$
5	$[15/\beta^2, 16/\beta^2)$	$(4, 0; \mathsf{A}_4)$
6	$[16/\beta^2, 17/\beta^2)$	$\theta_2(1, 5; \mathsf{C}_1) + (1-\theta_2)(4, 0; \mathsf{A}_4)$
7	$[17/\beta^2, 5/\beta)$	$(1, 5; \mathsf{C}_1)$
8	$[5/\beta, 18/\beta^2)$	$(1, 5; \mathsf{C}_1)$
9	$[18/\beta^2, 19/\beta^2)$	$(1, 5; \mathsf{C}_1)$
10	$[19/\beta^2, 20/\beta^2)$	$(1, 5; \mathsf{C}_1)$
11	$[20/\beta^2, 21/\beta^2)$	$\theta_3(1, 5; \mathsf{C}_1) + (1-\theta_3)(1, 6; \mathsf{C}_1)$
12	$[21/\beta^2, 6/\beta)$	$(1, 6; \mathsf{C}_1)$
13	$[6/\beta, 22/\beta^2)$	$(1, 6; \mathsf{C}_1)$
14	$[22/\beta^2, 23/\beta^2)$	$(1, 6; \mathsf{C}_1)$
15	$[23/\beta^2, 24/\beta^2)$	$(1, 6; \mathsf{C}_1)$
16	$[24/\beta^2, 7/\beta)$	$(1, 7; \mathsf{C}_1)$
17	$[7/\beta, 2)$	$\theta_4(1, 6; \mathsf{C}_1) + (1-\theta_4)(1, 7; \mathsf{C}_1)$
18	$[2, 25/\beta^2)$	$\theta_5(1, 6; \mathsf{C}_1) + (1-\theta_5)(2, 7; \mathsf{C}_2)$
19	$[25/\beta^2, 26/\beta^2)$	$(2, 7; \mathsf{C}_2)$
20	$[26/\beta^2, 27/\beta^2)$	$(2, 7; \mathsf{C}_2)$
21	$[27/\beta^2, 28/\beta^2)$	$\theta_6(2, 7; \mathsf{C}_2) + (1-\theta_6)(2, 8; \mathsf{C}_2)$

Row	Shift range	Variant and saving minorant
22	$[28/\beta^2, 8/\beta)$	$(2, 8; \mathsf{C}_2)$
23	$[8/\beta, 29/\beta^2)$	$(2, 8; \mathsf{C}_2)$
24	$[29/\beta^2, 30/\beta^2)$	$(2, 8; \mathsf{C}_2)$
25	$[30/\beta^2, 31/\beta^2)$	$(2, 8; \mathsf{C}_2)$
26	$[31/\beta^2, 9/\beta)$	$(2, 9; \mathsf{C}_2)$
27	$[9/\beta, 32/\beta^2)$	$\theta_7(2, 8; \mathsf{C}_2) + (1-\theta_7)(2, 9; \mathsf{C}_2)$
28	$[32/\beta^2, 33/\beta^2)$	$(2, 9; \mathsf{C}_2)$
29	$[33/\beta^2, 34/\beta^2)$	$(2, 9; \mathsf{C}_2)$
30	$[34/\beta^2, 35/\beta^2)$	$\theta_8(2, 9; \mathsf{C}_2) + (1-\theta_8)(2, 10; \mathsf{C}_2)$
31	$[35/\beta^2, 10/\beta)$	$(2, 10; \mathsf{C}_2)$
32	$[10/\beta, 36/\beta^2)$	$\theta_9(2, 10; \mathsf{C}_2) + (1-\theta_9)(3, 10; \mathsf{C}_3)$
33	$[36/\beta^2, 37/\beta^2)$	$(3, 10; \mathsf{C}_3)$
34	$[37/\beta^2, 3)$	$(3, 10; \mathsf{C}_3)$
35	$[3, 38/\beta^2)$	$(3, 10; \mathsf{C}_3)$
36	$[38/\beta^2, 11/\beta)$	$\theta_{10}(3, 10; \mathsf{C}_3) + (1-\theta_{10})(3, 11; \mathsf{C}_3)$
37	$[11/\beta, 39/\beta^2)$	$(3, 10; \mathsf{C}_3)$
38	$[39/\beta^2, 40/\beta^2)$	$(3, 11; \mathsf{C}_3)$
39	$[40/\beta^2, 41/\beta^2)$	$(3, 11; \mathsf{C}_3)$
40	$[41/\beta^2, 42/\beta^2)$	$(3, 11; \mathsf{C}_3)$
41	$[42/\beta^2, 12/\beta)$	$\theta_{11}(3, 11; \mathsf{C}_3) + (1-\theta_{11})(3, 12; \mathsf{C}_3)$
42	$[12/\beta, 43/\beta^2)$	$(3, 12; \mathsf{C}_3)$
43	$[43/\beta^2, \beta)$	$\theta_{12}(3, 12; \mathsf{C}_3) + (1-\theta_{12})(3, 13; \mathsf{C}_3)$

To use the table, first integrate the log-uniform shift density over each row, then multiply by that row's conditional option weights. The budgets are constant within a row, so its signed correction and first-batch minorant contribute fixed coefficients. Summing these contributions gives exactly the rank credits used in the main proof, through the coefficient definitions in (14). For each row $[l, h)$ and option of conditional weight v, compute $P = \ln(h/l)/\ln\beta$. Starting with $a_t = e_\beta(t)$ and $\mu = 0$, add $Pv[d(u,t) + s\mathbf{1}_{\{t \le q\}}]$ to a_t and $Pv\rho$ to μ, where the saving minorant is $sM_q - \rho\delta w$. The main text gives the complete finite-rank, zero-length, and tail checks. The file `expected-two-phase-`

`check.py`, using the same rational data in `expected-two-phase-certificate.json`, verifies the row coefficients and certificate inequalities from the displayed data.

Complete coefficient accumulation. Index the rows by i and the options in a row by j. Let the row be $[l_i, h_i)$ and its conditional option weights be v_{ij}. Put

$$P_i = \int_{l_i}^{h_i} \frac{du}{u \ln \beta} = \frac{\ln(h_i/l_i)}{\ln \beta}, \qquad \sum_i P_i = 1, \quad \sum_j v_{ij} = 1.$$

For the first identity, the antiderivative is $\ln u / \ln \beta$. Summing the consecutive row integrals then gives $\ln(\beta/1)/\ln \beta = 1$; each conditional option sum is one by the listed complementary mixing weights. For an option with budgets (q_{ij}, r_{ij}), the minorant coefficients are

$$(s_{ij}, \rho_{ij}) = \begin{cases} (1, q_{ij}), & \mathsf{A}_{q_{ij}}, \\ (c_*, d_* q_{ij}), & \mathsf{C}_{q_{ij}}, \end{cases} \qquad c_* = \frac{228683}{103683}, \quad d_* = \frac{250000}{103683}.$$

These rational values are obtained by substituting p_* in $c_* = (2 - p_*)/(1 - p_*)$ and $d_* = 2/(1 - p_*)$. Write k_{0i}, k_{1i}, k_{2i} for that row's constant geometric budgets and choose any $u_i \in [l_i, h_i)$. The full sums are

$$\begin{aligned} a_t &= e_\beta(t) + \sum_i P_i \sum_j v_{ij} \left[d_{q_{ij}, r_{ij}}(u_i, t) + s_{ij} \mathbf{1}_{\{t \le q_{ij}\}} \right], \\ \mu &= \sum_i P_i \sum_j v_{ij} \rho_{ij}, \\ D_0(k) &= \sum_i P_i \sum_j v_{ij} \left[q_{ij} \mathbf{1}_{\{q_{ij} < k\}} + r_{ij} \mathbf{1}_{\{r_{ij} < k\}} - k_{0i} \mathbf{1}_{\{k_{0i} < k\}} - k_{1i} \mathbf{1}_{\{k_{1i} < k\}} \right]. \end{aligned}$$

To verify that these budgets are constant throughout each half-open row, fix $j \in \{0, 1, 2\}$. Since the floor is nondecreasing, its minimum on $[l_i, h_i)$ occurs at l_i. Its largest value before the excluded right endpoint is $\lceil \beta^j h_i \rceil - 1$. Therefore the endpoint identity below certifies every shift in the row:

$$\lfloor \beta^j l_i \rfloor = \lceil \beta^j h_i \rceil - 1 = k_{ji}.$$

A worked row. In row 1, $1 \le u < 13/\beta^2$ gives $(k_0, k_1, k_2) = (1, 3, 12)$, and the sole option is $(q, r) = (3, 0)$ with minorant C_3. Its probability is

$$P_1 = \frac{\ln(13/\beta^2)}{\ln \beta} = \frac{\ln 13}{\ln \beta} - 2, \qquad 0.041455991537 < P_1 < 0.041455991538.$$

Substitution in (11), with these constant budgets, cancels both terms at rank three:

$$\begin{aligned} d_{3,0}(u, t) &= 1 - 3 + 4\mathbf{1}_{\{t>1\}} + 15\mathbf{1}_{\{t>3\}} - 15\mathbf{1}_{\{t>3\}} \\ &= -2 + 4\mathbf{1}_{\{t>1\}}. \end{aligned}$$

Consequently (14) and the zero-route correction (12) give this row's contributions as

$$\begin{aligned} \Delta a_t &= P_1 \left[-2 + 4\mathbf{1}_{\{t>1\}} + c_* \mathbf{1}_{\{t \le 3\}} \right], \\ \Delta \mu &= 3 d_* P_1, \\ \Delta D_0(k) &= P_1 \left[3\mathbf{1}_{\{3<k\}} - \mathbf{1}_{\{1<k\}} - 3\mathbf{1}_{\{3<k\}} \right] = -P_1 \mathbf{1}_{\{k>1\}}. \end{aligned}$$

In particular, its rank-one contribution is $P_1(c_* - 2)$, its contributions at ranks two and three are $P_1(c_* + 2)$, and every later contribution is $2P_1$. Rows with two options use the same calculation for each option and multiply the results by its conditional weight before adding. For example, row 2 contributes $P_2[\theta_1(c_* - 2) + (1 - \theta_1)(1 - 3)]$ to a_1 and $P_2[3d_*\theta_1 + 4(1 - \theta_1)]$ to μ. Each execution chooses one valid alternative. Consequently, the expected row contribution is the weighted average of the credit from each alternative.

References

[1] Uriel Feige, László Lovász, and Prasad Tetali. *Approximating Min-sum Set Cover*. *Algorithmica*, 40(4):219–234, 2004. `https://doi.org/10.1007/s00453-004-1110-5`. Primary journal manuscript: `https://tetali.math.gatech.edu/PUBLIS/mssc_final.pdf`. Lemma 5.5 adapts the first-hit viewpoint to circular interval demands and proves the ordering guarantee used here.

[2] Sanjoy Dasgupta. Analysis of a greedy active learning strategy. In *Advances in Neural Information Processing Systems 17*, pages 337–344, 2004. `https://papers.nips.cc/paper_files/paper/2004/hash/c61fbef63df5ff317aecdc3670094472-Abstract.html`.

[3] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*, 2nd edition. Wiley-Interscience, 2006. Chapters 2 and 5 cover the entropy chain rule, the log-sum inequality, Kraft's inequality, and binary prefix coding.

[4] Amin Karbasi and Morteza Zadimoghaddam. Constrained binary identification problem. In *30th International Symposium on Theoretical Aspects of Computer Science (STACS 2013)*, LIPIcs 20, pages 550–561, 2013. `https://doi.org/10.4230/LIPIcs.STACS.2013.550`.

[5] Micah Adler and Brent Heeringa. Approximating optimal binary decision trees. *Algorithmica*, 62:1112–1121, 2012. `https://doi.org/10.1007/s00453-011-9510-9`.

[6] Ferdinando Cicalese, Eduardo Sany Laber, and Aline Medeiros Saettler. Decision trees for function evaluation: simultaneous optimization of worst and expected cost. *Algorithmica*, 79(3):763–796, 2017. `https://doi.org/10.1007/s00453-016-0225-9`.

[7] Anupam Gupta, Viswanath Nagarajan, and R. Ravi. Approximation algorithms for optimal decision trees and adaptive TSP problems. *Mathematics of Operations Research*, 42(3):876–896, 2017. `https://doi.org/10.1287/moor.2016.0831`.

[8] Venkatesan T. Chakaravarthy, Vinayaka Pandit, Sambuddha Roy, Pranjal Awasthi, and Mukesh K. Mohania. Decision trees for entity identification: approximation algorithms and hardness results. *ACM Transactions on Algorithms*, 7(2), Article 15, 2011. `https://doi.org/10.1145/1921659.1921661`.

[9] Esther M. Arkin, Henk Meijer, Joseph S. B. Mitchell, David Rappaport, and Steven S. Skiena. Decision trees for geometric models. *International Journal of Computational Geometry and Applications*, 8(3):343–363, 1998. `https://doi.org/10.1142/S0218195998000175`.

[10] Jiří Matoušek. *Lectures on Discrete Geometry*. Graduate Texts in Mathematics 212, Springer, 2002. Standard convex-hull terminology and the planar form of Helly's theorem.

[11] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*, 3rd edition. Springer, 2008. Planar convex-hull algorithms and their running times.

[12] L. G. Khachiyan. A polynomial algorithm in linear programming. *Soviet Mathematics Doklady*, 20:191–194, 1979.

[13] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. Problem SP15 supplies strong NP-completeness of 3-Partition, including the restriction $B/4 < a_i < B/2$.

[14] Sariel Har-Peled and Mira Lee. Weighted geometric set cover problems revisited. *Journal of Computational Geometry*, 3(1):65–85, 2012. Exact minimum-weight planar halfplane cover: `https://doi.org/10.20382/jocg.v3i1a4`.

[15] Yurii Nesterov and Arkadii Nemirovskii. *Interior-Point Polynomial Algorithms in Convex Programming*. SIAM, 1994. Polynomial-time convex quadratic optimization; author lecture notes: `https://www2.isye.gatech.edu/~nemirovs/Lect_IPM.pdf`.

[16] Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of Computing*, 8(6):121–164, 2012. `https://theoryofcomputing.org/articles/v008a006/v008a006.pdf`.

[17] Yoav Freund and Robert E. Schapire. Adaptive game playing using multiplicative weights. *Games and Economic Behavior*, 29:79–103, 1999. Author manuscript: `https://cseweb.ucsd.edu/~yfreund/papers/games_long.pdf`.